DataStruct

目录

3 数据结构 1
3.1 线性结构、堆与哈希表 1
3.2 DSU 23
3.3 BIT 41
3.4 RMQ、静态区间查询与序列树 47
3.5 基础 SegTree 61
3.6 动态、可持久化与可合并线段树 86
3.7 李超树 124
3.8 平衡树与序列维护 138
3.9 分块、莫队与离线分治 186
3.10 其他高级结构 219
3.11 多维与嵌套数据结构 246

3. 数据结构

3.1 线性结构、堆与哈希表

3.1.1 对顶堆

动态分界 / 中位数 / Top-k

> ⚠ 如果需要删除元素，可以使用 `multiset` 或可删除堆替代 `priority_queue`

```cpp
template<class T>
// 先复制本页后文的 ErasableHeap 定义，再使用 MedianHeap。
struct MedianHeap {
    ErasableHeap<T> lo;
    ErasableHeap<T, greater<T>> hi;
    void balance() {
        while (lo.size() > hi.size() + 1) {
            T x = lo.top();
            lo.pop();
            hi.push(x);
        }
        while (lo.size() < hi.size()) {
            T x = hi.top();
            hi.pop();
            lo.push(x);
        }
    }
    void insert(const T &v) {
        if (lo.empty() || v ≤ lo.top()) lo.push(v);
        else hi.push(v);
        balance();
    }
    int size() const { return lo.size() + hi.size(); }
    bool empty() const { return lo.empty(); }
    const T &lower() { return lo.top(); }
    pair<T, T> middle() {
        if (lo.size() == hi.size()) return { lo.top(), hi.top() };
        return { lo.top(), lo.top() };
    }
    // 需要删除时
    void erase(const T &v) {
        if (v ≤ lo.top()) lo.erase(v);
        else hi.erase(v);
        balance();
    }
};
```

成员变量

对顶堆，维护两个堆：

- `lo`：大根堆，保存较小的一半元素
- `hi`：小根堆，保存较大的一半元素
- 始终满足：
  - `lo.size() == hi.size()` 或 `lo.size() == hi.size() + 1`
- 因此奇数个元素时，中位数始终位于 `lo.top()`

接口

- `insert(v)`：插入元素 `v`，均摊 $O(\log n)$
- `size()`：返回当前元素个数，$O(1)$
- `empty()`：判断是否为空，$O(1)$

- `erase(v)`：删除一个值为 `v` 的元素，要求当前结构非空且 `v` 存在，均摊 $O(\log n)$
- `lower()`：返回下中位数，要求非空，均摊 $O(1)$
- `middle()`：返回 `{下中位数，上中位数}`，要求非空，均摊 $O(1)$
  - 奇数个元素时两个值相同
  - 偶数个元素时分别为两个中间元素

注意事项

- `lower()`、`middle()` 都要求结构非空
- `erase(v)` 要求 `v` 当前确实存在；本结构本身不维护频率表，因此不会检查非法删除
- 如果题目中的删除操作不能保证元素存在，应额外维护 `map<T, int>` / `unordered_map<T, int>` 记录出现次数，删除前自行判断
- `ErasableHeap` 的 `size()` 是**逻辑大小**，而 `a.size()` 可能更大，因为尚未到堆顶的删除元素仍保存在物理堆中
- 因此大量“插入后删除、但被删元素长期不到堆顶”的情况下，实际内存不会立即释放；总空间与历史插入次数有关

删除操作不保证合法

如果输入可能尝试删除不存在的元素，应自行维护频率：

```
MedianHeap<int> med;
map<int, int> cnt;
auto insert = [&](int x) {
    med.insert(x);
    ++ cnt[x];
};
auto erase = [&](int x) {
    if (!cnt[x]) return;
    med.erase(x);
    if (!-- cnt[x]) cnt.erase(x);
};
```

TopKSum

- Top-k 元素和
- 第 k 大 / 第 k 小
- 动态 Top-k
- 滑动窗口 Top-k
- 固定比例元素和

```
// MAX=true: 维护最大的 k 个元素; MAX=false: 维护最小的 k 个元素
// 默认依赖 ErasableHeap; 若不需要删除, 可将 in / out 改为对应 priority_queue, 并删除 erase()
template<class T, bool MAX = true, class Sum = T>
struct TopKSum {
    using InCmp = conditional_t<MAX, greater<T>, less<T>>;
    using OutCmp = conditional_t<MAX, less<T>, greater<T>>;
    int k;
    Sum sum{};
    ErasableHeap<T, InCmp> in;   // 被选中的 k 个元素, 堆顶为其中最差者
    ErasableHeap<T, OutCmp> out; // 其余元素, 堆顶为其中最优者
    explicit TopKSum(int k_ = 0) : k(k_) {}
    // ===== 基础维护: insert() / erase() / set_k() 均依赖, 不建议删除 =====
    bool better(const T &x, const T &y) const {
        if constexpr (MAX) return x > y;
        else return x < y;
    }
    int need() const { return min(k, size()); }
    void move_in() {
        T x = out.top();
        out.pop();
        in.push(x);
        sum += Sum(x);
    }
    void move_out() {
        T x = in.top();
        in.pop();
        out.push(x);
        sum -= Sum(x);
    }
    void balance() {
```

```
        while (in.size() < need()) move_in();
        while (in.size() > need()) move_out();
    }
    // ===== 基础接口 =====
    int size() const { return in.size() + out.size(); }
    bool empty() const { return size() == 0; }
    Sum query() const { return sum; }
    const T &kth() { return in.top(); }
    void insert(const T &v) {
        if (k > 0 && (in.empty() || !better(in.top(), v))) {
            in.push(v);
            sum += Sum(v);
        } else out.push(v);
        balance();
    }
    // ===== 删除接口：若不需要删除，可删除本函数，并将 in / out 改为 priority_queue =====
    void erase(const T &v) {
        if (!in.empty() && !better(in.top(), v)) {
            in.erase(v);
            sum -= Sum(v);
        } else out.erase(v);
        balance();
    }
    // ===== 动态修改 k：若 k 固定，可直接删除 =====
    void set_k(int k_) {
        k = k_;
        balance();
    }
};
```

如果完全不需要删除，只需把：

```
ErasableHeap<T, InCmp> in;
ErasableHeap<T, OutCmp> out;
```

改为：

```
priority_queue<T, vector<T>, InCmp> in;
priority_queue<T, vector<T>, OutCmp> out;
```

然后删除：

```
void erase(const T &v) { ... }
```

成员变量

- `k`：需要维护的元素数量
- `sum`：当前被选中的 `min(k, size())` 个元素之和
- `in`：被选中的 Top-k 集合
  - `MAX = true` 时保存最大的 k 个元素，是小根堆
  - `MAX = false` 时保存最小的 k 个元素，是大根堆
  - 堆顶始终是当前被选中元素中最差的一个
- `out`：其余元素
  - 堆顶始终是未被选中元素中最优的一个
- `better(x, y)`：判断 `x` 是否比 `y` 更应该进入 Top-k
  - `MAX = true`：等价于 `x > y`
  - `MAX = false`：等价于 `x < y`

  例如维护最大的 k 个元素时：

```
out               in
其余元素        最大的 k 个元素
    ...  6 | 7  8  10
         ↑   ↑
     out.top  in.top
     最优落选   最差入选
```

始终保持所有 `in` 中元素不劣于所有 `out` 中元素

接口

- `TopKSum(k)`：构造空结构，维护 Top-k，要求 `k ⩾ 0`，$O(1)$
- `size()`：返回当前元素数量，$O(1)$
- `empty()`：判断是否为空，$O(1)$
- `insert(v)`：插入元素 `v`，均摊 $O(\log n)$
- `erase(v)`：删除一个值为 `v` 的元素，要求该元素当前存在，均摊 $O(\log n)$
- `query()`：返回当前最优 `min(k, n)` 个元素之和，$O(1)$
- `kth()`：返回第 k 优元素，要求 `1 ⩽ k ⩽ size()`，均摊 $O(\log n)$
- `set_k(k)`：动态修改需要维护的元素数量，要求 `k ⩾ 0`

其中：

- `TopKSum<i64> q(k);` 维护最大的 k 个元素之和
- `TopKSum<i64, false> q(k);` 维护最小的 k 个元素之和

注意事项

- 当当前元素数小于 `k` 时，`in` 保存全部元素，因此：`q.query()` 返回所有元素之和。也就是说实际维护数量为：`min(k, q.size())`，这样可以从空结构开始直接设置任意 `k`，无需等元素数量达到 `k` 后再修改
- `kth()` 只有在：`1 ⩽ k && k ⩽ size()` 时才表示真正的第 k 优元素。例如维护最大值时：

```
10 8 7 | 6 3 1
    ↑
  kth()
```

`kth()` 为第 k 大

- `erase(v)` 要求当前至少存在一个 `v`。底层 `ErasableHeap` 不检查非法删除
- 允许重复元素

例如：

```
5 5 5 3
```

维护最大的两个元素时，两个 `5` 都可以位于 `in`

- 删除边界值时，即使相同值同时存在于 `in` 和 `out`，代码优先删除 `in` 中的一个相同值，再由 `balance()` 补回需要的元素；因为相同值不可区分，不影响维护的多重集合与答案
- `Sum` 是求和使用的类型，默认等于 `T`
- 若不需要删除，建议按代码顶部说明改成 `priority_queue` 版本，可以删除懒删除带来的额外空间与常数
- 默认可删除版本的实际空间与历史插入次数有关，而不是当前元素数量，因为被懒删除但尚未到堆顶的元素不会立即从物理堆中移除

常见用法

最大的 k 个元素之和

```
TopKSum<i64> q(k);
for (i64 x : a) q.insert(x);
cout << q.query() << '\n';
```

最小的 k 个元素之和

```
TopKSum<i64, false> q(k);
for (i64 x : a) q.insert(x);
cout << q.query() << '\n';
```

动态插入和删除

```
TopKSum<i64> q(k);
q.insert(10);
q.insert(3);
q.insert(7);
cout << q.query() << '\n';
q.erase(7);
```

滑动窗口最大的 k 个元素之和

```
TopKSum<i64> q(k);
for (int i = 0; i < n; ++ i) {
    q.insert(a[i]);
```

```
    if (i ≥ len) q.erase(a[i - len]);
    if (i ≥ len - 1) {
        cout << q.query() << '\n';
    }
}
```

动态修改 k

```
q.set_k(new_k);
cout << q.query() << '\n';
```

例如维护所有元素中最大的前 10%，向上取整：

```
q.set_k((q.size() + 9) / 10);
```

若每次插入 / 删除后比例都要变化，就在操作后重新调用 `set_k()`

查询第 k 大 / 第 k 小

维护最大 k 个：

```
TopKSum<int> q(k);
for (int x : a) q.insert(x);
cout << q.kth() << '\n'; // 第 k 大
```

维护最小 k 个：

```
TopKSum<int, false> q(k);
for (int x : a) q.insert(x);
cout << q.kth() << '\n'; // 第 k 小
```

复杂度

设当前有 n 个元素

操作	时间复杂度
`size()` / `empty()`	$O(1)$
`query()`	$O(1)$
`insert()`	均摊 $O(\log n)$
`erase()`	均摊 $O(\log n)$
`kth()`	均摊 $O(\log n)$
`set_k(k')`	$O((\lvert k' - k \rvert + 1)\log n)$

固定 `k` 时，单次 `insert()` / `erase()` 后最多只需要跨两个堆移动常数个元素
若 `k` 只改变 $O(1)$，则 `set_k()` 为均摊 $O(\log n)$；最坏一次从 `0` 改为 `n` 等情况需要 $O(n \log n)$
空间复杂度：

- 可删除版：历史插入 q 次后最坏 $O(q)$
- `priority_queue` 无删除版：$O(n)$

可删除堆

支持任意元素懒删除的优先队列

```
template<class T, class Compare = less<T>>
struct ErasableHeap {
    priority_queue<T, vector<T>, Compare> a, del;
    int cnt{ 0 };
    void normalize() {
        while (!a.empty() && !del.empty() && a.top() == del.top()) {
            a.pop();
            del.pop();
        }
    }
    void push(const T &v) {
```

```
        a.push(v);
        ++ cnt;
    }
    void erase(const T &v) {
        del.push(v);
        -- cnt;
        normalize();
    }
    const T &top() {
        normalize();
        return a.top();
    }
    void pop() {
        normalize();
        a.pop();
        -- cnt;
        normalize();
    }
    int size() const { return cnt; }
    bool empty() const { return cnt == 0; }
};
```

成员变量

- `a`：实际插入元素的堆
- `del`：等待删除元素的堆
- `cnt`：当前**逻辑上仍存在**的元素数量
  当 `a.top() == del.top()` 时，通过 `normalize()` 同时弹出两个元素完成真正删除

```
ErasableHeap<int> mx;                // 大根堆
ErasableHeap<int, greater<int>> mn;  // 小根堆
```

接口

- `push(v)`：插入 `v`，$O(\log n)$
- `erase(v)`：删除一个值为 `v` 的元素，要求该元素当前存在，均摊 $O(\log n)$
- `top()`：返回当前堆顶，要求非空，均摊 $O(\log n)$，通常可视为均摊 $O(1)$ 次查询加懒删除
- `pop()`：删除当前堆顶，要求非空，均摊 $O(\log n)$
- `size()`：返回当前逻辑元素数量，$O(1)$
- `empty()`：判断逻辑上是否为空，$O(1)$

> **Important**
>
> `erase(v)` **不能判断** `v` **是否存在**。如果删除不存在的元素，`cnt` 和懒删除队列都会被破坏
> 因此调用方必须保证当前至少存在一个 `v`

二叉堆

```
template<class T, class Compare = less<T>>
struct BinaryHeap {
    Compare better;
    vector<T> a;
    explicit BinaryHeap(Compare better_ = Compare()) : better(better_) {}
    BinaryHeap(vector<T> a_, Compare better_ = Compare()) : better(better_), a(move(a_)) {
        for (int i = size() / 2 - 1; i >= 0; -- i) sift_down(i);
    }
    void sift_up(int x) {
        while (x > 0) {
            int p{ x - 1 >> 1 };
            if (!better(a[x], a[p])) break;
            swap(a[x], a[p]);
            x = p;
        }
    }
    void sift_down(int x) {
        while (true) {
```

```
            int y{ x }, l{ x << 1 | 1 }, r{ x + 1 << 1 };
            if (l < size() && better(a[l], a[y])) y = l;
            if (r < size() && better(a[r], a[y])) y = r;
            if (y == x) break;
            swap(a[x], a[y]);
            x = y;
        }
    }
    // ===== 基础接口 =====
    bool empty() const { return a.empty(); }
    int size() const { return int(a.size()); }
    const T &top() const { return a[0]; }
    void push(T v) {
        a.push_back(move(v));
        sift_up(size() - 1);
    }
    void pop() {
        swap(a[0], a.back());
        a.pop_back();
        if (!a.empty()) sift_down(0);
    }
};
```

成员变量

- `a`：完全二叉堆的顺序存储，节点 `x`：
  - 父节点为 `(x - 1) / 2`
  - 左儿子为 `2 * x + 1`
  - 右儿子为 `2 * x + 2`
- `better(x, y)`：判断 `x` 是否比 `y` 更优
  - 默认 `less<T>`：较小元素优先，为**小根堆**
  - `greater<T>`：较大元素优先，为**大根堆**

> **Important**
>
> `Compare` 的含义是“第一个参数是否应该排在第二个参数前面”，因此与 `priority_queue` 的默认表现相反：
>
> ```
> BinaryHeap<int> q;                 // 小根堆
> BinaryHeap<int, greater<int>> q;   // 大根堆
> ```

接口

- `BinaryHeap(better)`：构造空堆，可传入比较器，$O(1)$
- `BinaryHeap(a, better)`：由序列 `a` 使用 Floyd 建堆，$O(n)$
- `empty()`：判断堆是否为空，$O(1)$
- `size()`：返回元素数量，$O(1)$
- `top()`：返回当前最优元素，要求堆非空，$O(1)$
- `push(v)`：插入元素 `v`，$O(\log n)$
- `pop()`：删除当前最优元素，要求堆非空，$O(\log n)$

内部维护：

- `sift_up(x)`：从节点 `x` 向上恢复堆序，$O(\log n)$
- `sift_down(x)`：从节点 `x` 向下恢复堆序，$O(\log n)$

注意事项

- `top()` 和 `pop()` 要求堆非空
- 默认 `Compare = less<T>` 得到的是**小根堆**，与 `priority_queue<T>` 默认的大根堆相反
- `Compare` 应满足严格弱序；若：`better(x, y)` 为 `true`，表示 `x` 应比 `y` 更靠近堆顶
- 由整个数组初始化时，应直接使用：`BinaryHeap<int> q(a);` 建堆复杂度为 $O(n)$；逐个 `push()` 是 $O(n \log n)$
- `BinaryHeap(vector<T> a_)` 按值接收数组，因此：
  - 传左值时会复制；
  - 不再需要原数组时可：`BinaryHeap<int> q(move(a));` 避免复制
- 当前普通二叉堆只维护**元素值**，不维护元素身份或位置，因此不适合：
  - 按编号修改元素
  - 删除指定元素
  - `decrease-key / increase-key`

- 高效合并两个堆
  这些需求分别使用 `IndexedHeap`、可删除堆或可并堆
- 若只需要普通优先队列操作，应直接使用 STL `priority_queue`；这份实现主要用于需要手写、修改或扩展二叉堆内部操作的场景

常见用法

- 小根堆：`BinaryHeap<int> q;`
- 大根堆：`BinaryHeap<int, greater<int>> q;`

$O(n)$ 建堆

```
vector<int> a{ 5, 2, 7, 1, 3 };
BinaryHeap<int> q(move(a));
```

自定义优先级

```
struct Compare {
    bool operator()(const pair<int, int> &a, const pair<int, int> &b) const {
        return a.second < b.second;
    }
};
BinaryHeap<pair<int, int>, Compare> q;
```

此时 `second` 较小的元素优先
如果需要稳定处理相同优先级，应把次关键字也显式加入比较：

```
struct Compare {
    bool operator()(const pair<int, int> &a, const pair<int, int> &b) const {
        if (a.second != b.second) return a.second < b.second;
        return a.first < b.first;
    }
};
```

复杂度

操作	时间复杂度
构造空堆	$O(1)$
Floyd 建堆	$O(n)$
`empty()` / `size()`	$O(1)$
`top()`	$O(1)$
`push()`	$O(\log n)$
`pop()`	$O(\log n)$
`sift_up()` / `sift_down()`	$O(\log n)$

空间复杂度：$O(n)$

常见用法

小根堆

```
BinaryHeap<int> q;
```

大根堆

```
BinaryHeap<int, greater<int>> q;
```

自定义优先级

```
struct Compare {
    bool operator()(const pair<int, int> &a, const pair<int, int> &b) const {
        return a.second < b.second;
    }
};
BinaryHeap<pair<int, int>, Compare> q;
```

此时 `second` 较小的元素优先

索引堆

与普通二叉堆相比，额外维护稳定的元素编号 `id` 与堆位置之间的双向映射，因此可以按 `id` 在 $O(\log n)$ 内修改或删除任意元素

```cpp
template<class T, class Compare = less<T>>
struct IndexHeap {
    int n;
    Compare better;
    vector<int> heap, pos;
    vector<T> val;
    explicit IndexHeap(int n_, Compare better_ = Compare()) : n(n_), better(better_), pos(n, -1), val(n) {}
    IndexHeap(vector<T> val_, Compare better_ = Compare()) : n(val_.size()), better(better_), heap(n), pos(n), val(move(val_))
{
        ranges::iota(heap, 0);
        ranges::iota(pos, 0);
        for (int i = n / 2 - 1; i >= 0; -- i) sift_down(i);
    }
    // ===== 基础维护：所有修改操作依赖，不建议删除 =====
    bool better_id(int x, int y) const {
        if (better(val[x], val[y])) return true;
        if (better(val[y], val[x])) return false;
        return x < y;
    }
    void swap_node(int x, int y) {
        swap(heap[x], heap[y]);
        pos[heap[x]] = x;
        pos[heap[y]] = y;
    }
    void sift_up(int x) {
        while (x > 0) {
            int p{ x - 1 >> 1 };
            if (!better_id(heap[x], heap[p])) break;
            swap_node(x, p);
            x = p;
        }
    }
    void sift_down(int x) {
        while (true) {
            int y{ x }, l{ x << 1 | 1 }, r{ x + 1 << 1 };
            if (l < size() && better_id(heap[l], heap[y])) y = l;
            if (r < size() && better_id(heap[r], heap[y])) y = r;
            if (y == x) break;
            swap_node(x, y);
            x = y;
        }
    }
    void fix(int x) {
        if (x > 0 && better_id(heap[x], heap[x - 1 >> 1])) sift_up(x);
        else sift_down(x);
    }
    // ===== 基础接口 =====
    bool empty() const { return heap.empty(); }
    int size() const { return int(heap.size()); }
    bool contains(int id) const { return pos[id] != -1; }
    int top_id() const { return heap[0]; }
    const T &top() const { return val[heap[0]]; }
    void push(int id, T v) {
        val[id] = move(v);
        pos[id] = size();
        heap.push_back(id);
        sift_up(size() - 1);
    }
    void set(int id, T v) {
        val[id] = move(v);
        fix(pos[id]);
    }
    void erase(int id) {
        int x{ pos[id] };
        swap_node(x, size() - 1);
```

```
        heap.pop_back();
        pos[id] = -1;
        if (x < size()) fix(x);
    }
    void pop() { erase(top_id()); }
};
```

成员变量

- `n`：允许使用的 `id` 数量，合法编号为 `[0, n)`
- `heap[p]`：堆中位置 `p` 对应的元素 `id`
- `pos[id]`：编号 `id` 当前在 `heap` 中的位置；不存在时为 `-1`
- `val[id]`：编号 `id` 对应的权值
- `better(x, y)`：判断权值 `x` 是否比 `y` 更优
  - 默认 `less<T>`：较小权值优先，为**小根堆**
  - `greater<T>`：较大权值优先，为**大根堆**

始终维护：

```
heap[pos[id]] == id;
pos[heap[p]] == p;
```

对于当前存在的所有元素成立

> **Important**
>
> `id` 是元素的**稳定身份**，而 `pos[id]` 是会随堆调整不断变化的内部位置
> 调用接口时始终传 `id`，不要保存或使用某次操作后的堆位置

比较规则

`better_id(x, y)` 比较的是编号 `x`、`y` 对应的权值：`better(val[x], val[y])`
当两个权值在 `Compare` 意义下等价时，编号较小者优先，因此堆顶结果具有确定性
例如：

```
IndexHeap<int> q(n);                // 小根堆
IndexHeap<int, greater<int>> q(n);  // 大根堆
```

与普通 `BinaryHeap` 一样，`Compare` 表示“谁更优”，因此默认 `less<T>` 得到小根堆

接口

- `IndexHeap(n, better)`：构造空索引堆，允许 `id` 为 `[0, n)`，$O(n)$
- `IndexHeap(val, better)`：将 `val[i]` 作为编号 `i` 的权值，将全部 `0..n-1` 加入堆并使用 Floyd 建堆，$O(n)$
- `empty()`：判断当前是否为空，$O(1)$
- `size()`：返回当前堆中的元素数量，$O(1)$
- `contains(id)`：判断编号 `id` 当前是否在堆中，要求 `0 ≤ id < n`，$O(1)$
- `top_id()`：返回当前最优元素的编号，要求非空，$O(1)$
- `top()`：返回当前最优元素的权值，要求非空，$O(1)$
- `push(id, v)`：插入编号 `id`、权值 `v`，要求 `id` 合法且当前不存在，$O(\log n)$
- `set(id, v)`：将已存在的 `id` 的权值修改为 `v`，要求 `id` 当前存在，$O(\log n)$
- `erase(id)`：删除指定 `id`，要求 `id` 当前存在，$O(\log n)$
- `pop()`：删除当前最优元素，要求非空，$O(\log n)$

内部维护：

- `sift_up(p)`：从堆位置 `p` 向上恢复堆序，$O(\log n)$
- `sift_down(p)`：从堆位置 `p` 向下恢复堆序，$O(\log n)$
- `fix(p)`：权值改变或任意位置被替换后，根据实际失衡方向选择上浮或下沉，$O(\log n)$
- `swap_node(x, y)`：交换两个堆位置，并同步维护 `pos`，$O(1)$

注意事项

- 所有 `id` 均为 `0base`，范围固定为 `[0, n)`
- 同一个 `id` 在堆中最多出现一次
- `push(id, v)` 要求 `id` 当前不存在；若已经存在且只是修改权值，应使用：`q.set(id, v);`
- `set(id, v)` 与 `erase(id)` 要求 `id` 当前存在
- `top()`、`top_id()`、`pop()` 要求堆非空
- `erase(id)` 删除后：`q.contains(id) == false`
  但 `val[id]` 中仍可能保留旧值；只有 `contains(id)` 能表示该编号当前是否属于堆

- 删除后的 `id` 可以再次插入：

```
q.erase(id);
q.push(id, new_value);
```

- 不要直接修改：`q.val[id] = x;` 这不会恢复堆序。权值变化必须通过：`q.set(id, x);`
- 同样不要直接修改 `heap` 或 `pos`，否则会破坏双向映射
- `IndexHeap(n)` 会直接分配长度为 `n` 的 `pos` 和 `val`，因此构造本身为 $O(n)$，空间也与编号范围而不是当前堆大小有关。
- `T` 在空堆构造时需要能够默认构造，因为会创建：`vector<T> val(n);`
- 如果编号非常稀疏且范围极大，不适合直接使用这一实现；此时应改用哈希表维护 `id → pos`
- 两个索引堆不能像左偏树 / 斜堆一样高效合并；需要频繁合并堆时应使用可并堆

> Tip
>
> 普通二叉堆中，元素被交换后数组位置就失去了稳定意义；索引堆增加 `pos[id]` 的目的就是让外部始终通过稳定的 `id` 找到元素
> 因此它特别适合 `decrease-key`、`increase-key`、删除指定编号等普通 `priority_queue` 不直接支持的操作

常见用法

小根索引堆

```
IndexHeap<int> q(n);
q.push(3, 10);
q.push(5, 7);
q.push(1, 20);
cout << q.top_id() << ' ' << q.top() << '\n'; // 5 7
```

大根索引堆

```
IndexHeap<int, greater<int>> q(n);
```

修改指定元素

`set()` 同时支持权值变优和变差，无需区分 `decrease-key` / `increase-key`：

```
q.set(id, new_value);
```

例如小根堆中：

```
q.set(id, smaller); // decrease-key，可能向上移动
q.set(id, larger);  // increase-key，可能向下移动
```

`fix()` 会自行判断恢复方向

删除指定元素

```
if (q.contains(id)) q.erase(id);
```

删除时先用堆尾元素填补原位置，再根据新的局部关系决定上浮或下沉

$O(n)$ 建堆

若一开始所有编号 `0..n-1` 都存在：

```
vector<i64> a(n);
IndexHeap<i64> q(move(a));
```

此时：

```
q.val[id]
```

对应原数组中的 `a[id]`，全部编号通过 Floyd 方法在 $O(n)$ 内建堆

Dijkstra 的 decrease-key

索引堆最典型的用途之一是保证每个未确定节点在堆中至多出现一次。以节点下标作为索引用于 Dijkstra，并给出了 $O(m \log n)$ 的实现
松弛成功时可以写：

```
if (dist[v] > dist[u] + w) {
    dist[v] = dist[u] + w;
    if (q.contains(v)) q.set(v, dist[v]);
    else q.push(v, dist[v]);
}
```

相比常见的：

```
priority_queue<pair<i64, int>, ...>
```

反复插入新距离的懒删除写法，索引堆始终只保存每个 `id` 的当前权值

与普通二叉堆的区别

能力	`BinaryHeap`	`IndexHeap`
查询堆顶	✅	✅
插入	✅	✅
删除堆顶	✅	✅
$O(n)$ 建堆	✅	✅
保留稳定元素编号	❌	✅
判断指定 `id` 是否存在	❌	✅
修改指定元素	❌	✅
删除指定元素	❌	✅
`decrease-key` / `increase-key`	❌	✅
额外 `id` → `pos` 空间	❌	✅

所以两份板子的定位可以非常清楚：

- `BinaryHeap`：标准、极简的数组二叉堆
- `IndexHeap`：需要**稳定编号 + 任意元素修改/删除**时使用

复杂度

设允许的编号范围为 n，当前堆中有 m 个元素

操作	时间复杂度
构造空索引堆	$O(n)$
全量 Floyd 建堆	$O(n)$
`empty()` / `size()`	$O(1)$
`contains()`	$O(1)$
`top()` / `top_id()`	$O(1)$
`push()`	$O(\log m)$
`set()`	$O(\log m)$
`erase()`	$O(\log m)$
`pop()`	$O(\log m)$
`sift_up()` / `sift_down()` / `fix()`	$O(\log m)$

空间复杂度：$O(n)$

左偏树与斜堆

用于维护**可以快速合并的堆**

```
// LEFTIST=true: 左偏树; LEFTIST=false: 斜堆
// 两者只有合并后的维护规则不同，节点池与公开接口共用这一份实现
template<class T, bool LEFTIST = true, class Compare = less<T>>
struct MeldableHeap {
    struct Node {
        T val{};
        int ch[2]{}, dis{};
    };
```

```
    Compare better;
    vector<Node> tr{{}};
    // ===== 1. 节点创建：push() 依赖，不使用 push() 时可与 push() 一并删除 =====
    int new_node(const T &v) {
        tr.push_back({ v, {0, 0}, 1 });
        return int(tr.size()) - 1;
    }
    // ===== 2. 可并堆核心：push() / pop() 均依赖，不使用整个结构时才删除 =====
    int merge(int x, int y) {
        if (!x || !y) return x | y;
        if (better(tr[y].val, tr[x].val)) swap(x, y);
        tr[x].ch[1] = merge(tr[x].ch[1], y);
        if constexpr (LEFTIST) {
            if (tr[tr[x].ch[0]].dis < tr[tr[x].ch[1]].dis) {
                swap(tr[x].ch[0], tr[x].ch[1]);
            }
            tr[x].dis = tr[tr[x].ch[1]].dis + 1;
        } else {
            swap(tr[x].ch[0], tr[x].ch[1]);
        }
        return x;
    }
    // ===== 3. 插入扩展：不用时删除 push() 与 new_node() =====
    int push(int root, const T &v) { return merge(root, new_node(v)); }
    // ===== 4. 删除堆顶扩展：不用时只删除 pop() =====
    int pop(int root) { return merge(tr[root].ch[0], tr[root].ch[1]); }
    // ===== 5. 读取堆顶：不用时可单独删除 =====
    const T &top(int root) const { return tr[root].val; }
};
template<class T, class Compare = less<T>>
using LeftistHeap = MeldableHeap<T, true, Compare>;
template<class T, class Compare = less<T>>
using SkewHeap = MeldableHeap<T, false, Compare>;
```

接口说明

通过模板参数选择实现：

```
LeftistHeap<T>   // 左偏树
SkewHeap<T>      // 斜堆
```

两者共用相同的节点池与接口，只在 `merge()` 后的树形维护方式不同

成员变量

- `tr[0]`：空节点
- `tr[p].val`：节点 `p` 的值
- `tr[p].ch[0/1]`：左右儿子
- `tr[p].dis`：左偏树使用的右路径相关信息；斜堆不使用
- `root`：不存储在结构体内部，每个堆由一个外部整数根编号表示
- `better(x, y)`：判断 `x` 是否比 `y` 更优
  - 默认 `less<T>`：小根堆
  - `greater<T>`：大根堆

  一个 `MeldableHeap` 节点池可以同时维护多个堆：

```
LeftistHeap<int> heap;
int a{ 0 }, b{ 0 };
```

其中 `0` 表示空堆

接口

- `new_node(v)`：创建值为 `v` 的节点并返回编号，$O(1)$
- `merge(x, y)`：合并根为 `x`、`y` 的两个堆，返回新根
- `push(root, v)`：向 `root` 所表示的堆插入 `v`，返回新根
- `pop(root)`：删除堆顶并返回新根，要求 `root` $\neq 0$
- `top(root)`：返回堆顶元素，要求 `root` $\neq 0$，$O(1)$

> [!] Important

`push()`、`pop()`、`merge()` 都**返回新的根编号**，必须接住：

```
root = heap.push(root, x);
root = heap.pop(root);
root = heap.merge(root, other);
```

注意事项

- 默认 `less<T>` 是小根堆
- 空堆根编号为 `0`
- `top(root)` 和 `pop(root)` 要求 `root ≠ 0`
- `merge(x, y)` 是**破坏性合并**：
  - 要求 `x`、`y` 表示两个彼此独立的堆
  - 合并后应使用返回的新根
  - 不要继续把原来的 `x`、`y` 当作两个独立堆使用
- 特别不能执行：

```
root = heap.merge(root, root);
```

否则会破坏树结构

- `pop()` 只把原根从当前堆中移除，节点仍保留在节点池中，不会回收内存
- 因此空间复杂度取决于**历史创建节点数**，而不是当前堆中元素数
- 当前实现没有：
  - 删除任意节点
  - 修改已有节点权值
  - 节点回收

左偏树与斜堆的选择

结构	特点
左偏树	额外维护 `dis`，单次 `merge` 有 $O(\log n)$ 最坏保证
斜堆	不维护 `dis`，代码和常数更小，操作为均摊 $O(\log n)$
二叉堆	`push/pop` 简单，但两个普通堆不能高效合并

如果题目核心操作是：*不断把两个集合的堆合并*
应优先考虑左偏树或斜堆，而不是普通二叉堆

常见用法

单个可并堆

```
LeftistHeap<int> heap;
int root{ 0 };
root = heap.push(root, 3);
root = heap.push(root, 1);
root = heap.push(root, 5);
cout << heap.top(root) << '\n'; // 1
root = heap.pop(root);
```

合并两个堆

```
LeftistHeap<int> heap;
int a{ 0 }, b{ 0 };
a = heap.push(a, 1);
a = heap.push(a, 5);
b = heap.push(b, 2);
b = heap.push(b, 3);
a = heap.merge(a, b);
b = 0;
cout << heap.top(a) << '\n';
```

合并后令 `b = 0` 是很好的赛场习惯，可以避免误用旧根

大根可并堆

```
LeftistHeap<int, greater<int>> heap;
```

同时维护很多堆

```
LeftistHeap<int> heap;
vector<int> root(n);
root[u] = heap.push(root[u], x);
root[u] = heap.merge(root[u], root[v]);
root[v] = 0;
```

这种写法很适合并查集合并、树上合并等问题

Radix Heap

```
// 单调最小堆：要求每次 push(key, val) 满足 key ≥ last
// Key 必须为不超过 64 位的无符号整数
template<class T = int, class Key = u64>
struct RadixHeap {
    static_assert(is_unsigned_v<Key> && sizeof(Key) ≤ 8);
    static constexpr int B = numeric_limits<Key>::digits + 1;
    using P = pair<Key, T>;
    array<vector<P>, B> bucket;
    Key last{};
    int cnt{ 0 };
    // ===== 基础维护：push() / top() / pop() 均依赖，不建议删除 =====
    int bit_width(Key x) const {
        if (!x) return 0;
        if constexpr (sizeof(Key) ≤ 4) return 32 - __builtin_clz((unsigned)x);
        else return 64 - __builtin_clzll((unsigned long long)x);
    }
    int id(Key key) const { return bit_width(key ^ last); }
    void pull() {
        if (!bucket[0].empty()) return;
        int i{ 1 };
        while (bucket[i].empty()) ++ i;
        last = ranges::min_element(bucket[i], [](const P &a, const P &b) {
            return a.first < b.first;
        })->first;
        vector<P> cur;
        cur.swap(bucket[i]);
        for (auto &e : cur) bucket[id(e.first)].push_back(move(e));
    }
    // ===== 基础接口 =====
    bool empty() const { return cnt == 0; }
    int size() const { return cnt; }
    void push(Key key, T val) {
        bucket[id(key)].push_back({key, move(val)});
        ++ cnt;
    }
    const P &top() {
        pull();
        return bucket[0].back();
    }
    void pop() {
        pull();
        bucket[0].pop_back();
        -- cnt;
    }
};
```

成员变量

- `bucket[i]`：第 `i` 个桶，按照 `key ^ last` 的最高位进行划分
- `last`：最近一次被确定为堆顶的最小 `key`，初始为 `0`
- `cnt`：当前元素数量
- `B`：桶数量，为 `Key` 的二进制位数加 `1`
  - `u32`：`33` 个桶

  - `u64`：`65` 个桶
- `P = pair<Key, T>`：堆中元素，`first` 为排序键，`second` 为附带数据
  桶编号为：`id(key) = bit_width(key ^ last);`
  因此：
- `bucket[0]`：`key == last`
- `bucket[i]`：`key` 与 `last` 的最高不同位为第 `i - 1` 位

> Important
>
> Radix Heap 是**单调最小堆**。必须保证每次：`push(key, val);`
> 都满足：`key ≥ last;`
> 也就是新插入的键不能小于已经弹出的最小键

核心维护

当 `bucket[0]` 为空时，找到编号最小的非空桶 `i`：

1. 找出该桶中最小的 `key`
2. 将其设为新的 `last`
3. 将整个 `bucket[i]` 按新的 `last` 重新分桶
4. 此时至少一个元素进入 `bucket[0]`

```
last = min key in bucket[i];
```

由于重新分桶后，这些元素所在的桶编号都会严格减小，因此一个元素在整个生命周期中只会被重新分桶有限次

接口

- `RadixHeap<T, Key>()`：构造空 Radix Heap，`last = 0`，$O(1)$
- `empty()`：判断是否为空，$O(1)$
- `size()`：返回当前元素数量，$O(1)$
- `push(key, val)`：插入 `{key, val}`，要求 `key ≥ last`，$O(1)$
- `top()`：返回当前最小 `key` 对应的 `{key, val}`，要求非空，均摊 $O(W)$
- `pop()`：删除当前堆顶，要求非空，均摊 $O(W)$
  其中 W 为 `Key` 的二进制位数，通常：

```
u32: W = 32
u64: W = 64
```

因此固定整数类型下，桶数量是很小的常数

注意事项

- `Key` 必须是**无符号整数类型**，并且当前实现要求不超过 `64` 位：

```
RadixHeap<int, unsigned> q;
RadixHeap<int, u64> q;
```

- 不要使用：

```
RadixHeap<int, i64>
```

即使实际数据保证非负，也应直接使用 `u64` 作为键类型

- `push(key, val)` 必须满足：`key ≥ last` 违反该条件后，桶划分不再满足 Radix Heap 的单调性质，结果可能错误
- Radix Heap 不支持普通优先队列意义上的任意键序列，因此不能直接替代 `priority_queue`
  适合*弹出的 key 保证单调不减*，不适合：*弹出 100 后，未来仍可能插入 key = 50*
- `top()` 和 `pop()` 要求堆非空
- 相同 `key` 的元素都位于 `bucket[0]`，但这些元素之间**没有稳定顺序**。如果相同键下还要求按照其他字段排序，需要自行把次关键字编码进 `Key`，或者改用其他优先队列
- 当前版本不提供：
  - `erase()`
  - 任意元素修改
  - `decrease-key`
  - 最大堆
    Radix Heap 更常见的使用方式是直接插入新状态，并在取出时判断状态是否已经过期
- 与二叉堆不同，`push()` 本身就是 $O(1)$，因此若一开始有 n 个合法初始元素，直接逐个：

```
for (...) q.push(key, val);
```

总插入复杂度已经是 $O(n)$，没有必要再单独提供 `build()`

- `last` 表示最近一次 `pull()` 确定的最小键。调用 `top()` 后即使尚未 `pop()`，`last` 也可能已经更新为当前堆顶键

常见用法

基础使用

```
RadixHeap<int> q;
q.push(3, 0);
q.push(5, 1);
q.push(8, 2);
auto [key, id] = q.top();
q.pop();
```

由于初始：`last == 0`
这些插入都满足单调条件

自定义附带数据

排序只由 `Key` 决定，`T` 可以保存任意状态：

```
struct State {
    int u, from;
};
RadixHeap<State> q;
q.push(dis, {u, from});
```

Dijkstra

Radix Heap 最典型的应用是非负整数边权 Dijkstra。

```
vector<u64> dis(n, ULLONG_MAX);
RadixHeap<int> q;
dis[s] = 0;
q.push(0, s);
while (!q.empty()) {
    auto [d, u] = q.top();
    q.pop();
    if (d != dis[u]) continue;
    for (auto [v, w] : g[u]) if (d + w < dis[v]) {
        dis[v] = d + w;
        q.push(d + w, v);
    }
}
```

这里要求：`w ≥ 0`
因此当前有效状态满足：`nd = d + w ≥ d ≥ last`
符合 Radix Heap 的插入条件

> **Important**
>
> 使用懒删除 Dijkstra 时，应先跳过过期状态：`if (d != dis[u]) continue;` 再进行松弛
> 不应从一个已经过期的状态继续使用较小的 `dis[u]` 松弛，否则新得到的键可能小于当前 `last`，破坏 Radix Heap 的单调条件

与普通二叉堆的区别

特性	二叉堆	Radix Heap
任意可比较键	✅	❌
无符号整数键	✅	✅
插入键可任意变化	✅	❌
要求 `key ≥ last`	❌	✅
`push()`	$O(\log n)$	$O(1)$
`top/pop`	$O(\log n)$	均摊与整数位宽有关

特性	二叉堆	Radix Heap
自定义 `Compare`	✅	❌
Dijkstra	✅	整数键时适合
支持最大堆	✅	当前版本不支持

因此选择标准可以直接记为：

```
普通动态优先队列
→ BinaryHeap / priority_queue
非负整数 key + 弹出值单调不减
→ RadixHeap
```

复杂度

设：

- 当前 / 历史共插入 q 个元素
- `Key` 的位宽为 W
  每次重新分桶时，元素所在的桶编号会严格减小，因此每个元素最多被重新分桶 W 次

操作	时间复杂度
`empty()` / `size()`	$O(1)$
`push()`	$O(1)$
`top()`	均摊 $O(W)$
`pop()`	均摊 $O(W)$

单次 `top()` / `pop()` 在触发一个大桶重新分配时可能达到 $O(n+W)$，上述为整个操作序列上的均摊复杂度
总重新分桶复杂度为：$O(qW)$
对于固定的 `u32` / `u64`：$W = 32$ 或 64
因此实际实现中可将位宽看作较小常数
空间复杂度：$O(n+W)$
其中 n 为当前堆内元素数量
经典 one-level Radix Heap 用于边权为非负整数且最大边权为 C 的 Dijkstra 时，可得到 $O(m + n\log C)$ 的专门复杂度界；这是该结构最初提出时的主要用途。

3.1.2 Indexed List（下标双向链表）

```cpp
template<class T>
struct IndexedList {
    static constexpr int NONE{ -1 }, DEAD{ -2 };
    struct Node {
        T val;
        int pre{ NONE }, nxt{ NONE };
    };
    vector<Node> tr;
    int head{ NONE }, tail{ NONE }, cnt{ 0 };
    IndexedList() = default;
    explicit IndexedList(const vector<T> &a) {
        int n = a.size();
        tr.reserve(n);
        for (int i = 0; i < n; ++ i) {
            tr.push_back({ a[i], i - 1, i + 1 < n ? i + 1 : NONE });
        }
        cnt = n;
        if (n) {
            head = 0;
            tail = n - 1;
        }
    }
    bool contains(int p) const { return 0 <= p && p < tr.size() && tr[p].nxt != DEAD; }
    bool empty() const { return cnt == 0; }
    int size() const { return cnt; }
    T &operator[](int p) { return tr[p].val; }
    const T &operator[](int p) const { return tr[p].val; }
    int prev(int p) const { return contains(p) ? tr[p].pre : NONE; }
```

```
    int next(int p) const { return contains(p) ? tr[p].nxt : NONE; }
    T &front() { return tr[head].val; }
    const T &front() const { return tr[head].val; }
    T &back() { return tr[tail].val; }
    const T &back() const { return tr[tail].val; }
    int front_id() const { return head; }
    int back_id() const { return tail; }
    int push_back(const T &v) {
        int p = int(tr.size());
        tr.push_back({v, tail, NONE});
        if (tail != NONE) tr[tail].nxt = p;
        else head = p;
        tail = p;
        ++ cnt;
        return p;
    }
    bool erase(int p) {
        if (!contains(p)) return false;
        int l{ tr[p].pre }, r{ tr[p].nxt };
        if (l != NONE) tr[l].nxt = r;
        else head = r;
        if (r != NONE) tr[r].pre = l;
        else tail = l;
        tr[p].pre = tr[p].nxt = DEAD;
        -- cnt;
        return true;
    }
    void reserve(int n) { tr.reserve(n); }
};
```

成员变量

- `tr[p]`：编号 `p` 对应的节点，保存元素值及当前存活的前驱、后继编号
- `head`：当前首节点编号；空表时为 `NONE
- `tail`：当前尾节点编号；空表时为 `NONE
- `cnt`：当前存活节点数
- `NONE = -1`：不存在前驱、后继或首尾节点
- `DEAD = -2`：已删除节点标记
- 节点编号按创建顺序从 `0` 开始，创建后永久不变且不会复用

接口

- `IndexedList()`：构造空链表，$O(1)$
- `IndexedList(a)`：按 `a` 的顺序建表，初始节点编号为 `0..n-1`，$O(n)$
- `contains(p)`：判断编号 `p` 是否合法且仍存活，$O(1)$
- `empty()`：判断当前是否为空，$O(1)$
- `size()`：返回当前存活节点数，$O(1)$
- `operator[](p)`：访问编号 `p` 对应的值，不检查节点是否存活，$O(1)$
- `prev(p)`：返回存活节点 `p` 的前驱编号；不存在、非法或已删除时返回 `NONE`，$O(1)$
- `next(p)`：返回存活节点 `p` 的后继编号；不存在、非法或已删除时返回 `NONE`，$O(1)$
- `front()` / `back()`：访问当前首 / 尾节点的值，要求链表非空，$O(1)$
- `front_id()` / `back_id()`：返回当前首 / 尾节点编号；空表时返回 `NONE`，$O(1)$
- `push_back(v)`：在尾部加入值为 `v` 的新节点，返回新节点编号，均摊 $O(1)$
- `erase(p)`：删除编号 `p` 的节点；成功返回 `true`，非法或已经删除返回 `false`，$O(1)$
- `reserve(n)`：预留至少 `n` 个历史节点的存储空间，要求 `n ≥ 0`

注意事项

- 节点编号表示**创建时的永久编号**，不是当前链表中的排名；删除节点不会改变其他节点编号
- 删除节点后编号不会复用，因此空间取决于历史创建节点总数，而不是当前 `size()`
- `operator[]` 不检查节点是否存活；即使节点已经删除，其原来的 `val` 仍可通过编号访问
- `prev()` / `next()` 只能查询当前存活节点；对非法或已删除节点统一返回 `NONE`
- `push_back()` 可能导致 `vector` 扩容：**节点编号始终有效**，但此前取得的 `T &`、`Node *` 或迭代器可能失效；需要长期保存引用时应提前 `reserve()`

常见用法

删除元素并取得左右存活节点

```
IndexedList<int> lst(a);
int p = ...;
```

```
int l = lst.prev(p);
int r = lst.next(p);
lst.erase(p);
if (l != -1) {
    // l 为删除前最近的左侧存活节点
}
if (r != -1) {
    // r 为删除前最近的右侧存活节点
}
```

从头遍历当前所有存活节点

```
for (int p = lst.front_id(); p != -1; p = lst.next(p)) {
    cout << lst[p] << ' ';
}
```

保存稳定节点编号

```
int p = lst.push_back(x);
lst.push_back(y);
lst.erase(p);
// p 不会被后续新节点复用
```

3.1.3 SWAG（滑动窗口聚合）

```
template<class T>
struct GcdMerge {
    T operator()(const T &a, const T &b) const { return gcd(a, b); }
};
template<class T = int, class Merge = GcdMerge<T>>
struct SWAG {
    struct Node { T value, prod; };
    T identity;
    Merge merge;
    vector<Node> left, right;
    explicit SWAG(T identity_ = T{}, Merge merge_ = Merge()) : identity(identity_), merge(merge_) {}
    void push(const T &value) {
        T prod = right.empty() ? value : merge(right.back().prod, value);
        right.push_back({value, prod});
    }
    void pop() {
        if (left.empty()) {
            while (!right.empty()) {
                T value = right.back().value, prod = left.empty() ? value : merge(value, left.back().prod);
                right.pop_back();
                left.push_back({ value, prod });
            }
        }
        left.pop_back();
    }
    T query() const {
        T result = identity;
        if (!left.empty()) result = merge(result, left.back().prod);
        if (!right.empty()) result = merge(result, right.back().prod);
        return result;
    }
    bool empty() const { return left.empty() && right.empty(); }
    int size() const { return int(left.size() + right.size()); }
};
```

修改为矩阵乘法

设队列从前到后为

$$A1, A2, \ldots, Ak, A_1, A_2, \ldots, A_k$$

则 `query()` 返回

$$A1A2\cdots Ak.A_1A_2\cdots A_k$$

只需要提供矩阵乘法 `Merge`，并将单位元设为单位矩阵：

```
template<class T>
struct Multiply {
    T operator()(const T &a, const T &b) const { return a * b; }
};
using Mat = Matrix<i64>; // 依赖 2.8 的动态 Matrix
Mat I = Mat::E(n);
SWAG<Mat, Multiply<Mat>> q(I);
q.push(A);
q.push(B);
q.push(C);
Mat prod = q.query(); // A * B * C
q.pop();
prod = q.query();        // B * C
```

3.1.4 开放寻址高性能 HashMap

```
template<class Key, class Value, class Hash = hash<Key>>
struct FlatHashMap {
private:
    struct Slot {
        Key key{};
        Value val{};
        unsigned char state{ 0 }; // 0: 空，1: 有效，2: 墓碑
    };
    Hash hasher;
    vector<Slot> t;
    int siz{ 0 }, used{ 0 }; // used = 有效元素 + 墓碑
    size_t hash_key(const Key &key) const {
        static const u64 seed = chrono::steady_clock::now().time_since_epoch().count();
        u64 x = u64(hasher(key)) + seed + 0x9e3779b97f4a7c15ULL;
        x = (x ^ (x >> 30)) * 0xbf58476d1ce4e5b9ULL;
        x = (x ^ (x >> 27)) * 0x94d049bb133111ebULL;
        return size_t(x ^ (x >> 31));
    }
    // ===== 定位与扩容 =====
    int locate(const Key &key) const {
        int mask{ int(t.size()) - 1 }, p{ int(hash_key(key) & size_t(mask)) };
        while (t[p].state != 0) {
            if (t[p].state == 1 && t[p].key == key) return p;
            p = (p + 1) & mask;
        }
        return -1;
    }
    int insertion_pos(const Key &key) const {
        int mask{ int(t.size()) - 1 }, p{ int(hash_key(key) & size_t(mask)) }, dead{ -1 };
        while (t[p].state != 0) {
            if (t[p].state == 1 && t[p].key == key) return p;
            if (t[p].state == 2 && dead == -1) dead = p;
            p = (p + 1) & mask;
        }
        return dead == -1 ? p : dead;
    }
    // 仅供 rehash 使用，新表不存在重复键和墓碑
    void place(Key &&key, Value &&val) {
        int mask{ int(t.size()) - 1 }, p{ int(hash_key(key) & size_t(mask)) };
        while (t[p].state == 1) p = (p + 1) & mask;
        t[p].key = move(key);
        t[p].val = move(val);
        t[p].state = 1;
        ++ siz;
        ++ used;
    }
    void rehash(int cap) {
        vector<Slot> old{ move(t) };
```

```
        t = vector<Slot>(cap);
        siz = used = 0;
        for (auto &s : old) if (s.state == 1) {
            place(move(s.key), move(s.val));
        }
    }
    // used 达到约 70% 时：墓碑较多则原地清理，否则扩容
    bool prepare_insert() {
        int cap{ int(t.size()) };
        if (1LL * (used + 1) * 10 < 1LL * cap * 7) return false;
        if (1LL * (siz + 1) * 10 < 1LL * cap * 7) rehash(cap);
        else rehash(cap << 1);
        return true;
    }
public:
    explicit FlatHashMap(int expected = 0) {
        int cap{ 8 };
        while (1LL * cap < 2LL * expected) cap <<= 1;
        t.resize(cap);
    }
    // ===== 基础接口：查询 / 插入 / 下标访问 =====
    Value *find(const Key &key) {
        int p{ locate(key) };
        return p == -1 ? nullptr : &t[p].val;
    }
    const Value *find(const Key &key) const {
        int p{ locate(key) };
        return p == -1 ? nullptr : &t[p].val;
    }
    bool contains(const Key &key) const { return locate(key) != -1; }
    pair<Value *, bool> insert(Key key, Value val = Value()) {
        int p{ insertion_pos(key) };
        if (t[p].state == 1) return { &t[p].val, false };
        if (prepare_insert()) p = insertion_pos(key);
        if (t[p].state == 0) ++ used;
        t[p].key = move(key);
        t[p].val = move(val);
        t[p].state = 1;
        ++ siz;
        return { &t[p].val, true };
    }
    Value &operator[](const Key &key) { return *insert(key).first; }
    int size() const { return siz; }
    bool empty() const { return siz == 0; }
    // ===== 删除接口 =====
    bool erase(const Key &key) {
        int p{ locate(key) };
        if (p == -1) return false;
        t[p].state = 2;
        -- siz;
        // 墓碑达到容量的 1/4 时原地重建，避免查询链过长
        if (1LL * (used - siz) * 4 >= int(t.size())) rehash(int(t.size()));
        return true;
    }
};
```

接口说明

- `FlatHashMap(expected)`：构造哈希表；初始容量取不小于 `2 * expected` 的二次幂，最小为 `8`，构造 $O(C)$
- `find(key)`：存在时返回对应值的指针，否则返回 `nullptr`，期望 $O(1)$
- `contains(key)`：判断键是否存在，期望 $O(1)$
- `insert(key, val)`：键不存在时插入，返回 `{值指针, true}`；已存在时不修改原值并返回 `{原值指针, false}`，期望均摊 $O(1)$
- `operator[](key)`：返回对应值的引用；键不存在时插入 `Value{}`，期望均摊 $O(1)$
- `erase(key)`：删除键，存在时返回 `true`，否则返回 `false`，期望均摊 $O(1)$
- `size()` / `empty()`：元素数量与判空，$O(1)$

注意事项

- 容量始终为二的幂，通过 `hash_key(key) & (capacity - 1)` 定位；默认先对 `Hash` 的结果执行带随机种子的 splitmix64 混合，避免整型键低位相同导致大量冲突。仍可通过 `Hash` 模板参数自定义键哈希；此混合不能修复原始 `Hash` 对不同键返回相同值的问题
- `Key` 必须支持 `operator==`；当前实现还要求 `Key`、`Value` 可默认构造并可移动赋值，最适合整数、编码状态和其他轻量类型

- `Slot` 会为整个容量预先构造 `Key` 和 `Value`，因此不建议用很大的 `string`、`vector` 或其他重对象作为键值
- `state = 0/1/2` 分别表示空槽、有效元素、墓碑；查询遇到墓碑不能停止
- `used` 同时统计有效元素和墓碑。`used` 达到约 70 时扩容或清理墓碑，以保证线性探测不会形成过长的查询链
- `erase()` 产生墓碑；墓碑达到容量的 $1/4$ 时会按原容量重新哈希，不会缩小容量
- `insert()` 和 `erase()` 都可能触发 `rehash()`，此前通过 `find()`、`insert()` 或 `operator[]` 获得的值指针、引用会失效
- `insert(key, val)` 不会覆盖已经存在的值；需要修改时使用返回的指针或 `operator[]`
- 已知最大元素数量时应尽量传入 `expected`，通常可以避免运行过程中扩容
- 哈希表只保证期望复杂度；若哈希函数产生大量冲突，最坏仍会退化为线性时间

常见用法

计数

```
FlatHashMap<u64, int> mp(n);
for (u64 x : a) ++ mp[x];
if (int *p = mp.find(x)) {
    cout << *p << '\n';
}
```

判断是否存在并插入

```
auto [p, inserted] = mp.insert(x, value);
if (!inserted) {
    // x 原本已经存在，*p 为原值
}
```

自定义结构体键

```
struct GridKey {
    int x{}, y{};
    friend bool operator==(const GridKey &a, const GridKey &b) {
        return a.x == b.x && a.y == b.y;
    }
};
struct GridKeyHash {
    size_t operator()(const GridKey &p) const {
        u64 x = u64(uint32_t(p.x)) << 32 | uint32_t(p.y);
        return hash<u64>{}(x);
    }
};
FlatHashMap<GridKey, int, GridKeyHash> mp(n);
++ mp[{x, y}];
```

3.2 DSU

3.2.1 一般 DSU

```
struct DSU {
    int cc;
    vector<int> pa, siz;
    explicit DSU(int n) : cc(n), pa(n), siz(n, 1) { ranges::iota(pa, 0); }
    int find(int x) {
        while (x != pa[x]) x = pa[x] = pa[pa[x]];
        return x;
    }
    bool same(int x, int y) { return find(x) == find(y); }
    bool unite(int x, int y) {
        x = find(x), y = find(y);
        if (x == y) return false;
        if (siz[x] < siz[y]) swap(x, y);
        pa[y] = x;
        siz[x] += siz[y];
        -- cc;
        return true;
    }
    int size(int x) { return siz[find(x)]; }
    int count() const { return cc; }
```

```
    bool is_root(int x) const { return pa[x] == x; }
};
```

注意事项

- 默认点编号为 `0base`
- `siz[x]` 只有在 `x` 为根时有效；外部查询使用 `size(x)`
- 使用路径压缩，不能直接用于可撤销并查集
- 本板子采用按大小合并，不需要再维护秩
- 后面 3.2.2 二维与扩展域 DSU 直接依赖本板子的 `DSU`

3.2.2 二维与扩展域 DSU

依赖 3.2.1 一般 DSU

```
struct GridDSU {
    int n, m;
    DSU dsu;
    GridDSU(int n_, int m_) : n(n_), m(m_), dsu(n * m) {}
    int id(int x, int y) const { return x * m + y; }
    bool unite(int x1, int y1, int x2, int y2) { return dsu.unite(id(x1, y1), id(x2, y2)); }
    bool same(int x1, int y1, int x2, int y2) { return dsu.same(id(x1, y1), id(x2, y2)); }
};
template<int K>
struct DomainDSU {
    int n;
    DSU dsu;
    explicit DomainDSU(int n_) : n(n_), dsu(K * n) {}
    int id(int x, int state) const { return state * n + x; }
    // 加入 state(y) - state(x) ≡ delta (mod K)
    bool add_relation(int x, int y, int delta) {
        delta %= K;
        if (delta < 0) delta += K;
        // 已确定相对关系时先检查，避免冲突后污染并查集
        for (int s = 0; s < K; ++ s) {
            if (dsu.same(id(x, 0), id(y, s))) return s == delta;
        }
        for (int s = 0; s < K; ++ s) {
            dsu.unite(id(x, s), id(y, (s + delta) % K));
        }
        return true;
    }
};
```

接口说明

`GridDSU`

将二维坐标 `(x, y)` 映射为：

```
x * m + y
```

再交给普通 `DSU` 维护

- `GridDSU(n, m)`：建立 `n × m` 个位置，$O(nm)$
- `id(x, y)`：返回 `(x, y)` 的一维编号，$O(1)$
- `unite(x1, y1, x2, y2)`：合并两个位置，均摊 $O(\alpha(nm))$
- `same(x1, y1, x2, y2)`：判断两个位置是否连通，均摊 $O(\alpha(nm))$

`DomainDSU<K>`

每个对象 `x` 被拆成 `K` 个状态：

$$(x,0),(x,1),\ldots,(x,K-1)$$

`add_relation(x, y, delta)` 加入：

$$state(y) - state(x) \equiv delta \pmod{K}$$

即对所有 `s` 建立：

$$(x, s) \sim (y, s + delta \bmod K)$$

接口：

- `DomainDSU<K>(n)`：建立 `Kn` 个状态节点，$O(Kn)$
- `id(x, state)`：返回状态节点对应的一维编号，$O(1)$
- `add_relation(x, y, delta)`：加入关系；与已有约束冲突返回 `false`，否则返回 `true`，均摊 $O(K\alpha(Kn))$

注意事项

二维映射

- `GridDSU` 只负责二维编号映射，不会自动合并四邻域

扩展域

- `add_relation()` 的定义固定为 $state(y) - state(x) \equiv delta \pmod K$，关系方向不能写反
- `delta` 会自动归一化到 `[0,K)`
- 新关系冲突时返回 `false`，且不会加入该关系
- 扩展域占用 K 倍节点；若只是维护模 `K` 势能关系，带权并查集通常更省空间
- `K = 2` 常用于两类关系 / 二分约束，`K = 3` 常用于食物链一类三状态关系

常见用法

网格连通

```
GridDSU dsu(n, m);
if (x + 1 < n) dsu.unite(x, y, x + 1, y);
if (y + 1 < m) dsu.unite(x, y, x, y + 1);
```

二扩展域

```
DomainDSU<2> dsu(n);
if (!dsu.add_relation(x, y, delta)) { // 关系冲突
}
```

食物链三扩展域

```
DomainDSU<3> dsu(n);
if (!dsu.add_relation(x, y, delta)) { // 关系冲突
}
```

先按题目定义把关系转换成：

$$state(y) - state(x) \equiv delta \pmod 3$$

再调用接口

复杂度

`GridDSU`

操作	时间复杂度
构造	$O(nm)$
`id`	$O(1)$
`unite` / `same`	均摊 $O(\alpha(nm))$

空间复杂度：$O(nm)$

`DomainDSU<K>`

操作	时间复杂度
构造	$O(Kn)$
`id`	$O(1)$
`add_relation`	均摊 $O(K\alpha(Kn))$

空间复杂度：$O(Kn)$

3.2.3 连通块信息 DSU

```cpp
template<class Info, class Merge>
struct InfoDSU {
    vector<int> pa, siz;
    vector<Info> info;
    Merge merge;
    explicit InfoDSU(vector<Info> init, Merge merge_ = Merge())
        : pa(init.size()), siz(init.size(), 1), info(move(init)), merge(move(merge_)) {
        ranges::iota(pa, 0);
    }
    int find(int x) { return pa[x] == x ? x : pa[x] = find(pa[x]); }
    bool same(int x, int y) { return find(x) == find(y); }
    bool unite(int x, int y) {
        x = find(x), y = find(y);
        if (x == y) return false;
        if (siz[x] < siz[y]) swap(x, y);
        pa[y] = x;
        siz[x] += siz[y];
        info[x] = merge(info[x], info[y]);
        return true;
    }
    int size(int x) { return siz[find(x)]; }
    const Info &get(int x) { return info[find(x)]; }
};
```

接口说明

成员变量

- `pa[x]`：并查集森林中的父节点
- `siz[x]`：当 `x` 为根时，表示集合大小
- `info[x]`：当 `x` 为根时，表示整个集合的聚合信息
- `merge(a, b)`：合并两个连通块的信息

接口

- `InfoDSU(init, merge)`：建立并查集，第 `i` 个单点集合初始信息为 `init[i]`，$O(n)$
- `find(x)`：返回代表元，均摊 $O(\alpha(n))$
- `same(x, y)`：判断是否属于同一集合，均摊 $O(\alpha(n))$
- `unite(x, y)`：合并集合并维护聚合信息，均摊 $O(\alpha(n) + T_{\text{merge}})$
- `size(x)`：返回集合大小，均摊 $O(\alpha(n))$
- `get(x)`：返回所在集合的聚合信息，均摊 $O(\alpha(n))$

注意事项

- `siz[x]`、`info[x]` 仅当 `x` 为根时有效
- 外部查询连通块信息使用 `get(x)`，不要直接访问 `info[x]`
- 由于采用按大小合并，`merge(a, b)` 的左右参数顺序不固定；通常应满足交换律
- 若连通块信息的更新依赖具体连接边、端点或方向，仅有 `merge(info[x], info[y])` 通常不够，应针对题目重新设计合并接口
- 使用路径压缩，不能直接用于回滚

常见用法

连通块最大值

```cpp
struct MergeMax {
    int operator()(int x, int y) const { return max(x, y); }
};
InfoDSU<int, MergeMax> dsu(weight);
dsu.unite(u, v);
cout << dsu.get(u) << '\n';
```

连通块自定义信息

```cpp
struct ComponentInfo {
    i64 sum{};
    int vertices{};
```

```
};
struct MergeComponent {
    ComponentInfo operator()(const ComponentInfo &a, const ComponentInfo &b) const {
        return {a.sum + b.sum, a.vertices + b.vertices};
    }
};
vector<ComponentInfo> init(n);
for (int i = 0; i < n; ++ i) {
    init[i] = {weight[i], 1};
}
InfoDSU<ComponentInfo, MergeComponent> dsu(init);
dsu.unite(u, v);
ComponentInfo info = dsu.get(u);
```

复杂度

设一次 `Merge` 的复杂度为 T_{merge}

操作	时间复杂度
构造	$O(n)$
`find` / `same` / `size` / `get`	均摊 $O(\alpha(n))$
`unite`	均摊 $O(\alpha(n) + T_{\text{merge}})$

空间复杂度：$O(n)$ 加 `Info` 的存储

3.2.4 异或关系与二分图 DSU

维护约束：

$$val[u] \oplus val[v] = w$$

其中 `xr[x]` 表示 `val[x] ^ val[pa[x]]`，路径压缩后表示 `val[x] ^ val[root]`

```
template<class T = int>
struct XorDSU {
    int cc;
    vector<int> pa, siz;
    vector<T> xr;
    explicit XorDSU(int n) : cc(n), pa(n), siz(n, 1), xr(n, T{}) { ranges::iota(pa, 0); }
    int find(int x) {
        if (pa[x] == x) return x;
        int p{ pa[x] };
        pa[x] = find(p);
        xr[x] ^= xr[p];
        return pa[x];
    }
    bool same(int u, int v) { return find(u) == find(v); }
    // 加入 val[u] ^ val[v] = w; 与已有约束冲突时返回 false。
    bool add_relation(int u, int v, T w) {
        int ru{ find(u) }, rv{ find(v) };
        T xu{ xr[u] }, xv{ xr[v] };
        if (ru == rv) return (xu ^ xv) == w;
        if (siz[ru] < siz[rv]) {
            swap(ru, rv);
            swap(xu, xv);
        }
        pa[rv] = ru;
        xr[rv] = xu ^ xv ^ w;
        siz[ru] += siz[rv];
        -- cc;
        return true;
    }
    optional<T> relation(int u, int v) {
        int ru{ find(u) }, rv{ find(v) };
        if (ru != rv) return nullopt;
        return xr[u] ^ xr[v];
    }
    // ===== 以下接口若只需要加入约束和判冲突，可以直接删除 =====
```

```
    T potential(int x) {
        find(x);
        return xr[x];
    }
    int size(int x) { return siz[find(x)]; }
    int count() const { return cc; }
};
using BipartiteDSU = XorDSU<int>;
```

接口说明

成员变量

- `pa[x]`：并查集森林中的父节点
- `siz[x]`：当 `x` 为根时表示集合大小
- `xr[x]`：`val[x] ^ val[pa[x]]`；执行 `find(x)` 后为 `val[x] ^ val[root]`
- `cc`：当前连通块数量

接口

- `XorDSU<T>(n)`：建立 `n` 个独立节点，$O(n)$
- `find(x)`：返回代表元，同时维护路径上的异或势能，均摊 $O(\alpha(n))$
- `same(u, v)`：判断两点是否已有确定的相对关系，均摊 $O(\alpha(n))$
- `add_relation(u, v, w)`：加入 `val[u] ^ val[v] = w`；若与已有约束矛盾返回 `false`，否则返回 `true`，均摊 $O(\alpha(n))$
- `relation(u, v)`：若两点连通，返回 `val[u] ^ val[v]`；否则返回 `nullopt`，均摊 $O(\alpha(n))$
- `potential(x)`：返回 `val[x] ^ val[root]`，均摊 $O(\alpha(n))$
- `size(x)`：返回所在集合大小，均摊 $O(\alpha(n))$
- `count()`：返回当前集合数量，$O(1)$

注意事项

- `add_relation()` 的返回值表示约束是否自洽，不是“是否发生了集合合并”
- 重复加入已经存在且一致的关系仍返回 `true`
- `relation()` 使用 `optional<T>`，不要使用 `-1` 表示“不连通”，因为 `-1` 可能本身就是合法的按位异或结果
- `xr[x]` 会在路径压缩时同步更新；不能只压缩 `pa[x]`
- `potential(x)` 是相对当前代表元的势能，不是绝对的 `val[x]`；并查集合并后代表元可能改变
- 使用路径压缩，不能直接用于可撤销并查集
- 默认节点编号为 `0base`

二分图约束

二分图染色可以看作：

$$color[u] \oplus color[v] = 1$$

因此：

- 异色：`w = 1`
- 同色：`w = 0`
- 无向图中每加入一条普通边，就加入异色约束 `1`
- 自环会产生 `color[u] ^ color[u] = 1`，立即检测出矛盾

常见用法

维护异或关系

```
XorDSU<int> dsu(n);
if (!dsu.add_relation(u, v, w)) {
    // 与已有关系冲突
}
auto rel = dsu.relation(u, v);
if (rel) {
    cout << *rel << '\n';
}
```

动态判断二分图约束

```
BipartiteDSU dsu(n);
for (auto [u, v] : edges) {
    if (!dsu.add_relation(u, v, 1)) {
        cout << "No\n";
```

```
        return;
    }
}
cout << "Yes\n";
```

同色 / 异色混合约束

```
// type = 0: 要求同色
// type = 1: 要求异色
if (!dsu.add_relation(u, v, type)) {
    // 矛盾
}
```

3.2.5 带权 DSU（势能并查集）

维护约束：

$$val[v] - val[u] = d$$

其中 `dis[x]` 表示 `val[x] - val[pa[x]]`，路径压缩后表示 `val[x] - val[root]`

```
template<class T = i64>
struct WeightedDSU {
    int cc;
    vector<int> pa, siz;
    vector<T> dis;
    explicit WeightedDSU(int n) : cc(n), pa(n), siz(n, 1), dis(n, T{}) { ranges::iota(pa, 0); }
    static bool equal(const T &a, const T &b) {
        if constexpr (is_floating_point_v<T>) return abs(a - b) <= 1e-9;
        else return a == b;
    }
    int find(int x) {
        if (pa[x] == x) return x;
        int p{ pa[x] };
        pa[x] = find(p);
        dis[x] += dis[p];
        return pa[x];
    }
    bool same(int u, int v) { return find(u) == find(v); }
    // 加入 val[v] - val[u] = d; 与已有约束冲突时返回 false。
    bool add_relation(int u, int v, const T &d) {
        int ru{ find(u) }, rv{ find(v) };
        T du{ dis[u] }, dv{ dis[v] };
        if (ru == rv) return equal(dv - du, d);
        if (siz[ru] < siz[rv]) {
            pa[ru] = rv;
            dis[ru] = dv - du - d;
            siz[rv] += siz[ru];
        } else {
            pa[rv] = ru;
            dis[rv] = d + du - dv;
            siz[ru] += siz[rv];
        }
        -- cc;
        return true;
    }
    optional<T> diff(int u, int v) {
        int ru{ find(u) }, rv{ find(v) };
        if (ru != rv) return nullopt;
        return dis[v] - dis[u];
    }
    // ===== 以下接口若只需要加入约束和查询两点差值，可以直接删除 =====
    T potential(int x) {
        find(x);
        return dis[x];
    }
    int size(int x) { return siz[find(x)]; }
```

```
    int count() const { return cc; }
};
```

接口说明

成员变量

- `pa[x]`：并查集森林中的父节点
- `siz[x]`：当 `x` 为根时表示集合大小
- `dis[x]`：`val[x] - val[pa[x]]`；执行 `find(x)` 后为 `val[x] - val[root]`
- `cc`：当前连通块数量

接口

- `WeightedDSU<T>(n)`：建立 `n` 个独立节点，$O(n)$
- `find(x)`：返回代表元并同步维护势能，均摊 $O(\alpha(n))$
- `same(u, v)`：判断两点是否连通，均摊 $O(\alpha(n))$
- `add_relation(u, v, d)`：加入 $val[v]-val[u]=d$，若与已有约束冲突返回 `false`，否则返回 `true`，均摊 $O(\alpha(n))$
- `diff(u, v)`：连通时返回 $val[v]-val[u]$，否则返回 `nullopt`，均摊 $O(\alpha(n))$
- `potential(x)`：返回 `val[x] - val[root]`，均摊 $O(\alpha(n))$
- `size(x)`：返回所在集合大小，均摊 $O(\alpha(n))$
- `count()`：返回当前集合数量，$O(1)$

注意事项

- **方向固定为** $val[v]-val[u]=d$，调用 `add_relation(u, v, d)` 时不要把 `u`、`v` 或 `d` 的符号写反
- `add_relation()` 返回的是**约束是否自洽**；重复加入已有且一致的关系返回 `true`
- `dis[x]` 只在 `find(x)` 后表示 `x` 到当前根的完整势能差
- `potential(x)` 依赖当前代表元，不能当成绝对值保存后长期使用
- 使用路径压缩，不能直接用于可撤销 DSU
- `T` 的运算结果必须不会溢出；整数距离、前缀和等应先估计范围
- 浮点类型默认使用绝对误差 `1e-9` 判断冲突，精度要求不同应修改 `equal()`
- 自定义 `T` 时至少需要支持 `T{}`、`+=`、`+`、`-` 和 `==`
- 这一实现按加法形式书写，适用于整数、浮点数、二维偏移量等**可加交换群**

常见用法

区间和约束

设前缀和为 `s[i]`，若已知：

$$a_l+\cdots+a_r=sum$$

则：

$$s[r]-s[l-1]=sum$$

可以直接写：

```
WeightedDSU<i64> dsu(n + 1);
if (!dsu.add_relation(l - 1, r, sum)) {
    cout << "conflict\n";
}
auto ans = dsu.diff(l - 1, r);
if (ans) cout << *ans << '\n';
```

两点坐标差

```
struct Offset2D {
    i64 x{}, y{};
    Offset2D &operator+=(const Offset2D &o) {
        x += o.x;
        y += o.y;
        return *this;
    }
    friend Offset2D operator+(Offset2D a, const Offset2D &b) {
        return a += b;
    }
    friend Offset2D operator-(const Offset2D &a, const Offset2D &b) {
        return {a.x - b.x, a.y - b.y};
    }
    friend bool operator==(const Offset2D &a, const Offset2D &b) {
```

```
        return a.x == b.x && a.y == b.y;
    }
};
WeightedDSU<Offset2D> dsu(n);
dsu.add_relation(u, v, {dx, dy});
auto delta = dsu.diff(u, v);
if (delta) {
    cout << delta->x << ' ' << delta->y << '\n';
}
```

此时：

$$pos[v] - pos[u] = (dx, dy)$$

差分约束一致性

```
WeightedDSU<i64> dsu(n);
for (auto [u, v, d] : constraints) {
    if (!dsu.add_relation(u, v, d)) {
        cout << "No\n";
        return;
    }
}
```

3.2.6 可撤销 DSU

```
struct RollbackDSU {
    int cc;
    vector<int> pa, siz, his;
    explicit RollbackDSU(int n) : cc(n), pa(n), siz(n, 1) { ranges::iota(pa, 0); }
    // 不做路径压缩，使一次成功合并只修改一个父指针
    int find(int x) const {
        while (x != pa[x]) x = pa[x];
        return x;
    }
    bool same(int x, int y) const { return find(x) == find(y); }
    bool unite(int x, int y) {
        x = find(x), y = find(y);
        if (x == y) return false;
        if (siz[x] < siz[y]) swap(x, y);
        his.push_back(y);
        pa[y] = x;
        siz[x] += siz[y];
        -- cc;
        return true;
    }
    int size(int x) const { return siz[find(x)]; }
    int count() const { return cc; }
    int snapshot() const { return int(his.size()); }
    void rollback(int snap) {
        while (int(his.size()) > snap) {
            int y{ his.back() };
            his.pop_back();
            int x{ pa[y] };
            siz[x] -= siz[y];
            pa[y] = y;
            ++ cc;
        }
    }
};
```

接口说明

成员变量

- `pa[x]`：并查集森林中的父节点
- `siz[x]`：当 `x` 为根时表示所在集合大小
- `his`：成功合并的历史栈

- `cc`：当前连通块数量

接口

- `RollbackDSU(n)`：建立 `0 ... n - 1` 的单点集合，$O(n)$
- `find(x)`：返回当前代表元，最坏 $O(\log n)$
- `same(x, y)`：判断两点是否连通，最坏 $O(\log n)$
- `unite(x, y)`：按大小合并；成功返回 `true`，原本连通返回 `false`，最坏 $O(\log n)$
- `size(x)`：返回所在连通块大小，最坏 $O(\log n)$
- `count()`：返回当前连通块数量，$O(1)$
- `snapshot()`：返回当前历史位置，用作回滚快照，$O(1)$
- `rollback(snap)`：撤销 `snap` 之后的全部成功合并，每撤销一次合并 $O(1)$

注意事项

- **严禁路径压缩**。按大小合并保证并查集树高为 $O(\log n)$，同时使每次成功合并只需要记录一个修改
- `snapshot()` 返回的是**成功合并记录数**，不是调用 `unite()` 的次数
- `unite()` 发现两点已经连通时不会写入历史栈
- `rollback(snap)` 要求 `0 ⩽ snap ⩽ snapshot()`，通常只传入之前保存的快照
- 快照可以嵌套，但必须按照栈顺序回滚：先离开内层，再离开外层
- 回滚后，以前保存的代表元可能已经失效，需要重新调用 `find()`
- `siz[x]` 只有在 `x` 当前为根时有效
- 默认点编号为 `0base`

常见用法

临时修改后恢复

```
RollbackDSU dsu(n);
int snap = dsu.snapshot();
dsu.unite(u, v);
dsu.unite(x, y);
// 在临时状态上查询
bool connected = dsu.same(a, b);
dsu.rollback(snap);
```

时间线段树 / DFS 分治

```
void dfs(int p, int l, int r) {
    int snap{ dsu.snapshot() };
    for (auto [u, v] : edges[p]) {
        dsu.unite(u, v);
    }
    if (r - l == 1) {
        ans[l] = dsu.same(query_u[l], query_v[l]);
    } else {
        int m{ (l + r) / 2 };
        dfs(p << 1, l, m);
        dfs(p << 1 | 1, m, r);
    }
    dsu.rollback(snap);
}
```

3.2.7 可持久化 DSU

使用可持久化隐式线段树维护每个点的并查集状态：

- `val[x] < 0`：`x` 是根，集合大小为 `-val[x]`
- `val[x] ⩾ 0`：`val[x]` 是 `x` 的父节点
- 未出现在线段树中的点默认 `val[x] = -1`
  因此初始版本不需要真正建立整棵线段树

```
struct PersistentDSU {
private:
    struct Node {
        int ch[2]{};
        int val{ -1 };
    };
```

```
    int n;
    vector<Node> tr{{}};
    vector<int> root{ 0 }, cc;
    // 查询版本 rt 中点 x 的 parent / -size；未修改过的点默认为 -1
    int get(int p, int x) const {
        int l{ 0 }, r{ n };
        while (p && r - l > 1) {
            int m{ (l + r) / 2 };
            if (x < m) {
                p = tr[p].ch[0];
                r = m;
            } else {
                p = tr[p].ch[1];
                l = m;
            }
        }
        return p ? tr[p].val : -1;
    }
    // 在版本 p 上单点修改并返回新的线段树根
    int set_node(int p, int l, int r, int x, int val) {
        int q{ int(tr.size()) };
        tr.push_back(p ? tr[p] : Node{});
        if (r - l == 1) {
            tr[q].val = val;
            return q;
        }
        int m{ (l + r) / 2 };
        if (x < m) tr[q].ch[0] = set_node(tr[q].ch[0], l, m, x, val);
        else tr[q].ch[1] = set_node(tr[q].ch[1], m, r, x, val);
        return q;
    }
    // 不做路径压缩；按大小合并保证父链长度 O(log n)
    int find_root(int rt, int x) const {
        while (true) {
            int p{ get(rt, x) };
            if (p < 0) return x;
            x = p;
        }
    }
    int new_version(int rt, int cnt) {
        root.push_back(rt);
        cc.push_back(cnt);
        return int(root.size()) - 1;
    }
public:
    explicit PersistentDSU(int n_, int q = 0) : n(n_), cc{ n } {
        if (q > 0) {
            int lg{};
            while ((1LL << lg) < n) ++ lg;
            tr.reserve(1 + 2LL * q * (lg + 1));
            root.reserve(q + 1);
            cc.reserve(q + 1);
        }
    }
    int versions() const { return int(root.size()); }
    int find(int ver, int x) const { return find_root(root[ver], x); }
    bool same(int ver, int x, int y) const { return find_root(root[ver], x) == find_root(root[ver], y); }
    int size(int ver, int x) const {
        int rt{ root[ver] };
        x = find_root(rt, x);
        return -get(rt, x);
    }
    int count(int ver) const { return cc[ver]; }
    // 从版本 ver 合并 x、y，并始终追加一个新版本
    int unite(int ver, int x, int y) {
        int rt{ root[ver] };
        x = find_root(rt, x);
        y = find_root(rt, y);
        if (x == y) return new_version(rt, cc[ver]);
        int sx{ -get(rt, x) }, sy{ -get(rt, y) };
        if (sx < sy) {
```

```
            swap(x, y);
            swap(sx, sy);
        }
        rt = set_node(rt, 0, n, y, x);
        rt = set_node(rt, 0, n, x, -(sx + sy));
        return new_version(rt, cc[ver] - 1);
    }
    // ===== 若题目每次操作一定执行 unite()，以下接口可直接删除 =====
    int copy_version(int ver) { return new_version(root[ver], cc[ver]); }
};
```

接口说明

状态表示

版本 `0` 为初始版本，此时所有点互不连通
底层使用一个可持久化数组表示 DSU：

- 值 `< 0`：当前点为根，绝对值为集合大小
- 值 `≥ 0`：当前点的父节点编号
- 从未修改的位置隐式取值 `-1`
  因此不需要构造初始线段树

接口

- `PersistentDSU(n, q)`：建立 `n` 个点；`q` 可选，用于预留最多约 `q` 次版本操作的空间，构造本身 $O(1)$
- `versions()`：返回当前版本数量，$O(1)$
- `find(ver, x)`：返回版本 `ver` 中 `x` 的代表元，$O(\log^2 n)$
- `same(ver, x, y)`：判断指定版本中两点是否连通，$O(\log^2 n)$
- `size(ver, x)`：返回指定版本中 `x` 所在集合大小，$O(\log^2 n)$
- `count(ver)`：返回指定版本的连通块数量，$O(1)$
- `unite(ver, x, y)`：从版本 `ver` 合并两点，追加并返回新版本编号，$O(\log^2 n)$
- `copy_version(ver)`：追加一个与 `ver` 完全相同的版本并返回新版本编号，$O(1)$

注意事项

- 构造后初始版本编号为 `0`
- `unite(ver, x, y)` 始终产生一个新版本；即使两点原本已经连通，也会追加一个内容相同的版本
- 可以从任意历史版本继续调用 `unite()`，因此自然支持版本分叉
- `copy_version(ver)` 只是新增一个根引用，不复制底层节点
- 不使用路径压缩；按大小合并保证 DSU 父链长度为 $O(\log n)$
- 查询一次父节点需要在线段树中进行一次 $O(\log n)$ 单点查询，因此 `find()` 为 $O(\log^2 n)$
- 一次成功 `unite()` 只持久化修改两个位置：新根的大小与旧根的父亲，共新增 $O(\log n)$ 个节点
- `find(ver, x)` 返回的代表元只对该版本有意义；不同版本中同一集合的代表元不必相同
- 默认点编号为 `0base`
- 要求 `n > 0`、`0 ≤ x < n`、`0 ≤ ver < versions()`
- 构造参数 `q` 只用于 `reserve()`，给小或不给都不影响正确性

常见用法

从历史版本继续分叉

```
PersistentDSU dsu(4, 4);

int v1 = dsu.unite(0, 0, 1);   // {0,1}, {2}, {3}
int v2 = dsu.unite(0, 2, 3);   // 从 v0 分叉：{0}, {1}, {2,3}
int v3 = dsu.unite(v1, 1, 2);  // {0,1,2}, {3}

cout << dsu.same(0, 0, 1) << '\n';   // 0
cout << dsu.same(v1, 0, 1) << '\n';  // 1
cout << dsu.same(v2, 0, 1) << '\n';  // 0
cout << dsu.same(v2, 2, 3) << '\n';  // 1
cout << dsu.same(v3, 0, 2) << '\n';  // 1
```

每次操作对应一个版本

```
vector<int> ver(q + 1);
ver[0] = 0;

for (int i = 1; i ≤ q; ++ i) {
```

```
    if (op[i] == 1) {
        ver[i] = dsu.unite(ver[k], x, y);
    } else {
        ver[i] = dsu.copy_version(ver[k]);
    }
}
```

题目操作编号和内部版本编号只有在“每次操作恰好创建一个版本”时才能直接一一对应；更稳妥的方式是像上面一样显式保存 `ver[i]`

3.2.8 可撤销异或关系与二分图 DSU

维护可撤销约束：

$$val[u] \oplus val[v] = w$$

其中 `xr[x]` 表示：

$$val[x] \oplus val[pa[x]]$$

```
template<class T = int>
struct RollbackXorDSU {
    struct Change {
        int y;
        bool conflict;
    };
    int cc, bad{};
    vector<int> pa, siz;
    vector<T> xr;
    vector<Change> his;
    explicit RollbackXorDSU(int n) : cc(n), pa(n), siz(n, 1), xr(n, T{}) { ranges::iota(pa, 0); }
    // 不做路径压缩；返回 {根, val[x] ^ val[root]}
    pair<int, T> find(int x) const {
        T d{};
        while (x != pa[x]) {
            d ^= xr[x];
            x = pa[x];
        }
        return {x, d};
    }
    bool same(int u, int v) const { return find(u).first == find(v).first; }
    // 加入 val[u] ^ val[v] = w；返回该约束是否与当前已有约束一致
    bool add_relation(int u, int v, T w) {
        auto [ru, xu]{ find(u) };
        auto [rv, xv]{ find(v) };
        if (ru == rv) {
            bool ok{ (xu ^ xv) == w };
            his.push_back({-1, !ok});
            bad += !ok;
            return ok;
        }
        if (siz[ru] < siz[rv]) {
            swap(ru, rv);
            swap(xu, xv);
        }
        his.push_back({rv, false});
        pa[rv] = ru;
        xr[rv] = xu ^ xv ^ w;
        siz[ru] += siz[rv];
        --cc;
        return true;
    }
    // ===== 回滚接口：线段树分治等算法依赖 =====
    int snapshot() const { return int(his.size()); }
    void rollback(int snap) {
        while (int(his.size()) > snap) {
            auto [y, conflict]{ his.back() };
            his.pop_back();
            if (y == -1) {
                bad -= conflict;
                continue;
```

```
            }
            int x{ pa[y] };
            siz[x] -= siz[y];
            pa[y] = y;
            xr[y] = T{};
            ++ cc;
        }
    }
    bool valid() const { return bad == 0; }
    int conflicts() const { return bad; }
    // ===== 扩展查询 =====
    optional<T> relation(int u, int v) const {
        auto [ru, xu]{ find(u) };
        auto [rv, xv]{ find(v) };
        if (ru != rv) return nullopt;
        return xu ^ xv;
    }
    T potential(int x) const { return find(x).second; }
    int size(int x) const { return siz[find(x).first]; }
    int count() const { return cc; }
};
// 二分图专用封装：可直接接入只调用 unite(u, v) 的线段树分治
struct RollbackBipartiteDSU : RollbackXorDSU<int> {
    using RollbackXorDSU<int>::RollbackXorDSU;
    bool unite(int u, int v) { return add_relation(u, v, 1); }
};
```

接口说明

成员变量

- `pa[x]`：并查集森林中的父节点
- `siz[x]`：当 `x` 为根时表示集合大小
- `xr[x]`：`val[x] ^ val[pa[x]]`
- `cc`：当前连通块数量
- `bad`：当前活跃约束中产生的矛盾数量
- `his`：加入约束产生的历史记录

接口

- `RollbackXorDSU<T>(n)`：建立 `n` 个独立节点，$O(n)$
- `find(x)`：返回 `{root, val[x] ^ val[root]}`，最坏 $O(\log n)$
- `same(u, v)`：判断两点是否已有确定的相对关系，最坏 $O(\log n)$
- `add_relation(u, v, w)`：加入 $val[u] \oplus val[v] = w$，若这条约束与当前已有约束冲突返回 `false`，否则返回 `true`，最坏 $O(\log n)$
- `snapshot()`：保存当前完整约束状态，$O(1)$
- `rollback(snap)`：回滚到指定快照，每撤销一条约束 $O(1)$
- `valid()`：判断当前所有活跃约束是否整体自洽，$O(1)$
- `conflicts()`：返回当前矛盾约束数量，$O(1)$
- `relation(u, v)`：连通时返回 `val[u] ^ val[v]`，否则返回 `nullopt`，最坏 $O(\log n)$
- `potential(x)`：返回 `val[x] ^ val[root]`，最坏 $O(\log n)$
- `size(x)`：返回所在集合大小，最坏 $O(\log n)$
- `count()`：返回当前连通块数量，$O(1)$

二分图封装

`RollbackBipartiteDSU` 将每条普通无向边解释为：

$$color[u] \oplus color[v] = 1$$

因此：

```
dsu.unite(u, v);
```

等价于：

```
dsu.add_relation(u, v, 1);
```

可以直接替换 3.9.6 时间线段树分治与可撤销维护 使用的普通 `RollbackDSU`

二分图关系

对于染色 `color[x] ∈ {0,1}`：

- 异色：$color[u] \oplus color[v] = 1$
- 同色：$color[u] \oplus color[v] = 0$
  普通无向图要求每条边两端异色，所以动态二分图直接使用 `RollbackBipartiteDSU`

注意事项

- 不能使用路径压缩；只使用按大小合并，保证树高为 $O(\log n)$
- `xr[x]` 始终只保存 `x` 到直接父节点的异或关系，`find()` 查询过程中累积到根
- `add_relation()` 的返回值表示这一条新约束是否与当前状态一致，不是是否真的进行了集合合并
- 两点已经连通且关系一致时仍然会写入一条历史记录，因为 `snapshot()` 维护的是完整的“约束加入状态”
- 两点已经连通但关系矛盾时，不修改并查集结构，而是令 `bad ++`
- 因此 `rollback()` 不仅需要撤销集合合并，还必须撤销冲突状态
- `snapshot()` 等于当前活跃的 `add_relation()` 历史记录数，而不是成功合并次数
- 即使当前 `bad > 0`，后续约束仍然必须正常处理；不能因为当前已经矛盾就提前跳过，否则回滚掉原冲突后状态会不完整
- `valid()` 判断的是整个约束系统是否无矛盾；本实现不维护“每个连通块各自是否矛盾”
- 当前约束系统存在矛盾时，`relation()` / `potential()` 仍会返回并查集生成森林确定的关系，但它不一定满足所有活跃约束；通常只在相关约束状态合法时使用
- `T` 需要支持 `T{}`、`^`、`^=` 和 `==`
- 默认点编号为 `0base`

常见用法

可撤销异或约束

```
RollbackXorDSU<int> dsu(n);

int snap{ dsu.snapshot() };

dsu.add_relation(0, 1, 3);
dsu.add_relation(1, 2, 5);

auto w = dsu.relation(0, 2);
if (w) {
    cout << *w << '\n'; // 3 ^ 5
}

dsu.rollback(snap);
```

动态二分图

可以直接与 3.9.6 时间线段树分治与可撤销维护 组合：

```
TimeSeg<array<int, 2>> dc(q);

for (auto [l, r, u, v] : intervals) {
    dc.add(l, r, {u, v});
}

RollbackBipartiteDSU dsu(n);
vector<bool> ans(q);

dc.run(dsu, [](auto &cur, const auto &e) {
    cur.unite(e[0], e[1]);
}, [&](int t, RollbackBipartiteDSU &cur) {
    if (op[t] == '?') {
        ans[t] = cur.valid();
    }
});
```

这里时间线段树仍然调用：

```
dsu.unite(u, v);
```

但对于 `RollbackBipartiteDSU`，其含义已经变为加入异色约束

动态同色 / 异色关系

若边本身带 `w`，不能使用仅保存 `(u,v)` 的普通 `DynamicConnectivity::Edge`，应将时间树中的边扩展为：

```
struct Edge {
    int u, v, w;
};
```

进入节点时：

```
dsu.add_relation(u, v, w);
```

叶子查询：

```
bool ok = dsu.valid();
```

复杂度

操作	时间复杂度
构造	$O(n)$
`find` / `same`	最坏 $O(\log n)$
`add_relation`	最坏 $O(\log n)$
`relation` / `potential` / `size`	最坏 $O(\log n)$
`snapshot` / `valid` / `conflicts` / `count`	$O(1)$
撤销一条约束记录	$O(1)$

`rollback(snap)` 若撤销 k 条约束，复杂度为 $O(k)$
空间复杂度：$O(n+h)$，其中 h 为当前历史栈中的约束记录数

3.2.9 可撤销带权 DSU（势能并查集）

维护可撤销约束：

$$val[v]-val[u]=d$$

其中 `dis[x]` 表示：

$$val[x]-val[pa[x]]$$

```
template<class T = i64>
struct RollbackWeightedDSU {
    struct Change {
        int y;
        bool conflict;
    };
    int cc, bad{};
    vector<int> pa, siz;
    vector<T> dis;
    vector<Change> his;
    explicit RollbackWeightedDSU(int n) : cc(n), pa(n), siz(n, 1), dis(n, T{}) { ranges::iota(pa, 0); }
    static bool equal(const T &a, const T &b) {
        if constexpr (is_floating_point_v<T>) return abs(a - b) <= 1e-9;
        else return a == b;
    }
    // 不做路径压缩；返回 {根, val[x] - val[root]}
    pair<int, T> find(int x) const {
        T d{};
        while (x != pa[x]) {
            d += dis[x];
            x = pa[x];
        }
        return {x, d};
    }
    bool same(int u, int v) const { return find(u).first == find(v).first; }
    // 加入 val[v] - val[u] = d; 返回该约束是否与当前已有约束一致
    bool add_relation(int u, int v, const T &d) {
        auto [ru, du]{ find(u) };
        auto [rv, dv]{ find(v) };
        if (ru == rv) {
```

```
            bool ok{ equal(dv - du, d) };
            his.push_back({-1, !ok});
            bad += !ok;
            return ok;
        }
        if (siz[ru] < siz[rv]) {
            his.push_back({ru, false});
            pa[ru] = rv;
            dis[ru] = dv - du - d;
            siz[rv] += siz[ru];
        } else {
            his.push_back({rv, false});
            pa[rv] = ru;
            dis[rv] = d + du - dv;
            siz[ru] += siz[rv];
        }
        -- cc;
        return true;
    }
    // ===== 回滚接口：线段树分治等算法依赖，不建议删除 =====
    int snapshot() const { return int(his.size()); }
    void rollback(int snap) {
        while (int(his.size()) > snap) {
            auto [y, conflict]{ his.back() };
            his.pop_back();
            if (y == -1) {
                bad -= conflict;
                continue;
            }
            int x{ pa[y] };
            siz[x] -= siz[y];
            pa[y] = y;
            dis[y] = T{};
            ++ cc;
        }
    }
    bool valid() const { return bad == 0; }
    int conflicts() const { return bad; }
    // ===== 扩展查询：若只判断当前约束系统是否合法，以下代码可直接删除 =====
    optional<T> diff(int u, int v) const {
        auto [ru, du]{ find(u) };
        auto [rv, dv]{ find(v) };
        if (ru != rv) return nullopt;
        return dv - du;
    }
    T potential(int x) const { return find(x).second; }
    int size(int x) const { return siz[find(x).first]; }
    int count() const { return cc; }
};
```

接口说明

成员变量

- `pa[x]`：并查集森林中的父节点
- `siz[x]`：当 `x` 为根时表示集合大小
- `dis[x]`：`val[x] - val[pa[x]]`
- `cc`：当前连通块数量
- `bad`：当前活跃约束中的矛盾数量
- `his`：所有加入约束对应的历史记录

接口

- `RollbackWeightedDSU<T>(n)`：建立 `n` 个独立节点，$O(n)$
- `find(x)`：返回 `{root, val[x] - val[root]}`，最坏 $O(\log n)$
- `same(u, v)`：判断两点是否连通，最坏 $O(\log n)$
- `add_relation(u, v, d)`：加入 $val[v] - val[u] = d$，若与已有约束冲突返回 `false`，否则返回 `true`，最坏 $O(\log n)$
- `snapshot()`：保存当前完整约束状态，$O(1)$
- `rollback(snap)`：撤销到指定快照，每撤销一条约束 $O(1)$
- `valid()`：判断当前所有活跃约束是否整体自洽，$O(1)$

- `conflicts()`：返回当前矛盾约束数量，$O(1)$
- `diff(u, v)`：连通时返回 `val[v] - val[u]`，否则返回 `nullopt`，最坏 $O(\log n)$
- `potential(x)`：返回 `val[x] - val[root]`，最坏 $O(\log n)$
- `size(x)`：返回所在集合大小，最坏 $O(\log n)$
- `count()`：返回当前连通块数量，$O(1)$

注意事项

- 不能使用路径压缩；按大小合并保证并查集父链长度为 $O(\log n)$
- 约束方向始终固定为：$val[v] - val[u] = d$，调用时不能交换 `u/v` 后仍使用原来的 `d`
- `dis[x]` 只保存当前节点到直接父节点的势能差，`find()` 查询时沿父链累积
- 两个不同集合总能通过调整根之间的势能加入一条新关系，因此真正的冲突只会发生在两点已经连通时
- 同根且一致的关系也会写入历史记录，使 `snapshot()` 能准确对应完整约束状态
- 同根但冲突时不改变树结构，只令 `bad ++`
- `rollback()` 必须同时恢复并查集结构和 `bad`
- 即使当前已经存在矛盾，也不能跳过后续 `add_relation()`，否则之后回滚原冲突时会丢失本应存在的状态
- `valid()` 判断的是整个当前约束系统；不维护每个连通块单独的合法性
- 约束系统存在冲突时，`diff()` / `potential()` 返回的是维护森林确定的势能，不保证满足全部活跃约束
- 浮点类型默认使用绝对误差 `1e-9` 判断关系是否一致；精度要求特殊时修改 `equal()`
- 自定义 `T` 至少需要支持 `T{}`、`+=`、`+`、`-` 和 `==`
- 整数势能需要预先估计累加范围，必要时使用 `i64` / `i128`
- 默认点编号为 `0base`

常见用法

临时差分约束

```
RollbackWeightedDSU<i64> dsu(n);

int snap{ dsu.snapshot() };

dsu.add_relation(0, 1, 5);  // val[1] - val[0] = 5
dsu.add_relation(1, 2, -2); // val[2] - val[1] = -2

auto d = dsu.diff(0, 2);
if (d) {
    cout << *d << '\n'; // 3
}

dsu.rollback(snap);
```

判断动态差分约束是否矛盾

时间线段树中的约束需要包含 `d`：

```
struct Edge {
    int u, v;
    i64 d;
};
```

进入时间线段树节点时：

```
int snap{ dsu.snapshot() };

for (auto [u, v, d] : seg[p]) {
    dsu.add_relation(u, v, d);
}
```

叶子查询：

```
ans[t] = dsu.valid();
```

退出节点：

```
dsu.rollback(snap);
```

动态相对坐标

```
struct Offset2D {
    i64 x{}, y{};
    Offset2D &operator+=(const Offset2D &o) {
        x += o.x;
        y += o.y;
        return *this;
    }
    friend Offset2D operator+(Offset2D a, const Offset2D &b) { return a += b; }
    friend Offset2D operator-(const Offset2D &a, const Offset2D &b) { return {a.x - b.x, a.y - b.y}; }
    friend bool operator==(const Offset2D &a, const Offset2D &b) { return a.x == b.x && a.y == b.y; }
};
RollbackWeightedDSU<Offset2D> dsu(n);
dsu.add_relation(u, v, {dx, dy});
auto d = dsu.diff(u, v);
```

表示：

$$pos[v] - pos[u] = (dx, dy)$$

3.3 BIT

3.3.1 基础 BIT

```
template<class T>
struct BIT {
    int n;
    vector<T> t;
    explicit BIT(int n_ = 0) : n(n_), t(n + 1) {}
    explicit BIT(const vector<T> &a) : BIT(int(a.size())) {
        for (int i = 1; i <= n; ++ i) {
            t[i] += a[i - 1];
            int j{ i + (i & -i) };
            if (j <= n) t[j] += t[i];
        }
    }
    void add(int x, const T &v) { for (++ x; x <= n; x += x & -x) t[x] += v; }
    T sum(int r) const {
        T res{};
        for (; r > 0; r -= r & -r) res += t[r];
        return res;
    }
    T query(int l, int r) const { return sum(r) - sum(l); }
    // 返回最大的 r，满足 sum(r) <= limit，O(log_n)
    // 要求所有权值非负
    int max_right(const T &limit) const {
        int x{ 0 }, step{ 1 };
        T cur{};
        while (step <= n / 2) step <<= 1;
        for (; step; step >>= 1) if (x + step <= n && cur + t[x + step] <= limit) {
            cur += t[x += step];
        }
        return x;
    }
    // 权值 BIT 中返回第 k(从 1 开始) 个元素所在的 0base 下标，O(log_n)
    int kth(const T &k) const { return max_right(k - T(1)); }
};
```

区间加、区间求和 BIT

使用差分数组和两棵基础 `BIT`

```
template<class T>
struct RangeBIT {
    int n;
    BIT<T> b1, b2;
    explicit RangeBIT(int n_ = 0) : n(n_), b1(n + 1), b2(n + 1) {}
```

```
    explicit RangeBIT(const vector<T> &a) : n(int(a.size())) {
        vector<T> d(n + 1), w(n + 1);
        for (int i = 0; i < n; ++ i) d[i] = a[i] - (i ? a[i - 1] : T{});
        if (n) d[n] = T{} - a.back();
        for (int i = 0; i <= n; ++ i) w[i] = d[i] * T(i);
        b1 = BIT<T>(d);
        b2 = BIT<T>(w);
    }
private:
    void add_point(int x, const T &v) {
        b1.add(x, v);
        b2.add(x, v * T(x));
    }
public:
    void add(int l, int r, const T &v) {
        add_point(l, v);
        add_point(r, T{} - v);
    }
    T sum(int r) const { return b1.sum(r) * T(r) - b2.sum(r); }
    T query(int l, int r) const { return sum(r) - sum(l); }
};
```

前缀最大值 BIT

```
template<class T>
struct MaxBIT {
    int n;
    T e; // 初始聚合值
    vector<T> t;
    explicit MaxBIT(int n_ = 0, T e_ = numeric_limits<T>::lowest()) : n(n_), e(e_), t(n + 1, e) {}
    // 单点执行 chmax
    void apply(int x, const T &v) { for (++ x; x <= n; x += x & -x) t[x] = max(t[x], v); }
    // 查询 [0, r) 最大值
    T prefix(int r) const {
        T res{ e };
        for (; r > 0; r -= r & -r) res = max(res, t[r]);
        return res;
    }
};
```

修改为其他前缀运算

只需修改：

1. 单位元 `e`
2. `apply()` 和 `prefix()` 中的运算
   其中 `e` 必须满足：

$$\mathrm{op}(e, x) = x$$

常见运算如下：

运算	`e`	合并方式
最大值	`numeric_limits<T>::lowest()`	`max(x, y)`
最小值	`numeric_limits<T>::max()`	`min(x, y)`
GCD	`T{}`，即 `0`	`gcd(x, y)`
按位或	`T{}`，即 `0`	`x | y`
按位与	全 `1`	`x & y`

3.3.2 基础 BIT2D：单点加、矩形和

```
template<class T>
struct BIT2D {
    int n, m;
```

```
    vector<vector<T>> t;
    BIT2D(int n_ = 0, int m_ = 0) : n(n_), m(m_), t(n + 1, vector<T>(m + 1)) {}
    explicit BIT2D(const vector<vector<T>> &a)
        : n(a.size()), m(n ? a[0].size() : 0), t(n + 1, vector<T>(m + 1)) {
        for (int i = 1; i <= n; ++ i) {
            for (int j = 1; j <= m; ++ j) t[i][j] = a[i - 1][j - 1];
        }
        for (int i = 1; i <= n; ++ i) {
            for (int j = 1; j <= m; ++ j) {
                int k{ j + (j & -j) };
                if (k <= m) t[i][k] += t[i][j];
            }
        }
        for (int i = 1; i <= n; ++ i) {
            int k{ i + (i & -i) };
            if (k <= n) {
                for (int j = 1; j <= m; ++ j) t[k][j] += t[i][j];
            }
        }
    }
    void add(int x, int y, const T &v) {
        for (++ x; x <= n; x += x & -x) {
            for (int j = y + 1; j <= m; j += j & -j) t[x][j] += v;
        }
    }
    T prefix(int x, int y) const {
        T res{};
        for (; x > 0; x -= x & -x) {
            for (int j = y; j > 0; j -= j & -j) res += t[x][j];
        }
        return res;
    }
    T query(int x1, int y1, int x2, int y2) const {
        return prefix(x2, y2) - prefix(x1, y2)
             - prefix(x2, y1) + prefix(x1, y1);
    }
};
```

前缀最大值 BIT2D

```
template<class T>
struct MaxBIT2D {
    int n, m;
    T e;
    vector<vector<T>> t;
    MaxBIT2D(int n_ = 0, int m_ = 0, T e_ = numeric_limits<T>::lowest())
        : n(n_), m(m_), e(e_), t(n + 1, vector<T>(m + 1, e)) {}
    void apply(int x, int y, const T &v) {
        for (++ x; x <= n; x += x & -x) {
            for (int j = y + 1; j <= m; j += j & -j) t[x][j] = max(t[x][j], v);
        }
    }
    T prefix(int x, int y) const {
        T res{ e };
        for (; x > 0; x -= x & -x) {
            for (int j = y; j > 0; j -= j & -j) res = max(res, t[x][j]);
        }
        return res;
    }
};
```

修改为其他前缀运算

只需修改单位元 `e` 和两处聚合运算

`e` 必须满足：

$$\mathrm{op}(e, a) = a$$

常见修改：

运算	e	聚合
最大值	numeric_limits<T>::lowest()	max(a, b)
最小值	numeric_limits<T>::max()	min(a, b)
GCD	T{}，即 0	gcd(a, b)
按位或	T{}，即 0	a | b
按位与	~T{}	a & b

矩形加、单点查询

使用二维差分。为了允许在 x = n 或 y = m 处加入右边界/下边界差分，内部 BIT 开到 (n + 1) × (m + 1)

```
template<class T>
struct RectAddPoint {
    BIT2D<T> bit;
private:
    void corner(int x, int y, const T &v) { bit.add(x, y, v); }
public:
    RectAddPoint(int n, int m) : bit(n + 1, m + 1) {}
    void add(int x1, int y1, int x2, int y2, const T &v) {
        corner(x1, y1, v);
        corner(x1, y2, T{} - v);
        corner(x2, y1, T{} - v);
        corner(x2, y2, v);
    }
    T get(int x, int y) const { return bit.prefix(x + 1, y + 1); }
};
```

矩形加、矩形求和

二维差分中，前缀矩形和为：

$$\sum_{i<x,\, j<y} d_{i,j}(x-i)(y-j)$$

展开后分别维护：

$$d, \quad id, \quad jd, \quad ijd$$

因此需要四棵二维 BIT

```
template<class T>
struct RectAddRectSum {
    BIT2D<T> b00, b10, b01, b11;
private:
    void corner(int x, int y, const T &v) {
        b00.add(x, y, v);
        b10.add(x, y, v * T(x));
        b01.add(x, y, v * T(y));
        b11.add(x, y, v * T(x) * T(y));
    }
public:
    RectAddRectSum(int n, int m) : b00(n + 1, m + 1), b10(n + 1, m + 1),
          b01(n + 1, m + 1), b11(n + 1, m + 1) {}
    void add(int x1, int y1, int x2, int y2, const T &v) {
        corner(x1, y1, v);
        corner(x1, y2, T{} - v);
        corner(x2, y1, T{} - v);
        corner(x2, y2, v);
    }
    T prefix(int x, int y) const {
        return b00.prefix(x, y) * T(x) * T(y) - b10.prefix(x, y) * T(y)
             - b01.prefix(x, y) * T(x) + b11.prefix(x, y);
    }
    T query(int x1, int y1, int x2, int y2) const {
        return prefix(x2, y2) - prefix(x1, y2) - prefix(x2, y1) + prefix(x1, y1);
```

```
    }
};
```

注意事项

- 所有坐标均为 `0base`
- 所有矩形均使用半开区间 `[x1, x2) × [y1, y2)`
- `prefix(x, y)` 中的 `x, y` 表示矩形长度，不包含坐标 `x` 和 `y`
- 所有非初始化操作复杂度均为 $\log n \log m$

3.3.3 离线二维树状数组

解决问题： 第一维已离散、第二维坐标很大且修改点稀疏时，处理二维点修改、二维前缀和与矩形和

外层对第一维建立 BIT；每个 BIT 节点只离散实际可能被修改的第二维坐标

```
template<class T>
struct OfflineBIT2D {
    int n;
    vector<vector<i64>> ys;
    vector<vector<T>> t;
    explicit OfflineBIT2D(int n_ = 0) : n(n_), ys(n + 1), t(n + 1) {}
    OfflineBIT2D(int n_, const vector<pair<int, i64>> &points) : OfflineBIT2D(n_) {
        for (auto [x, y] : points) reserve_point(x, y);
        build();
    }
    // ===== 离线预处理：所有可能修改的点登记完成后调用 build() =====
    void reserve_point(int x, i64 y) {
        for (++ x; x <= n; x += x & -x) ys[x].push_back(y);
    }
    void build() {
        for (int x = 1; x <= n; ++ x) {
            auto &v{ ys[x] };
            ranges::sort(v.begin(), v.end());
            v.erase(ranges::unique(v).begin(), v.end());
            t[x].assign(v.size() + 1, T{});
        }
    }
    // ===== 修改与查询：build() 后使用 =====
    void add(int x, i64 y, const T &v) {
        for (++ x; x <= n; x += x & -x) {
            int p{ int(ranges::lower_bound(ys[x], y) - ys[x].begin()) + 1 };
            for (; p < int(t[x].size()); p += p & -p) t[x][p] += v;
        }
    }
    T prefix(int x, i64 y) const {
        T res{};
        for (; x > 0; x -= x & -x) {
            int p{ int(ranges::lower_bound(ys[x], y) - ys[x].begin()) };
            for (; p > 0; p -= p & -p) res += t[x][p];
        }
        return res;
    }
    T query(int x1, i64 y1, int x2, i64 y2) const {
        return prefix(x2, y2) - prefix(x1, y2) - prefix(x2, y1) + prefix(x1, y1);
    }
};
```

接口说明

成员变量

- `n`：第一维离散后的长度
- `ys[x]`：外层 BIT 节点 `x` 实际需要维护的第二维坐标
- `t[x]`：节点 `x` 内部的一维 BIT

接口

- `OfflineBIT2D(n)`：建立第一维长度为 `n` 的空结构
- `OfflineBIT2D(n, points)`：登记 `points` 中所有可能修改的点并自动执行 `build()`

- `reserve_point(x, y)`：预登记之后可能调用 `add()` 的坐标，不产生实际修改，$O(\log n)$
- `build()`：对每个外层 BIT 节点分别离散第二维
- `add(x, y, v)`：令点 `(x, y)` 的值增加 `v`
- `prefix(x, y)`：查询 $[0, x) \times (-\infty, y)$ 中的值之和
- `query(x1, y1, x2, y2)`：查询 $[x_1, x_2) \times [y_1, y_2)$ 中的值之和

注意事项

- 第一维必须已经转换为 `0base` 的 `[0, n)` 下标
- 若原始第一维坐标也很大，应先在外部离散化；本结构只自动离散第二维
- 第二维使用 `i64`，可以直接处理很大的正负坐标
- 所有以后可能调用

```
add(x, y, ...)
```

的 `(x, y)` 都必须在 `build()` 前登记

- 查询所使用的 `y` 不需要登记，因为查询通过 `lower_bound()` 找到严格小于 `y` 的已登记坐标
- 同一点可以重复登记，`build()` 会自动去重
- `build()` 后不要继续 `reserve_point()`；新增坐标不会被加入已经建立的内部 BIT
- `add()` 默认传入的点已经登记，不检查坐标是否真实存在
- 所有区间均为半开区间
- `T` 需要支持零初始化、`+=` 和减法
- 若所有二维坐标范围本来就很小，直接使用稠密 `BIT2D` 更简单且常数更低

常见用法

已知全部可能修改点

```
vector<pair<int, i64>> points;

for (auto [x, y] : possible_updates) {
    points.push_back({x, y});
}

OfflineBIT2D<i64> bit(n, points);

bit.add(x, y, delta);
i64 ans = bit.query(x1, y1, x2, y2);
```

扫描操作后统一建树

```
OfflineBIT2D<int> bit(n);

for (auto [x, y] : possible_updates) {
    bit.reserve_point(x, y);
}

bit.build();

for (auto &op : operations) {
    // add / query
}
```

二维偏序计数

按某一维排序后，将另一维作为 BIT 的第一维或第二维：

```
bit.add(x, y, 1);
int cnt = bit.prefix(x, y);
```

具体是否包含当前点，应根据排序和同值处理方式调整。

复杂度

设共登记 M 个点
一次 `reserve_point()` 会进入 $O(\log n)$ 个外层 BIT 节点，因此所有内部离散坐标总数为

$$O(M \log n)$$

操作	时间复杂度
`reserve_point`	$O(\log n)$
全部 `build`	$O(M \log n \log M)$
`add`	$O(\log n \log M)$
`prefix`	$O(\log n \log M)$
`query`	$O(\log n \log M)$

空间复杂度：

$$O(M \log n)$$

两维规模同阶时，单次修改和查询通常简写为 $O(\log^2 n)$

3.4 RMQ、静态区间查询与序列树

3.4.1 ST 表

```
struct OR {
    template<class T>
    T operator()(const T &a, const T &b) const { return a | b; }
};
struct AND {
    template<class T>
    T operator()(const T &a, const T &b) const { return a & b; }
};
struct MAX {
    template<class T>
    T operator()(const T &a, const T &b) const { return max(a, b); }
};
struct MIN {
    template<class T>
    T operator()(const T &a, const T &b) const { return min(a, b); }
};
struct GCD {
    template<class T>
    T operator()(T a, T b) const { return gcd(a, b); }
};
template<class T, class Merge>
struct ST {
    int n{};
    Merge merge;
    vector<vector<T>> st;
    explicit ST(vector<T> a, Merge m = Merge()) : merge(m) { build(move(a)); }
    void build(vector<T> a) {
        n = int(a.size());
        assert(n > 0);
        int k{ __lg(n) + 1 };
        st.assign(k, {});
        st[0] = move(a);
        for (int i = 1; i < k; ++ i) {
            int len{ 1 << i }, half{ len >> 1 };
            st[i].resize(n - len + 1);
            for (int j = 0; j + len <= n; ++ j) st[i][j] = merge(st[i - 1][j], st[i - 1][j + half]);
        }
    }
    T query(int l, int r) const {
        if (l > r) swap(l, r);
        int k{ __lg(r - l + 1) };
        return merge(st[k][l], st[k][r - (1 << k) + 1]);
    }
};
```

Argmin / Argmax

修改比较器即可切换 `Argmin` / `Argmax`

```cpp
template<class T, class Cmp = less<T>>
struct ArgST {
    int n{};
    bool pre_l{ true };
    Cmp cmp;
    vector<T> a;
    vector<vector<int>> st;
    explicit ArgST(vector<T> a_, bool pl = true, Cmp c = Cmp()) : pre_l(pl), cmp(c) { build(move(a_)); }
    void build(vector<T> a_) {
        a = move(a_);
        n = a.size();
        assert(n > 0);
        int k{ __lg(n) + 1 };
        st.assign(k, {});
        st[0].resize(n);
        ranges::iota(st[0], 0);
        for (int j = 1; j < k; ++ j) {
            int len{ 1 << j }, half{ len >> 1 };
            st[j].resize(n - len + 1);
            for (int i = 0; i + len <= n; ++ i) st[j][i] = better(st[j - 1][i], st[j - 1][i + half]);
        }
    }
    int queryIndex(int l, int r) const {
        if (l > r) swap(l, r);
        int k{ __lg(r - l + 1) };
        return better(st[k][l], st[k][r - (1 << k) + 1]);
    }
    const T &query(int l, int r) const { return a[queryIndex(l, r)]; }
    const T &at(int i) const { return a[i]; }
private:
    int better(int i, int j) const {
        if (i < 0) return j;
        if (j < 0) return i;
        if (cmp(a[i], a[j])) return i;
        if (cmp(a[j], a[i])) return j;
        return pre_l ? min(i, j) : max(i, j);
    }
};
```

接口说明

成员变量

- `n`：数组长度
- `merge`：区间合并函数
- `st[k][i]`：长度为 `2^k`、左端点为 `i` 的区间信息
- `ArgST::a`：保存原数组，避免外部数组失效
- `ArgST::prefer_left`：最优值相同时是否返回更靠左的位置

接口

- `ST(vector<T> a, Merge m = Merge())`：用数组 `a` 建表，$O(n \log n)$
- `build(vector<T> a)`：重新建表，$O(n \log n)$
- `query(l, r)`：查询闭区间 `[l, r]` 的合并值，$O(1)$
- `ArgST(vector<T> a, Cmp c = Cmp(), bool preferLeft = true)`：用比较器建位置表，$O(n \log n)$
- `queryIndex(l, r)`：返回闭区间 `[l, r]` 内最优元素下标，$O(1)$
- `ArgST::query(l, r)`：返回闭区间 `[l, r]` 内最优元素引用，$O(1)$
- `at(i)`：返回原数组第 `i` 个元素引用，$O(1)$

注意事项

- 下标为 `0base`，查询区间为闭区间 `[l, r]`
- `ArgST` 默认 `less<T>`，即返回最小值位置；若要最大值，使用 `greater<T>`

常见用法

区间极值位置

```cpp
ArgST<int> st(a); // 最大值改为 ArgST<int, greater<int>> st(a);
int p{ st.queryIndex(l, r) }, val{ st.query(l, r) };
```

结构体自定义 merge/cmp

ST

```
struct Info {
    i64 key = 0;
    int id = -1;
};
struct MinKey {
    Info operator()(const Info &a, const Info &b) const {
        if (a.key != b.key) return a.key < b.key ? a : b;
        return a.id < b.id ? a : b;
    }
};
vector<Info> a(n);
for (int i = 0; i < n; ++ i) {
    cin >> a[i].key;
    a[i].id = i;
}
ST<Info, MinKey> st(a);
Info res{ st.query(l, r) };
int pos{ res.id };
```

ArgST

```
struct Info {
    i64 key = 0;
    int id = -1;
};
struct CmpKey {
    bool operator()(const Info &a, const Info &b) const {
        if (a.key != b.key) return a.key < b.key;
        return a.id < b.id;
    }
};
vector<Info> a(n);
for (int i = 0; i < n; ++ i) cin >> a[i].key;
ArgST<Info, CmpKey> st(a);
int pos{ st.queryIndex(l, r) };
Info res{ st.query(l, r) };
```

3.4.2 二维 ST 表

```
template<class T, class Merge>
struct SparseTable2D {
    int n, m, kn, km;
    Merge merge;
    vector<vector<T>> st;
    explicit SparseTable2D(const vector<vector<T>> &a, Merge merge_ = Merge())
        : n(int(a.size())), m(n ? int(a[0].size()) : 0), merge(move(merge_)) {
        assert(n > 0 && m > 0);
        for (const auto &row : a) assert(int(row.size()) == m);
        kn = __lg(n) + 1;
        km = __lg(m) + 1;
        st.resize(kn * km);
        // st[x * km + y] 连续存储所有 2^x * 2^y 子矩形
        st[0].resize(n * m);
        for (int i = 0; i < n; ++ i) {
            copy(a[i].begin(), a[i].end(), st[0].begin() + i * m);
        }
        // 先沿列方向倍增
        for (int y = 1; y < km; ++ y) {
            int len{ 1 << y }, half{ len >> 1 };
            int w{ m - len + 1 }, pw{ m - half + 1 };
            auto &cur{ st[y] };
            const auto &pre{ st[y - 1] };
            cur.resize(n * w);
            for (int i = 0; i < n; ++ i) {
```

```
                for (int j = 0; j < w; ++ j) {
                    cur[i * w + j] = merge(pre[i * pw + j], pre[i * pw + j + half]);
                }
            }
        }
        // 再沿行方向倍增
        for (int x = 1; x < kn; ++ x) {
            int len{ 1 << x }, half{ len >> 1 }, h{ n - len + 1 };
            for (int y = 0; y < km; ++ y) {
                int w{ m - (1 << y) + 1 };
                auto &cur{ st[x * km + y] };
                const auto &pre{ st[(x - 1) * km + y] };
                cur.resize(h * w);
                for (int i = 0; i < h; ++ i) {
                    for (int j = 0; j < w; ++ j) {
                        cur[i * w + j] = merge(pre[i * w + j], pre[(i + half) * w + j]);
                    }
                }
            }
        }
    }
    T query(int x1, int y1, int x2, int y2) const {
        assert(0 <= x1 && x1 < x2 && x2 <= n);
        assert(0 <= y1 && y1 < y2 && y2 <= m);
        int kx{ __lg(x2 - x1) }, ky{ __lg(y2 - y1) }, hx{ 1 << kx }, wy{ 1 << ky };
        int xa{ x2 - hx }, ya{ y2 - wy }, w{ m - wy + 1 };
        const auto &v{ st[kx * km + ky] };
        auto get = [&](int x, int y) -> const T & {
            return v[x * w + y];
        };
        return merge(
            merge(get(x1, y1), get(x1, ya)),
            merge(get(xa, y1), get(xa, ya))
        );
    }
};
```

- `SparseTable2D<T, Merge>(a, merge)`：对非空矩阵 `a` 建立二维 ST 表，$O(nm \log n \log m)$
- `query(x1, y1, x2, y2)`：查询半开矩形 `[x1,x2) × [y1,y2)`，$O(1)$
- `Merge`：通常使用 `min`、`max`、`gcd`、按位与、按位或等幂等运算

注意事项

- 二维 ST 的空间常数很大，实际存储元素数为

$$\sum_{x=0}^{\lfloor \log_2 n \rfloor} \sum_{y=0}^{\lfloor \log_2 m \rfloor} (n - 2^x + 1)(m - 2^y + 1)$$

  大规模矩阵使用前应先估算内存
- 若 `T` 是较大的结构体，二维 ST 的内存会进一步放大；只需要位置时可以考虑仅保存下标

二维最小值

```
struct Min {
    int operator()(int a, int b) const { return min(a, b); }
};
SparseTable2D<int, Min> st(a);
int ans = st.query(x1, y1, x2, y2);
```

同时返回二维最小值及位置

```
struct GridInfo {
    i64 value{};
    int x{ -1 }, y{ -1 };
};

struct MinGridInfo {
    GridInfo operator()(const GridInfo &a, const GridInfo &b) const {
        return tie(a.value, a.x, a.y) < tie(b.value, b.x, b.y) ? a : b;
    }
};
```

```
vector<vector<GridInfo>> a(n, vector<GridInfo>(m));
for (int i = 0; i < n; ++ i) {
    for (int j = 0; j < m; ++ j) {
        a[i][j] = { value[i][j], i, j };
    }
}

SparseTable2D<GridInfo, MinGridInfo> st(a);
GridInfo ans = st.query(x1, y1, x2, y2);
```

猫树（Disjoint Sparse Table）

```
template<class T, class Merge>
struct DisjointSparseTable {
    int n, lg;
    Merge merge;
    vector<T> a;
    vector<vector<T>> st;
    explicit DisjointSparseTable(vector<T> a_, Merge merge_ = Merge())
        : n(int(a_.size())), lg(n <= 1 ? 0 : __lg(n - 1) + 1),
          merge(move(merge_)), a(move(a_)), st(lg, vector<T>(n)) {
        assert(n > 0);
        // 第 k 层每块长度为 2^(k+1)，分别维护中点向左、向右的区间积
        for (int k = 0; k < lg; ++ k) {
            int half{ 1 << k }, len{ half << 1 };
            for (int l = 0; l < n; l += len) {
                int mid{ min(l + half, n) }, r{ min(l + len, n) };
                st[k][mid - 1] = a[mid - 1];
                for (int i = mid - 2; i >= l; -- i) {
                    st[k][i] = merge(a[i], st[k][i + 1]);
                }
                if (mid < r) {
                    st[k][mid] = a[mid];
                    for (int i = mid + 1; i < r; ++ i) {
                        st[k][i] = merge(st[k][i - 1], a[i]);
                    }
                }
            }
        }
    }
    T query(int l, int r) const {
        assert(0 <= l && l < r && r <= n);
        if (r - l == 1) return a[l];
        int k{ __lg(l ^ (r - 1)) };
        return merge(st[k][l], st[k][r - 1]);
    }
};
```

- `DisjointSparseTable<T, Merge>(a, merge)`：对非空静态数组建立猫树，$O(n \log n)$
- `query(l, r)`：查询半开区间 `[l,r)`，$O(1)$
- `Merge`：只要求满足结合律，不要求幂等，也不要求交换律

注意事项

- 猫树查询时 `merge(st[k][l], st[k][r-1])` 的顺序不能交换，因此可以正确处理字符串连接、矩阵乘法等非交换结合运算
- 猫树单元素区间直接返回 `a[l]`；否则 `l ^ (r - 1)` 一定非零，可以安全调用 `__lg()`
- `Merge::operator()` 应能够在 `const` 查询中调用，通常写成：

```
T operator()(const T &a, const T &b) const;
```

猫树求区间和

```
DisjointSparseTable<i64, plus<i64>> st(a);
i64 ans = st.query(l, r);
```

猫树处理非交换运算

例如字符串拼接：

```
struct Merge {
    string operator()(const string &a, const string &b) const {
        return a + b;
    }
};

DisjointSparseTable<string, Merge> st(a);
string ans = st.query(l, r);
```

矩阵乘法同理，只要 `merge(a, b)` 表示严格的 `a * b` 即可

3.4.3 笛卡尔树

```
template<class T, class Compare = less<T>>
struct CartesianTree {
    int n, root{ -1 };
    vector<int> parent, left, right;
    explicit CartesianTree(const vector<T> &a, Compare better = Compare())
        : n(int(a.size())), parent(n, -1), left(n, -1), right(n, -1) {
        vector<int> stk;
        stk.reserve(n);
        for (int i = 0; i < n; ++ i) {
            int last{ -1 };
            while (!stk.empty() && better(a[i], a[stk.back()])) {
                last = stk.back();
                stk.pop_back();
            }
            if (!stk.empty()) {
                parent[i] = stk.back();
                right[stk.back()] = i;
            }
            if (last != -1) {
                parent[last] = i;
                left[i] = last;
            }
            stk.push_back(i);
        }
        if (!stk.empty()) root = stk.front();
    }
};
```

成员变量

- `n`：元素个数
- `root`：根节点下标；空数组时为 `-1`
- `parent[i]`：节点 `i` 的父亲；根节点为 `-1`
- `left[i]` / `right[i]`：节点 `i` 的左右儿子；不存在时为 `-1`
- 节点编号就是原数组下标，因此整棵树的中序遍历固定为 `0,1,...,n-1`

接口

- `CartesianTree(a, better)`：$O(n)$ 构造由 `better` 决定堆序的笛卡尔树
- 默认 `less<T>`：最小笛卡尔树
- 使用 `greater<T>`：最大笛卡尔树

注意事项

- `Compare` 应满足严格弱序；`better(x,y)` 表示 `x` 的优先级严格高于 `y`
- 当前只在新元素**严格更优**时弹栈，因此等价元素不会弹出。默认最小笛卡尔树中，相同最小值会优先选择**更靠左**的位置
- 因此默认最小笛卡尔树中，对任意非空区间 `[l,r)`：$\mathrm{LCA}(l, r-1)$ 就是区间最小值的位置；有多个最小值时返回最靠左的一个。最大笛卡尔树同理对应区间最大值
- 每棵子树在原数组中都对应一个连续区间；节点 `u` 是该区间按 `Compare` 选出的最优元素
- 若节点 `u` 的左右子树大小分别为 `L,R`，则在当前重复值规则下，以 `u` 作为所选极值位置的子数组数量为 $(L+1)(R+1)$
- 笛卡尔树可能完全退化成长度为 `n` 的链；需要遍历整棵树时，大数据不要直接使用递归 DFS
- `root = stk.front()` 是构造后的根：最终单调栈中的第一个元素没有父亲
- 笛卡尔树把 RMQ 转为 LCA；而经典的 LCA 欧拉序算法则把 LCA 转为 RMQ，两者方向不要混淆

常见用法

最小笛卡尔树

```
CartesianTree<int> ct(a);

int root = ct.root;
int lc = ct.left[root];
int rc = ct.right[root];
```

最大笛卡尔树

```
CartesianTree<int, greater<int>> ct(a);
```

RMQ 转 LCA

先在笛卡尔树上建立 LCA 结构：

```
CartesianTree<int> ct(a);

// 在 ct.left / ct.right 构成的树上预处理 LCA。
int pos = lca(l, r - 1);
int minimum = a[pos];
```

自定义比较规则

```
struct CartesianKey {
    i64 value{};
    int id{};
};

struct BetterCartesianKey {
    bool operator()(const CartesianKey &a, const CartesianKey &b) const {
        return tie(a.value, a.id) < tie(b.value, b.id);
    }
};

vector<CartesianKey> a(n);
CartesianTree<CartesianKey, BetterCartesianKey> ct(a);

int id = a[ct.root].id;
```

如果需要显式规定重复值的优先级，最稳妥的方法就是像上面一样把编号一起放入比较键

3.4.4 划分树

只需要静态区间第 `k` 小时使用

```
template<class T>
struct PartitionTree {
private:
    int n, lg;
    vector<vector<T>> tr;
    vector<vector<int>> pref;
    void build(int d, int l, int r, const vector<T> &sorted) {
        if (r - l <= 1) return;
        int m{ (l + r) / 2 };
        const T &pivot{ sorted[m - 1] };
        // 左半部分固定需要 m-l 个元素，重复 pivot 时精确控制放左边的数量
        int same{ m - l };
        for (int i = l; i < r; ++ i) {
            if (tr[d][i] < pivot) -- same;
        }
        int x{ l }, y{ m };
        for (int i = l; i < r; ++ i) {
            bool left{
                tr[d][i] < pivot ||
                (!(pivot < tr[d][i]) && same -- > 0)
```

```
            };
            pref[d][i + 1] = pref[d][i] + left;
            tr[d + 1][left ? x ++ : y ++] = tr[d][i];
        }
        build(d + 1, l, m, sorted);
        build(d + 1, m, r, sorted);
    }
    T kth(int d, int l, int r, int ql, int qr, int k) const {
        if (r - l == 1) return tr[d][l];
        int m{ (l + r) / 2 };
        int before{ pref[d][ql] - pref[d][l] };
        int cnt{ pref[d][qr] - pref[d][ql] };
        if (k < cnt) {
            int nl{ l + before };
            return kth(d + 1, l, m, nl, nl + cnt, k);
        }
        int nl{ m + (ql - l) - before };
        return kth(d + 1, m, r, nl, nl + (qr - ql - cnt), k - cnt);
    }
public:
    explicit PartitionTree(const vector<T> &a)
        : n(int(a.size())), lg(n <= 1 ? 0 : __lg(n - 1) + 1),
          tr(lg + 1, vector<T>(n)), pref(lg, vector<int>(n + 1)) {
        tr[0] = a;
        // sorted 只在建树时需要，不作为成员长期保存
        vector<T> sorted(a);
        sort(sorted.begin(), sorted.end());
        build(0, 0, n, sorted);
    }
    T kth(int l, int r, int k) const {
        assert(0 <= l && l < r && r <= n);
        assert(0 <= k && k < r - l);
        return kth(0, 0, n, l, r, k);
    }
};
```

- `PartitionTree(a)`：预处理静态数组，$O(n \log n)$
- `kth(l,r,k)`：返回 `[l,r)` 中第 `k` 小的值，`k` 为 `0base`，$O(\log n)$
  划分树只保留最核心的区间第 `k` 小接口；需要复杂值域统计时优先使用下面的小波矩阵

小波矩阵（离散值 + 压位）

```
template<class T>
struct WaveletMatrix {
private:
    struct BitVector {
        int words{};
        vector<u64> bit;
        vector<int> sum;
        BitVector() = default;
        explicit BitVector(int n) : words((n + 63) >> 6), bit(words), sum(words + 1) {}
        void set(int i) { bit[i >> 6] |= 1ULL << (i & 63); }
        void build() {
            for (int i = 0; i < words; ++ i) {
                sum[i + 1] = sum[i] + popcount(bit[i]);
            }
        }
        // [0,i) 中 1 的数量
        int rank1(int i) const {
            int w{ i >> 6 }, b{ i & 63 };
            return sum[w] + (b ? popcount(bit[w] & ((1ULL << b) - 1)) : 0);
        }
    };
    int n, lg;
    vector<T> xs;
    vector<int> zero;
    vector<BitVector> mat;
    int kth_id(int l, int r, int k) const {
```

```
        int id{};
        for (int d = 0; d < lg; ++ d) {
            int bit{ lg - 1 - d };
            int one_l{ mat[d].rank1(l) };
            int one_r{ mat[d].rank1(r) };
            int zeros{ (r - l) - (one_r - one_l) };
            if (k < zeros) {
                l -= one_l;
                r -= one_r;
            } else {
                k -= zeros;
                id |= 1 << bit;
                l = zero[d] + one_l;
                r = zero[d] + one_r;
            }
        }
        return id;
    }
    // 统计离散编号 < upper 的元素个数
    int count_id_less(int l, int r, int upper) const {
        assert(0 ⩽ l && l ⩽ r && r ⩽ n);
        if (upper ⩽ 0) return 0;
        if (upper ⩾ int(xs.size())) return r - l;
        int ans{};
        for (int d = 0; d < lg; ++ d) {
            int bit{ lg - 1 - d }, one_l{ mat[d].rank1(l) };
            int one_r{ mat[d].rank1(r) }, zeros{ (r - l) - (one_r - one_l) };
            if (upper >> bit & 1) {
                ans += zeros;
                l = zero[d] + one_l;
                r = zero[d] + one_r;
            } else {
                l -= one_l;
                r -= one_r;
            }
        }
        return ans;
    }
public:
    explicit WaveletMatrix(const vector<T> &a) : n(int(a.size())), xs(a) {
        sort(xs.begin(), xs.end());
        // 只要求 T 定义严格弱序 <，不额外要求 ==
        xs.erase(ranges::unique(xs, [](const T &a, const T &b) {
            return !(a < b) && !(b < a);
        }).begin(), xs.end());
        int sigma{ int(xs.size()) };
        lg = sigma ⩽ 1 ? 0 : __lg(sigma - 1) + 1;
        zero.resize(lg);
        mat.reserve(lg);
        vector<int> cur(n), nxt(n);
        for (int i = 0; i < n; ++ i) {
            cur[i] = lower_bound(xs.begin(), xs.end(), a[i]) - xs.begin();
        }
        // 从高位到低位稳定划分
        for (int d = 0; d < lg; ++ d) {
            int bit{ lg - 1 - d };
            mat.emplace_back(n);
            for (int i = 0; i < n; ++ i) {
                if (cur[i] >> bit & 1) mat.back().set(i);
            }
            mat.back().build();
            zero[d] = n - mat.back().rank1(n);
            int p0{ 0 }, p1{ zero[d] };
            for (int x : cur) {
                if (x >> bit & 1) nxt[p1 ++] = x;
                else nxt[p0 ++] = x;
            }
            cur.swap(nxt);
        }
    }
    // ===== 顺序统计 =====
```

```
    T kth(int l, int r, int k) const {
        assert(0 <= l && l < r && r <= n);
        assert(0 <= k && k < r - l);
        return xs[kth_id(l, r, k)];
    }
    // ===== 频次统计 =====
    int count_less(int l, int r, const T &x) const {
        int upper{ int(lower_bound(xs.begin(), xs.end(), x) - xs.begin()) };
        return count_id_less(l, r, upper);
    }
    int count_leq(int l, int r, const T &x) const {
        int upper{ int(upper_bound(xs.begin(), xs.end(), x) - xs.begin()) };
        return count_id_less(l, r, upper);
    }
    int count(int l, int r, const T &x) const {
        assert(0 <= l && l <= r && r <= n);
        int id{ int(lower_bound(xs.begin(), xs.end(), x) - xs.begin()) };
        if (id == int(xs.size()) || x < xs[id]) return 0;
        for (int d = 0; d < lg; ++ d) {
            int a{ mat[d].rank1(l) }, b{ mat[d].rank1(r) };
            if (id >> (lg - 1 - d) & 1) l = zero[d] + a, r = zero[d] + b;
            else l -= a, r -= b;
        }
        return r - l;
    }
    int count_range(int l, int r, const T &lo, const T &hi) const {
        assert(!(hi < lo));
        return count_less(l, r, hi) - count_less(l, r, lo);
    }
    // ===== 前驱后继：若不需要可以整段删除 =====
    optional<T> predecessor(int l, int r, const T &x) const {
        int k{ count_less(l, r, x) };
        if (k == 0) return nullopt;
        return kth(l, r, k - 1);
    }
    optional<T> successor(int l, int r, const T &x) const {
        int k{ count_leq(l, r, x) };
        if (k == r - l) return nullopt;
        return kth(l, r, k);
    }
    optional<T> next_ge(int l, int r, const T &x) const {
        int k{ count_less(l, r, x) };
        if (k == r - l) return nullopt;
        return kth(l, r, k);
    }
};
```

小波矩阵

- `WaveletMatrix(a)`：离散化并建立小波矩阵
- `kth(l,r,k)`：返回 `[l,r)` 中第 `k` 小，`k` 为 `0base`
- `count_less(l,r,x)`：统计 `[l,r)` 中 `< x` 的数量
- `count_leq(l,r,x)`：统计 `[l,r)` 中 `≤ x` 的数量
- `count(l,r,x)`：统计 `[l,r)` 中等于 `x` 的数量
- `count_range(l,r,lo,hi)`：统计位置 `[l,r)`、值域 `[lo,hi)` 中的元素数量
- `predecessor(l,r,x)`：返回 `< x` 的最大值，不存在返回 `nullopt`
- `successor(l,r,x)`：返回 `> x` 的最小值，不存在返回 `nullopt`
- `next_ge(l,r,x)`：返回 `≥ x` 的最小值，不存在返回 `nullopt`

除构造外，上述查询均为 $O(\log \sigma)$，其中 σ 为不同值数量

注意事项

- 两个结构均为静态结构；原数组修改后必须重新构造
- 所有位置区间均采用 `0base` 左闭右开 `[l,r)`
- `kth(l,r,k)` 中 `k` 为 `0base`，合法范围为 `0 ≤ k < r-l`
- 划分树中不能简单按 `≤ pivot` 分割。出现重复值时必须通过 `same` 精确控制进入左半部分的 `pivot` 数量，否则左右子树大小会错误
- 划分树的 `pref[d][i]` 是第 `d` 层位置前缀中进入左儿子的元素数量，用它把原查询区间映射到下一层
- 小波矩阵先把值离散为 `[0,\sigma)`，因此可以直接处理负数以及任意可排序类型
- 本版 `T` 只要求能够通过 `<` 建立严格弱序，不要求定义 `==`
- `count_range(l,r,lo,hi)` 的值域也是半开区间 `[lo,hi)`

- `predecessor()` / `successor()` 均为严格前驱、严格后继；如果需要 `≥ x` 的非严格后继使用 `next_ge()`
- 小波矩阵本身只维护离散编号和位向量，因此当前版本只能查询**数量与值**，不维护区间元素和。若需要“前 `k` 小之和 / 值域和”，需要在每层额外维护前缀和
- 小波矩阵中的 `BitVector` 使用 `u64` 压位，并每 `64` 位保存一次前缀 popcount；相比每层保存 `n+1` 个 `int` 前缀数组，内存常数显著降低
- 两个结构都可以构造空数组，但所有 `kth()` 查询必须保证区间非空

常见用法

区间第 k 小

```
WaveletMatrix<int> wm(a);
int x = wm.kth(l, r, k);
```

区间中位数

下中位数：

```
int median = wm.kth(l, r, (r - l - 1) / 2);
```

上中位数：

```
int median = wm.kth(l, r, (r - l) / 2);
```

值域频次

统计 `[l,r)` 中满足 `low ≤ a[i] < high` 的元素：

```
int cnt = wm.count_range(l, r, low, high);
```

统计某个值出现次数：

```
int cnt = wm.count(l, r, x);
```

排名

区间中严格小于 `x` 的元素数量就是 `x` 的 `0base` lower-bound 排名：

```
int rank = wm.count_less(l, r, x);
```

前驱与后继

```
auto pre = wm.predecessor(l, r, x); // 最大的 < x
auto nxt = wm.successor(l, r, x);   // 最小的 > x

if (pre) cout << *pre << '\n';
if (nxt) cout << *nxt << '\n';
```

如果题目需要最小的 `≥ x`：

```
auto nxt = wm.next_ge(l, r, x);
```

自定义可排序结构体

```
struct WaveletKey {
    i64 value{};
    int id{};
    friend bool operator<(const WaveletKey &a, const WaveletKey &b) {
        return tie(a.value, a.id) < tie(b.value, b.id);
    }
};

vector<WaveletKey> a(n);
WaveletMatrix<WaveletKey> wm(a);

WaveletKey x = wm.kth(l, r, k);
```

如果只希望按 `value` 查询而不区分 `id`，不要把 `id` 加入排序键

复杂度

设不同值数量为 σ

结构 / 操作	时间复杂度
划分树构造	$O(n\log n)$
划分树 `kth`	$O(\log n)$
小波矩阵离散化与构造	$O(n\log n)$
小波矩阵 `kth`	$O(\log \sigma)$
小波矩阵计数查询	$O(\log \sigma)$
小波矩阵前驱 / 后继	$O(\log \sigma)$

划分树空间复杂度为 $O(n\log n)$
小波矩阵的位向量部分约占 $O(n\log \sigma)$ bits，并额外维护每 `64` 位一个前缀计数；离散值表占 $O(\sigma)$ 个 `T`

3.4.5 状压 RMQ

对静态数组按比较器查询任意非空区间的最优值或最优位置，支持 $O(1)$ 查询，并在存在多个最优值时返回最左位置

```
template<class T, class Cmp = less<T>>
struct RMQ {
private:
    static constexpr int B{ 64 };
    int n;
    Cmp cmp;
    vector<T> a;
    vector<u64> stk;
    vector<vector<int>> st;
    // 返回两个位置中更优的位置；等价时取更靠左的位置
    int better(int x, int y) const {
        if (cmp(a[x], a[y])) return x;
        if (cmp(a[y], a[x])) return y;
        return min(x, y);
    }
    // 查询同一块内 [l,r) 的最优位置
    int small(int l, int r) const {
        u64 s{ stk[r - 1] >> (l % B) };
        return l + __builtin_ctzll(s);
    }
public:
    explicit RMQ(vector<T> a_, Cmp cmp_ = Cmp())
        : n(int(a_.size())), cmp(move(cmp_)), a(move(a_)), stk(n) {
        assert(n > 0);
        int m{ (n + B - 1) / B };
        // 每个前缀维护块内单调栈；第 i 位表示对应位置仍在栈中
        for (int b = 0; b < m; ++ b) {
            int l{ b * B }, r{ min(n, l + B) };
            u64 s{};
            for (int i = l; i < r; ++ i) {
                while (s) {
                    int j{ 63 - __builtin_clzll(s) };
                    if (!cmp(a[i], a[l + j])) break;
                    s ^= 1ULL << j;
                }
                s |= 1ULL << (i - l);
                stk[i] = s;
            }
        }
        // 对完整块的最优位置建立 ST
        int lg{ __lg(m) + 1 };
        st.resize(lg);
        st[0].resize(m);
        for (int b = 0; b < m; ++ b) {
            st[0][b] = small(b * B, min(n, (b + 1) * B));
        }
        for (int k = 1; k < lg; ++ k) {
```

```
            int len{ m - (1 << k) + 1 };
            st[k].resize(max(0, len));
            for (int b = 0; b < len; ++ b) {
                st[k][b] = better(st[k - 1][b], st[k - 1][b + (1 << (k - 1))]);
            }
        }
    }
    int query_index(int l, int r) const {
        assert(0 ≤ l && l < r && r ≤ n);
        int lb{ l / B }, rb{ (r - 1) / B };
        if (lb == rb) return small(l, r);
        // 左右两个不完整块
        int ans{ better(small(l, (lb + 1) * B), small(rb * B, r)) };
        // 中间所有完整块
        int x{ lb + 1 }, y{ rb };
        if (x < y) {
            int k{ __lg(y - x) };
            ans = better(ans, better(st[k][x], st[k][y - (1 << k)]));
        }
        return ans;
    }
    const T &query(int l, int r) const { return a[query_index(l, r)]; }
};
```

模板参数

- `Cmp`：严格比较器，`cmp(x,y) = true` 表示 `x` 比 `y` 更优
  - 默认 `less<T>`：查询最小值
  - 使用 `greater<T>`：查询最大值

接口

- `RMQ(a, cmp)`：对非空静态数组建立 RMQ
- `query_index(l,r)`：返回 `[l,r)` 中最优元素的原数组下标，$O(1)$
- `query(l,r)`：返回 `[l,r)` 中最优元素的常引用，$O(1)$
  当多个元素在 `Cmp` 下等价时，两种查询都选择最左位置

注意事项

- 数组下标为 `0base`，所有查询均为左闭右开 `[l,r)`
- 构造数组必须非空，查询必须满足 `0 ≤ l < r ≤ n`
- 这是静态 RMQ；构造完成后不能修改内部数组
- `Cmp` 必须满足严格弱序，并且其调用运算符应能在 `const` 环境下调用
- 当前弹栈条件为新元素严格更优，等价元素不会弹出；结合 `better()` 的下标比较，可以保证相同最优值返回最左位置
- 该结构只支持“根据比较器选出最优元素”，不支持区间和、异或、矩阵乘法等一般结合运算
- `query()` 返回内部 `a` 中元素的引用；RMQ 对象销毁后该引用失效
- 块长固定为 `64`，因此块内单调栈状态可以由一个 `u64` 保存
- `small(l,r)` 只会在 `l`、`r-1` 属于同一个块时调用；不能单独拿它查询跨块区间
- `__builtin_ctzll()` 的参数不会为 `0`：因为 `r-1` 本身一定仍存在于 `stk[r-1]` 中，并且 `l ≤ r-1`
- 相比普通 ST，这个结构更适合需要大量 RMQ、尤其是还需要返回极值位置的场景；普通 ST 的代码更短，应按需求选择

原理简记

每个长度至多为 `64` 的块独立维护单调栈
对于位置 `r-1`，`stk[r-1]` 记录块前缀 `[block_l,r)` 的单调栈状态。删除所有位于 `l` 左侧的位置后：

```
stk[r - 1] >> (l % B)
```

最低的剩余 `1` 所代表的位置，就是 `[l,r)` 的最左最优位置，因此块内 RMQ 可以通过一次 `ctz` 完成
跨块查询拆成：

```
左侧不完整块 + 中间完整块 + 右侧不完整块
```

左右两部分由块内状态 $O(1)$ 查询，中间完整块由块级 ST $O(1)$ 查询

常见用法

区间最小值及其位置

```
RMQ<int> rmq(a);

int p{ rmq.query_index(l, r) };
int mn{ rmq.query(l, r) };
```

区间最大值

```
RMQ<int, greater<int>> rmq(a);

int p{ rmq.query_index(l, r) };
int mx{ rmq.query(l, r) };
```

相同最小值取最左位置

```
vector<int> a{ 3, 1, 2, 1 };

RMQ<int> rmq(a);
int p{ rmq.query_index(0, 4) }; // 1
```

结构体按字段选择最优元素

```
struct Info {
    i64 key{};
    int id{};
};

struct MinKey {
    bool operator()(const Info &a, const Info &b) const {
        return a.key < b.key;
    }
};

RMQ<Info, MinKey> rmq(a);

int p{ rmq.query_index(l, r) };
const Info &res{ rmq.query(l, r) };
```

这里 `Cmp` 只比较 `key`，因此 `key` 相等时由 RMQ 自动选择原数组中更靠左的位置
如果希望 `id` 本身参与优先级，则改为：

```
struct MinKey {
    bool operator()(const Info &a, const Info &b) const {
        return tie(a.key, a.id) < tie(b.key, b.id);
    }
};
```

复杂度

设块长 $B = 64$，块数

$$m = \left\lceil \frac{n}{B} \right\rceil$$

更严格地写：

操作	时间复杂度
块内状态预处理	$O(n)$
块级 ST 预处理	$O(m \log m)$
`query_index(l,r)`	$O(1)$
`query(l,r)`	$O(1)$

空间复杂度：

$$O(n + m \log m)$$

代入 $m = \lceil n/64 \rceil$：

$$O\left(n + \frac{n}{64} \log \frac{n}{64}\right)$$

在固定 64 位机器模型下，$\log m$ 有固定上界，因此该实现通常记作**线性预处理、线性空间、$O(1)$ 查询**
若一次 `Cmp` 比较的复杂度为 $O(k)$，则预处理和查询中的比较开销相应乘上 $O(k)$

3.5 基础 SegTree

3.5.1 区间加、区间和

```
struct SegAddSum {
    int n;
    vector<i64> tr, lz;
    explicit SegAddSum(int n_) : n(n_), tr(4 * n + 4), lz(4 * n + 4) {}
    explicit SegAddSum(const vector<i64> &a) : SegAddSum(int(a.size())) {
        if (n) build(1, 0, n, a);
    }
private:
    void pull(int p) { tr[p] = tr[p << 1] + tr[p << 1 | 1]; }
    void apply_node(int p, int l, int r, i64 v) {
        tr[p] += v * (r - l);
        lz[p] += v;
    }
    void push(int p, int l, int r) {
        if (lz[p] == 0 || r - l == 1) return;
        int m{(l + r) / 2};
        apply_node(p << 1, l, m, lz[p]);
        apply_node(p << 1 | 1, m, r, lz[p]);
        lz[p] = 0;
    }
    void build(int p, int l, int r, const vector<i64> &a) {
        if (r - l == 1) return void(tr[p] = a[l]);
        int m{(l + r) / 2};
        build(p << 1, l, m, a);
        build(p << 1 | 1, m, r, a);
        pull(p);
    }
    void set(int p, int l, int r, int x, i64 v) {
        if (r - l == 1) {
            tr[p] = v;
            lz[p] = 0;
            return;
        }
        push(p, l, r);
        int m{(l + r) / 2};
        if (x < m) set(p << 1, l, m, x, v);
        else set(p << 1 | 1, m, r, x, v);
        pull(p);
    }
    void add(int p, int l, int r, int ql, int qr, i64 v) {
        if (qr <= l || r <= ql) return;
        if (ql <= l && r <= qr) return apply_node(p, l, r, v);
        push(p, l, r);
        int m{(l + r) / 2};
        add(p << 1, l, m, ql, qr, v);
        add(p << 1 | 1, m, r, ql, qr, v);
        pull(p);
    }
    i64 query(int p, int l, int r, int ql, int qr) {
        if (qr <= l || r <= ql) return 0;
        if (ql <= l && r <= qr) return tr[p];
        push(p, l, r);
        int m{(l + r) / 2};
        return query(p << 1, l, m, ql, qr) + query(p << 1 | 1, m, r, ql, qr);
    }
public:
    void set(int x, i64 v) { set(1, 0, n, x, v); }
    void add(int l, int r, i64 v) { if (l < r) add(1, 0, n, l, r, v); }
    i64 query(int l, int r) { return l < r ? query(1, 0, n, l, r) : 0; }
    i64 all() const { return n ? tr[1] : 0; }
};
```

通用线段树

默认给出区间加、区间乘、区间赋值、区间和。标记统一表示仿射变换

$$x \mapsto mul \cdot x + add$$

因此：

- 区间加 v：`{1, v}`
- 区间乘 v：`{v, 0}`
- 区间赋值 v：`{0, v}`

```
struct Tag {
    i64 mul{1}, add{0};
    static Tag plus(i64 v) { return {1, v}; }
    static Tag times(i64 v) { return {v, 0}; }
    static Tag assign(i64 v) { return {0, v}; }
    bool empty() const { return mul == 1 && add == 0; }
    // 已有变换之后再执行 t
    void apply(const Tag &t) {
        mul *= t.mul;
        add = add * t.mul + t.add;
    }
};
struct Info {
    i64 sum{0};
    Info(i64 x = 0) : sum(x) {}
    void apply(const Tag &t, int l, int r) {
        sum = sum * t.mul + t.add * (r - l);
    }
};
Info operator+(const Info &a, const Info &b) { return Info(a.sum + b.sum); }

struct NoTag {};

template<class Info, class Tag = NoTag>
struct LazySegTree {
    static constexpr bool has_lazy = !is_same_v<Tag, NoTag>;
    int n;
    vector<Info> tr;
    vector<Tag> lz;
    LazySegTree(int n_, const Info &value = Info()) : LazySegTree(vector<Info>(n_, value)) {}
    template<class T>
    explicit LazySegTree(const vector<T> &a) : n(a.size()), tr(4 * n + 4), lz(has_lazy ? 4 * n + 4 : 0) {
        if (n) build(1, 0, n, a);
    }
private:
    template<class T>
    void build(int p, int l, int r, const vector<T> &a) {
        if (r - l == 1) return void(tr[p] = Info(a[l]));
        int mid{ l + r >> 1 };
        build(p << 1, l, mid, a);
        build(p << 1 | 1, mid, r, a);
        pull(p);
    }
    void pull(int p) { tr[p] = tr[p << 1] + tr[p << 1 | 1]; }
    void apply_node(int p, int l, int r, const Tag &tag) {
        if constexpr (has_lazy) {
            tr[p].apply(tag, l, r);
            lz[p].apply(tag);
        }
    }
    void push(int p, int l, int r) {
        if constexpr (has_lazy) {
            if (lz[p].empty() || r - l == 1) return;
            int mid{ l + r >> 1 };
            apply_node(p << 1, l, mid, lz[p]);
            apply_node(p << 1 | 1, mid, r, lz[p]);
            lz[p] = Tag();
        }
    }
    void set(int p, int l, int r, int x, const Info &value) {
```

```
        if (r - l == 1) {
            tr[p] = value;
            if constexpr (has_lazy) lz[p] = Tag();
            return;
        }
        push(p, l, r);
        int mid{ l + r >> 1 };
        if (x < mid) set(p << 1, l, mid, x, value);
        else set(p << 1 | 1, mid, r, x, value);
        pull(p);
    }
    void apply(int p, int l, int r, int ql, int qr, const Tag &tag) {
        if (qr <= l || r <= ql) return;
        if (ql <= l && r <= qr) return apply_node(p, l, r, tag);
        push(p, l, r);
        int mid{ l + r >> 1 };
        apply(p << 1, l, mid, ql, qr, tag);
        apply(p << 1 | 1, mid, r, ql, qr, tag);
        pull(p);
    }
    Info query(int p, int l, int r, int ql, int qr) {
        if (qr <= l || r <= ql) return Info();
        if (ql <= l && r <= qr) return tr[p];
        push(p, l, r);
        int mid{ l + r >> 1 };
        return query(p << 1, l, mid, ql, qr) + query(p << 1 | 1, mid, r, ql, qr);
    }
    template<class F>
    int find_first(int p, int l, int r, int ql, int qr, F &pred) {
        if (qr <= l || r <= ql) return -1;
        if (ql <= l && r <= qr && !pred(tr[p])) return -1;
        if (r - l == 1) return l;
        push(p, l, r);
        int mid{ l + r >> 1 };
        int res = find_first(p << 1, l, mid, ql, qr, pred);
        if (res == -1) res = find_first(p << 1 | 1, mid, r, ql, qr, pred);
        return res;
    }
    template<class F>
    int find_last(int p, int l, int r, int ql, int qr, F &pred) {
        if (qr <= l || r <= ql) return -1;
        if (ql <= l && r <= qr && !pred(tr[p])) return -1;
        if (r - l == 1) return l;
        push(p, l, r);
        int mid{ l + r >> 1 };
        int res = find_last(p << 1 | 1, mid, r, ql, qr, pred);
        if (res == -1) res = find_last(p << 1, l, mid, ql, qr, pred);
        return res;
    }
public:
    void set(int x, const Info &value) { set(1, 0, n, x, value); }
    template<class U = Tag>
    enable_if_t<!is_same_v<U, NoTag>> apply(int l, int r, const U &tag) {
        if (l < r) apply(1, 0, n, l, r, tag);
    }
    Info query(int l, int r) { return l < r ? query(1, 0, n, l, r) : Info(); }
    Info all() const { return n ? tr[1] : Info(); }
    template<class F>
    int find_first(int l, int r, F pred) { return l < r ? find_first(1, 0, n, l, r, pred) : -1; }
    template<class F>
    int find_last(int l, int r, F pred) { return l < r ? find_last(1, 0, n, l, r, pred) : -1; }
};
```

短版常见修改

如果题目仍然是区间加，通常无需抄泛型版，只修改 `SegAddSum` 的 `tr`、`pull()`、`apply_node()` 和 `query()` 即可

维护信息	`pull()`	区间加如何作用	能否直接修改短版
区间和	两儿子求和	`sum += v * len`	是
区间最大值	`max`	`mx += v`	是

维护信息	`pull()`	区间加如何作用	能否直接修改短版
区间最小值	`min`	`mn += v`	是
和 + 最大/最小	分别合并	`sum += v*len`，`mx/mn += v`	是
最大值及出现次数	最大值合并，相等次数相加	`mx += v`，次数不变	是
平方和	分别维护 $\sum a_i, \sum a_i^2$	用二项式展开	是
下标加权和 $\sum ia_i$	两儿子求和	加上 $v\sum i$	是
GCD	`gcd`	不能由原 GCD 推出	否
最大子段和	四元组区间合并	区间加后最优子段长度未知	否
AND / OR / XOR	对应位运算	普通加法会产生进位	否
大于某值的个数	次数合并	加法会改变值域分布	否

区间加、区间最大值

只改三处：

```
void pull(int p) { tr[p] = max(tr[p << 1], tr[p << 1 | 1]); }
void apply_node(int p, int l, int r, i64 v) {
    tr[p] += v;
    lz[p] += v;
}
i64 query(int p, int l, int r, int ql, int qr) {
    if (qr <= l || r <= ql) return -INF;
    if (ql <= l && r <= qr) return tr[p];
    push(p, l, r);
    int m{(l + r) / 2};
    return max(query(p << 1, l, m, ql, qr), query(p << 1 | 1, m, r, ql, qr));
}
```

区间最小值同理，把 `max` / `-INF` 改为 `min` / `INF`

区间加，同时维护和、最小值、最大值

把 `tr` 改成：

```
struct Node {
    i64 sum{0}, mn{INF}, mx{-INF};
    Node() = default;
    Node(i64 x) : sum(x), mn(x), mx(x) {}
    Node(i64 s, i64 lo, i64 hi) : sum(s), mn(lo), mx(hi) {}
};
vector<Node> tr;
```

修改：

```
Node merge(const Node &a, const Node &b) {
    return { a.sum + b.sum, min(a.mn, b.mn), max(a.mx, b.mx) };
}
void pull(int p) { tr[p] = merge(tr[p << 1], tr[p << 1 | 1]); }
void apply_node(int p, int l, int r, i64 v) {
    tr[p].sum += v * (r - l);
    tr[p].mn += v;
    tr[p].mx += v;
    lz[p] += v;
}
```

叶子改为：

```
tr[p] = Node(a[l]);
```

`query()` 返回 `Node`，区间外返回 `Node()`，合并改为 `merge()`

区间加、区间最大值及最大值出现次数

```
struct Node {
    i64 mx{-INF};
    int cnt{0};
```

```
    Node() = default;
    Node(i64 x) : mx(x), cnt(1) {}
    Node(i64 v, int c) : mx(v), cnt(c) {}
};
Node merge(const Node &a, const Node &b) {
    if (a.mx > b.mx) return a;
    if (a.mx < b.mx) return b;
    return {a.mx, a.cnt + b.cnt};
}
```

上面的 Node 已包含二参数构造函数，它对应以下写法（不要重复添加）：

```
Node(i64 mx_, int cnt_) : mx(mx_), cnt(cnt_) {}
```

此时：

```
void pull(int p) { tr[p] = merge(tr[p << 1], tr[p << 1 | 1]); }
void apply_node(int p, int l, int r, i64 v) {
    tr[p].mx += v;
    lz[p] += v;
}
```

区间加不会改变同一区间内各元素的大小关系，因此最大值出现次数不变

区间加、区间和与平方和

维护

$$S_1 = \sum a_i, \quad S_2 = \sum a_i^2$$

所有数加上 v 后：

$$S_2' = S_2 + 2vS_1 + nv^2$$

```
struct Node { i64 sum{0}, sq{0}; };
void pull(int p) {
    tr[p].sum = tr[p << 1].sum + tr[p << 1 | 1].sum;
    tr[p].sq = tr[p << 1].sq + tr[p << 1 | 1].sq;
}
void apply_node(int p, int l, int r, i64 v) {
    i64 len{r - l};
    tr[p].sq += 2 * v * tr[p].sum + v * v * len;
    tr[p].sum += v * len;
    lz[p] += v;
}
```

注意必须先更新 `sq`，再更新 `sum`
更高次幂和同理使用

$$(a+v)^k = \sum_{j=0}^{k} \binom{k}{j} a^j v^{k-j}$$

区间加、下标加权和

维护

$$S = \sum a_i, \qquad W = \sum i \cdot a_i$$

由于内部下标为 `0base`，区间 `[l,r)` 的下标和为

$$\sum_{i=l}^{r-1} i = \frac{(l+r-1)(r-l)}{2}$$

```
struct Node { i64 sum{0}, weighted{0}; };
void apply_node(int p, int l, int r, i64 v) {
    i64 len{r - l};
    i64 index_sum{i64(l + r - 1) * len / 2};
    tr[p].sum += v * len;
    tr[p].weighted += v * index_sum;
    lz[p] += v;
}
```

这也是通用版将 `Info::apply()` 传入 `l,r` 而不只传 `len` 的原因

不能直接套区间加懒标记的常见信息

区间 GCD

参考 3.5.2 非递归线段树（ZKW 风格） 中的 *RangeAddGCD*

最大子段和

参考 3.5.2 非递归线段树（ZKW 风格） 中的 *区间最大子段和*

AND / OR / XOR

普通“区间加”会发生二进制进位，因此仅保存区间 `AND/OR/XOR` 无法 $O(1)$ 推出加法后的结果
但如果修改本身也是 XOR，则是另一套懒标记。例如维护区间 XOR：

```
void apply_node(int p, int l, int r, i64 v) {
    if ((r - l) & 1) tr[p] ^= v;
    lz[p] ^= v;
}
```

因为偶数个相同的 `v` 异或后抵消

接口说明

- `LazySegTree(a)`：由能转换为 `Info` 的数组建树
- `LazySegTree(n,value)`：建立长度为 `n`、每个位置均为 `value` 的树
- `set(x,value)`：单点赋值
- `apply(l,r,tag)`：对 `[l,r)` 追加标记
- `query(l,r)`：查询 `[l,r)` 的聚合信息
- `all()`：查询整个数组的聚合信息
- `find_first(l,r,pred)`：从左向右寻找第一个可能满足 `pred` 的位置
- `find_last(l,r,pred)`：从右向左寻找第一个可能满足 `pred` 的位置

注意事项

- 全部下标为 `0base`，区间统一为 `[l,r)`
- `Info()` 必须是区间合并的单位元
- `Tag()` 必须表示空标记
- `operator+` 必须按左区间、右区间的顺序合并；最大子段和等信息不可交换顺序
- `Tag::apply(t)` 表示“已有标记之后再执行 `t`”，复合标记时顺序不能写反
- 默认仿射 `Tag` 只保证当前 `sum` 信息正确；若同时维护 `min/max`，特别是允许负数乘法时，需要重新推导 `Info::apply()`
- 若题目取模，应在 `Tag::apply()`、`Info::apply()` 和 `operator+` 中统一处理
- 先估算 `sum * mul`、`v * v * len` 等中间结果是否溢出；必要时使用 `i128` 或取模类
- `find_first/find_last` 中，`pred(info)==false` 必须保证该整段不存在答案，例如维护 `mx` 时寻找第一个 `a[i]≥x` 可使用 `info.mx≥x`
- 不要因为“线段树能维护某信息”就认为它也能支持任意懒修改；关键是必须能够仅根据当前节点信息和标记，在 $O(1)$ 内得到修改后的节点信息

常见用法

查询第一个不小于 `x` 的位置

需要让 `Info` 维护区间最大值 `mx`：

```
int p = seg.find_first(l, r, [&](const Info &info) {
    return info.mx ≥ x;
});
```

若不存在返回 `-1`

3.5.2 非递归线段树（ZKW 风格）

解决问题： 只需单点修改、区间聚合和树上二分时，使用紧凑的非递归线段树；相比递归线段树代码更短、常数更小

```
template<class T, class Merge>
struct IterSegTree {
    int n, base{1};
    T identity;
    Merge merge;
    vector<T> tr;
```

```
    IterSegTree(int n_, T identity_, Merge merge_ = Merge())
        : n(n_), identity(move(identity_)), merge(move(merge_)) {
        while (base < n) base <<= 1;
        tr.assign(base << 1, identity);
    }
    IterSegTree(const vector<T> &a, T identity_, Merge merge_ = Merge())
        : IterSegTree(int(a.size()), move(identity_), move(merge_)) {
        for (int i = 0; i < n; ++ i) tr[base + i] = a[i];
        for (int p = base - 1; p; -- p) pull(p);
    }
    void pull(int p) { tr[p] = merge(tr[p << 1], tr[p << 1 | 1]); }
    void set(int x, const T &v) {
        assert(0 <= x && x < n);
        int p{base + x};
        tr[p] = v;
        for (p >>= 1; p; p >>= 1) pull(p);
    }
    const T &get(int x) const { assert(0 <= x && x < n); return tr[base + x]; }
    T query(int l, int r) const {
        assert(0 <= l && l <= r && r <= n);
        T left{identity}, right{identity};
        for (l += base, r += base; l < r; l >>= 1, r >>= 1) {
            if (l & 1) left = merge(left, tr[l ++]);
            if (r & 1) right = merge(tr[-- r], right);
        }
        return merge(left, right);
    }
    const T &all() const { return tr[1]; }
    // ===== 树上二分：若不需要，可直接删除 max_right() / min_left() =====
    // check(identity) 必须为 true，且随着区间扩展只能从 true 变为 false。
    template<class F>
    int max_right(int l, F check) const {
        assert(0 <= l && l <= n && check(identity));
        if (l == n) return n;
        int p{l + base};
        T cur{identity};
        do {
            while (!(p & 1)) p >>= 1;
            T nxt{merge(cur, tr[p])};
            if (!check(nxt)) {
                while (p < base) {
                    p <<= 1;
                    nxt = merge(cur, tr[p]);
                    if (check(nxt)) {
                        cur = nxt;
                        ++ p;
                    }
                }
                return min(n, p - base);
            }
            cur = nxt;
            ++ p;
        } while ((p & -p) != p);
        return n;
    }
    template<class F>
    int min_left(int r, F check) const {
        assert(0 <= r && r <= n && check(identity));
        if (r == 0) return 0;
        int p{r + base};
        T cur{identity};
        do {
            -- p;
            while (p > 1 && (p & 1)) p >>= 1;
            T nxt{merge(tr[p], cur)};
            if (!check(nxt)) {
                while (p < base) {
                    p = p << 1 | 1;
                    nxt = merge(tr[p], cur);
                    if (check(nxt)) {
                        cur = nxt;
```

```
                    -- p;
                }
            }
            return p + 1 - base;
        }
        cur = nxt;
    } while ((p & -p) != p);
    return 0;
  }
};
```

接口说明

- `IterSegTree(n, identity, merge)`：建立长度为 `n`、所有位置均为单位元的线段树，$O(n)$
- `IterSegTree(a, identity, merge)`：由数组建树，$O(n)$
- `set(x,v)`：令 `a[x]=v`，$O(\log n)$
- `get(x)`：返回 `a[x]`，$O(1)$
- `query(l,r)`：按原顺序聚合 `[l,r)`，$O(\log n)$
- `all()`：返回整个 `[0,n)` 的聚合信息，$O(1)$
- `max_right(l,check)`：返回最大的 `r`，使 `check(query(l,r))` 成立，$O(\log n)$
- `min_left(r,check)`：返回最小的 `l`，使 `check(query(l,r))` 成立，$O(\log n)$

注意事项

- 下标为 `0base`，区间统一使用 `[l,r)`
- `merge` 必须满足结合律，`identity` 必须同时满足 `merge(identity,x)=merge(x,identity)=x`
- `merge` 不要求满足交换律；`query()`、`max_right()`、`min_left()` 都保持原数组顺序，因此可以维护矩阵乘积、字符串连接等信息
- `[n,base)` 的补充叶子全部为 `identity`，不会影响实际区间答案
- `max_right/min_left` 要求 `check(identity)=true`
- 树上二分还要求判定具有单调性：区间不断扩展时 `check` 只能从 `true` 变成 `false`
- `max_right(l,check)` 返回的是**最后仍满足条件的右边界**；若加入位置 `p` 后第一次失败，则返回 `p`
- `min_left(r,check)` 对称地返回**最靠左仍满足条件的左边界**
- 若需要区间修改，优先使用懒标记线段树，不建议为了非递归而强行加入传统 ZKW 开区间懒标记技巧

常见用法

区间和

```
struct Add {
    i64 operator()(i64 x, i64 y) const { return x + y; }
};
IterSegTree<i64, Add> seg(a, 0LL);
seg.set(x, v);
i64 sum = seg.query(l, r);
```

区间最大值

```
struct Max {
    i64 operator()(i64 x, i64 y) const { return max(x, y); }
};
IterSegTree<i64, Max> seg(a, i64(-4E18));
i64 mx = seg.query(l, r);
```

区间最大子段和

```
struct MaxSubarrayInfo {
    static constexpr i64 NEG_INF = -4'000'000'000'000'000'000LL;
    i64 sum{0};
    i64 pre{NEG_INF}, suf{NEG_INF}, best{NEG_INF};
    MaxSubarrayInfo() = default;
    MaxSubarrayInfo(i64 x) : sum(x), pre(x), suf(x), best(x) {}
    MaxSubarrayInfo(i64 sum_, i64 pre_, i64 suf_, i64 best_)
        : sum(sum_), pre(pre_), suf(suf_), best(best_) {}
};
struct MergeMaxSubarray {
    MaxSubarrayInfo operator()(const MaxSubarrayInfo &a, const MaxSubarrayInfo &b) const {
        if (a.best == MaxSubarrayInfo::NEG_INF) return b;
        if (b.best == MaxSubarrayInfo::NEG_INF) return a;
        return { a.sum + b.sum, max(a.pre, a.sum + b.pre),
```

```
            max(b.suf, b.sum + a.suf), max({a.best, b.best, a.suf + b.pre})
        };
    }
};
```

> 有负数时怎么找第一个前缀和不小于 `x`？

这个 `Info` 里的：`pre` 恰好就是答案
因为对于 `[l,r)`：`info.pre`
表示：

$$\max_{l\le k<r}\sum_{i=l}^{k} a_i$$

随着 `r` 右移，`pre` 只可能不减，所以可以直接：

```
int p = seg.max_right(l, [&](const MaxSubarrayInfo &info) {
    return info.pre < x;
});
```

那么 `p` 就是第一个满足

$$a_l + a_{l+1} + \cdots + a_p \ge x$$

的位置
哪怕数组中存在负数也成立

区间加、区间 GCD

设

$$d_i = a_i - a_{i-1}$$

则：

$$\gcd(a_l, a_{l+1}, \ldots, a_{r-1}) = \gcd\left(a_l, d_{l+1}, \ldots, d_{r-1}\right)$$

而 `[l,r)` 全部加 `v` 只改变：

$$d_l \mathrel{+}= v, \qquad d_r \mathrel{-}= v$$

```
struct Gcd {
    i64 operator()(i64 x, i64 y) const { return gcd(x, y); }
};
```

然后 `IterSegTree<i64, Gcd> seg(diff, 0LL);` 即可维护出差分数组
完整实现：

```
struct Gcd {
    i64 operator()(i64 x, i64 y) const { return gcd(x, y); }
};
struct RangeAddGCD {
    int n;
    BIT<i64> bit;
    vector<i64> diff;
    IterSegTree<i64, Gcd> seg;
    static vector<i64> make_diff(const vector<i64> &a) {
        int n{int(a.size())};
        vector<i64> d(n + 1);
        for (int i = 0; i < n; ++ i) {
            d[i] = a[i] - (i ? a[i - 1] : 0);
        }
        if (n) d[n] = -a[n - 1];
        return d;
    }
    explicit RangeAddGCD(const vector<i64> &a)
        : n(a.size()), bit(), diff(make_diff(a)), seg(diff, 0LL) {
        bit = BIT<i64>(diff);
    }
    void add_diff(int x, i64 v) {
        diff[x] += v;
        bit.add(x, v);
        seg.set(x, diff[x]);
```

```
    }
    void add(int l, int r, i64 v) { add_diff(l, v); add_diff(r, -v); }
    i64 get(int x) const { return bit.sum(x + 1); }
    i64 query(int l, int r) const { assert(l < r); return gcd(abs(get(l)), seg.query(l + 1, r)); }
};
```

找 [l,n) 中第一个 a[i] ≥ x

维护区间最大值。对于某段：

```
mx < x
```

表示这一整段都没有答案

```
int p = seg.max_right(l, [&](i64 mx) {
    return mx < x;
});
if (p == n) {
    // 不存在
} else {
    // p 为第一个满足 a[p] ≥ x 的位置
}
```

同理，找 [l,r) 中第一个 a[i]≥x：

```
int p = seg.max_right(l, [&](i64 mx) {
    return mx < x;
});
if (p ≥ r) {
    // [l,r) 中不存在
}
```

找 [0,n) 中第一个前缀和不小于 x

要求数组元素均非负，使前缀和单调不减
维护区间和，并令：

```
sum < x
```

表示当前前缀还没有达到目标

```
int p;
if (x ≤ 0) {
    p = 0;
} else {
    p = seg.max_right(0, [&](i64 sum) {
        return sum < x;
    });
}
if (p == n) {
    // 总和仍小于 x，不存在
} else {
    // p 为最小位置，使 a[0] + ... + a[p] ≥ x
}
```

例如：

```
a = [2, 4, 1, 5]
前缀和 = [2, 6, 7, 12]
```

当 x=7 时：

```
seg.max_right(0, [&](i64 sum) {
    return sum < 7;
});
```

返回 2
如果数组允许负数， sum < x 不再具有单调性，不能这样二分；此时需要维护最大前缀和等额外信息

找 `[0,r)` 中最后一个 `a[i] ≥ x`

维护区间最大值，从右向左使用 `min_left()`：

```
int p = seg.min_left(r, [&](i64 mx) {
    return mx < x;
}) - 1;
if (p == -1) {
    // [0,r) 中不存在
} else {
    // p 为最后一个满足 a[p] ≥ x 的位置
}
```

自定义区间信息

只需令 `T` 表示整个区间的信息，并让 `Merge` 表示左右区间合并

```
struct Info {
    i64 sum{};
    int len{};
};
struct Merge {
    Info operator()(const Info &a, const Info &b) const {
        return {a.sum + b.sum, a.len + b.len};
    }
};
vector<Info> init(n);
for (int i = 0; i < n; ++ i) init[i] = {a[i], 1};
IterSegTree<Info, Merge> seg(init, Info{});
Info info = seg.query(l, r);
```

常见单位元：

聚合信息	`merge(a,b)`	`identity`
区间和	`a + b`	`0`
区间最大值	`max(a,b)`	`-INF`
区间最小值	`min(a,b)`	`INF`
GCD	`gcd(a,b)`	`0`
XOR	`a ^ b`	`0`
矩阵乘积	`a * b`	单位矩阵

复杂度

操作	时间复杂度
建树	$O(n)$
`set()` / `query()` / `max_right()` / `min_left()`	$O(\log n)$
`get()` / `all()`	$O(1)$

空间复杂度：$O(n)$
`RangeAddGCD` 从差分数组直接线性构建 BIT，初始化总时间 O(n)，区间修改和查询仍为 O(log n)。

3.5.3 函数复合线段树

解决问题： 每个位置表示一个函数或状态转移，支持单点修改，并查询区间内按照下标顺序依次执行这些函数后的复合结果
依赖：

- 3.5.2 非递归线段树（ZKW 风格） 中的 `IterSegTree<T, Merge>`
- `IterSegTree` 必须保持左右区间的合并顺序，即 `merge(left, right)` 表示**左区间后接右区间**

原理

设每个位置维护一个函数：

$$f_0, f_1, \ldots, f_{n-1}$$

查询 `[l,r)` 时，从左到右依次执行：

$$x \xrightarrow{f_l} f_l(x) \xrightarrow{f_{l+1}} f_{l+1}(f_l(x)) \to \cdots$$

因此整个区间对应的复合函数为：

$$f_{r-1} \circ f_{r-2} \circ \cdots \circ f_l$$

函数复合满足结合律：

$$(h \circ g) \circ f = h \circ (g \circ f)$$

因此可以直接作为普通线段树的区间信息
若：

```
left  = 左区间对应的复合函数
right = 右区间对应的复合函数
```

那么两个相邻区间合并后，应先执行 `left`，再执行 `right`：

```
merge(left, right) = right ∘ left
```

函数复合通常不满足交换律，因此左右顺序不能颠倒

仿射函数

最常见的情况是每个位置维护仿射函数：

$$f(x) = ax + b$$

```
struct Affine {
    i64 a{1}, b{0};
    i64 operator()(i64 x) const {
        return a * x + b;
    }
};
struct ComposeAffine {
    Affine operator()(const Affine &left, const Affine &right) const {
        // 返回 right(left(x))
        return { right.a * left.a, right.a * left.b + right.b };
    }
};
```

设：

$$f(x) = a_1 x + b_1$$

$$g(x) = a_2 x + b_2$$

则：

$$g(f(x)) = a_2(a_1 x + b_1) + b_2 = (a_2 a_1)x + (a_2 b_1 + b_2)$$

所以：

```
compose(f, g) = { g.a * f.a, g.a * f.b + g.b };
```

默认的：

```
Affine{}
```

表示恒等函数：

$$f(x) = x$$

因此正好是函数复合的单位元

使用方法

建树

```
vector<Affine> f(n);
for (int i = 0; i < n; ++ i) {
    f[i] = {a[i], b[i]};
```

```
}
IterSegTree<Affine, ComposeAffine> seg(f, Affine{});
```

查询区间复合函数

```
Affine g = seg.query(l, r);
```

此时：

$$g = f_{r-1} \circ f_{r-2} \circ \cdots \circ f_l$$

若要计算初始值 `x` 经过整个区间后的结果：

```
i64 ans = seg.query(l, r)(x);
```

单点修改

将位置 `p` 修改为：

$$f_p(x) = ax + b$$

```
seg.set(p, Affine{a, b});
```

模意义下的仿射函数

实际竞赛中经常维护：

$$f(x) = ax + b \pmod{M}$$

若已有取模类型 `Z`，直接将 `i64` 改成 `Z`：

```
struct Affine {
    Z a{1}, b{0};
    Z operator()(Z x) const { return a * x + b; }
};
struct ComposeAffine {
    Affine operator()(const Affine &left, const Affine &right) const {
        return { right.a * left.a, right.a * left.b + right.b };
    }
};
```

其余线段树代码与调用方式不变

如何推广

仿射函数只是最简单的一类可复合信息
如果每个位置表示某种状态转移，并满足：

1. 单个转移能用固定大小的信息表示
2. 两段连续区间的转移能够快速复合
3. 复合满足结合律
4. 存在“不执行任何操作”的单位转移
   那么就可以直接把这种转移作为 `IterSegTree` 的 `T`
   常见形式包括：

- 仿射变换
- 矩阵变换
- 置换复合
- 固定状态数的 DP 转移
- 自动机状态转移
- 其他对复合封闭的小型函数
  例如状态转移可以表示为：

$$v' = Mv$$

则每个位置存一个矩阵，区间合并维护矩阵乘积即可
核心转换方式为：

```
单点函数 / 状态转移
        ↓
压缩为固定大小的信息 T
```

```
        ↓
定义相邻区间的复合 Merge
        ↓
使用 IterSegTree<T, Merge>
```

注意事项

- `merge(left, right)` 表示左区间后接右区间，因此函数复合应返回 `right(left(x))`
- 函数复合满足结合律，但通常不满足交换律，左右区间不能交换
- `IterSegTree::query()` 必须保持原区间顺序，不能使用只适用于交换运算的简化写法
- 单位元必须表示恒等函数，即对任意 `x` 有 $id(x) = x$
- 函数族必须对复合封闭；若两函数复合后无法继续用相同的固定大小信息表示，就不能直接套本方法
- 若一次 `Merge` 的复杂度为 $O(C)$，线段树操作复杂度也需要额外乘上 C
- 模意义下优先直接使用取模类型保存参数，避免在复合过程中漏取模
- 若题目维护的是"若干函数在某点的最优值"，而不是按顺序执行函数复合，通常不属于本方法

复杂度

设一次函数复合 `Merge` 的复杂度为 $O(C)$

操作	时间复杂度
建树	$O(Cn)$
单点修改 / 区间函数复合	$O(C \log n)$
将结果函数作用于一个值	取决于函数表示，仿射函数为 $O(1)$

空间复杂度：$O(n)$

3.5.4 势能线段树与非线性区间修改

解决问题： 区间修改无法像普通 Lazy 一样直接作用于节点，但修改会使元素快速进入稳定状态，或在某些节点上能退化成统一的简单修改时，利用剪枝与势能分析获得均摊复杂度。

基本思想

普通懒标记要求区间修改能够根据节点信息直接在 $O(1)$ 内更新节点
对于：

$$a_i \leftarrow f(a_i)$$

如果无法直接更新，可以尝试：

```
当前节点完全不会变化    → 直接 return
当前节点能整段等价处理  → 直接修改节点并 return
否则                    → 递归左右儿子
```

只要能够证明每次真正向下递归都会显著降低某种势能，就不需要保证每次修改严格 $O(\log n)$，仍然可以获得较好的总复杂度
典型操作：

- 区间开方
- 区间取模
- 区间整除
- 其他会快速收缩值域的非线性修改

区间开方、区间和

对于非负整数：

$$a_i \leftarrow \lfloor \sqrt{a_i} \rfloor$$

当区间最大值满足：

$$mx \leq 1$$

整段已经稳定，以后继续开方都不会发生变化

```
struct SqrtSegTree {
    struct Node { i64 sum{0}, mx{0}; };
    int n;
```

```
    vector<Node> tr;
    explicit SqrtSegTree(const vector<i64> &a) : n(a.size()), tr(4 * n) {
        assert(n > 0);
        build(1, 0, n, a);
    }
    static i64 floor_sqrt(i64 x) {
        i64 y{static_cast<i64>(sqrtl(static_cast<long double>(x)))};
        while ((__int128_t)(y + 1) * (y + 1) <= x) ++ y;
        while ((__int128_t)y * y > x) -- y;
        return y;
    }
    void pull(int p) {
        tr[p].sum = tr[p << 1].sum + tr[p << 1 | 1].sum;
        tr[p].mx = max(tr[p << 1].mx, tr[p << 1 | 1].mx);
    }
    void build(int p, int l, int r, const vector<i64> &a) {
        if (r - l == 1) {
            tr[p] = {a[l], a[l]};
            return;
        }
        int m{(l + r) / 2};
        build(p << 1, l, m, a);
        build(p << 1 | 1, m, r, a);
        pull(p);
    }
    void apply_sqrt(int p, int l, int r, int ql, int qr) {
        if (qr <= l || r <= ql || tr[p].mx <= 1) return;
        if (r - l == 1) {
            tr[p].sum = tr[p].mx = floor_sqrt(tr[p].mx);
            return;
        }
        int m{(l + r) / 2};
        apply_sqrt(p << 1, l, m, ql, qr);
        apply_sqrt(p << 1 | 1, m, r, ql, qr);
        pull(p);
    }
    i64 query(int p, int l, int r, int ql, int qr) const {
        if (qr <= l || r <= ql) return 0;
        if (ql <= l && r <= qr) return tr[p].sum;
        int m{(l + r) / 2};
        return query(p << 1, l, m, ql, qr) + query(p << 1 | 1, m, r, ql, qr);
    }
    void apply_sqrt(int l, int r) { assert(0 <= l && l <= r && r <= n); apply_sqrt(1, 0, n, l, r); }
    i64 query(int l, int r) const { assert(0 <= l && l <= r && r <= n); return query(1, 0, n, l, r); }
};
```

非负整数连续开方后会很快进入 `0/1`，单个元素只有 $O(\log \log A)$ 次有效变化

区间取模、区间和

修改：

$$a_i \leftarrow a_i \bmod v$$

若节点：

$$mx < v$$

则整个节点都不会变化：

```
if (tr[p].mx < v) return;
```

若取模真正改变一个非负整数 x，则：

$$x \bmod v < \frac{x}{2}$$

因此一个元素在没有重新赋大值时，最多发生 $O(\log A)$ 次有效取模

```
struct ModSegTree {
    struct Node { i64 sum{0}, mx{0}; };
```

```
    int n;
    vector<Node> tr;
    explicit ModSegTree(const vector<i64> &a) : n(a.size()), tr(4 * n) {
        assert(n > 0);
        build(1, 0, n, a);
    }
    void pull(int p) {
        tr[p].sum = tr[p << 1].sum + tr[p << 1 | 1].sum;
        tr[p].mx = max(tr[p << 1].mx, tr[p << 1 | 1].mx);
    }
    void build(int p, int l, int r, const vector<i64> &a) {
        if (r - l == 1) {
            tr[p] = {a[l], a[l]};
            return;
        }
        int m{(l + r) / 2};
        build(p << 1, l, m, a);
        build(p << 1 | 1, m, r, a);
        pull(p);
    }
    void mod(int p, int l, int r, int ql, int qr, i64 v) {
        if (qr <= l || r <= ql || tr[p].mx < v) return;
        if (r - l == 1) {
            tr[p].sum %= v;
            tr[p].mx = tr[p].sum;
            return;
        }
        int m{(l + r) / 2};
        mod(p << 1, l, m, ql, qr, v);
        mod(p << 1 | 1, m, r, ql, qr, v);
        pull(p);
    }
    void set(int p, int l, int r, int x, i64 v) {
        if (r - l == 1) {
            tr[p] = {v, v};
            return;
        }
        int m{(l + r) / 2};
        if (x < m) set(p << 1, l, m, x, v);
        else set(p << 1 | 1, m, r, x, v);
        pull(p);
    }
    i64 query(int p, int l, int r, int ql, int qr) const {
        if (qr <= l || r <= ql) return 0;
        if (ql <= l && r <= qr) return tr[p].sum;
        int m{(l + r) / 2};
        return query(p << 1, l, m, ql, qr) + query(p << 1 | 1, m, r, ql, qr);
    }
    void mod(int l, int r, i64 v) { assert(0 <= l && l <= r && r <= n && v > 0); mod(1, 0, n, l, r, v); }
    void set(int x, i64 v) { assert(0 <= x && x < n && v >= 0); set(1, 0, n, x, v); }
    i64 query(int l, int r) const { assert(0 <= l && l <= r && r <= n); return query(1, 0, n, l, r); }
};
```

`set()` 会重新提高某个位置的势能，但一次重新赋值最多重新产生 $O(\log A)$ 次有效取模机会

非线性修改退化为统一增量

对于：

$$a_i \leftarrow f(a_i)$$

定义：

$$d(x) = f(x) - x$$

如果能够证明当前节点内所有元素的 $d(x)$ 相同，那么整个非线性修改等价于：

$$a_i \leftarrow a_i + d$$

因此可以直接使用普通区间加懒标记
如果 $d(x)$ 在当前值域上单调，只需要检查：

```
i64 dl = f(mn) - mn;
i64 dr = f(mx) - mx;
if (dl == dr) {
    apply_add(p, dl, r - l);
    return;
}
```

因为此时 `[mn,mx]` 中所有值的变化量都相同

开方

$$d(x) = \lfloor \sqrt{x} \rfloor - x$$

可以使用：

```
i64 dl = floor_sqrt(mn) - mn;
i64 dr = floor_sqrt(mx) - mx;
```

整除

对于非负整数和 $v \geq 2$：

$$d(x) = \left\lfloor \frac{x}{v} \right\rfloor - x$$

可以使用：

```
i64 dl = mn / v - mn;
i64 dr = mx / v - mx;
```

区间加、区间整除

这是“统一变化量”技巧的典型高级应用
支持：

- 区间加
- 区间向下整除
- 区间和、最小值、最大值
  要求所有元素始终非负，除数满足 $v \geq 2$

```
struct DivSegTree {
    static constexpr i64 INF = 4'000'000'000'000'000'000LL;
    struct Node { i64 sum{0}, mn{INF}, mx{-INF}, add{0}; };
    int n;
    vector<Node> tr;
    explicit DivSegTree(const vector<i64> &a) : n(a.size()), tr(4 * n) {
        assert(n > 0);
        build(1, 0, n, a);
    }
    void pull(int p) {
        tr[p].sum = tr[p << 1].sum + tr[p << 1 | 1].sum;
        tr[p].mn = min(tr[p << 1].mn, tr[p << 1 | 1].mn);
        tr[p].mx = max(tr[p << 1].mx, tr[p << 1 | 1].mx);
    }
    void apply_add(int p, i64 v, int len) {
        tr[p].sum += v * len;
        tr[p].mn += v;
        tr[p].mx += v;
        tr[p].add += v;
    }
    void push(int p, int l, int r) {
        if (tr[p].add == 0 || r - l == 1) return;
        int m{(l + r) / 2};
        apply_add(p << 1, tr[p].add, m - l);
        apply_add(p << 1 | 1, tr[p].add, r - m);
        tr[p].add = 0;
    }
    void build(int p, int l, int r, const vector<i64> &a) {
        if (r - l == 1) {
            tr[p] = {a[l], a[l], a[l], 0};
```

```
            return;
        }
        int m{(l + r) / 2};
        build(p << 1, l, m, a);
        build(p << 1 | 1, m, r, a);
        pull(p);
    }
    void add(int p, int l, int r, int ql, int qr, i64 v) {
        if (qr <= l || r <= ql) return;
        if (ql <= l && r <= qr) {
            apply_add(p, v, r - l);
            return;
        }
        push(p, l, r);
        int m{(l + r) / 2};
        add(p << 1, l, m, ql, qr, v);
        add(p << 1 | 1, m, r, ql, qr, v);
        pull(p);
    }
    void divide(int p, int l, int r, int ql, int qr, i64 v) {
        if (qr <= l || r <= ql || tr[p].mx == 0) return;
        if (ql <= l && r <= qr) {
            i64 dl{tr[p].mn / v - tr[p].mn}, dr{tr[p].mx / v - tr[p].mx};
            if (dl == dr) {
                apply_add(p, dl, r - l);
                return;
            }
        }
        push(p, l, r);
        int m{(l + r) / 2};
        divide(p << 1, l, m, ql, qr, v);
        divide(p << 1 | 1, m, r, ql, qr, v);
        pull(p);
    }
    Node query(int p, int l, int r, int ql, int qr) {
        if (qr <= l || r <= ql) return {};
        if (ql <= l && r <= qr) return tr[p];
        push(p, l, r);
        int m{(l + r) / 2};
        Node a{query(p << 1, l, m, ql, qr)};
        Node b{query(p << 1 | 1, m, r, ql, qr)};
        return { a.sum + b.sum, min(a.mn, b.mn), max(a.mx, b.mx), 0 };
    }
    void add(int l, int r, i64 v) { assert(0 <= l && l <= r && r <= n); add(1, 0, n, l, r, v); }
    void divide(int l, int r, i64 v) { assert(0 <= l && l <= r && r <= n && v >= 2); divide(1, 0, n, l, r, v); }
    Node query(int l, int r) { assert(0 <= l && l < r && r <= n); return query(1, 0, n, l, r); }
};
```

接口说明

- `SqrtSegTree::apply_sqrt(l,r)`：逐点执行 $a_i \leftarrow \lfloor \sqrt{a_i} \rfloor$
- `SqrtSegTree::query(l,r)`：区间和
- `ModSegTree::mod(l,r,v)`：逐点执行 `a[i] %= v`
- `ModSegTree::set(x,v)`：单点赋值
- `ModSegTree::query(l,r)`：区间和
- `DivSegTree::add(l,r,v)`：区间加
- `DivSegTree::divide(l,r,v)`：逐点执行 `a[i] /= v`
- `DivSegTree::query(l,r)`：返回区间 `sum/mn/mx`

注意事项

- 三类技巧都依赖具体修改的数学性质，不能把“递归到不能递归为止”当成通用均摊保证
- `sqrt/mod/divide` 这里均按非负整数处理
- `ModSegTree` 要求 `v > 0`
- `DivSegTree` 要求 `v ≥ 2`，并要求所有区间加之后数组仍保持非负
- C++ 负数整数除法是向零截断，不等于数学意义上的向下取整
- `floor_sqrt()` 使用浮点数初估后再用 `__int128_t` 校正，避免大整数舍入误差
- “稳定节点剪枝”和“统一变化量整段处理”可以同时使用，后者属于进一步优化
- 所有 `sum`、`v * len` 等运算仍需保证不会超出 `i64`

复杂度

区间查询为 $O(\log n)$
区间开方中，单个非负整数最多经历 $O(\log\log A)$ 次有效变化，因此总复杂度可按：

$$O((n\log\log A+q)\log n)$$

估计
区间取模中，每次真正修改都会使元素至少减半；包含单点重新赋值时，常用宽松总复杂度为：

$$O((n+q)\log n\log A)$$

`DivSegTree` 单次最坏可能访问大量节点；它依赖统一变化量剪枝与整除造成的值域收缩获得均摊效率，不应视为普通 Lazy 的严格 $O(\log n)$ 修改

3.5.5 Segment Tree Beats

解决问题： 维护区间 `chmin/chmax/add` 与区间和、最小值、最大值，通过记录严格次大值、严格次小值和极值出现次数，使部分非线性修改能够直接停在内部节点

原理

考虑修改：

$$a_i\leftarrow\min(a_i,x)$$

若节点最大值为 `mx1`，严格次大值为 `mx2`
当：

$$mx_1\le x$$

时整段都不会变化
当：

$$mx_2<x<mx_1$$

时，只有所有等于 `mx1` 的元素会被修改为 `x`
若最大值出现 `mxc` 次，则：

$$sum\leftarrow sum+(x-mx_1)\cdot mxc$$

因此无需递归到儿子
只有：

$$x\le mx_2$$

时，受影响的值不止最大值这一层，才继续向下递归
`chmax` 完全对称，因此需要同时维护：

```
mx1 / mx2 / mxc    最大值 / 严格次大值 / 最大值次数
mn1 / mn2 / mnc    最小值 / 严格次小值 / 最小值次数
```

```
struct SegTreeBeats {
    static constexpr i64 INF = 4'000'000'000'000'000'000LL;
    struct Node {
        i64 sum{0}, mx1{-INF}, mx2{-INF};
        i64 mn1{INF}, mn2{INF}, add{0};
        int mxc{0}, mnc{0};
    };
    int n;
    vector<Node> tr;
    explicit SegTreeBeats(const vector<i64> &a) : n(a.size()), tr(4 * n) {
        assert(n > 0);
        build(1, 0, n, a);
    }
    Node merge_node(const Node &a, const Node &b) const {
        Node c;
        c.sum = a.sum + b.sum;
        c.mx1 = max(a.mx1, b.mx1);
        c.mxc = (a.mx1 == c.mx1 ? a.mxc : 0) + (b.mx1 == c.mx1 ? b.mxc : 0);
        c.mx2 = max(a.mx1 == c.mx1 ? a.mx2 : a.mx1, b.mx1 == c.mx1 ? b.mx2 : b.mx1);
        c.mn1 = min(a.mn1, b.mn1);
        c.mnc = (a.mn1 == c.mn1 ? a.mnc : 0) + (b.mn1 == c.mn1 ? b.mnc : 0);
        c.mn2 = min(a.mn1 == c.mn1 ? a.mn2 : a.mn1, b.mn1 == c.mn1 ? b.mn2 : b.mn1);
        return c;
```

```
}
void pull(int p) { tr[p] = merge_node(tr[p << 1], tr[p << 1 | 1]); }
void apply_add(int p, i64 v, int len) {
    Node &x{tr[p]};
    x.sum += v * len;
    x.mx1 += v;
    if (x.mx2 != -INF) x.mx2 += v;
    x.mn1 += v;
    if (x.mn2 != INF) x.mn2 += v;
    x.add += v;
}
// ===== chmin: 若完全不需要区间取 min，可删除本函数及 range_chmin() =====
void apply_chmin(int p, i64 v) {
    Node &x{tr[p]};
    if (x.mx1 <= v) return;
    x.sum += (v - x.mx1) * x.mxc;
    if (x.mn1 == x.mx1) x.mn1 = v;
    else if (x.mn2 == x.mx1) x.mn2 = v;
    x.mx1 = v;
}
// ===== chmax: 若完全不需要区间取 max，可删除本函数及 range_chmax() =====
void apply_chmax(int p, i64 v) {
    Node &x{tr[p]};
    if (x.mn1 >= v) return;
    x.sum += (v - x.mn1) * x.mnc;
    if (x.mx1 == x.mn1) x.mx1 = v;
    else if (x.mx2 == x.mn1) x.mx2 = v;
    x.mn1 = v;
}
void push(int p, int l, int r) {
    if (r - l == 1) return;
    int m{(l + r) / 2};
    // 必须先下传区间加。
    if (tr[p].add != 0) {
        apply_add(p << 1, tr[p].add, m - l);
        apply_add(p << 1 | 1, tr[p].add, r - m);
        tr[p].add = 0;
    }
    // 再让儿子的极值满足父节点当前的上下界。
    for (int c : {p << 1, p << 1 | 1}) {
        if (tr[c].mx1 > tr[p].mx1) apply_chmin(c, tr[p].mx1);
        if (tr[c].mn1 < tr[p].mn1) apply_chmax(c, tr[p].mn1);
    }
}
void build(int p, int l, int r, const vector<i64> &a) {
    if (r - l == 1) {
        tr[p].sum = tr[p].mx1 = tr[p].mn1 = a[l];
        tr[p].mxc = tr[p].mnc = 1;
        return;
    }
    int m{(l + r) / 2};
    build(p << 1, l, m, a);
    build(p << 1 | 1, m, r, a);
    pull(p);
}
// ===== 区间加：若不需要，可删除 range_add() / add() / Node::add =====
void range_add(int p, int l, int r, int ql, int qr, i64 v) {
    if (qr <= l || r <= ql) return;
    if (ql <= l && r <= qr) {
        apply_add(p, v, r - l);
        return;
    }
    push(p, l, r);
    int m{(l + r) / 2};
    range_add(p << 1, l, m, ql, qr, v);
    range_add(p << 1 | 1, m, r, ql, qr, v);
    pull(p);
}
// ===== 区间 chmin =====
void range_chmin(int p, int l, int r, int ql, int qr, i64 v) {
    if (qr <= l || r <= ql || tr[p].mx1 <= v) return;
```

```
        if (ql <= l && r <= qr && tr[p].mx2 < v) {
            apply_chmin(p, v);
            return;
        }
        push(p, l, r);
        int m{(l + r) / 2};
        range_chmin(p << 1, l, m, ql, qr, v);
        range_chmin(p << 1 | 1, m, r, ql, qr, v);
        pull(p);
    }
    // ===== 区间 chmax =====
    void range_chmax(int p, int l, int r, int ql, int qr, i64 v) {
        if (qr <= l || r <= ql || tr[p].mn1 >= v) return;
        if (ql <= l && r <= qr && tr[p].mn2 > v) {
            apply_chmax(p, v);
            return;
        }
        push(p, l, r);
        int m{(l + r) / 2};
        range_chmax(p << 1, l, m, ql, qr, v);
        range_chmax(p << 1 | 1, m, r, ql, qr, v);
        pull(p);
    }
    Node query(int p, int l, int r, int ql, int qr) {
        if (qr <= l || r <= ql) return {};
        if (ql <= l && r <= qr) return tr[p];
        push(p, l, r);
        int m{(l + r) / 2};
        return merge_node(query(p << 1, l, m, ql, qr), query(p << 1 | 1, m, r, ql, qr));
    }
    void add(int l, int r, i64 v) { assert(0 <= l && l <= r && r <= n); range_add(1, 0, n, l, r, v); }
    void chmin(int l, int r, i64 v) { assert(0 <= l && l <= r && r <= n); range_chmin(1, 0, n, l, r, v); }
    void chmax(int l, int r, i64 v) { assert(0 <= l && l <= r && r <= n); range_chmax(1, 0, n, l, r, v); }
    Node query(int l, int r) { assert(0 <= l && l < r && r <= n); return query(1, 0, n, l, r); }
};
```

接口说明

- `add(l,r,v)`：区间加 v
- `chmin(l,r,v)`：逐点执行 $a_i \leftarrow \min(a_i, v)$
- `chmax(l,r,v)`：逐点执行 $a_i \leftarrow \max(a_i, v)$
- `query(l,r).sum`：区间和
- `query(l,r).mx1`：区间最大值
- `query(l,r).mn1`：区间最小值
- `mx2/mn2`：严格次大值 / 严格次小值，不存在时为哨兵
- `mxc/mnc`：最大值 / 最小值出现次数

可裁剪版本

若只需要：

```
add + chmin + sum / max
```

可以删除：

```
mn1 / mn2 / mnc
apply_chmax()
range_chmax()
chmax()
push() 中向儿子执行 chmax 的部分
```

只保留：

```
sum / mx1 / mx2 / mxc / add
```

若只需要：

```
add + chmax + sum / min
```

则完全对称地删除最大值一侧

注意事项

- `chmin` 能整段执行的关键条件是 $mx_2 < v < mx_1$
- `chmax` 能整段执行的关键条件是 $mn_1 < v < mn_2$
- 若 $mx_1 \le v$ 或 $mn_1 \ge v$，修改完全无效，应立即剪枝
- `mx2/mn2` 必须是严格次大 / 次小值，而不是允许和第一极值相等的第二个元素
- 下传必须先处理 `add`，再使用父节点当前的 `mx1/mn1` 限制儿子
- `Node{}` 同时作为查询空段单位元，依赖 `mx1=-INF`、`mn1=INF` 等哨兵设计
- Beats 的均摊证明与允许的操作集合紧密相关，不要随意加入新的非线性懒标记
- 所有区间和与 `(v - mx1) * mxc` 等运算必须保证不溢出

常见用法

```
SegTreeBeats seg(a);
seg.chmin(l, r, x);
seg.chmax(l, r, y);
seg.add(l, r, d);
auto info = seg.query(l, r);
cout << info.sum << ' ' << info.mn1 << ' ' << info.mx1 << '\n';
```

复杂度

区间加和普通查询为 $O(\log n)$
`chmin/chmax` 单次最坏可能访问大量节点，但在标准 Beats 操作序列中具有均摊保证，常用保守总复杂度写为：

$$O((n+q)\log^2 n)$$

空间复杂度为 $O(n)$

3.5.6 按位拆分线段树

解决问题： 区间按位修改难以直接维护整数整体信息时，把每个二进制位独立拆开，维护每一位的 `1` 个数，使区间 XOR、AND、OR 等操作重新变成普通懒标记修改
依赖：

- 3.5.1 SegTree `LazySegTree<Info, Tag>`
- `Info` 支持区间合并
- `Info::apply(tag,l,r)` 表示把标记作用于整个节点
- 以下约定 `Tag::apply(t)` 表示：**当前已有变换之后，再执行 `t`**
  若现有 `LazySegTree` 的标记合成方向相反，只需交换 `Tag` 的复合顺序

原理

设某一位在长度为 `len` 的区间中有 `ones` 个 `1`
执行区间 XOR 时，如果掩码这一位为 `1`，所有 `0/1` 翻转，因此：

$$ones \leftarrow len - ones$$

如果这一位没有被 XOR，则保持不变
因此只要对每个二进制位分别维护 `1` 的数量，就能同时恢复：

- 区间算术和
- 区间异或和
- 每一位出现次数
- 其他依赖位计数的信息

区间 XOR

```
template<int B = 61>
struct XorTag {
    u64 mask{0};
    bool empty() const { return mask == 0; }
    void apply(const XorTag &t) { mask ^= t.mask; }
};
template<int B = 61>
struct XorInfo {
    array<int, B> ones{};
```

```
    XorInfo() = default;
    explicit XorInfo(u64 x) {
        for (int b = 0; b < B; ++ b) ones[b] = x >> b & 1ULL;
    }
    void apply(const XorTag<B> &t, int l, int r) {
        int len{r - l};
        for (int b = 0; b < B; ++ b) if (t.mask >> b & 1ULL) {
            ones[b] = len - ones[b];
        }
    }
};
template<int B>
XorInfo<B> operator+(const XorInfo<B> &a, const XorInfo<B> &b) {
    XorInfo<B> c;
    for (int bit = 0; bit < B; ++ bit) c.ones[bit] = a.ones[bit] + b.ones[bit];
    return c;
}
```

初始化：

```
using Info = XorInfo<31>;
using Tag = XorTag<31>;
vector<Info> init;
init.reserve(n);
for (u64 x : a) init.emplace_back(x);
// 按已有 LazySegTree 的构造接口建树。
LazySegTree<Info, Tag> seg(init);
```

区间 XOR：

```
seg.apply(l, r, Tag{mask});
```

具体修改接口名按已有 `LazySegTree` 为准

从位计数恢复答案

区间算术和

若第 b 位有 `ones[b]` 个 `1`，贡献为：

$$ones_b \cdot 2^b$$

```
template<int B>
__int128_t bit_sum(const XorInfo<B> &info) {
    __int128_t ans{0};
    for (int b = 0; b < B; ++ b) {
        ans += (__int128_t)info.ones[b] << b;
    }
    return ans;
}
```

使用：

```
auto info = seg.query(l, r);
__int128_t sum = bit_sum(info);
```

区间异或和

某一位出现奇数次时，该位的区间异或结果为 `1`：

```
template<int B>
u64 bit_xor(const XorInfo<B> &info) {
    u64 ans{0};
    for (int b = 0; b < B; ++ b) if (info.ones[b] & 1) {
        ans |= 1ULL << b;
    }
```

```
    return ans;
}
```

一般 AND / OR / XOR 变换

若只支持 XOR，每一位只有：

```
保持
翻转
```

两种状态
如果同时需要：

```
AND
OR
XOR
```

则每一位实际上可能出现四种布尔变换：

变换	$f(0)$	$f(1)$
保持	0	1
翻转	1	0
赋 0	0	0
赋 1	1	1

因此可以使用两个掩码：

```
to0 = f(0)
to1 = f(1)
```

完整表示每一位的变换

通用按位标记

```
template<int B = 61>
struct BitTag {
    static_assert(1 ≤ B && B ≤ 64);
    static constexpr u64 full_mask() {
        if constexpr (B == 64) return ~0ULL;
        else return (1ULL << B) - 1;
    }
    // 当前变换对输入 0 / 1 的输出。
    u64 to0{0};
    u64 to1{full_mask()};
    bool empty() const { return to0 == 0 && to1 == full_mask(); }
    static BitTag bit_and(u64 mask) {
        mask &= full_mask();
        return {0, mask};
    }
    static BitTag bit_or(u64 mask) {
        mask &= full_mask();
        return {mask, full_mask()};
    }
    static BitTag bit_xor(u64 mask) {
        mask &= full_mask();
        return {mask, full_mask() ^ mask};
    }
    // 当前变换为 f，新加入变换为 g，合成为 g(f(x))。
    void apply(const BitTag &g) {
        u64 n0{((~to0 & g.to0) | (to0 & g.to1)) & full_mask()};
        u64 n1{((~to1 & g.to0) | (to1 & g.to1)) & full_mask()};
        to0 = n0;
        to1 = n1;
```

```
    }
};
```

对应区间信息：

```
template<int B = 61>
struct BitInfo {
    array<int, B> ones{};
    BitInfo() = default;
    explicit BitInfo(u64 x) {
        for (int b = 0; b < B; ++ b) ones[b] = x >> b & 1ULL;
    }
    void apply(const BitTag<B> &t, int l, int r) {
        int len{ r - l };
        for (int b = 0; b < B; ++ b) {
            int z{int(t.to0 >> b & 1ULL)}, o{int(t.to1 >> b & 1ULL)};
            if (!z && !o) ones[b] = 0;
            else if (z && o) ones[b] = len;
            else if (z && !o) ones[b] = len - ones[b];
        }
    }
};
template<int B>
BitInfo<B> operator+(const BitInfo<B> &a, const BitInfo<B> &b) {
    BitInfo<B> c;
    for (int bit = 0; bit < B; ++ bit) { c.ones[bit] = a.ones[bit] + b.ones[bit]; }
    return c;
}
```

其中：

```
BitTag<B>::bit_and(mask)
BitTag<B>::bit_or(mask)
BitTag<B>::bit_xor(mask)
```

分别表示区间 AND、OR、XOR
因此可以在同一棵 LazySegTree 中任意混合这些区间按位修改

原理推广

按位拆分的核心不是 XOR，而是：

> 如果一个整数操作的各二进制位彼此独立，就可以把一个整数状态拆成 B 个小状态分别维护

常见判断方式：

```
整数整体难以更新
        ↓
操作是否对每一位独立？
        ↓ 是
维护每位 0/1 数量或其他小状态
        ↓
把原修改变成每位上的简单函数
        ↓
使用普通 LazySegTree
```

这种方法与势能线段树、Segment Tree Beats 不同：这里不存在均摊复杂度，修改本身仍是普通的懒标记

接口说明

基础 XOR 版：

- `XorInfo<B>`：维护各二进制位的 `1` 个数
- `XorTag<B>{mask}`：区间 XOR `mask`
- `bit_sum(info)`：从位计数恢复区间算术和
- `::bit_xor(info)`（全局限定避免与 std::bit_xor 重名）：从位计数恢复区间异或和

通用版：

- `BitTag<B>::bit_and(mask)`：区间 AND
- `BitTag<B>::bit_or(mask)`：区间 OR

- `BitTag<B>::bit_xor(mask)`：区间 XOR
- `BitInfo<B>`：同时支持上述三种修改

注意事项

- `B` 必须满足 $1 \le B \le 64$
- `B=64` 时不能直接计算 `1ULL << 64`，因此 `full_mask()` 必须使用 `if constexpr`
- 若数据只使用较低位，应尽量缩小 `B`，因为时间和空间复杂度都与 `B` 成正比
- `ones[b]` 最大为 n，通常 `int` 足够；区间算术和可能超过 `i64`，必要时使用 `__int128_t`
- 通用 `BitTag` 的复合方向必须与现有 `LazySegTree` 的懒标记合成方向一致
- 如果只查询区间 XOR 而不需要区间算术和，通常没必要维护全部位计数，可以使用更简单的区间 XOR 聚合
- 按位拆分适用于各位相互独立的操作；若操作会产生跨位进位，例如普通加法，就不能直接按这种方式拆分

复杂度

设维护 B 个二进制位
建树：

$$O(Bn)$$

一次区间修改或查询：

$$O(B\log n)$$

空间复杂度：

$$O(Bn)$$

当 B 是固定常数时，也可以视为普通线段树的 $O(\log n)$ 操作

3.6 动态、可持久化与可合并线段树

3.6.1 动态开点线段树

```
template<class Info, class Coord = i64>
struct DynamicSegTree {
    struct Node {
        int ch[2]{};
        Info info{};
    };
    Coord lo, hi;
    vector<Node> tr{ Node{} };
    int root{ 0 };
    explicit DynamicSegTree(Coord lo_, Coord hi_, size_t reserve_nodes = 0) : lo(lo_), hi(hi_) {
        assert(lo < hi);
        if (reserve_nodes) tr.reserve(reserve_nodes + 1);
    }
    // ===== 节点创建与维护：modify / query 依赖，不建议删除 =====
    int new_node() {
        tr.emplace_back();
        return int(tr.size()) - 1;
    }
    const Info &info(int p) const {
        static const Info e{};
        return p ? tr[p].info : e;
    }
    void pull(int p) {
        tr[p].info = info(tr[p].ch[0]) + info(tr[p].ch[1]);
    }
    // ===== 单点修改：modify() 为通用叶子修改，set() 为覆盖赋值 =====
    template<class F>
    int modify(int p, Coord l, Coord r, Coord x, F &f) {
        if (!p) p = new_node();
        if (l + 1 == r) {
            f(tr[p].info);
            return p;
        }
        Coord m = midpoint(l, r);
        int d = x >= m;
        int q = modify(tr[p].ch[d], d ? m : l, d ? r : m, x, f);
        tr[p].ch[d] = q;
```

```
        pull(p);
        return p;
    }
    template<class F>
    void modify(Coord x, F &&f) {
        assert(lo <= x && x < hi);
        root = modify(root, lo, hi, x, f);
    }
    void set(Coord x, const Info &v) {
        modify(x, [v](Info &a) { a = v; });
    }
    // ===== 查询：未创建节点视为 Info{}，查询不会创建新节点 =====
    Info query(int p, Coord l, Coord r, Coord ql, Coord qr) const {
        if (!p || qr <= l || r <= ql) return Info{};
        if (ql <= l && r <= qr) return tr[p].info;
        Coord m = midpoint(l, r);
        return query(tr[p].ch[0], l, m, ql, qr) + query(tr[p].ch[1], m, r, ql, qr);
    }
    Info query(Coord l, Coord r) const {
        assert(lo <= l && l <= r && r <= hi);
        return query(root, lo, hi, l, r);
    }
    Info get(Coord x) const {
        assert(lo <= x && x < hi);
        return query(x, x + 1);
    }
    const Info &all() const { return info(root); }
    int nodes() const { return int(tr.size()) - 1; }
};
```

接口说明

模板参数与成员

- `Info`：节点维护的信息类型，要求 `Info{}` 为合并单位元，并定义 `Info + Info`
- `Coord`：坐标类型，默认 `i64`
- `[lo, hi)`：整棵线段树维护的整数值域
- `root`：根节点编号，`0` 表示空树
- `tr`：节点池，`tr[0]` 为哨兵空节点

接口

- `DynamicSegTree(lo, hi, reserve_nodes = 0)`：维护整数值域 `[lo, hi)`；第三个参数可预留节点空间，构造本身 $O(1)$
- `modify(x, f)`：对位置 `x` 的叶节点执行 `f(Info&)`，$O(\log(hi - lo))$
- `set(x, v)`：令位置 `x` 的信息变为 `v`，$O(\log(hi - lo))$
- `query(l, r)`：查询 `[l, r)` 的合并信息，$O(\log(hi - lo))$
- `get(x)`：查询单点 `x`，$O(\log(hi - lo))$
- `all()`：返回整棵树的信息，$O(1)$
- `nodes()`：当前实际创建的节点数，$O(1)$

注意事项

- 坐标和值域均采用**整数左闭右开区间 `[l, r)`**
- 没有 `build()`；只有访问到的修改路径才会创建节点
- `Info{}` 必须是合并单位元，即应满足：

```
Info{} + x == x
x + Info{} == x
```

因此未开点区域天然表示“默认空信息”

- `query()` **不会开点**。只有 `modify()` / `set()` 会增加节点
- 当前版本删除一个值后，即使节点重新变成 `Info{}`，也**不会释放已经创建的节点**。节点回收留到之后的“可合并权值线段树”统一处理
- 不支持区间修改，因此没有 `Tag / apply / push`。动态开点 + lazy 是另一层扩展，不建议让基础板子变臃肿
- 使用 `midpoint(l, r)` 而不是 `(l + r) / 2`，避免大坐标时 `l + r` 溢出
- 一次修改最多创建树高数量的新节点，因此可根据操作数提前：

```
seg.tr.reserve(...);
```

更推荐直接通过构造函数的 `reserve_nodes` 参数完成

常见用法

区间和

```
struct Info {
    i64 sum{};
};
Info operator+(const Info &a, const Info &b) {
    return { a.sum + b.sum };
}
```

建立一个巨大值域：

```
DynamicSegTree<Info> seg(-1E18, i64(1E18) + 1);
```

单点加：

```
seg.modify(x, [&](Info &a) { a.sum += v; });
```

区间和：

```
i64 ans = seg.query(l, r).sum;
```

即查询 `[l, r)`

单点覆盖

```
seg.set(x, { v });
```

等价于：

```
seg.modify(x, [&](Info &a) { a = { v }; });
```

动态维护出现次数

```
struct Info {
    int cnt{};
};
Info operator+(const Info &a, const Info &b) {
    return { a.cnt + b.cnt };
}
```

插入 / 删除一个值：

```
seg.modify(x, [&](Info &a) { ++ a.cnt; });
seg.modify(x, [&](Info &a) { -- a.cnt; });
```

查询某个值域内元素数量：

```
int cnt = seg.query(l, r).cnt;
```

不过如果进一步需要：

```
kth
rank
前驱 / 后继
merge
split
```

就不要继续膨胀这一份板子了，那些统一放到之后的 可合并权值线段树

估算预留节点数

若有 `q` 次单点修改、值域树高约为 `B`：

```
DynamicSegTree<Info> seg(0, 1LL << 60, size_t(q) * 61);
```

这是一个保守上界；不同修改路径通常会共享节点，所以实际节点数一般更少

复杂度

设值域宽度为

$$W = hi - lo$$

操作	时间复杂度
构造	O(1)
modify() / set() / get() / query()	O(log W)
all()	O(1)

若进行了 `q` 次单点修改，空间复杂度最坏为

$$O(q \log W)$$

而不是普通线段树的 $O(W)$

3.6.2 可持久化线段树

```
template<class Info>
struct PersistentSegTree {
    struct Node {
        int ch[2]{};
        Info info{};
    };
    int n{};
    vector<Node> tr{ Node{} };
    vector<int> roots;
    // 空初始版本：所有位置均为 Info{}
    explicit PersistentSegTree(int n_, size_t reserve_nodes = 0) : n(n_), roots{ 0 } {
        assert(n > 0);
        if (reserve_nodes) tr.reserve(reserve_nodes + 1);
    }
    // 从任意随机访问 range / view 建立第 0 个版本
    template<ranges::random_access_range R>
    requires ranges::sized_range<R>
          && convertible_to<ranges::range_reference_t<R>, Info>
    explicit PersistentSegTree(R &&a, size_t reserve_nodes = 0)
        : n(int(ranges::size(a))) {
        assert(n > 0);
        if (reserve_nodes) tr.reserve(reserve_nodes + 1);
        roots.push_back(build(0, n, a));
    }
    // ===== 节点创建与维护：build / modify 依赖，不建议删除 =====
    int new_node(Node x = {}) {
        tr.push_back(move(x));
        return int(tr.size()) - 1;
    }
    int clone(int p) { return new_node(tr[p]); }
    void pull(int p) {
        tr[p].info = tr[tr[p].ch[0]].info + tr[tr[p].ch[1]].info;
    }
    // ===== 初始建树：使用空初始版本时可删除 build() 与 range 构造函数 =====
    template<ranges::random_access_range R>
    int build(int l, int r, R &a) {
        int p = new_node();
        if (r - l == 1) {
            tr[p].info = *(ranges::begin(a) + l);
            return p;
        }
        int m = midpoint(l, r);
        tr[p].ch[0] = build(l, m, a);
        tr[p].ch[1] = build(m, r, a);
        pull(p);
        return p;
    }
```

```
    // ===== 路径复制修改：可持久化核心，不建议删除 =====
    template<class F>
    int modify(int p, int l, int r, int x, F &f) {
        int q = clone(p);
        if (r - l == 1) {
            invoke(f, tr[q].info);
            return q;
        }
        int m = midpoint(l, r);
        int d = x >= m;
        int c = modify(tr[q].ch[d], d ? m : l, d ? r : m, x, f);
        tr[q].ch[d] = c;
        pull(q);
        return q;
    }
    template<class F>
    int modify(int ver, int x, F &&f) {
        assert(0 <= ver && ver < versions());
        assert(0 <= x && x < n);
        roots.push_back(modify(roots[ver], 0, n, x, f));
        return versions() - 1;
    }
    int set(int ver, int x, const Info &v) {
        return modify(ver, x, [v](Info &a) { a = v; });
    }
    // ===== 查询：只读任意历史版本，不创建节点 =====
    Info query(int p, int l, int r, int ql, int qr) const {
        if (!p || qr <= l || r <= ql) return Info{};
        if (ql <= l && r <= qr) return tr[p].info;
        int m = midpoint(l, r);
        return query(tr[p].ch[0], l, m, ql, qr) + query(tr[p].ch[1], m, r, ql, qr);
    }
    Info query(int ver, int l, int r) const {
        assert(0 <= ver && ver < versions());
        assert(0 <= l && l <= r && r <= n);
        return query(roots[ver], 0, n, l, r);
    }
    Info get(int ver, int x) const {
        assert(0 <= x && x < n);
        return query(ver, x, x + 1);
    }
    const Info &all(int ver) const {
        assert(0 <= ver && ver < versions());
        return tr[roots[ver]].info;
    }
    // ===== 版本管理：需要“查询操作也生成一个版本”时很方便 =====
    int copy_version(int ver) {
        assert(0 <= ver && ver < versions());
        roots.push_back(roots[ver]);
        return versions() - 1;
    }
    // 按预计单点修改次数预留空间
    void reserve_updates(size_t q) {
        size_t per = bit_width(static_cast<unsigned>(n - 1)) + 1;
        tr.reserve(tr.size() + q * per);
        roots.reserve(roots.size() + q);
    }
    int versions() const { return int(roots.size()); }
    int nodes() const { return int(tr.size()) - 1; }
};
```

接口说明

模板参数与成员变量

- `Info`：线段树节点维护的信息
- `Info{}`：必须是合并运算的单位元
- `tr`：所有历史版本共享的节点池，`tr[0]` 是空节点
- `roots[v]`：版本 `v` 的根节点
- `n`：维护下标范围 `[0, n)`

其中每个版本实际上只保存一个根：

```
roots[0] ─┐
          ├─ 大量共享旧节点
roots[1] ─┤
          ├─ 仅修改路径上的节点不同
roots[2] ─┘
```

构造

空初始版本：

```
PersistentSegTree<Info> seg(n);
```

此时第 `0` 个版本的所有位置都等价于：

```
Info{}
```

从初始序列建树：

```
PersistentSegTree<Info> seg(a);
```

也支持 `view`，因此不必为了转换成 `Info` 再额外构造数组：

```
PersistentSegTree<Info> seg(
    a | views::transform([](i64 x) {
        return Info{ x };
    })
);
```

修改与版本

`modify(ver, x, f)`：以版本 `ver` 为基础修改位置 `x`，产生一个新版本，返回新版本编号

```
int nv = seg.modify(ver, x, [&](Info &a) { a.sum += d; });
```

原来的 `ver` 不受影响
`set(ver, x, v)`：覆盖位置 `x`：

```
int nv = seg.set(ver, x, Info{ v });
```

`copy_version(ver)`：创建一个完全相同的新版本，但不复制任何树节点：

```
int nv = seg.copy_version(ver);
```

这个接口对“每次操作都必须产生一个编号版本”的问题尤其方便

查询

`query(ver, l, r)`：查询版本 `ver` 的 `[l, r)`

```
auto ans = seg.query(ver, l, r);
```

`get(ver, x)`：查询某版本单点

```
auto value = seg.get(ver, x);
```

`all(ver)`：查询某版本整棵树维护的信息

```
auto info = seg.all(ver);
```

`versions()`：当前版本数
`nodes()`：节点池实际创建的节点数量
`reserve_updates(q)`：已知接下来大约有 `q` 次单点修改时预留节点空间，避免节点池频繁扩容

注意事项

- 区间统一为 `[l, r)`，下标为 `0base`
- 当前版本只做单点修改；不加入 `Tag / push()`。可持久化区间修改需要处理共享节点上的 lazy，下一个“可持久化懒标记线段树”再单独整理

- `modify(ver, ...)` 可以从任意历史版本继续生成新版本，因此不仅能形成版本链，也能形成版本树：

```
0
├── 1
│   ├── 2
│   └── 3
└── 4
```

- `Info{}` 必须满足合并单位元要求：

$$e + x = x + e = x$$

  否则空初始版本以及查询越界部分都会产生错误信息
- 未发生修改的子树会在多个版本之间共享，因此绝对不能原地修改旧节点。所有会改变节点内容的操作必须先经过 `clone()`
- 也正因为节点可能同时被很多历史版本引用，本板子不能使用简单 free list 回收节点。节点回收留给不存在共享所有权问题的可合并权值线段树
- `copy_version()` 只复制根编号，不创建树节点
- `query()` 完全只读，不会创建节点

常见用法

1. 可持久化数组：单点赋值 + 历史查询

最简单的 `Info`：

```
struct Info {
    i64 sum{};
};
Info operator+(const Info &a, const Info &b) {
    return { a.sum + b.sum };
}
```

从原数组建立版本 `0`：

```
vector<i64> a(n);
PersistentSegTree<Info> seg(
    a | views::transform([](i64 x) {
        return Info{ x };
    })
);
```

从版本 `v` 修改：

```
int nv = seg.set(v, p, { x });
```

查询历史版本：

```
i64 x = seg.get(v, p).sum;
i64 sum = seg.query(v, l, r).sum;
```

2. 在任意历史版本上分叉

例如：

```
int v1 = seg.set(0, 3, { 10 });
int v2 = seg.set(v1, 5, { 20 });
int v3 = seg.set(v1, 7, { 30 });
```

版本关系是：

```
0
└── v1
    ├── v2
    └── v3
```

其中：

```
seg.get(v2, 7);
seg.get(v3, 5);
```

互不影响

3. 按操作编号建立版本

如果每个修改操作恰好生成一个版本，可以直接利用版本编号：

```
for (int i : views::iota(0, q)) {
    int p;
    i64 x;
    cin >> p >> x;
    seg.set(i, p, { x });
}
```

于是：

```
版本 0 = 初始状态
版本 1 = 第 1 次修改后
版本 2 = 第 2 次修改后
...
```

若某个操作只是查询，但题意仍要求产生一个新版本：

```
seg.copy_version(old_ver);
```

常见 `Info`

区间和

```
struct Info {
    i64 sum{};
};
Info operator+(const Info &a, const Info &b) {
    return { a.sum + b.sum };
}
```

单点加：

```
int nv = seg.modify(ver, p, [&](Info &a) {
    a.sum += x;
});
```

单点赋值：

```
int nv = seg.set(ver, p, { x });
```

同时维护和、最小值、最大值

空节点必须是单位元，所以最小值与最大值不能默认设成 `0`：

```
struct Info {
    i64 sum{};
    i64 mn{ INF };
    i64 mx{ -INF };
};
Info operator+(const Info &a, const Info &b) {
    return {
        a.sum + b.sum,
        min(a.mn, b.mn),
        max(a.mx, b.mx)
    };
}
```

初始化一个实际元素：

```
auto make_info = [](i64 x) {
    return Info{ x, x, x };
};
```

可以直接用 `views::transform` 建树：

```
PersistentSegTree<Info> seg(
    a | views::transform([&](i64 x) {
        return make_info(x);
    })
);
```

查询：

```
auto res = seg.query(ver, l, r);
i64 sum = res.sum;
i64 mn = res.mn;
i64 mx = res.mx;
```

维护最大公约数

`0` 正好是 `gcd` 的单位元：

```
struct Info {
    i64 g{};
};
Info operator+(const Info &a, const Info &b) {
    return { gcd(a.g, b.g) };
}
```

因此：

```
auto g = seg.query(ver, l, r).g;
```

可以直接查询任意历史版本的区间 gcd

维护最大子段和

这种 `Info` 最容易犯错，因为空区间不能简单当成全 `0` 节点。可以显式维护 `empty`：

```
struct Info {
    i64 sum{};
    i64 pre{}, suf{}, ans{};
    bool empty{ true };
    Info() = default;
    explicit Info(i64 x) : sum(x), pre(x), suf(x), ans(x), empty(false) {}
};
Info operator+(const Info &a, const Info &b) {
    if (a.empty) return b;
    if (b.empty) return a;
    Info c;
    c.empty = false;
    c.sum = a.sum + b.sum;
    c.pre = max(a.pre, a.sum + b.pre);
    c.suf = max(b.suf, b.sum + a.suf);
    c.ans = max({ a.ans, b.ans, a.suf + b.pre });
    return c;
}
```

于是初始序列仍然可以漂亮地写成：

```
PersistentSegTree<Info> seg(
    a | views::transform([](i64 x) {
        return Info(x);
    })
);
```

之后：

```
i64 ans = seg.query(ver, l, r).ans;
```

就是某个历史版本 `[l, r)` 的最大子段和
这个例子其实也说明了一个原则：

> 可持久化只改变“节点如何产生”，不会改变普通线段树的 `Info` 合并逻辑

维护频数与权值和

有时某个坐标代表一个值，希望同时记录出现次数与权值和：

```
struct Info {
    int cnt{};
    i64 sum{};
};
Info operator+(const Info &a, const Info &b) {
    return {
        a.cnt + b.cnt,
        a.sum + b.sum
    };
}
```

加入一个值：

```
int nv = seg.modify(ver, x, [&](Info &a) {
    ++ a.cnt;
    a.sum += x;
});
```

删除：

```
int nv = seg.modify(ver, x, [&](Info &a) {
    -- a.cnt;
    a.sum -= x;
});
```

这样已经能处理“某个历史版本的值域计数 / 值域和”
不过一旦要进一步写：

```
两版本作差
kth
rank
前驱
后继
```

就应该切换到主席树

常见套路

历史版本回退

其实完全不需要真正“撤销”：

```
int current = old_version;
```

之后继续：

```
current = seg.set(current, p, { x });
```

即可从那个历史状态重新分叉

查询操作也产生版本

某些题目的第 `i` 次操作对应版本 `i`，即使本次没有修改：

```
if (op == 1) {
    seg.set(ver, p, { x });
} else {
    cout << seg.query(ver, l, r).sum << '\n';
    seg.copy_version(ver);
}
```

这样始终保持：

```
操作编号 = 新版本编号
```

会比在外面维护复杂映射干净很多

预留节点

假设之后大约有 `q` 次单点修改：

```
seg.reserve_updates(q);
```

一次修改只复制根到叶的一条路径
若维护长度为 n 的序列，每次修改创建的节点数至多约为：

$$\lceil \log_2 n \rceil + 1$$

因此节点数通常可以预估为：

$$N_{\text{initial}} + q\,(\lceil \log_2 n \rceil + 1)$$

其中从完整初始数组建树时：

$$N_{\text{initial}} = 2n - 1$$

如果初始版本为空树：

$$N_{\text{initial}} = 0$$

`reserve_updates()` 内部利用 `bit_width()` 自动估计这部分空间

复杂度

设序列长度为 n
从完整初始数组建树：

$$O(n)$$

单次 `modify()` 或 `set()`：

$$O(\log n)$$

并新建：

$$O(\log n)$$

个节点
单点查询：

$$O(\log n)$$

区间查询：

$$O(\log n)$$

`all()`：

$$O(1)$$

`copy_version()`：

$$O(1)$$

且不创建任何树节点
若初始建树后进行了 (q) 次单点修改，总空间复杂度为：

$$O(n + q \log n)$$

若初始版本为空、之后仅通过单点修改动态产生节点，则为：

$$O(q \log n)$$

版本根数组本身额外占用：

$$(V)$$

其中 V 为版本数量

3.6.3 主席树

```
template<class T = i64>
struct ChairmanTree {
    struct Node {
        int ch[2]{};
        int cnt{};
    };
    vector<T> val;
    vector<Node> tr{ Node{} };
    vector<int> root{ 0 };
    bool prefix_built{ false };
    // build_prefix = true: a 既作为待离散化值域，也建立前缀版本
    // build_prefix = false: 只离散化 a，常用于树上路径等自定义版本关系
    template<ranges::input_range R>
    explicit ChairmanTree(R &&a, bool build_prefix = true, size_t reserve_nodes = 0) {
        vector<T> seq;
        if constexpr (ranges::sized_range<R>) seq.reserve(ranges::size(a));
        for (auto &&x : a) seq.emplace_back(x);
        val = seq;
        ranges::sort(val);
        val.erase(ranges::unique(val).begin(), val.end());
        assert(!val.empty());
        if (reserve_nodes) tr.reserve(reserve_nodes + 1);
        prefix_built = build_prefix;
        if (build_prefix) {
            root.resize(seq.size() + 1);
            if (!reserve_nodes) reserve_updates(seq.size());
            for (int i : views::iota(0, int(seq.size()))) {
                root[i + 1] = add_root(root[i], seq[i]);
            }
        }
    }
    // ===== 节点创建与单点路径复制：主席树核心，不建议删除 =====
    int new_node(Node x = {}) {
        tr.push_back(x);
        return int(tr.size()) - 1;
    }
    int clone(int p) { return new_node(tr[p]); }
    int id(const T &x) const {
        return int(ranges::lower_bound(val, x) - val.begin());
    }
    int add(int p, int l, int r, int x, int delta) {
        int q = clone(p);
        tr[q].cnt += delta;
        if (r - l == 1) return q;
        int m = midpoint(l, r);
        int d = x >= m;
        tr[q].ch[d] = add(tr[q].ch[d], d ? m : l, d ? r : m, x, delta);
        return q;
    }
    // 从任意 root 产生新 root；x 必须已存在于离散值域中
    int add_root(int p, const T &x, int delta = 1) {
        int k = id(x);
        assert(k < int(val.size()) && val[k] == x);
        return add(p, 0, val.size(), k, delta);
    }
    // ===== 两版本差分查询：前缀区间 [l, r) 的基础 =====
    int prefix_count(int a, int b, int l, int r, int x) const {
        if (x <= l) return 0;
        if (r <= x) return tr[b].cnt - tr[a].cnt;
        int m = midpoint(l, r);
        return prefix_count(tr[a].ch[0], tr[b].ch[0], l, m, x) + prefix_count(tr[a].ch[1], tr[b].ch[1], m, r, x);
    }
    int prefix_count(int a, int b, int x) const { return prefix_count(a, b, 0, val.size(), x); }
    int kth_index(int a, int b, int k) const {
        int l = 0, r = val.size();
        while (r - l > 1) {
            int m = midpoint(l, r);
            int ca = tr[a].ch[0], cb = tr[b].ch[0];
            int left = tr[cb].cnt - tr[ca].cnt;
            if (k < left) {
```

```
                a = ca;
                b = cb;
                r = m;
            } else {
                k -= left;
                a = tr[a].ch[1];
                b = tr[b].ch[1];
                l = m;
            }
        }
        return l;
    }
    // ===== 静态区间接口：[l, r)，k 为 0base =====
    int size(int l, int r) const {
        assert_prefix(l, r);
        return tr[root[r]].cnt - tr[root[l]].cnt;
    }
    T kth(int l, int r, int k) const {
        assert_prefix(l, r);
        assert(0 <= k && k < size(l, r));
        return val[kth_index(root[l], root[r], k)];
    }
    T kth_largest(int l, int r, int k) const { return kth(l, r, size(l, r) - 1 - k); }
    int count_less(int l, int r, const T &x) const {
        assert_prefix(l, r);
        int p = ranges::lower_bound(val, x) - val.begin();
        return prefix_count(root[l], root[r], p);
    }
    int count_leq(int l, int r, const T &x) const {
        assert_prefix(l, r);
        int p = int(ranges::upper_bound(val, x) - val.begin());
        return prefix_count(root[l], root[r], p);
    }
    int count(int l, int r, const T &lo, const T &hi) const {
        assert_prefix(l, r);
        if (!(lo < hi)) return 0;
        int L = ranges::lower_bound(val, lo) - val.begin();
        int R = ranges::lower_bound(val, hi) - val.begin();
        return prefix_count(root[l], root[r], R) - prefix_count(root[l], root[r], L);
    }
    int frequency(int l, int r, const T &x) const { return count_leq(l, r, x) - count_less(l, r, x); }
    // 排名按竞赛中常见定义：小于 x 的元素数 + 1
    int rank(int l, int r, const T &x) const { return count_less(l, r, x) + 1; }
    optional<T> predecessor(int l, int r, const T &x) const {
        int k = count_less(l, r, x);
        if (!k) return nullopt;
        return kth(l, r, k - 1);
    }
    optional<T> successor(int l, int r, const T &x) const {
        int k = count_leq(l, r, x);
        if (k == size(l, r)) return nullopt;
        return kth(l, r, k);
    }
    // ===== 树上路径四版本 kth：不需要时可删除 =====
    // root[u] + root[v] - root[lca] - root[parent(lca)]
    T kth_path(int a, int b, int c, int d, int k) const {
        int total = tr[a].cnt + tr[b].cnt - tr[c].cnt - tr[d].cnt;
        assert(0 <= k && k < total);
        int l = 0, r = val.size();
        while (r - l > 1) {
            int m = midpoint(l, r);
            int al = tr[a].ch[0], bl = tr[b].ch[0];
            int cl = tr[c].ch[0], dl = tr[d].ch[0];
            int left = tr[al].cnt + tr[bl].cnt - tr[cl].cnt - tr[dl].cnt;
            if (k < left) {
                a = al;
                b = bl;
                c = cl;
                d = dl;
                r = m;
            } else {
```

```
                k -= left;
                a = tr[a].ch[1];
                b = tr[b].ch[1];
                c = tr[c].ch[1];
                d = tr[d].ch[1];
                l = m;
            }
        }
        return val[l];
    }
    // ===== 辅助接口 =====
    void reserve_updates(size_t q) {
        size_t h = bit_width(unsigned(val.size() - 1)) + 1;
        tr.reserve(tr.size() + q * h);
    }
    int nodes() const { return int(tr.size()) - 1; }
private:
    void assert_prefix(int l, int r) const {
        assert(prefix_built);
        assert(0 <= l && l <= r && r < int(root.size()));
    }
};
```

这版和一般可持久化线段树相比做了两个重要特化：

1. 不再泛化 `Info`，只维护 `cnt`，因为主席树的核心就是**权值域频数版本差分**
2. 不建立完整零树，`0` 号节点直接代表整棵空树；每插入一个元素仅复制一条路径

接口说明

成员变量

- `val`：排序去重后的权值数组
- `tr`：所有版本共享的节点池，`tr[0]` 表示空节点
- `root[i]`：前缀 `[0, i)` 对应的版本根
- `prefix_built`：是否建立了标准前缀版本
  标准构造：

```
ChairmanTree seg(a);
```

版本含义为：

```
root[0]: 空集
root[1]: a[0]
root[2]: a[0..1]
...
root[i]: a[0..i)
```

所以区间 `[l, r)` 的权值集合就是：

```
root[r] - root[l]
```

查询接口

- `kth(l, r, k)`：返回 `[l, r)` 中第 `k` 小，`k` 为 `0base`
- `kth_largest(l, r, k)`：返回第 `k` 大，`k` 为 `0base`
- `count_less(l, r, x)`：统计 `< x`
- `count_leq(l, r, x)`：统计 `<= x`
- `count(l, r, lo, hi)`：统计值落在 `[lo, hi)` 的元素数
- `frequency(l, r, x)`：统计 `x` 出现次数
- `rank(l, r, x)`：返回 `x` 的排名，即“小于 `x` 的数量 + 1”
- `predecessor(l, r, x)`：严格小于 `x` 的最大值，不存在返回 `nullopt`
- `successor(l, r, x)`：严格大于 `x` 的最小值，不存在返回 `nullopt`
- `size(l, r)`：区间元素数量
  上述查询的时间复杂度均为：

$$O(\log m)$$

其中 m 是不同权值数量

自定义版本接口

- `add_root(p, x, delta)`：从根 `p` 出发，将 `x` 的频数增加 `delta`，返回新根
- `kth_path(a, b, c, d, k)`：对四个根的线性组合

$$a+b-c-d$$

求第 `k` 小
单次 `add_root()` 的复杂度为：

$$O(\log m)$$

并新增：

$$O(\log m)$$

个节点

注意事项

- 数组区间统一使用 `[l, r)`，数组下标为 `0base`
- `kth()` 的 `k` 也是 `0base`：

```
seg.kth(l, r, 0);          // 最小值
seg.kth(l, r, r - l - 1);  // 最大值
```

- `rank()` 则沿用常见题面定义，从 `1` 开始
- `count(l, r, lo, hi)` 的值域区间也是 `[lo, hi)`
- 主席树查询依赖：

$$\mathrm{cnt}(root_r)-\mathrm{cnt}(root_l)$$

因此标准前缀版本只能用于**静态序列**；原数组发生修改后，后面的所有前缀版本都会受到影响
- `add_root()` 的 `x` 必须已经存在于初始化时的 `val` 中。若后续可能插入其他值，构造时必须把所有可能出现的值提前加入离散值域
- 如果只需要静态区间第 `k` 小，核心真正必需的只有：

```
Node
add()
add_root()
kth_index()
kth()
```

`count / rank / predecessor / successor / kth_path` 都可以按题目裁掉

- 动态单点修改 + 区间第 `k` 小不是普通主席树直接擅长的模型；传统做法通常转为 BIT 套动态权值线段树等结构。

常见用法

静态区间第 `k` 小

```
vector<i64> a(n);
ChairmanTree seg(a);
i64 ans = seg.kth(l, r, k);
```

例如第 `3` 小：

```
i64 ans = seg.kth(l, r, 2);
```

区间排名

求 `x` 在 `[l, r)` 中的排名：

```
int rk = seg.rank(l, r, x);
```

其含义为：

$$1+\#\{a_i<x\}$$

因此即使 `x` 本身不存在，也能得到它插入后的排名

前驱与后继

```
if (auto x = seg.predecessor(l, r, v)) {
    // *x 为严格小于 v 的最大值
}
if (auto x = seg.successor(l, r, v)) {
    // *x 为严格大于 v 的最小值
}
```

如果题目保证一定存在，可以直接：

```
i64 pre = *seg.predecessor(l, r, v);
```

值域计数

查询 `[l, r)` 中：

小于 `x`

```
int cnt = seg.count_less(l, r, x);
```

不大于 `x`

```
int cnt = seg.count_leq(l, r, x);
```

落在 `[lo, hi)`

```
int cnt = seg.count(l, r, lo, hi);
```

这套接口经常可以直接替代额外二分：

```
第 k 小
排名
频数
值域计数
前驱后继
```

本质都只是对同一份版本差分频数树的不同下降方式

树上路径第 `k` 小

这是主席树非常经典的另一套版本组织方式
“LCA + 主席树”的树上路径点权第 k 大
先只离散化所有点权，不建立数组前缀：

```
ChairmanTree seg(w, false);
```

令：

```
root[u] = 从树根到 u 的路径上所有点权形成的主席树
```

DFS 时：

```
root[u] = seg.add_root(root[fa], w[u]);
```

例如：

```
vector<int> root(n);
auto dfs = Y([&](auto self, int u, int fa) -> void {
    root[u] = seg.add_root(fa == -1 ? 0 : root[fa], w[u]);
    for (int v : g[u]) if (v != fa) {
        self(v, u);
    }
});
```

设：

```
int p = lca(u, v);
```

```
int pp = parent[p];
```

则路径 `u → v` 上的点权集合为：

$$root_u + root_v - root_p - root_{\text{parent}(p)}$$

于是：

```
int rp = parent[p] == -1 ? 0 : root[parent[p]];
i64 ans = seg.kth_path(root[u], root[v], root[p], rp, k);
```

这里的 `k` 同样为 `0base`
为什么是四个版本很好记：

```
root[u] + root[v]
```

会把 LCA 及其祖先算两次，因此减去：

```
root[lca] + root[parent(lca)]
```

恰好保留路径上的所有节点一次

常见维护扩展：同时维护权值和

这个功能很有价值，但**不建议默认塞进核心主席树**
原因很现实：节点从：

```
struct Node {
    int ch[2];
    int cnt;
};
```

扩成：

```
struct Node {
    int ch[2]{};
    int cnt{};
    i64 sum{};
};
```

大量节点下会明显增加内存
如果题目需要：

- 前 `k` 小之和
- 后 `k` 大之和
- 小于某值的元素和
- 区间值域和
  再增加即可

修改节点

在 `add()` 中加入：

```
tr[q].sum += i64(val[x]) * delta;
```

即：

```
int add(int p, int l, int r, int x, int delta) {
    int q = clone(p);
    tr[q].cnt += delta;
    tr[q].sum += i64(val[x]) * delta;
    if (r - l == 1) return q;
    int m = midpoint(l, r);
    int d = x >= m;
    tr[q].ch[d] = add(tr[q].ch[d], d ? m : l, d ? r : m, x, delta);
    return q;
}
```

前 `k` 小元素和

这里的 `k` 表示取多少个元素，范围为：

$$0 \le k \le r - l$$

```
i64 sum_k_smallest_root(int a, int b, int k) const {
    i64 ans = 0;
    int l = 0, r = val.size();
    while (r - l > 1) {
        int m = midpoint(l, r);
        int al = tr[a].ch[0];
        int bl = tr[b].ch[0];
        int left = tr[bl].cnt - tr[al].cnt;
        if (k ≤ left) {
            a = al;
            b = bl;
            r = m;
        } else {
            ans += tr[bl].sum - tr[al].sum;
            k -= left;
            a = tr[a].ch[1];
            b = tr[b].ch[1];
            l = m;
        }
    }
    return ans + i64(val[l]) * k;
}
i64 sum_k_smallest(int l, int r, int k) const {
    assert_prefix(l, r);
    assert(0 ≤ k && k ≤ size(l, r));
    return sum_k_smallest_root(root[l], root[r], k);
}
```

查询前 `k` 大之和也无需重新写下降逻辑：

```
i64 sum_all(int l, int r) const {
    return tr[root[r]].sum - tr[root[l]].sum;
}
i64 sum_k_largest(int l, int r, int k) const {
    return sum_all(l, r) - sum_k_smallest(l, r, size(l, r) - k);
}
```

单次仍然只下降一条值域路径：

$$O(\log m)$$

这一套路在“选若干最小/最大值求代价”类问题里非常实用

常见套路：区间不同数个数

这个也是经典“主席树”应用，但它和上面的权值主席树不是同一种版本含义
从右向左建立版本：对于每个后缀，只让每一种值的最左一次出现位置为 `1`。如果当前值之前已经在更右侧位置 `p` 被标记，则先把 `p` 改成 `0`，再把当前位置改成 `1`。这样版本 `root[l]` 查询位置区间 `[l,r]` 的和，就是原数组 `[l,r]` 中不同数的个数
例如：

```
a = [1, 2, 1, 3]
```

从位置 `0` 看：

```
1 1 0 1
```

只保留：

```
1 的第一次出现
2 的第一次出现
3 的第一次出现
```

于是 `[0,4)` 的和就是 `3`
这个套路本质是：

> 值决定“更新哪个旧位置”，而线段树下标维护的是“数组位置”

因此不建议硬塞进当前 `ChairmanTree<T>` 类；直接用上一份通用的可持久化线段树实现更自然

复杂度

设：

- 原序列长度为 n
- 不同权值数量为 m
  离散化：

$$O(n \log n)$$

建立全部前缀版本：

$$O(n \log m)$$

每插入一个元素新增节点数：

$$O(\log m)$$

因此总节点数 / 总空间复杂度：

$$O(n \log m)$$

单次第 k 小(大) / 单次排名查询 / 单次值域计数 / 单次频数查询 / 单次前驱 / 后继 / 树上路径第 k 小：

$$O(\log m)$$

从父节点版本建立一个新的树上节点版本：

$$O(\log m)$$

如果增加权值和后查询前 k 小 / 大元素和，单次复杂度仍为：

$$O(\log m)$$

3.6.4 可合并权值线段树

```
template<integral Coord = i64>
struct MergeableValueSegTree {
    struct Node {
        int ch[2]{};
        int cnt{};
    };
    static_assert(!same_as<Coord, bool>);
    Coord lo, hi;
    vector<Node> tr{ Node{} };
    vector<int> free_nodes;
    explicit MergeableValueSegTree(Coord lo_, Coord hi_, size_t reserve_nodes = 0) : lo(lo_), hi(hi_) {
        assert(lo < hi);
        if (reserve_nodes) tr.reserve(reserve_nodes + 1);
    }
    // ===== 节点池与维护：所有操作依赖，不建议删除 =====
    int size(int p) const { return p ? tr[p].cnt : 0; }
    int new_node() {
        if (free_nodes.empty()) {
            tr.emplace_back();
            return int(tr.size()) - 1;
        }
        int p = free_nodes.back();
        free_nodes.pop_back();
        tr[p] = {};
        return p;
    }
    void recycle_node(int p) {
        assert(p);
        tr[p] = {};
        free_nodes.push_back(p);
    }
    int normalize(int p) {
        if (p && tr[p].cnt == 0) {
            recycle_node(p);
            return 0;
```

```
        }
        return p;
    }
    void pull(int p) { tr[p].cnt = size(tr[p].ch[0]) + size(tr[p].ch[1]); }
    // ===== 单点增删：权值多重集基础，不建议删除 =====
    int add_node(int p, Coord l, Coord r, Coord x, int delta) {
        if (!p) p = new_node();
        if (l + 1 == r) {
            tr[p].cnt += delta;
            assert(tr[p].cnt >= 0);
            return normalize(p);
        }
        Coord m = midpoint(l, r);
        int d = x >= m;
        int q = add_node(tr[p].ch[d], d ? m : l, d ? r : m, x, delta);
        tr[p].ch[d] = q;
        pull(p);
        return normalize(p);
    }
    void add(int &p, Coord x, int delta = 1) {
        assert(lo <= x && x < hi);
        p = add_node(p, lo, hi, x, delta);
    }
    template<ranges::input_range R>
    int build(R &&a) {
        int p = 0;
        for (auto &&x : a) add(p, static_cast<Coord>(x));
        return p;
    }
    // ===== 顺序统计：若只需要 merge / split，可按需删除部分接口 =====
    int count_node(int p, Coord l, Coord r, Coord ql, Coord qr) const {
        if (!p || qr <= l || r <= ql) return 0;
        if (ql <= l && r <= qr) return tr[p].cnt;
        Coord m = midpoint(l, r);
        return count_node(tr[p].ch[0], l, m, ql, qr) + count_node(tr[p].ch[1], m, r, ql, qr);
    }
    int count(int p, Coord l, Coord r) const {
        l = max(l, lo);
        r = min(r, hi);
        return l < r ? count_node(p, lo, hi, l, r) : 0;
    }
    int count_less(int p, Coord x) const {
        if (x <= lo) return 0;
        if (hi <= x) return size(p);
        return count_node(p, lo, hi, lo, x);
    }
    int count_leq(int p, Coord x) const {
        if (x < lo) return 0;
        if (x >= hi - 1) return size(p);
        return count_less(p, x + 1);
    }
    int frequency(int p, Coord x) const {
        if (x < lo || hi <= x) return 0;
        return count(p, x, x + 1);
    }
    Coord kth(int p, int k) const {
        assert(0 <= k && k < size(p));
        Coord l = lo, r = hi;
        while (l + 1 < r) {
            Coord m = midpoint(l, r);
            int q = tr[p].ch[0];
            int left = size(q);
            if (k < left) {
                p = q;
                r = m;
            } else {
                k -= left;
                p = tr[p].ch[1];
                l = m;
            }
        }
```

```
        return l;
    }
    Coord kth_largest(int p, int k) const {
        assert(0 <= k && k < size(p));
        return kth(p, size(p) - 1 - k);
    }
    int rank(int p, Coord x) const { return count_less(p, x) + 1; }
    optional<Coord> predecessor(int p, Coord x) const {
        int k = count_less(p, x);
        if (!k) return nullopt;
        return kth(p, k - 1);
    }
    optional<Coord> successor(int p, Coord x) const {
        int k = count_leq(p, x);
        if (k == size(p)) return nullopt;
        return kth(p, k);
    }
    // ===== 合并：破坏性地将 b 合入 a，结束后 b = 0 =====
    int merge_node(int a, int b, Coord l, Coord r) {
        if (!a) return b;
        if (!b) return a;
        assert(a != b);
        if (l + 1 == r) {
            tr[a].cnt += tr[b].cnt;
            recycle_node(b);
            return a;
        }
        Coord m = midpoint(l, r);
        int lc = merge_node(tr[a].ch[0], tr[b].ch[0], l, m);
        int rc = merge_node(tr[a].ch[1], tr[b].ch[1], m, r);
        tr[a].ch[0] = lc;
        tr[a].ch[1] = rc;
        pull(a);
        recycle_node(b);
        return a;
    }
    void merge(int &a, int &b) {
        assert(!a || a != b);
        a = merge_node(a, b, lo, hi);
        b = 0;
    }
    // ===== 按值分裂：返回 { < x, ≥ x }，原根被消费并置 0 =====
    pair<int, int> split_value_node(int p, Coord l, Coord r, Coord x) {
        if (!p) return {};
        if (r <= x) return { p, 0 };
        if (x <= l) return { 0, p };
        Coord m = midpoint(l, r);
        int lc = tr[p].ch[0], rc = tr[p].ch[1];
        auto [a0, b0] = split_value_node(lc, l, m, x);
        auto [a1, b1] = split_value_node(rc, m, r, x);
        int q = new_node();
        tr[p].ch[0] = a0;
        tr[p].ch[1] = a1;
        pull(p);
        tr[q].ch[0] = b0;
        tr[q].ch[1] = b1;
        pull(q);
        p = normalize(p);
        q = normalize(q);
        return { p, q };
    }
    pair<int, int> split_by_value(int &p, Coord x) {
        auto res = split_value_node(p, lo, hi, x);
        p = 0;
        return res;
    }
    // ===== 按排名分裂：前 k 小 / 其余，k 表示元素数量 =====
    pair<int, int> split_rank_node(int p, Coord l, Coord r, int k) {
        if (!p) return {};
        if (k <= 0) return { 0, p };
        if (k >= size(p)) return { p, 0 };
```

```
        if (l + 1 == r) {
            int q = new_node();
            tr[q].cnt = tr[p].cnt - k;
            tr[p].cnt = k;
            return { p, q };
        }
        Coord m = midpoint(l, r);
        int lc = tr[p].ch[0], rc = tr[p].ch[1];
        int left = size(lc);
        int q = new_node();
        if (k < left) {
            auto [a, b] = split_rank_node(lc, l, m, k);
            tr[p].ch[0] = a;
            tr[p].ch[1] = 0;
            pull(p);
            tr[q].ch[0] = b;
            tr[q].ch[1] = rc;
            pull(q);
        } else if (k == left) {
            tr[p].ch[0] = lc;
            tr[p].ch[1] = 0;
            pull(p);
            tr[q].ch[0] = 0;
            tr[q].ch[1] = rc;
            pull(q);
        } else {
            auto [a, b] = split_rank_node(rc, m, r, k - left);
            tr[p].ch[0] = lc;
            tr[p].ch[1] = a;
            pull(p);
            tr[q].ch[0] = 0;
            tr[q].ch[1] = b;
            pull(q);
        }
        return { p, q };
    }
    pair<int, int> split_by_rank(int &p, int k) {
        assert(0 <= k && k <= size(p));
        auto res = split_rank_node(p, lo, hi, k);
        p = 0;
        return res;
    }
    // ===== 整棵删除：若不会主动丢弃集合，可删除 =====
    void clear_node(int p) {
        if (!p) return;
        int lc = tr[p].ch[0], rc = tr[p].ch[1];
        clear_node(lc);
        clear_node(rc);
        recycle_node(p);
    }
    void clear(int &p) {
        clear_node(p);
        p = 0;
    }
    int nodes() const { return int(tr.size()) - 1 - int(free_nodes.size()); }
};
```

成员变量

- `[lo, hi)`：整个整数权值域
- `tr`：共享节点池，`tr[0]` 为永久空节点
- `free_nodes`：已经释放、可以复用的节点编号
- `Node::cnt`：当前节点对应值域中的元素总数
- 每一个 `int root` 表示一个独立的权值多重集
  例如：

```
MergeableValueSegTree seg(0, 1'000'000'001LL);
vector<int> root(n);
```

此时 `root[u] == 0` 表示第 `u` 个集合为空

基础修改

单点加入一个元素：

```
seg.add(root, x);
```

加入多个：

```
seg.add(root, x, cnt);
```

删除一个：

```
seg.add(root, x, -1);
```

要求删除后频数不能为负
从一个 range / view 建立一棵树：

```
int root = seg.build(a);
```

也可以：

```
int root = seg.build(a | views::filter([](int x) {
    return x >= 0;
}));
```

顺序统计

集合大小：

```
int n = seg.size(root);
```

第 `k` 小，`k` 为 `0base`：

```
auto x = seg.kth(root, k);
```

第 `k` 大：

```
auto x = seg.kth_largest(root, k);
```

严格小于 `x` 的数量：

```
int cnt = seg.count_less(root, x);
```

不大于 `x` 的数量：

```
int cnt = seg.count_leq(root, x);
```

值域 `[l, r)` 中的数量：

```
int cnt = seg.count(root, l, r);
```

某个值出现次数：

```
int cnt = seg.frequency(root, x);
```

排名采用竞赛中常见定义：

```
int rk = seg.rank(root, x);
```

其含义为：

$$\#\{v \mid v < x\}$$

前驱、后继：

```
auto pre = seg.predecessor(root, x);
auto nxt = seg.successor(root, x);
```

不存在时返回 `nullopt`

合并

```
seg.merge(a, b);
```

执行后：

```
a = 原 a ∪ 原 b
b = 0
```

这里的并是**多重集相加**：
如果：

```
a: {1, 1, 3}
b: {1, 2}
```

合并后：

```
a: {1, 1, 1, 2, 3}
b: {}
```

这是破坏性合并，不是持久化操作

按值分裂

```
auto [a, b] = seg.split_by_value(root, x);
```

得到：

```
a: 所有 < x 的元素
b: 所有 ≥ x 的元素
root: 0
```

例如：

```
root = {1, 2, 2, 4, 7}
split_by_value(root, 4)
a = {1, 2, 2}
b = {4, 7}
```

这里 `x` 不要求当前集合中存在

按排名分裂

```
auto [a, b] = seg.split_by_rank(root, k);
```

得到：

```
a: 最小的 k 个元素
b: 剩余元素
root: 0
```

其中 `k` 表示**数量**，不是 `0base` 下标
例如：

```
root = {1, 2, 2, 2, 5}
k = 3
```

得到：

```
a = {1, 2, 2}
```

```
b = {2, 5}
```

因此如果切分点落在重复值内部，同一个权值可以同时存在于两棵结果树中
这也是按排名分裂与按值分裂最重要的区别

注意事项

所有权是这一份板子最重要的规则

每个非零节点只能被一棵当前有效的树拥有
因此不要写：

```
int b = a;
```

然后分别修改：

```
seg.add(a, x);
seg.add(b, y); // 错误
```

因为这只是复制了根编号，两棵“树”实际上共享同一批节点。之后 `merge / split / clear` 可能把另一棵树仍在使用的节点回收
如果需要：

```
一个版本复制成两个版本
两边以后互不影响
```

应该使用上一份可持久化线段树，而不是这一份

`merge()` 会消费第二棵树

```
seg.merge(a, b);
```

之后：

```
assert(b == 0);
```

不要保存并继续使用旧的 `b`
这种接口故意模拟了“move”语义，可以有效避免节点所有权混乱

`split()` 会消费原根

```
auto [a, b] = seg.split_by_value(root, x);
```

之后：

```
assert(root == 0);
```

同理：

```
auto [a, b] = seg.split_by_rank(root, k);
```

也会消费 `root`

删除到空时会自动回收整条无用路径

例如：

```
seg.add(root, x);
seg.add(root, x, -1);
```

如果这棵树原本只有一个 `x`，最终：

```
root == 0
```

沿途节点都会进入 `free_nodes`
因此不需要额外手动清理空节点

值域必须统一

同一个 `MergeableValueSegTree` 中的所有根天然拥有相同 `[lo, hi)`，因此可以直接合并
不要把不同对象中的根节点编号拿来混用

不支持 lazy

这里维护的是权值多重集，不是普通位置线段树
典型操作是：

```
插入 / 删除一个值
统计
kth
merge
split
```

如果需要区间加、区间赋值等懒标记操作，那已经不是这份结构最合适的用途

常见用法

树上子树权值集合合并

这是这份结构最经典的应用之一
每个节点先建立只包含自己权值的树：

```
MergeableValueSegTree seg(0, V);
vector<int> root(n);
for (int u : views::iota(0, n)) {
    seg.add(root[u], a[u]);
}
```

后序 DFS：

```
auto dfs = Y([&](auto self, int u, int fa) -> void {
    for (int v : g[u]) if (v != fa) {
        self(v, u);
        seg.merge(root[u], root[v]);
    }
    // 此时 root[u] = u 整棵子树的权值多重集
    if (seg.size(root[u]) >= k) {
        ans[u] = seg.kth_largest(root[u], k - 1);
    }
});
```

注意：

```
seg.merge(root[u], root[v]);
```

之后 `root[v] == 0`
因此必须在子节点答案已经统计完成以后再合并，正好符合树形 DP 的后序过程

提取一个值域

想把 `[L, R)` 整段从 `root` 中拿出来：

```
auto [left, ge_l] = seg.split_by_value(root, L);
auto [mid, right] = seg.split_by_value(ge_l, R);
```

此时：

```
left  = (-∞, L)
mid   = [L, R)
right = [R, +∞)
```

处理 `mid`：

```
// ...
```

重新拼回：

```
seg.merge(left, mid);
seg.merge(left, right);
root = left;
```

这其实把 `split_by_value()` 变成了一种很方便的权值区间剪切工具

从一个集合搬走最小的 `k` 个

```
auto [take, rest] = seg.split_by_rank(a, k);
a = rest;
seg.merge(b, take);
```

执行后，原来 `a` 中最小的 `k` 个元素被整体搬入 `b`

搬走最大的 `k` 个

设：

```
int n = seg.size(a);
```

那么：

```
auto [rest, take] = seg.split_by_rank(a, n - k);
a = rest;
seg.merge(b, take);
```

即可把最大的 `k` 个元素搬走

搬走某个值域

例如把 `a` 中 `[L, R)` 的所有元素移动到 `b`：

```
auto [left, ge_l] = seg.split_by_value(a, L);
auto [mid, right] = seg.split_by_value(ge_l, R);
a = left;
seg.merge(a, right);
seg.merge(b, mid);
```

这个套路在“集合之间批量转移满足某权值条件的元素”时非常方便

离散化后使用

如果真实权值范围很大，但所有可能值都能提前获得，可以先压成 `[0, m)`，缩短树高

```
vector<i64> xs = a;
ranges::sort(xs);
xs.erase(ranges::unique(xs).begin(), xs.end());
auto id = [&](i64 x) {
    return int(ranges::lower_bound(xs, x) - xs.begin());
};
MergeableValueSegTree<int> seg(0, xs.size());
```

插入：

```
seg.add(root, id(x));
```

查询第 `k` 小真实值：

```
i64 x = xs[seg.kth(root, k)];
```

如果需要提前收集修改值，也可以直接组合 view：

```
vector<i64> xs;
ranges::copy(
    queries
    | views::filter([](const auto &q) {
        return q.type == 1;
    })
    | views::transform([](const auto &q) {
```

```
        return q.x;
    }),
    back_inserter(xs)
);
```

然后排序去重即可

常见维护扩展：同时维护权值和

很多题不仅需要数量，还需要：

```
值域元素和
前 k 小元素和
前 k 大元素和
```

这时非常值得给 `Node` 再加一个 `sum`
例如：

```
struct Node {
    int ch[2]{};
    int cnt{};
    i64 sum{};
};
```

修改 `pull()`

```
i64 sum(int p) const {
    return p ? tr[p].sum : 0;
}
void pull(int p) {
    tr[p].cnt = size(tr[p].ch[0]) + size(tr[p].ch[1]);
    tr[p].sum = sum(tr[p].ch[0]) + sum(tr[p].ch[1]);
}
```

修改单点叶子

将：

```
tr[p].cnt += delta;
```

改为：

```
tr[p].cnt += delta;
tr[p].sum += i64(x) * delta;
```

修改 `merge()` 的叶子

```
tr[a].cnt += tr[b].cnt;
tr[a].sum += tr[b].sum;
```

内部节点仍然统一 `pull()` 即可

按排名分裂时叶子要额外处理

原来：

```
int q = new_node();
tr[q].cnt = tr[p].cnt - k;
tr[p].cnt = k;
```

需要改成：

```
int q = new_node();
int c = tr[p].cnt;
tr[p].cnt = k;
tr[p].sum = i64(l) * k;
tr[q].cnt = c - k;
tr[q].sum = i64(l) * (c - k);
```

因为两个结果树都可能在同一个权值叶子上保留若干个元素

前 `k` 小元素和

增加：

```
i64 sum_k_smallest(int p, int k) const {
    assert(0 <= k && k <= size(p));
    if (!k) return 0;
    i64 ans = 0;
    Coord l = lo, r = hi;
    while (l + 1 < r) {
        Coord m = midpoint(l, r);
        int q = tr[p].ch[0];
        int left = size(q);
        if (k <= left) {
            p = q;
            r = m;
        } else {
            ans += sum(q);
            k -= left;
            p = tr[p].ch[1];
            l = m;
        }
    }
    return ans + i64(l) * k;
}
```

于是前 `k` 大元素和可以直接写成：

```
i64 sum_k_largest(int p, int k) const {
    assert(0 <= k && k <= size(p));
    return sum(p) - sum_k_smallest(p, size(p) - k);
}
```

另一种常见维护：不同权值个数

如果题目需要：

```
当前集合有多少种不同权值
```

可以增加：

```
int distinct{};
```

内部节点：

```
tr[p].distinct = (tr[p].ch[0] ? tr[tr[p].ch[0]].distinct : 0) + (tr[p].ch[1] ? tr[tr[p].ch[1]].distinct : 0);
```

叶子：

```
tr[p].distinct = tr[p].cnt > 0;
```

`merge()` 的叶子累加频数后：

```
tr[a].distinct = 1;
```

`split_by_rank()` 如果把一个重复值拆到两边，则两个叶子的 `distinct` 都应为 `1`

复杂度

设整个整数权值域宽度为 W
树高为：

$$H = O(\log W)$$

若提前离散化后共有 m 个不同坐标，则：

$$H = O(\log m)$$

单点插入 / 删除

$$O(H)$$

最多访问一条根到叶路径

区间计数 / 第 k 小 / 第 k 大 / 排名 / 前驱 / 后继

$$O(H)$$

按值分裂

只需复制分界值经过的一条路径，因此：

$$O(H)$$

并最多额外产生：

$$O(H)$$

个节点

按排名分裂

同样只沿着第 kk 个元素所在路径递归：

$$O(H)$$

并最多额外产生：

$$O(H)$$

个节点

合并

设两棵树当前分别拥有 S_a, S_b 个有效节点
只有在两边对应节点都非空时才继续递归，因此单次最坏可以写为：

$$O(\min(S_a, S_b))$$

而且每个真正重叠的 `b` 节点在这次破坏性合并后都会被回收
在树形 DP 一类"子树只向父亲合并、源树之后不再使用"的场景中，这也是线段树合并能够非常高效的关键

清空一棵树

设该树当前有 S 个节点：

$$O(S)$$

空间

若进行了 q 次彼此独立的单点插入而没有任何回收，最多产生：

$$O(qH)$$

个节点

每次分裂额外产生：

$$O(H)$$

个节点
但 `merge()`、删除到空以及 `clear()` 都会把废弃节点加入 `free_nodes`，之后 `new_node()` 会优先复用，因此实际节点池大小由历史最大同时存活节点数主导，而不是所有操作累计产生的节点数

3.6.5 可持久化懒标记线段树

```
template<class Info, class Tag>
struct PersistentLazySegTree {
    struct Node {
        array<int, 2> ch{};
        Info info{};
        Tag tag{};
        bool has_tag{};
    };
    int n{};
    vector<Node> tr{ Node{} };
```

```
    vector<int> roots;
    explicit PersistentLazySegTree(int n_, const Info &v = Info{}, size_t reserve_nodes = 0) : n(n_) {
        assert(n > 0);
        if (reserve_nodes) tr.reserve(reserve_nodes + 1);
        roots.push_back(build_same(0, n, v));
    }
    template<ranges::input_range R>
    requires constructible_from<Info, ranges::range_reference_t<R>>
    explicit PersistentLazySegTree(R &&a, size_t reserve_nodes = 0) {
        vector<Info> init;
        if constexpr (ranges::sized_range<R>) init.reserve(ranges::size(a));
        for (auto &&x : a) init.emplace_back(forward<decltype(x)>(x));
        n = init.size();
        assert(n > 0);
        if (reserve_nodes) tr.reserve(reserve_nodes + 1);
        roots.push_back(build(0, n, init));
    }
    // ===== 节点创建与维护：所有修改依赖，不建议删除 =====
    int new_node(Node x = {}) {
        tr.push_back(move(x));
        return int(tr.size()) - 1;
    }
    int clone(int p) { return new_node(tr[p]); }
    void pull(int p) { tr[p].info = tr[tr[p].ch[0]].info + tr[tr[p].ch[1]].info; }
    // 当前 Tag 后再执行 v；Tag::apply(v) 必须遵守同一顺序
    static void append_tag(Tag &cur, bool &has, const Tag &v) {
        if (has) {
            cur.apply(v);
        } else {
            cur = v;
            has = true;
        }
    }
    // ===== 懒标记维护：push() 下传前必须复制共享儿子 =====
    void apply(int p, int l, int r, const Tag &v) {
        tr[p].info.apply(v, l, r);
        append_tag(tr[p].tag, tr[p].has_tag, v);
    }
    // 返回是否发生过下传；若返回 true，则两个儿子都已经是当前版本独占节点
    bool push(int p, int l, int r) {
        if (!tr[p].has_tag || r - l == 1) return false;
        int m = midpoint(l, r);
        Tag t = tr[p].tag;
        int lc = clone(tr[p].ch[0]), rc = clone(tr[p].ch[1]);
        tr[p].ch = { lc, rc };
        apply(lc, l, m, t);
        apply(rc, m, r, t);
        tr[p].tag = Tag{};
        tr[p].has_tag = false;
        return true;
    }
    // ===== 建树：若只保留一种初始化方式，可删除另一组 build =====
    int build_same(int l, int r, const Info &v) {
        int p = new_node();
        if (r - l == 1) {
            tr[p].info = v;
            return p;
        }
        int m = midpoint(l, r);
        int lc = build_same(l, m, v), rc = build_same(m, r, v);
        tr[p].ch = { lc, rc };
        pull(p);
        return p;
    }
    int build(int l, int r, const vector<Info> &a) {
        int p = new_node();
        if (r - l == 1) {
            tr[p].info = a[l];
            return p;
        }
        int m = midpoint(l, r);
```

```
        int lc = build(l, m, a), rc = build(m, r, a);
        tr[p].ch = { lc, rc };
        pull(p);
        return p;
    }
    // ===== 区间修改：可持久化 lazy 核心，不建议删除 =====
    void range_apply_owned(int p, int l, int r, int ql, int qr, const Tag &v) {
        if (ql <= l && r <= qr) {
            apply(p, l, r, v);
            return;
        }
        bool pushed = push(p, l, r);
        int m = midpoint(l, r);
        if (ql < m) {
            int c = tr[p].ch[0];
            if (!pushed) c = clone(c);
            range_apply_owned(c, l, m, ql, qr, v);
            tr[p].ch[0] = c;
        }
        if (m < qr) {
            int c = tr[p].ch[1];
            if (!pushed) c = clone(c);
            range_apply_owned(c, m, r, ql, qr, v);
            tr[p].ch[1] = c;
        }
        pull(p);
    }
    int range_apply(int ver, int l, int r, const Tag &v) {
        assert(0 <= ver && ver < versions());
        assert(0 <= l && l <= r && r <= n);
        if (l == r) return copy_version(ver);
        int q = clone(roots[ver]);
        range_apply_owned(q, 0, n, l, r, v);
        roots.push_back(q);
        return versions() - 1;
    }
    // ===== 单点修改：若不需要 set / 任意叶子修改，可整段删除 =====
    template<class F>
    void modify_owned(int p, int l, int r, int x, F &f) {
        if (r - l == 1) {
            tr[p].tag = Tag{};
            tr[p].has_tag = false;
            invoke(f, tr[p].info);
            return;
        }
        bool pushed = push(p, l, r);
        int m = midpoint(l, r);
        int d = x >= m;
        int c = tr[p].ch[d];
        if (!pushed) c = clone(c);
        modify_owned(c, d ? m : l, d ? r : m, x, f);
        tr[p].ch[d] = c;
        pull(p);
    }
    template<class F>
    int modify(int ver, int x, F &&f) {
        assert(0 <= ver && ver < versions());
        assert(0 <= x && x < n);
        int q = clone(roots[ver]);
        modify_owned(q, 0, n, x, f);
        roots.push_back(q);
        return versions() - 1;
    }
    int set(int ver, int x, const Info &v) {
        return modify(ver, x, [v](Info &a) {
            a = v;
        });
    }
    // ===== 只读查询：不 push、不复制节点 =====
    // up 表示尚未作用到当前节点的祖先标记。
    // 当前节点 tag 一定早于祖先尚未下传的 up，因此组合顺序为 tag → up。
```

```
    static pair<Tag, bool> next_tag(const Node &node, Tag up, bool has_up) {
        Tag next{};
        bool has_next = false;
        if (node.has_tag) append_tag(next, has_next, node.tag);
        if (has_up) append_tag(next, has_next, up);
        return { next, has_next };
    }
    Info query_node(int p, int l, int r, int ql, int qr, Tag up = {}, bool has_up = false) const {
        if (qr <= l || r <= ql) return Info{};
        if (ql <= l && r <= qr) {
            Info res = tr[p].info;
            if (has_up) res.apply(up, l, r);
            return res;
        }
        auto [next, has_next] = next_tag(tr[p], up, has_up);
        int m = midpoint(l, r);
        return query_node(tr[p].ch[0], l, m, ql, qr, next, has_next)
             + query_node(tr[p].ch[1], m, r, ql, qr, next, has_next);
    }
    Info query(int ver, int l, int r) const {
        assert(0 <= ver && ver < versions());
        assert(0 <= l && l <= r && r <= n);
        if (l == r) return Info{};
        return query_node(roots[ver], 0, n, l, r);
    }
    Info get(int ver, int x) const {
        assert(0 <= x && x < n);
        return query(ver, x, x + 1);
    }
    const Info &all(int ver) const {
        assert(0 <= ver && ver < versions());
        return tr[roots[ver]].info;
    }
    // ===== 查找接口 =====
    // pred 必须满足：pred(整段信息) == false 时，该段一定不存在答案
    template<class F>
    int find_first_node(int p, int l, int r, int ql, int qr, Tag up, bool has_up, F &pred) const {
        if (qr <= l || r <= ql) return -1;
        if (ql <= l && r <= qr) {
            Info cur = tr[p].info;
            if (has_up) cur.apply(up, l, r);
            if (!invoke(pred, cur)) return -1;
        }
        if (r - l == 1) return l;
        auto [next, has_next] = next_tag(tr[p], up, has_up);
        int m = midpoint(l, r);
        int res = find_first_node(tr[p].ch[0], l, m, ql, qr, next, has_next, pred);
        if (res == -1) {
            res = find_first_node(tr[p].ch[1], m, r, ql, qr, next, has_next, pred);
        }
        return res;
    }
    template<class F>
    int find_first(int ver, int l, int r, F &&pred) const {
        assert(0 <= ver && ver < versions());
        assert(0 <= l && l <= r && r <= n);
        if (l == r) return -1;
        return find_first_node(roots[ver], 0, n, l, r, Tag{}, false, pred);
    }
    template<class F>
    int find_last_node(int p, int l, int r, int ql, int qr, Tag up, bool has_up, F &pred) const {
        if (qr <= l || r <= ql) return -1;
        if (ql <= l && r <= qr) {
            Info cur = tr[p].info;
            if (has_up) cur.apply(up, l, r);
            if (!invoke(pred, cur)) return -1;
        }
        if (r - l == 1) return l;
        auto [next, has_next] = next_tag(tr[p], up, has_up);
        int m = midpoint(l, r);
        int res = find_last_node(tr[p].ch[1], m, r, ql, qr, next, has_next, pred);
```

```
        if (res == -1) {
            res = find_last_node(tr[p].ch[0], l, m, ql, qr, next, has_next, pred);
        }
        return res;
    }
    template<class F>
    int find_last(int ver, int l, int r, F &&pred) const {
        assert(0 <= ver && ver < versions());
        assert(0 <= l && l <= r && r <= n);
        if (l == r) return -1;
        return find_last_node(roots[ver], 0, n, l, r, Tag{}, false, pred);
    }
    // ===== 版本与空间管理 =====
    int copy_version(int ver) {
        assert(0 <= ver && ver < versions());
        roots.push_back(roots[ver]);
        return versions() - 1;
    }
    void reserve_nodes(size_t extra) { tr.reserve(tr.size() + extra); }
    int versions() const { return int(roots.size()); }
    int nodes() const { return int(tr.size()) - 1; }
};
```

接口说明

`Info / Tag` 约定

`Info` 需要支持：

```
Info operator+(const Info &a, const Info &b);
void Info::apply(const Tag &v, int l, int r);
```

其中 `Info{}` 必须是合并单位元
`Tag` 需要支持：

```
void Tag::apply(const Tag &v);
```

而且**组合顺序非常重要**：

```
tag.apply(v)
```

表示：

> 已经存在的 `tag` 先执行，再执行新的 `v`

也就是：

$$v \circ tag$$

这一约定不能反过来，否则赋值、仿射变换等非交换标记都会错

版本修改

`range_apply(ver, l, r, tag)`：从版本 `ver` 出发，对 `[l,r)` 施加标记并生成新版本
时间复杂度：

$$O(\log n)$$

新增节点数量：

$$O(\log n)$$

`modify(ver, x, f)`：从历史版本 `ver` 出发修改一个叶子并生成新版本

$$O(\log n)$$

`set(ver, x, info)` / `get(ver, x)`

$$O(\log n)$$

`copy_version(ver)`：生成内容完全相同的新版本，不复制任何树节点

$$O(1)$$

查询

`query(ver, l, r)`：查询历史版本 `ver` 的 `[l,r)`

$$O(\log n)$$

并且：

$$\text{新增节点数} = 0$$

查询单点

$$O(\log n)$$

`all(ver)`：查询整个版本的根信息

$$O(1)$$

查找

`find_first(ver, l, r, pred)`：寻找 `[l,r)` 中最靠左、其所在路径没有被 `pred` 剪掉的位置
`find_last()` 同理寻找最靠右位置
在常见能够有效剪枝的维护信息下：

$$O(\log n)$$

若 `pred` 不具有正确的区间剪枝性质，则不仅可能复杂度退化，逻辑本身也可能不成立

注意事

- 区间统一使用 `[l,r)`，下标为 `0ba生新分支
- `push()` 绝不能直接修改旧儿子。旧儿子可能仍被其他历史版本引用，所以必须先 `clone()`
- 查询不能使用普通 Lazy Segment Tree 那种破坏性的 `push()`。本板子通过携带“祖先尚未下传的标记”完成只读查询，因此查询不会产生节点
- `has_tag` 独立于 `Tag` 保存，避免强迫 `Tag{}` 必须能够判断“是否为空”。`Tag{}` 只要求可以默认构造
- `Info{}` 必须是 `operator+` 的单位元
- `set()` 和 `modify()` 到达叶子时会清除叶子的旧懒标记，因为叶子已经没有需要继续下传的子树
- 不建议加入节点回收：多个版本可能共同引用同一节点，普通 free list 无法判断一个节点是否仍被其他版本使用
- 如果题目只有单点修改而没有区间修改，应使用前一份普通可持久化线段树，代码和常数都会更小

常见用法

区间加 + 区间和

这是最简洁的一组 `Info / Tag`：

```
struct Tag {
    i64 add{};
    void apply(const Tag &t) {
        add += t.add;
    }
};
struct Info {
    i64 sum{};
    Info() = default;
    Info(i64 x) : sum(x) {}
    void apply(const Tag &t, int l, int r) {
        sum += t.add * (r - l);
    }
};
Info operator+(const Info &a, const Info &b) {
    return { a.sum + b.sum };
}
```

直接从数组或 view 建树：

```
PersistentLazySegTree<Info, Tag> seg(
    a | views::transform([](i64 x) {
        return Info(x);
    })
);
```

版本 `0` 加：

```
int v1 = seg.range_apply(0, l, r, Tag{ x });
```

继续从 `v1` 修改：

```
int v2 = seg.range_apply(v1, ql, qr, Tag{ y });
```

旧版本依旧可查：

```
i64 a0 = seg.query(0, l, r).sum;
i64 a1 = seg.query(v1, l, r).sum;
i64 a2 = seg.query(v2, l, r).sum;
```

同时维护 `sum / min / max`

```
struct Tag {
    i64 add{};
    void apply(const Tag &t) {
        add += t.add;
    }
};
struct Info {
    i64 sum{};
    i64 mn{ INF };
    i64 mx{ -INF };
    void apply(const Tag &t, int l, int r) {
        sum += t.add * (r - l);
        mn += t.add;
        mx += t.add;
    }
};
Info operator+(const Info &a, const Info &b) {
    return {
        a.sum + b.sum,
        min(a.mn, b.mn),
        max(a.mx, b.mx)
    };
}
```

初始化：

```
PersistentLazySegTree<Info, Tag> seg(
    a | views::transform([](i64 x) {
        return Info{ x, x, x };
    })
);
```

查询：

```
auto res = seg.query(ver, l, r);
i64 sum = res.sum;
i64 mn = res.mn;
i64 mx = res.mx;
```

找第一个不小于 `x` 的位置

维护 `mx` 后：

```
int p = seg.find_first(ver, l, r, [&](const Info &a) {
    return a.mx ≥ x;
});
```

最后一个不小于 `x`：

```
int p = seg.find_last(ver, l, r, [&](const Info &a) {
    return a.mx ≥ x;
});
```

这里 `mx < x` 能保证整段没有答案，所以满足剪枝要求。

区间赋值 + 区间加

这一组很适合检查 `Tag::apply()` 的组合方向

```
struct Tag {
    optional<i64> set;
    i64 add{};
    void apply(const Tag &t) {
        if (t.set) {
            set = t.set;
            add = t.add;
        } else {
            add += t.add;
        }
    }
};
struct Info {
    i64 sum{}, mn{ INF }, mx{ -INF };
    void apply(const Tag &t, int l, int r) {
        if (t.set) {
            i64 x = *t.set;
            sum = x * (r - l);
            mn = mx = x;
        }
        sum += t.add * (r - l);
        mn += t.add;
        mx += t.add;
    }
};
Info operator+(const Info &a, const Info &b) {
    return {
        a.sum + b.sum,
        min(a.mn, b.mn),
        max(a.mx, b.mx)
    };
}
```

区间赋值：

```
int nv = seg.range_apply(ver, l, r, Tag{ x, 0 });
```

区间加：

```
int nv = seg.range_apply(ver, l, r, Tag{ nullopt, x });
```

若已有：

```
先赋值 5
再加 3
```

最终必须等价于：

```
赋值 8
```

而：

```
先加 3
再赋值 5
```

最终必须等价于：

```
赋值 5
```

这就是为什么 `Tag::apply()` 的先后顺序不能随便写

仿射修改

若已经有取模类 `Z`，可以维护：

$$x \leftarrow ax + b$$

```
struct Tag {
    Z mul{ 1 }, add{};
    void apply(const Tag &t) {
        add = add * t.mul + t.add;
        mul *= t.mul;
    }
};
struct Info {
    Z sum{};
    void apply(const Tag &t, int l, int r) {
        sum = sum * t.mul + t.add * (r - l);
    }
};
Info operator+(const Info &a, const Info &b) {
    return { a.sum + b.sum };
}
```

这里若当前标记为：
`f(x)=ax+b`
新标记为：
$g(x) = cx + d$
那么组合后应该是：

$$g(f(x)) = acx + bc + d$$

所以对应代码正是：

```
add = add * t.mul + t.add;
mul *= t.mul;
```

这也是验证 `Tag::apply()` 是否写反的最好方法之一

分叉版本

```
int v1 = seg.range_apply(0, 0, 5, Tag{ 3 });
int v2 = seg.range_apply(v1, 2, 8, Tag{ 4 });
int v3 = seg.range_apply(v1, 1, 6, Tag{ 10 });
```

版本关系：

```
0
└── v1
    ├── v2
    └── v3
```

`v2` 和 `v3` 共享没有被修改的绝大多数节点，但之后彼此完全独立

每次操作都要求产生版本

修改：

```
seg.range_apply(ver, l, r, tag);
```

查询操作如果同样占一个版本：

```
auto ans = seg.query(ver, l, r);
seg.copy_version(ver);
```

于是始终可以保持：

```
操作编号 = 新版本编号
```

`reserve`

与普通可持久化线段树相比，这一版一次区间修改的节点常数会更

- 边界路径需要复制
- 遇到尚未下传的 Tag 时，两个儿子都必须复制
  所以若操作很多，建议提前：

```
seg.reserve_nodes(estimated_nodes);
```

不要机械地按普通主席树“一次修改只开一条链”的常数预估。

复杂度

设序列长度为 n
初始完整建树：

$$O(n)$$

初始节点数：

$$2n-1$$

单次区间修改 / 单点修改：

$$O(\log n)$$

单次区间修改新增节点数：

$$O(\log n)$$

查询整棵树 / 复制一个版本：

$$O(1)$$

若初始建树后进行了 q 次区间修改，总空间复杂度为：

$$O(n+q\log n)$$

版本根数组额外占用：

$$O(V)$$

其中v 为版本数量
对于满足正确区间剪枝条件的 `find_first / find_last`，通常为 $(\log n)$
若谓词不能有效剪枝，最坏可能退化到：

$$O(n)$$

> 修改时：要改共享儿子，先 clone
> 查询时：不要 push，把祖先 Tag 带下去

3.7 李超树

3.7.1 离散横坐标 Li Chao

```
template<class T = i64, bool MIN = true, class V = conditional_t<is_integral_v<T>, i128, T>>
struct LiChao {
    struct Line {
        T k{}, b{};
        V get(T x) const { return V(k) * V(x) + V(b); }
    };
    int n;
    vector<T> xs;
    vector<Line> line{{}};
    vector<int> tr;
    explicit LiChao(vector<T> x, int max_lines = 0) : xs(move(x)) {
        if constexpr (is_floating_point_v<T>) for (T x : xs) assert(isfinite(x));
        ranges::sort(xs);
        xs.erase(ranges::unique(xs).begin(), xs.end());
        n = int(xs.size());
        assert(n > 0);
        tr.assign(4 * n + 4, 0);
        if (max_lines > 0) line.reserve(max_lines + 1);
    }
private:
    // ===== 基础比较：插入与查询共同依赖，不建议删除 =====
    static V empty_value() {
        if constexpr (numeric_limits<V>::has_infinity) {
            return MIN ? numeric_limits<V>::infinity() : -numeric_limits<V>::infinity();
```

```
        } else {
            return MIN ? numeric_limits<V>::max() : numeric_limits<V>::lowest();
        }
    }
    V calc(int id, int x) const { return id ? line[id].get(xs[x]) : empty_value(); }
    V diff(int u, int v, int x) const {
        return (V(line[u].k) - V(line[v].k)) * V(xs[x]) + (V(line[u].b) - V(line[v].b));
    }
    bool better(int u, int v, int x) const {
        if (!u) return false;
        if (!v) return true;
        if constexpr (is_integral_v<T>) {
            V a = calc(u, x), b = calc(v, x);
            if (a != b) return MIN ? a < b : a > b;
        } else {
            V d{ diff(u, v, x) };
            if (d != V{}) return MIN ? d < V{} : d > V{};
        }
        return u < v;
    }
    int best(int u, int v, int x) const { return better(u, v, x) ? u : v; }
    int index(T x) const {
        auto it = ranges::lower_bound(xs, x);
        assert(it != xs.end() && *it == x);
        return int(it - xs.begin());
    }
    // ===== Li Chao 核心：只插完整直线 =====
    void insert(int p, int l, int r, int u) {
        int v{ tr[p] };
        if (!v) return void(tr[p] = u);
        if (line[u].k == line[v].k) {
            if (line[u].b != line[v].b) {
                bool take{ MIN ? line[u].b < line[v].b : line[u].b > line[v].b };
                if (take) tr[p] = u;
            } else if (u < v) {
                tr[p] = u;
            }
            return;
        }
        int m{ (l + r) >> 1 };
        if (better(u, v, m)) swap(u, v);
        tr[p] = v;
        if (r - l == 1) return;
        if (better(u, v, l)) insert(p << 1, l, m, u);
        else insert(p << 1 | 1, m, r, u);
    }
    int query_index(int x) const {
        int p{ 1 }, l{}, r{ n }, ans{};
        while (true) {
            ans = best(ans, tr[p], x);
            if (r - l == 1) return ans;
            int m{ (l + r) >> 1 };
            if (x < m) p <<= 1, r = m;
            else p = p << 1 | 1, l = m;
        }
    }
public:
    // ===== 基础接口 =====
    int add_line(T k, T b) {
        if constexpr (is_floating_point_v<T>) assert(isfinite(k) && isfinite(b));
        line.push_back({ k, b });
        int id{ int(line.size()) - 1 };
        insert(1, 0, n, id);
        return id;
    }
    int query_id(T x) const { return query_index(index(x)); }
    V query(T x) const {
        int p{ index(x) };
        return calc(query_index(p), p);
    }
    pair<V, int> query_info(T x) const {
```

```
        int p{ index(x) }, id{ query_index(p) };
        return { calc(id, p), id };
    }
    // ===== 重置：对象只使用一次时可删除 =====
    void clear() {
        ranges::fill(tr, 0);
        line.resize(1);
    }
};
```

模板参数

- `T`：横坐标、斜率、截距类型，默认 `i64`
- `MIN = true`：维护最小值；`MIN = false`：维护最大值
- `V`：函数值计算类型。整数 `T` 默认使用 `i128`，避免常见的 `k * x + b` 溢出

接口

- `LiChao(xs, max_lines = 0)`：对查询横坐标排序去重并建树，$O(n \log n)$
- `add_line(k, b)`：插入完整直线 `y = kx + b`，返回从 `1` 开始的直线编号，$O(\log n)$
- `query(x)`：返回 `x` 处最优值，要求 `x` 存在于构造时的 `xs` 中，$O(\log n)$
- `query_id(x)`：返回 `x` 处最优直线编号；没有直线时返回 `0`，$O(\log n)$
- `query_info(x)`：返回 `{最优值, 直线编号}`，只进行一次定位和树上查询，$O(\log n)$
- `clear()`：删除所有直线并保留离散横坐标，$O(n)$

注意事项

- 只支持**完整直线**。若需要区间有效直线，整数连续值域使用 `SegmentLiChao`，浮点离散横坐标使用 `DiscreteRealLiChao`
- 多条直线计算结果完全相同时返回编号较小者
- 内部直接比较两条直线之差 `(k1 - k2) * x + (b1 - b2)`；等斜率时直接比较截距

浮点版本

```
using ld = long double;
LiChao<ld, true, ld> seg(xs);   // 最小值
LiChao<ld, false, ld> seg2(xs); // 最大值
```

浮点时额外注意：

- `query(x)` 要求 `x` 与构造时某个 `xs[i]` 的 `long double` **实际值完全相同**。最稳妥的是保存全部查询横坐标后，之后继续用原来的 `xs[i]` 查询
- Li Chao 内部不要使用 `EPS` 判断斜率相同或函数值相同。非常接近但不相等的两条直线仍可能交换优劣
- `xs`、`k`、`b` 应为有限值，不要传入 `NaN` 或无穷
- 若所有浮点查询点已知但还需要区间插线，应改用 `DiscreteRealLiChao`，而不是给本结构增加区间接口

常见用法

整数最小值

```
vector<i64> xs = { 1, 3, 7, 10 };
LiChao seg(xs, m);
seg.add_line(k, b);
i128 ans = seg.query(7);
```

最大值

```
LiChao<i64, false> seg(xs);
seg.add_line(k, b);
auto [ans, id] = seg.query_info(x);
```

浮点横坐标

```
vector<ld> xs(q);
LiChao<ld, true, ld> seg(xs, m);
seg.add_line(k, b);
ld ans = seg.query(xs[i]);
```

复杂度

设离散横坐标数量为 `n`，插入直线数为 `L`

操作	时间复杂度
构造	$O(n \log n)$
`add_line()` / `query()` / `query_id()` / `query_info()`	$O(\log n)$
`clear()`	$O(n)$

空间复杂度：$O(n + L)$

3.7.2 整数连续域 Segment Li Chao

```
template<class T = i64, bool MIN = true, class V = conditional_t<is_integral_v<T>, i128, T>>
struct SegmentLiChao {
    static_assert(is_integral_v<T>);
    struct Line {
        T k{}, b{};
        V get(T x) const { return V(k) * V(x) + V(b); }
    };
    struct Node {
        int ch[2]{};
        int line{};
    };
    T xl, xr;
    int root{ 1 };
    vector<Line> line{{}};
    vector<Node> tr{{}, {}};
    explicit SegmentLiChao(T xl_, T xr_, int max_lines = 0, int max_nodes = 0) : xl(xl_), xr(xr_) {
        assert(xl < xr);
        if (max_lines > 0) line.reserve(max_lines + 1);
        if (max_nodes > 0) tr.reserve(max_nodes + 2);
    }
private:
    // ===== 基础比较与节点创建 =====
    static V empty_value() {
        if constexpr (numeric_limits<V>::has_infinity) {
            return MIN ? numeric_limits<V>::infinity() : -numeric_limits<V>::infinity();
        } else {
            return MIN ? numeric_limits<V>::max() : numeric_limits<V>::lowest();
        }
    }
    int new_node() {
        tr.push_back({});
        return int(tr.size()) - 1;
    }
    V calc(int id, T x) const { return id ? line[id].get(x) : empty_value(); }
    V diff(int u, int v, T x) const {
        return (V(line[u].k) - V(line[v].k)) * V(x) + (V(line[u].b) - V(line[v].b));
    }
    bool better(int u, int v, T x) const {
        if (!u) return false;
        if (!v) return true;
        if constexpr (is_integral_v<T>) {
            V a = calc(u, x), b = calc(v, x);
            if (a != b) return MIN ? a < b : a > b;
        } else {
            V d{ diff(u, v, x) };
            if (d != V{}) return MIN ? d < V{} : d > V{};
        }
        return u < v;
    }
    int best(int u, int v, T x) const { return better(u, v, x) ? u : v; }
    // ===== 整线插入核心：add_line() 与区间插线共同依赖 =====
    void insert(int p, T l, T r, int u) {
        int v{ tr[p].line };
        if (!v) return void(tr[p].line = u);
        if (line[u].k == line[v].k) {
            if (line[u].b != line[v].b) {
                bool take{ MIN ? line[u].b < line[v].b : line[u].b > line[v].b };
                if (take) tr[p].line = u;
```

```
            } else if (u < v) {
                tr[p].line = u;
            }
            return;
        }
        T m{ midpoint(l, r) };
        if (better(u, v, m)) swap(u, v);
        tr[p].line = v;
        if (r == l + 1) return;
        int d{ better(u, v, l) ? 0 : 1 };
        T ql{ d ? m : l }, qr{ d ? r : m };
        int c{ tr[p].ch[d] };
        if (!c) tr[p].ch[d] = c = new_node();
        insert(c, ql, qr, u);
    }
    // ===== 区间插线：若只插完整直线，可删除 update() 与三参数区间接口 =====
    int update(int p, T l, T r, T ql, T qr, int u) {
        if (qr <= l || r <= ql) return p;
        if (!p) p = new_node();
        if (ql <= l && r <= qr) {
            insert(p, l, r, u);
            return p;
        }
        T m{ midpoint(l, r) };
        int lc{ tr[p].ch[0] }, rc{ tr[p].ch[1] };
        lc = update(lc, l, m, ql, qr, u);
        rc = update(rc, m, r, ql, qr, u);
        tr[p].ch[0] = lc;
        tr[p].ch[1] = rc;
        return p;
    }
public:
    // ===== 基础接口 =====
    int add_line(T k, T b) {
        line.push_back({ k, b });
        int id{ int(line.size()) - 1 };
        insert(root, xl, xr, id);
        return id;
    }
    int add_line(T k, T b, T l, T r) {
        if (l > r) swap(l, r);
        line.push_back({ k, b });
        int id{ int(line.size()) - 1 };
        l = max(l, xl);
        r = min(r, xr);
        if (l < r) root = update(root, xl, xr, l, r, id);
        return id;
    }
    int query_id(T x) const {
        assert(xl <= x && x < xr);
        int p{ root }, ans{};
        T l{ xl }, r{ xr };
        while (p) {
            ans = best(ans, tr[p].line, x);
            if (r == l + 1) break;
            T m{ midpoint(l, r) };
            if (x < m) p = tr[p].ch[0], r = m;
            else p = tr[p].ch[1], l = m;
        }
        return ans;
    }
    V query(T x) const { return calc(query_id(x), x); }
    pair<V, int> query_info(T x) const {
        int id{ query_id(x) };
        return { calc(id, x), id };
    }
    // ===== 重置：对象只使用一次时可删除 =====
    void clear() {
        line.resize(1);
        tr.assign(2, {});
        root = 1;
```

```
    }
};
```

模板参数

- `T`：整数横坐标、斜率和截距类型，默认 `i64`；该版本要求 `T` 为整数类型
- `MIN = true`：维护最小值；`MIN = false`：维护最大值
- `V`：函数值计算类型，默认整数时使用 `i128`

接口

- `SegmentLiChao(xl, xr, max_lines = 0, max_nodes = 0)`：维护全部整数横坐标 `x ∈ [xl, xr)`，构造 $O(1)$
- `add_line(k, b)`：在整个值域插入 `y = kx + b`，返回直线编号，$O(\log C)$
- `add_line(k, b, l, r)`：只在整数横坐标 `[l, r)` 上生效；自动与总值域求交，返回直线编号，$O(\log^2 C)$
- `query(x)`：查询 `x` 处最优值，$O(\log C)$
- `query_id(x)`：查询最优直线编号；无覆盖时返回 `0`，$O(\log C)$
- `query_info(x)`：返回 `{最优值, 直线编号}`，$O(\log C)$
- `clear()`：清空所有动态节点和直线，保留值域，复杂度与已分配节点数线性相关
  其中 `C = xr - xl`

注意事项

- 总值域和区间插线都使用半开区间 `[l, r)`
- 若题目给出闭区间 `[l, r]` 且 `r + 1` 不会溢出，可写 `add_line(k, b, l, r + 1)`
- 采用动态开点，不需要为整个值域申请 `4 * C` 个节点，适合非常大的整数值域
- `new_node()` 会 `push_back()`，因此不要在调用它时长期保存 `tr[p]` 内元素的引用；`vector` 扩容后引用会失效
- 不提供“两整数端点自动生成线段”的接口。两整数端点确定的斜率未必是整数，直接用 `T` 相除会截断
- 多条直线值完全相同时返回编号较小者

常见用法

整个整数值域插线

```
SegmentLiChao seg(-1000000000LL, 1000000001LL, m);
seg.add_line(k, b);
i128 ans = seg.query(x);
```

闭区间 `[l, r]` 生效

```
seg.add_line(k, b, l, r + 1);
```

最大值

```
SegmentLiChao<i64, false> seg(L, R);
seg.add_line(k, b, l, r);
auto [ans, id] = seg.query_info(x);
```

复杂度

设整数值域长度为 `C = xr - xl`

操作	时间复杂度
`add_line(k, b)` / `query()` / `query_id()` / `query_info()`	$O(\log C)$
`add_line(k, b, l, r)`	$O(\log^2 C)$

动态节点空间最坏约为：

- 只插完整直线：$O(L \log C)$
- 含区间插线：$O(L \log^2 C)$

3.7.3 离散浮点横坐标 Discrete Real Li Chao

```
template<bool MIN = true>
struct DiscreteRealLiChao {
    static constexpr ld E = MIN ? numeric_limits<ld>::infinity() : -numeric_limits<ld>::infinity();
    struct Line {
```

```
        ld k{}, b{};
        ld get(ld x) const { return k * x + b; }
    };
    int n;
    vector<ld> xs;
    vector<Line> line{{}};
    vector<int> tr;
    explicit DiscreteRealLiChao(vector<ld> x, int max_lines = 0) : xs(move(x)) {
        for (ld x : xs) assert(isfinite(x));
        ranges::sort(xs);
        xs.erase(ranges::unique(xs).begin(), xs.end());
        n = int(xs.size());
        assert(n > 0);
        tr.assign(4 * n + 4, 0);
        if (max_lines > 0) line.reserve(max_lines + 1);
    }
private:
    // ===== 基础比较：插入与查询共同依赖 =====
    ld calc(int id, int x) const { return id ? line[id].get(xs[x]) : E; }
    ld diff(int u, int v, int x) const {
        return (line[u].k - line[v].k) * xs[x] + (line[u].b - line[v].b);
    }
    bool better(int u, int v, int x) const {
        if (!u) return false;
        if (!v) return true;
        ld d{ diff(u, v, x) };
        if (d != 0) return MIN ? d < 0 : d > 0;
        return u < v;
    }
    int best(int u, int v, int x) const { return better(u, v, x) ? u : v; }
    int index(ld x) const {
        auto it = ranges::lower_bound(xs, x);
        assert(it != xs.end() && *it == x);
        return int(it - xs.begin());
    }
    pair<int, int> range(ld l, ld r) const {
        if (l > r) swap(l, r);
        int ql{ int(ranges::lower_bound(xs, l) - xs.begin()) };
        int qr{ int(ranges::upper_bound(xs, r) - xs.begin()) };
        return { ql, qr };
    }
    // ===== 整线插入核心：完整直线与区间直线共同依赖 =====
    void insert(int p, int l, int r, int u) {
        int v{ tr[p] };
        if (!v) return void(tr[p] = u);
        if (line[u].k == line[v].k) {
            if (line[u].b != line[v].b) {
                bool take{ MIN ? line[u].b < line[v].b : line[u].b > line[v].b };
                if (take) tr[p] = u;
            } else if (u < v) {
                tr[p] = u;
            }
            return;
        }
        int m{ (l + r) >> 1 };
        if (better(u, v, m)) swap(u, v);
        tr[p] = v;
        if (r - l == 1) return;
        if (better(u, v, l)) insert(p << 1, l, m, u);
        else insert(p << 1 | 1, m, r, u);
    }
    // ===== 区间插线：若只插完整直线，可删除 update()、range() 与对应公开接口 =====
    void update(int p, int l, int r, int ql, int qr, int u) {
        if (qr <= l || r <= ql) return;
        if (ql <= l && r <= qr) return insert(p, l, r, u);
        int m{ (l + r) >> 1 };
        update(p << 1, l, m, ql, qr, u);
        update(p << 1 | 1, m, r, ql, qr, u);
    }
    int query_index(int x) const {
        int p{ 1 }, l{}, r{ n }, ans{};
```

```
        while (true) {
            ans = best(ans, tr[p], x);
            if (r - l == 1) return ans;
            int m{ (l + r) >> 1 };
            if (x < m) p <<= 1, r = m;
            else p = p << 1 | 1, l = m;
        }
    }
public:
    // ===== 基础接口 =====
    int add_line(ld k, ld b) {
        assert(isfinite(k) && isfinite(b));
        line.push_back({ k, b });
        int id{ int(line.size()) - 1 };
        insert(1, 0, n, id);
        return id;
    }
    // ===== 有限有效域：若只插完整直线，可删除以下两个插入接口 =====
    int add_line(ld k, ld b, ld l, ld r) {
        assert(isfinite(k) && isfinite(b) && isfinite(l) && isfinite(r));
        line.push_back({ k, b });
        int id{ int(line.size()) - 1 };
        auto [ql, qr] = range(l, r);
        if (ql < qr) update(1, 0, n, ql, qr, id);
        return id;
    }
    int add_seg(ld x0, ld y0, ld x1, ld y1) {
        assert(isfinite(x0) && isfinite(y0) && isfinite(x1) && isfinite(y1));
        if (x0 > x1) swap(x0, x1), swap(y0, y1);
        if (x0 == x1) {
            ld y{ MIN ? min(y0, y1) : max(y0, y1) };
            return add_line(0, y, x0, x0);
        }
        ld k{ (y1 - y0) / (x1 - x0) };
        return add_line(k, y0 - k * x0, x0, x1);
    }
    // ===== 查询 =====
    int query_id(ld x) const { return query_index(index(x)); }
    ld query(ld x) const {
        int p{ index(x) };
        return calc(query_index(p), p);
    }
    pair<ld, int> query_info(ld x) const {
        int p{ index(x) }, id{ query_index(p) };
        return { calc(id, p), id };
    }
    // ===== 重置：对象只使用一次时可删除 =====
    void clear() {
        ranges::fill(tr, 0);
        line.resize(1);
    }
};
```

模板参数

- `MIN = true`：维护最小值
- `MIN = false`：维护最大值

接口

- `DiscreteRealLiChao(xs, max_lines = 0)`：收集、排序、去重所有允许查询的浮点横坐标，$O(n \log n)$
- `add_line(k, b)`：在所有离散横坐标插入 `y = kx + b`，$O(\log n)$
- `add_line(k, b, l, r)`：只在满足 `l ≤ xs[i] ≤ r` 的离散点上生效，$O(\log^2 n)$
- `add_seg(x0, y0, x1, y1)`：插入两端点之间的有限线段，仅覆盖端点横坐标范围内的离散点，$O(\log^2 n)$
- `query(x)`：返回 `x` 处最优值，要求 `x` 精确存在于 `xs`，$O(\log n)$
- `query_id(x)`：返回最优插入对象编号；无覆盖时返回 `0`，$O(\log n)$
- `query_info(x)`：返回 `{最优值, 编号}`，$O(\log n)$
- `clear()`：清空所有已插入对象，保留离散横坐标，$O(n)$

注意事项

- `query(x)` 中的 `x` 必须与构造时某个 `xs[i]` 的 `long double` 实际值完全相同；不要重新通过另一套浮点计算构造一个“数学上应该相等”的值
- 内部区间树维护的是离散下标 `[0, n)`；`add_line(k, b, l, r)` 的用户语义仍是闭区间 `[l, r]`，通过 `lower_bound / upper_bound` 映射到离散下标半开区间
- 浮点 Li Chao 内部不使用 `EPS`。等斜率只在 `k` 实际相等时处理；直线比较使用实际 `long double` 结果
- `xs`、直线参数、线段端点均应为有限值。极端量级仍可能使浮点乘加上溢
- 多条候选值完全相同时返回编号较小者

竖直线段

若 `x0 == x1`，线段不是函数。模板把它解释为：

- 最小值版只在离散点 `x0` 上加入 `min(y0, y1)`
- 最大值版只在离散点 `x0` 上加入 `max(y0, y1)`
  如果 `x0` 不在 `xs` 中，这条竖直线段不会影响任何查询

与其他版本的关系

- 只插完整直线时，直接使用泛型 `LiChao<ld, MIN, ld>` 更短
- 浮点查询横坐标无法提前收集且只插完整直线时，使用 `RealLiChao`

常见用法

区间有效直线

```
DiscreteRealLiChao<> seg(xs, m);
seg.add_line(k, b, l, r);
ld ans = seg.query(xs[i]);
```

两端点有限线段

```
int id = seg.add_seg(x0, y0, x1, y1);
auto [ans, best_id] = seg.query_info(xs[i]);
```

最大值

```
DiscreteRealLiChao<false> seg(xs);
seg.add_line(k, b, l, r);
```

复杂度

设离散横坐标数量为 `n`，插入对象数为 `L`

操作	时间复杂度
构造	$O(n \log n)$
`add_line(k, b)` / 查询	$O(\log n)$
`add_line(k, b, l, r)` / `add_seg()`	$O(\log^2 n)$
`clear()`	$O(n)$

空间复杂度：$O(n + L)$

3.7.4 连续实数域 Real Li Chao

```
template<bool MIN = true, int DEPTH = 80>
struct RealLiChao {
    static_assert(DEPTH >= 0);
    static constexpr ld E = MIN ? numeric_limits<ld>::infinity() : -numeric_limits<ld>::infinity();
    struct Line {
        ld k{}, b{};
        ld get(ld x) const { return k * x + b; }
    };
    struct Node {
        int ch[2]{};
        int line{};
        vector<int> terminal; // 达到深度/浮点分辨率上限时保留候选，避免丢失答案
    };
    ld xl, xr;
```

```
    int root{ 1 };
    vector<Line> line{{}};
    vector<Node> tr{{}, {}};
    explicit RealLiChao(ld xl_, ld xr_, int max_lines = 0, int max_nodes = 0) : xl(xl_), xr(xr_) {
        assert(isfinite(xl) && isfinite(xr) && xl < xr);
        if (max_lines > 0) line.reserve(max_lines + 1);
        if (max_nodes > 0) tr.reserve(max_nodes + 2);
    }
private:
    // ===== 基础比较与节点创建：以下代码被全部操作依赖，不建议删除 =====
    int new_node() {
        tr.push_back({});
        return int(tr.size()) - 1;
    }
    ld calc(int id, ld x) const { return id ? line[id].get(x) : E; }
    ld diff(int u, int v, ld x) const {
        return (line[u].k - line[v].k) * x + (line[u].b - line[v].b);
    }
    bool better(int u, int v, ld x) const {
        if (!u) return false;
        if (!v) return true;
        ld d{ diff(u, v, x) };
        if (d != 0) return MIN ? d < 0 : d > 0;
        return u < v;
    }
    int best(int u, int v, ld x) const { return better(u, v, x) ? u : v; }
    // ===== 连续实数 Li Chao 核心：只插完整直线 =====
    void insert(int p, ld l, ld r, int dep, int u) {
        int v{ tr[p].line };
        if (!v) return void(tr[p].line = u);
        if (line[u].k == line[v].k) {
            if (line[u].b != line[v].b) {
                bool take{ MIN ? line[u].b < line[v].b : line[u].b > line[v].b };
                if (take) tr[p].line = u;
            } else if (u < v) {
                tr[p].line = u;
            }
            return;
        }
        ld m{ midpoint(l, r) };
        if (better(u, v, m)) swap(u, v);
        tr[p].line = v;
        if (dep == DEPTH || m == l || m == r) {
            tr[p].terminal.push_back(u);
            return;
        }
        int d{ better(u, v, l) ? 0 : 1 };
        ld ql{ d ? m : l }, qr{ d ? r : m };
        int c{ tr[p].ch[d] };
        if (!c) tr[p].ch[d] = c = new_node();
        insert(c, ql, qr, dep + 1, u);
    }
public:
    // ===== 基础接口 =====
    int add_line(ld k, ld b) {
        assert(isfinite(k) && isfinite(b));
        line.push_back({ k, b });
        int id{ int(line.size()) - 1 };
        insert(root, xl, xr, 0, id);
        return id;
    }
    int query_id(ld x) const {
        assert(isfinite(x) && xl <= x && x <= xr);
        int p{ root }, ans{}, dep{};
        ld l{ xl }, r{ xr };
        while (p) {
            ans = best(ans, tr[p].line, x);
            for (int id : tr[p].terminal) ans = best(ans, id, x);
            ld m{ midpoint(l, r) };
            if (dep == DEPTH || m == l || m == r) break;
            if (x <= m) p = tr[p].ch[0], r = m;
```

```
            else p = tr[p].ch[1], l = m;
            ++ dep;
        }
        return ans;
    }
    ld query(ld x) const { return calc(query_id(x), x); }
    pair<ld, int> query_info(ld x) const {
        int id{ query_id(x) };
        return { calc(id, x), id };
    }
    // ===== 重置：对象只使用一次时可删除 =====
    void clear() {
        line.resize(1);
        tr.assign(2, {});
        root = 1;
    }
};
```

接口说明

模板参数

- `MIN = true`：维护最小值；`MIN = false`：维护最大值
- `DEPTH`：最多二分层数，默认 `80`

接口

- `RealLiChao(xl, xr, max_lines = 0, max_nodes = 0)`：建立连续实数值域 `[xl, xr]`，$O(1)$
- `add_line(k, b)`：在整个实数值域插入 `y = kx + b`，返回直线编号，$O(DEPTH)$
- `query(x)`：查询任意 `x ∈ [xl, xr]` 的最优值，$O(DEPTH + K)$
- `query_id(x)`：返回最优直线编号；无直线时返回 `0`，$O(DEPTH + K)$
- `query_info(x)`：返回 `{最优值, 直线编号}`，$O(DEPTH + K)$
- `clear()`：清空所有直线和动态节点，保留值域

注意事项

- 连续实数没有天然叶子，因此使用 `DEPTH` 控制递归深度，并在 `midpoint(l, r)` 已经等于端点时提前停止
- 默认 `DEPTH = 80` 对常见 `ld` 数据范围通常足够；如果值域跨度极大且需要区分非常接近的查询位置，可以适当增大
- 内部不使用 `EPS` 判断斜率、交点或函数值相等
- `xl`、`xr`、查询横坐标、斜率、截距都应是有限值；极端数值仍可能使浮点乘加上溢
- 多条候选值实际计算完全相同时返回编号较小者

常见用法

连续实数最小值

```
RealLiChao<> seg(-1E9L, 1E9L, m);
seg.add_line(k, b);
ld ans = seg.query(x);
```

最大值

```
RealLiChao<false> seg(L, R);
seg.add_line(k, b);
auto [ans, id] = seg.query_info(x);
```

增加二分深度

```
RealLiChao<true, 120> seg(L, R);
```

复杂度

操作	时间复杂度
`add_line()`	$O(DEPTH)$
`query()` / `query_id()` / `query_info()`	$O(DEPTH + K)$

最坏空间复杂度：$O(L * DEPTH)$

深度截断节点保存未能继续下沉的候选直线，查询时逐条比较；因此调小 DEPTH 不会直接丢弃这些候选，但最坏查询可能退化为线性扫描。浮点运算仍受舍入误差影响，输入还须保证中间求值和比较不溢出。

这里 K 为本次查询路径末端实际扫描的候选数。保留候选是正确性要求，不以丢弃直线换速度；已知所有查询横坐标时优先用 3.7.1/3.7.3 的离散版，避免深度截断候选扫描。

3.7.5 可持久化整数域 Persistent Li Chao

```
template<class T = i64, bool MIN = true, class V = conditional_t<is_integral_v<T>, i128, T>>
struct PersistentLiChao {
    static_assert(is_integral_v<T>);
    struct Line {
        T k{}, b{};
        V get(T x) const { return V(k) * V(x) + V(b); }
    };
    struct Node {
        int ch[2]{};
        int line{};
    };
    T xl, xr;
    vector<Line> line{{}};
    vector<Node> tr{{}};
    vector<int> root{ 0 };
    explicit PersistentLiChao(T xl_, T xr_, int max_lines = 0, int max_nodes = 0) : xl(xl_), xr(xr_) {
        assert(xl < xr);
        if (max_lines > 0) {
            line.reserve(max_lines + 1);
            root.reserve(max_lines + 1);
        }
        if (max_nodes > 0) tr.reserve(max_nodes + 1);
    }
private:
    // ===== 基础比较与路径复制 =====
    static V empty_value() {
        if constexpr (numeric_limits<V>::has_infinity) {
            return MIN ? numeric_limits<V>::infinity() : -numeric_limits<V>::infinity();
        } else {
            return MIN ? numeric_limits<V>::max() : numeric_limits<V>::lowest();
        }
    }
    int clone(int p) {
        tr.push_back(p ? tr[p] : Node{});
        return int(tr.size()) - 1;
    }
    V calc(int id, T x) const { return id ? line[id].get(x) : empty_value(); }
    V diff(int u, int v, T x) const {
        return (V(line[u].k) - V(line[v].k)) * V(x) + (V(line[u].b) - V(line[v].b));
    }
    bool better(int u, int v, T x) const {
        if (!u) return false;
        if (!v) return true;
        if constexpr (is_integral_v<T>) {
            V a = calc(u, x), b = calc(v, x);
            if (a != b) return MIN ? a < b : a > b;
        } else {
            V d{ diff(u, v, x) };
            if (d != V{}) return MIN ? d < V{} : d > V{};
        }
        return u < v;
    }
    int best(int u, int v, T x) const { return better(u, v, x) ? u : v; }
    // ===== 持久化整线插入 =====
    int insert(int p, T l, T r, int u) {
        int q{ clone(p) }, v{ tr[q].line };
        if (!v) {
            tr[q].line = u;
            return q;
        }
        if (line[u].k == line[v].k) {
```

```
            if (line[u].b != line[v].b) {
                bool take{ MIN ? line[u].b < line[v].b : line[u].b > line[v].b };
                if (take) tr[q].line = u;
            } else if (u < v) {
                tr[q].line = u;
            }
            return q;
        }
        T m{ midpoint(l, r) };
        if (better(u, v, m)) swap(u, v);
        tr[q].line = v;
        if (r == l + 1) return q;
        int d{ better(u, v, l) ? 0 : 1 };
        T ql{ d ? m : l }, qr{ d ? r : m };
        tr[q].ch[d] = insert(tr[q].ch[d], ql, qr, u);
        return q;
    }
    // ===== 持久化区间插线：若只插完整直线，可删除 update() 与对应公开接口 =====
    int update(int p, T l, T r, T ql, T qr, int u) {
        if (qr <= l || r <= ql) return p;
        if (ql <= l && r <= qr) return insert(p, l, r, u);
        int q{ clone(p) };
        T m{ midpoint(l, r) };
        tr[q].ch[0] = update(tr[q].ch[0], l, m, ql, qr, u);
        tr[q].ch[1] = update(tr[q].ch[1], m, r, ql, qr, u);
        return q;
    }
    int query_root(int p, T x) const {
        int ans{};
        T l{ xl }, r{ xr };
        while (p) {
            ans = best(ans, tr[p].line, x);
            if (r == l + 1) break;
            T m{ midpoint(l, r) };
            if (x < m) p = tr[p].ch[0], r = m;
            else p = tr[p].ch[1], l = m;
        }
        return ans;
    }
public:
    // ===== 版本接口：每次插入创建一个新版本 =====
    int add_line(int ver, T k, T b) {
        assert(0 <= ver && ver < int(root.size()));
        line.push_back({ k, b });
        int id{ int(line.size()) - 1 };
        root.push_back(insert(root[ver], xl, xr, id));
        assert(id == int(root.size()) - 1);
        return id;
    }
    int add_line(int ver, T k, T b, T l, T r) {
        assert(0 <= ver && ver < int(root.size()));
        if (l > r) swap(l, r);
        line.push_back({ k, b });
        int id{ int(line.size()) - 1 };
        l = max(l, xl);
        r = min(r, xr);
        root.push_back(l < r ? update(root[ver], xl, xr, l, r, id) : root[ver]);
        assert(id == int(root.size()) - 1);
        return id;
    }
    int query_id(int ver, T x) const {
        assert(0 <= ver && ver < int(root.size()));
        assert(xl <= x && x < xr);
        return query_root(root[ver], x);
    }
    V query(int ver, T x) const { return calc(query_id(ver, x), x); }
    pair<V, int> query_info(int ver, T x) const {
        int id{ query_id(ver, x) };
        return { calc(id, x), id };
    }
    int versions() const { return int(root.size()); }
```

```
    // ===== 重置：对象只使用一次时可删除 =====
    void clear() {
        line.resize(1);
        tr.resize(1);
        root.assign(1, 0);
    }
};
```

接口说明

模板参数

- `T`：整数横坐标、斜率和截距类型，默认 `i64`
- `MIN = true`：维护最小值；`MIN = false`：维护最大值
- `V`：函数值类型，整数默认使用 `i128`

版本语义

- 版本 `0` 是空树
- 每次 `add_line()` 都创建一个新版本，不修改原版本
- 为了减少现场管理，本模板保持：**新版本编号 = 本次插入对象编号**
- 可以从任意历史版本继续插入，因此版本可以分叉

接口

- `PersistentLiChao(xl, xr, max_lines = 0, max_nodes = 0)`：维护整数值域 `[xl, xr)`
- `add_line(ver, k, b)`：从版本 `ver` 创建新版本并插入完整直线，返回新版本编号，$O(\log C)$
- `add_line(ver, k, b, l, r)`：从版本 `ver` 创建新版本，并令直线只在 `[l, r)` 生效，返回新版本编号，$O(\log^2 C)$
- `query(ver, x)`：查询版本 `ver` 在 `x` 处的最优值，$O(\log C)$
- `query_id(ver, x)`：返回该版本最优插入对象编号；无覆盖时为 `0`，$O(\log C)$
- `query_info(ver, x)`：返回 `{最优值, 对象编号}`，$O(\log C)$
- `versions()`：返回当前版本数量，包含版本 `0`，$O(1)$
- `clear()`：恢复为只有空版本 `0` 的状态
  其中 `C = xr - xl`

注意事项

- 总值域和区间插线均使用半开区间 `[l, r)`
- 从旧版本分叉完全合法：

```
int v1 = seg.add_line(0, k1, b1);
int v2 = seg.add_line(v1, k2, b2);
int v3 = seg.add_line(v1, k3, b3);
```

此时 `v2` 与 `v3` 分别继承 `v1`，互不影响

- 即使区间插线与总值域没有交集，也仍然创建一个新版本，只是新版本根与旧版本相同；这样可以继续保持"版本编号 = 插入对象编号"
- 路径复制仅复制发生修改的节点，旧版本节点始终不修改
- 多条直线值完全相同时返回编号较小者

浮点版本

这个版本与浮点的差异较大，**不能只把 `T` 改成 `ld`**。整数实现依赖：

```
r == l + 1
```

作为精确叶子，并依赖 `[l, r)` 的整数区间分解

浮点 `x` 可以提前收集

这是最推荐的持久化浮点方案：

```
收集全部会查询的 x
→ 排序去重得到 xs
→ 树上维护下标区间 [0, n)
→ 比较位置使用 xs[l] / xs[mid]
```

具体修改原则：

- `xl, xr` 删除，增加 `vector<ld> xs` 与 `n`
- `insert(p, l, r, u)` 中的 `l / m` 仍是整数下标，但 `better()` 使用 `xs[x]` 计算函数值

- 叶子条件改为 `r - l == 1`，这是离散下标的精确叶子
- 浮点区间 `[L, R]` 通过 `lower_bound(xs, L)` 与 `upper_bound(xs, R)` 映射成下标半开区间 `[ql, qr)`，再复用当前持久化 `update()`
- 查询时必须要求 `x` 精确存在于 `xs`，与 `DiscreteRealLiChao` 相同
  这样可以同时得到：

```
浮点横坐标
+ 持久化
+ 精确的离散区间有效域
```

这是需要区间插线时的首选

浮点 `x` 无法提前收集，只插完整直线

可以按 `RealLiChao` 修改成连续实数持久化：

```
T xl, xr       → ld xl, xr
T k, b         → ld k, b
V               → ld
整数叶子        → DEPTH / midpoint 停止条件
insert()        → 增加 dep 参数
```

停止条件改为：

```
ld m = midpoint(l, r);
if (dep == DEPTH || m == l || m == r) return q;
```

路径复制逻辑本身无需改变，因此完整直线仍可持久化

常见用法

链式建立历史版本

```
PersistentLiChao seg(L, R);
int v1 = seg.add_line(0, k1, b1);
int v2 = seg.add_line(v1, k2, b2);
i128 ans1 = seg.query(v1, x);
i128 ans2 = seg.query(v2, x);
```

从历史版本分叉

```
int v1 = seg.add_line(0, k1, b1);
int v2 = seg.add_line(v1, k2, b2);
int v3 = seg.add_line(v1, k3, b3);
```

持久化区间直线

```
int nv = seg.add_line(ver, k, b, l, r);
auto [ans, id] = seg.query_info(nv, x);
```

复杂度

设整数值域长度为 `C = xr - xl`

操作	时间复杂度	新增节点
完整直线	$O(\log C)$	$O(\log C)$
区间直线	$O(\log^2 C)$	$O(\log^2 C)$
查询	$O(\log C)$	$O(1)$

3.8 平衡树与序列维护

3.8.1 块状链表

解决问题： 维护动态序列，支持按位置访问、单点 / 批量插入、单点 / 区间删除。相比隐式平衡树实现更短，操作复杂度约为 $O(\sqrt{n})$

```
块状链表
    动态长度
    ↓
按“位置”分块
插入 / 删除 / 访问 / 批量编辑

ODT
    固定长度
    ↓
按“连续相等值”分段
区间赋值 / 整段变换 / 枚举常值段
```

```cpp
template<class T, int B = 700>
struct BlockList {
    static_assert(B >= 2);
    using Block = vector<T>;
    using Iter = typename list<Block>::iterator;
    using CIter = typename list<Block>::const_iterator;
    list<Block> blocks;
    int n{};
    explicit BlockList(const vector<T> &a = {}) { build(a); }
    int size() const { return n; }
    bool empty() const { return n == 0; }
private:
    // ===== 建块：构造和 rebuild() 依赖，不建议删除 =====
    void build(const vector<T> &a) {
        blocks.clear();
        n = a.size();
        for (int l = 0; l < n; l += B) {
            blocks.emplace_back(a.begin() + l, a.begin() + min(n, l + B));
        }
        if (blocks.empty()) {
            blocks.emplace_back();
        } else if (blocks.size() > 1 && int(blocks.back().size()) < B / 2) {
            auto last = prev(blocks.end());
            auto pre = prev(last);
            pre->insert(pre->end(),
                make_move_iterator(last->begin()),
                make_move_iterator(last->end()));
            blocks.erase(last);
        }
    }
    // ===== 定位：所有按位置操作依赖，不建议删除 =====
    pair<Iter, int> locate(int pos) {
        auto it = blocks.begin();
        while (next(it) != blocks.end() && pos >= int(it->size())) {
            pos -= it->size();
            ++ it;
        }
        return {it, pos};
    }
    pair<CIter, int> locate(int pos) const {
        auto it = blocks.begin();
        while (next(it) != blocks.end() && pos >= int(it->size())) {
            pos -= it->size();
            ++ it;
        }
        return {it, pos};
    }
    // ===== 块分裂与重平衡：插入 / 删除依赖，不建议删除 =====
    // 保证全局位置 pos 成为块边界，返回 pos 开始的块
    Iter split_at(int pos) {
        if (pos == n) return blocks.end();
        auto [it, k] = locate(pos);
        if (k == 0) return it;
        Block right(
            make_move_iterator(it->begin() + k),
            make_move_iterator(it->end())
        );
        it->erase(it->begin() + k, it->end());
```

```
        return blocks.insert(next(it), move(right));
    }
    Iter split_big(Iter it) {
        if (int(it->size()) <= 2 * B) return next(it);
        int k = it->size() / 2;
        Block right(
            make_move_iterator(it->begin() + k),
            make_move_iterator(it->end())
        );
        it->erase(it->begin() + k, it->end());
        return blocks.insert(next(it), move(right));
    }
    Iter rebalance(Iter it) {
        if (it == blocks.end()) return it;
        while (blocks.size() > 1) {
            if (it->empty()) {
                auto nxt = next(it);
                if (nxt != blocks.end()) {
                    blocks.erase(it);
                    it = nxt;
                } else {
                    auto pre = prev(it);
                    blocks.erase(it);
                    it = pre;
                }
                continue;
            }
            if (int(it->size()) > 2 * B) {
                split_big(it);
                return it;
            }
            if (int(it->size()) >= B / 2) return it;
            auto nxt = next(it);
            if (nxt != blocks.end()) {
                it->insert(it->end(),
                    make_move_iterator(nxt->begin()),
                    make_move_iterator(nxt->end()));
                blocks.erase(nxt);
            } else {
                auto pre = prev(it);
                pre->insert(pre->end(),
                    make_move_iterator(it->begin()),
                    make_move_iterator(it->end()));
                blocks.erase(it);
                it = pre;
            }
        }
        return it;
    }
    void fix_pos(int pos) {
        if (n == 0) return;
        pos = clamp(pos, 0, n - 1);
        auto [it, k] = locate(pos);
        rebalance(it);
    }
public:
    // ===== 基础接口：按位置访问 / 单点插入删除 =====
    T &operator[](int pos) {
        assert(0 <= pos && pos < n);
        auto [it, k] = locate(pos);
        return (*it)[k];
    }
    const T &operator[](int pos) const {
        assert(0 <= pos && pos < n);
        auto [it, k] = locate(pos);
        return (*it)[k];
    }
    void insert(int pos, T value) {
        assert(0 <= pos && pos <= n);
        auto [it, k] = locate(pos);
        it->insert(it->begin() + k, move(value));
```

```
        ++ n;
        split_big(it);
    }
    T erase(int pos) {
        assert(0 <= pos && pos < n);
        auto [it, k] = locate(pos);
        T value = move((*it)[k]);
        it->erase(it->begin() + k);
        -- n;
        if (n == 0) {
            blocks.clear();
            blocks.emplace_back();
        } else {
            rebalance(it);
        }
        return value;
    }
    // ===== 批量操作：若只需要单点插删，可以删除以下两个接口与 split_at() =====
    void insert(int pos, vector<T> values) {
        assert(0 <= pos && pos <= n);
        if (values.empty()) return;
        int m = values.size();
        auto at = split_at(pos);
        if (n == 0 && blocks.size() == 1 && blocks.front().empty()) {
            blocks.clear();
            at = blocks.end();
        }
        for (int l = 0; l < m; l += B) {
            int r = min(m, l + B);
            Block block(
                make_move_iterator(values.begin() + l),
                make_move_iterator(values.begin() + r)
            );
            blocks.insert(at, move(block));
        }
        n += m;
        if (pos > 0) fix_pos(pos - 1);
        fix_pos(pos);
        fix_pos(pos + m - 1);
        if (pos + m < n) fix_pos(pos + m);
    }
    void erase(int l, int r) {
        assert(0 <= l && l <= r && r <= n);
        if (l == r) return;
        auto last = split_at(r);
        auto first = split_at(l);
        blocks.erase(first, last);
        n -= r - l;
        if (n == 0) {
            blocks.clear();
            blocks.emplace_back();
            return;
        }
        if (l > 0) fix_pos(l - 1);
        if (l < n) fix_pos(l);
    }
    // ===== 遍历与重构：调试、输出或题目需要顺序扫描时保留 =====
    template<class F>
    void for_each(int l, int r, F f) const {
        assert(0 <= l && l <= r && r <= n);
        if (l == r) return;
        auto [it, k] = locate(l);
        int len = r - l;
        while (len) {
            int take = min(len, int(it->size()) - k);
            for (int i = 0; i < take; ++ i) f((*it)[k + i]);
            len -= take;
            ++ it;
            k = 0;
        }
    }
```

```
    vector<T> to_vector() const {
        vector<T> a;
        a.reserve(n);
        for (const auto &block : blocks) {
            a.insert(a.end(), block.begin(), block.end());
        }
        return a;
    }
    void rebuild() { build(to_vector()); }
};
```

接口说明

- `BlockList(a)`：由序列 `a` 构造
- `size()` / `empty()`：当前元素个数 / 是否为空，$O(1)$
- `operator[](pos)`：访问下标 `pos`，$O(n/B)$
- `insert(pos, value)`：在原下标 `pos` 前插入一个元素
- `erase(pos)`：删除并返回下标 `pos` 的元素
- `insert(pos, values)`：在 `pos` 前批量插入序列 `values`
- `erase(l, r)`：删除 `[l,r)`
- `for_each(l,r,f)`：顺序访问 `[l,r)` 中每个元素，不修改结构
- `to_vector()`：恢复完整连续序列
- `rebuild()`：重新按目标块长分块

块大小维护

结构大致维持：

```
B / 2 ≤ 块大小 ≤ 2B
```

边界和只有一个块时允许例外
删除后如果某块过小，就与相邻块合并；合并后过大再重新分裂。这一点很重要——参考资料中的块状链表同样专门强调需要合并，否则结构可能退化。

注意事项

- 所有下标均为 `0base`，区间统一为 `[l,r)`
- `insert(pos, ...)` 表示插在原 `pos` 之前；`pos == size()` 即尾插
- `B` 应取约 $\Theta(\sqrt{n})$。常见数据规模下 `500~1000` 通常都可以，再根据常数调整
- `operator[]` 只需要扫描块，不扫描块内元素，因此为 $O(n/B)$
- 单点插入 / 删除还需要移动块内 `vector`，因此有额外 $O(B)$
- 批量操作比逐个调用 `insert/erase` 更重要：完整块可以整体加入 / 删除
- 如果题目大量需要区间翻转、区间加、区间聚合等操作，通常更适合隐式 FHQ Treap，而不是继续给这个结构堆懒标记
- `rebuild()` 是 $O(n)$，不应每次操作后调用；它适合调试或非常特殊的周期性重构策略

常见用法

动态数组

```
BlockList<int> a({1, 2, 3, 4});
a.insert(2, 10);
// [1, 2, 10, 3, 4]
a.erase(1);
// [1, 10, 3, 4]
a[2] = 100;
// [1, 10, 100, 4]
```

批量插入和区间删除

这正是块状链表相对普通 `vector` 更值得使用的情况：

```
vector<int> b{5, 6, 7, 8};
a.insert(pos, b);
a.erase(l, r);
```

如果 `b` 后面不再使用，可以避免一次复制：

```
a.insert(pos, move(b));
```

文本编辑器

经典应用就是动态文本

```
BlockList<char> text;
vector<char> s(input.begin(), input.end());
text.insert(cursor, move(s));
text.erase(cursor, cursor + len);
text.for_each(cursor, cursor + len, [&](char c) {
    cout << c;
});
```

整体输出 / 对拍

```
vector<int> a = blocks.to_vector();
```

如果经过大量自定义操作，怀疑块分布不好，也可以：

```
blocks.rebuild();
```

特化：区间统计

块状链表本身没有维护区间信息。如果只是偶尔查询，可以直接遍历：

```
i64 sum = 0;
a.for_each(l, r, [&](int x) {
    sum += x;
});
```

如果这种区间查询非常频繁，就不应该继续用这个基础版本，应给块维护统计信息，或者直接换其他数据结构

复杂度

设当前长度为 n，目标块长为 B

操作	时间复杂度
构造	$O(n)$
`size / empty`	$O(1)$
`operator[]`	$O(n/B)$
单点 `insert / erase`	$O(n/B+B)$
批量插入 m 个元素	$O(n/B+B+m)$
删除 `[l,r)`	$O(n/B+B+k)$
`for_each(l,r)`	$O(n/B+r-l)$
`to_vector / rebuild`	$O(n)$

其中 k 为删除涉及的块数；对于有非平凡析构函数的 `T`，删除元素本身还要计入对应析构开销
取 $B=\Theta(\sqrt{n})$ 时，单点插删约为 $O(\sqrt{n})$

3.8.2 ODT（珂朵莉树）

```
template<class T, class Equal = equal_to<T>>
struct ODT {
    using Map = map<int, T>;
    using Iter = typename Map::iterator;
    int n{};
    Map mp;
    Equal equal;
    explicit ODT(const vector<T> &a = {}, Equal equal_ = Equal())
        : n(a.size()), equal(move(equal_)) {
        for (int i = 0; i < n; ++ i) {
            if (i == 0 || !same(a[i - 1], a[i])) {
                mp.emplace_hint(mp.end(), i, a[i]);
            }
        }
    }
private:
```

```
    bool same(const T &a, const T &b) const { return equal(a, b); }
    // ===== 相邻等值段合并：所有修改操作依赖，不建议删除 =====
    void normalize(int l, int r) {
        if (mp.empty()) return;
        auto it = mp.lower_bound(l);
        if (it != mp.begin()) --it;
        while (it != mp.end()) {
            auto nxt = next(it);
            if (nxt == mp.end() || nxt->first > r) break;
            if (same(it->second, nxt->second)) mp.erase(nxt);
            else it = nxt;
        }
    }
public:
    // ===== 核心操作：split / assign / apply =====
    // 保证 x 成为区间左端点；x == n 时返回 end()
    Iter split(int x) {
        assert(0 <= x && x <= n);
        if (x == n) return mp.end();
        auto it = prev(mp.upper_bound(x));
        if (it->first == x) return it;
        return mp.emplace_hint(next(it), x, it->second);
    }
    void assign(int l, int r, const T &value) {
        assert(0 <= l && l <= r && r <= n);
        if (l == r) return;
        auto last = split(r);
        auto first = split(l);
        mp.erase(first, last);
        mp.emplace_hint(last, l, value);
        normalize(l, r);
    }
    // f(T &) 必须把一个常值段整体映射为另一个常值
    template<class F>
    void apply(int l, int r, F f) {
        assert(0 <= l && l <= r && r <= n);
        if (l == r) return;
        auto last = split(r);
        auto first = split(l);
        for (auto it = first; it != last; ++it) f(it->second);
        normalize(l, r);
    }
    // ===== 只读查询：不会 split，不会制造额外断点 =====
    const T &get(int pos) const {
        assert(0 <= pos && pos < n);
        return prev(mp.upper_bound(pos))->second;
    }
    template<class F>
    void for_each(int l, int r, F f) const {
        assert(0 <= l && l <= r && r <= n);
        if (l == r) return;
        auto it = prev(mp.upper_bound(l));
        while (it != mp.end() && it->first < r) {
            auto nxt = next(it);
            int L = max(l, it->first);
            int R = min(r, nxt == mp.end() ? n : nxt->first);
            f(L, R, it->second);
            it = nxt;
        }
    }
    int interval_count() const { return mp.size(); }
};
```

表示方式

若：

```
mp = {
    {0, 2},
    {4, 7},
```

```
    {9, 3}
};
```

且 `n = 12`，表示：

```
[0, 4)  → 2
[4, 9)  → 7
[9, 12) → 3
```

核心接口

- `split(x)`：保证存在以 `x` 为左端点的段；`x == n` 返回 `end()`
- `assign(l,r,value)`：将 `[l,r)` 全部赋为 `value`
- `apply(l,r,f)`：对 `[l,r)` 覆盖到的每个常值段执行一次 `f(T&)`
- `get(pos)`：单点查询
- `for_each(l,r,f)`：枚举 `[l,r)` 内所有常值段，回调参数为 `(L,R,value)`
- `interval_count()`：当前常值段数量

`apply()` 的限制

例如：

```
odt.apply(l, r, [&](i64 &x) {
    x += delta;
});
```

是合法的，因为一个原本相等的整段修改后仍然相等
但不能用它表示：

```
a[i] += i
```

因为同一常值段内部不同位置会得到不同结果，原来的一个 ODT 节点不能继续表示整段

注意事项

- 所有下标为 `0base`，区间统一为 `[l,r)`
- ODT 维护的是定长数组的等值区间，不支持像 BlockList 那样插入 / 删除位置
- ODT 的性能核心不是“名字叫珂朵莉树”，而是当前等值段数量很少
- `assign()` 会大量删除断点，因此有大量区间赋值时通常表现很好
- 如果只有区间加而没有赋值，原有不同值段通常不会减少，ODT 很可能逐渐失去优势
- 对抗数据可以让段数达到 $\Theta(n)$，一次操作也可能退化到 $\Theta(n)$
- 每次修改后都会尝试合并相邻等值段，以尽量压低段数
- `Equal` 只负责判断两个值是否相等，所以基础 ODT 不要求 `T` 可以排序
- 如果只需要区间赋值和遍历，不需要数值运算，`T` 可以是颜色、状态甚至结构体

常见用法

区间赋值

ODT 最核心、也是最适合它的操作：

```
ODT<int> odt(a);
odt.assign(l, r, color);
```

区间加

区间加只是 `apply()` 的一个特例，没有必要永久写进核心结构：

```
odt.apply(l, r, [&](i64 &x) {
    x += delta;
});
```

如果题目大量使用，可以临时包装：

```
auto add = [&](int l, int r, i64 x) {
    odt.apply(l, r, [&](i64 &v) {
        v += x;
```

```
    });
};
```

区间和

无需写进 ODT：

```
i64 sum = 0;
odt.for_each(l, r, [&](int L, int R, i64 value) {
    sum += value * (R - L);
});
```

区间某个值出现次数

```
int cnt = 0;
odt.for_each(l, r, [&](int L, int R, int value) {
    if (value == x) cnt += R - L;
});
```

区间最值

```
i64 mn = INF;
i64 mx = -INF;
odt.for_each(l, r, [&](int L, int R, i64 value) {
    mn = min(mn, value);
    mx = max(mx, value);
});
```

区间第 k 小

这是经典 ODT 题型，但依赖 `T` 可以排序，因此更适合放在常用方法中，而不是让核心 ODT 强制要求 `<` `k` 为 `0base`：

```
vector<pair<i64, int>> a;
odt.for_each(l, r, [&](int L, int R, i64 value) {
    a.push_back({value, R - L});
});
ranges::sort(a);
for (auto [value, cnt] : a) {
    if (k < cnt) {
        answer = value;
        break;
    }
    k -= cnt;
}
```

如果题目频繁使用，可以包装成：

```
auto kth = [&](int l, int r, int k) {
    vector<pair<i64, int>> a;
    odt.for_each(l, r, [&](int L, int R, i64 value) {
        a.push_back({value, R - L});
    });
    ranges::sort(a);
    for (auto [value, cnt] : a) {
        if (k < cnt) return value;
        k -= cnt;
    }
    assert(false);
    return i64{};
};
```

区间幂次和

经典珂朵莉树题常见：

$$\sum_{i=l}^{r-1} a_i^k \pmod{mod}$$

无需给 ODT 增加特化接口：

```
i64 ans = 0;
odt.for_each(l, r, [&](int L, int R, i64 value) {
    ans = (ans + i64(R - L) * qpow(value, k, mod)) % mod;
});
```

自定义值类型

例如维护颜色和附加信息，只做整体赋值：

```
struct Info {
    int color{};
    i64 value{};
    bool operator==(const Info &) const = default;
};
ODT<Info> odt(a);
odt.assign(l, r, {color, value});
```

甚至不需要定义 `<`

观察是否开始退化

调试时很实用：

```
cerr << odt.interval_count() << '\n';
```

如果本来以为只有几十个段，结果迅速膨胀到接近 `n`，就说明这道题并不适合 ODT

复杂度

设当前共有 m 个等值段，一次操作实际覆盖 k 个段

操作	时间复杂度
`split` / `get`	$O(\log m)$
`assign` / `apply` / `for_each`	$O(\log m + k)$
`interval_count`	$O(1)$

空间复杂度为 $O(m)$，最坏 $m = O(n)$
特化查询则另外计算自身开销。例如区间第 `k` 小若覆盖 k_s 个常值段，需要：

$$O(\log m + k_s \log k_s)$$

构造时断点按下标递增，使用 end() 插入提示，建表 O(n)；split/assign 复用已找到的相邻迭代器作提示，避免第二次树搜索。区间操作的最坏复杂度仍取决于遍历段数，不保证随机数据以外的快速表现。

3.8.3 FHQ Treap 有序多重集

```
template<class T, class Compare = less<T>>
struct FHQTreap {
    struct Node {
        int ch[2]{}, size{}, cnt{};
        unsigned pri{};
        T value{};
    };
    Compare comp;
    mt19937 rng;
    vector<Node> tr{Node{}};
    vector<int> bin;
    int root{};
    explicit FHQTreap(
        unsigned seed = unsigned(chrono::steady_clock::now().time_since_epoch().count()),
        Compare comp_ = Compare()
    ) : comp(move(comp_)), rng(seed) {}
    explicit FHQTreap(
        vector<T> a,
        unsigned seed = unsigned(chrono::steady_clock::now().time_since_epoch().count()),
        Compare comp_ = Compare()
    ) : comp(move(comp_)), rng(seed) {
```

```
        tr.reserve(a.size() + 1);
        build(move(a));
    }
private:
    int node_size(int p) const { return tr[p].size; }
    bool equal(const T &a, const T &b) const { return !comp(a, b) && !comp(b, a); }
    // ===== 节点创建与维护：所有修改操作依赖，不建议删除 =====
    int new_node(const T &value, int cnt = 1) {
        int p;
        if (bin.empty()) {
            p = tr.size();
            tr.push_back({});
        } else {
            p = bin.back();
            bin.pop_back();
        }
        tr[p] = Node{{}, cnt, cnt, unsigned(rng()), value};
        return p;
    }
    void recycle(int p) {
        tr[p] = Node{};
        bin.push_back(p);
    }
    void pull(int p) {
        tr[p].size = node_size(tr[p].ch[0]) + tr[p].cnt + node_size(tr[p].ch[1]);
    }
    // ===== 批量建树：若只从空树逐个 insert，可删除该函数与 vector 构造函数 =====
    void build(vector<T> a) {
        if (a.empty()) return;
        ranges::sort(a, comp);
        vector<int> st;
        st.reserve(a.size());
        for (int l = 0, r; l < a.size(); l = r) {
            for (r = l + 1; r < a.size() && equal(a[l], a[r]); ++ r) {}
            int x = new_node(a[l], r - l), last = 0;
            while (!st.empty() && tr[st.back()].pri > tr[x].pri) {
                last = st.back();
                st.pop_back();
            }
            tr[x].ch[0] = last;
            if (!st.empty()) tr[st.back()].ch[1] = x;
            st.push_back(x);
        }
        root = st.front();
        vector<int> order{root};
        order.reserve(tr.size() - 1);
        for (int i = 0; i < order.size(); ++ i) {
            int p = order[i];
            if (tr[p].ch[0]) order.push_back(tr[p].ch[0]);
            if (tr[p].ch[1]) order.push_back(tr[p].ch[1]);
        }
        for (int p : order | views::reverse) pull(p);
    }
    // ===== FHQ 核心：insert / erase / unite 依赖，不建议删除 =====
    // < value | ≥ value
    pair<int, int> split_less(int p, const T &value) {
        if (!p) return {0, 0};
        if (comp(tr[p].value, value)) {
            auto [a, b] = split_less(tr[p].ch[1], value);
            tr[p].ch[1] = a;
            pull(p);
            return {p, b};
        }
        auto [a, b] = split_less(tr[p].ch[0], value);
        tr[p].ch[0] = b;
        pull(p);
        return {a, p};
    }
    // ≤ value | > value
    pair<int, int> split_leq(int p, const T &value) {
        if (!p) return {0, 0};
```

```
        if (!comp(value, tr[p].value)) {
            auto [a, b] = split_leq(tr[p].ch[1], value);
            tr[p].ch[1] = a;
            pull(p);
            return {p, b};
        }
        auto [a, b] = split_leq(tr[p].ch[0], value);
        tr[p].ch[0] = b;
        pull(p);
        return {a, p};
    }
    // 要求 a 中所有键严格小于 b 中所有键
    int merge(int a, int b) {
        if (!a || !b) return a | b;
        if (tr[a].pri < tr[b].pri) {
            tr[a].ch[1] = merge(tr[a].ch[1], b);
            pull(a);
            return a;
        }
        tr[b].ch[0] = merge(a, tr[b].ch[0]);
        pull(b);
        return b;
    }
    bool erase_node(int &p, const T &value) {
        if (!p) return false;
        if (comp(value, tr[p].value)) {
            bool ok = erase_node(tr[p].ch[0], value);
            if (ok) pull(p);
            return ok;
        }
        if (comp(tr[p].value, value)) {
            bool ok = erase_node(tr[p].ch[1], value);
            if (ok) pull(p);
            return ok;
        }
        if (tr[p].cnt > 1) {
            -- tr[p].cnt;
            pull(p);
        } else {
            int q = p;
            p = merge(tr[p].ch[0], tr[p].ch[1]);
            recycle(q);
        }
        return true;
    }
public:
    // ===== 基础接口：默认操作 root；传入 int &rt 可在同一节点池维护多棵树 =====
    int size(int p) const { return node_size(p); }
    int size() const { return size(root); }
    bool empty(int p) const { return p == 0; }
    bool empty() const { return root == 0; }
    void reserve(int nodes) {
        tr.reserve(nodes + 1);
        bin.reserve(nodes);
    }
    void clear() {
        tr.resize(1);
        tr[0] = Node{};
        bin.clear();
        root = 0;
    }
    void insert(int &p, const T &value) {
        auto [a, c] = split_less(p, value);
        auto [b, d] = split_leq(c, value);
        if (b) {
            ++ tr[b].cnt;
            pull(b);
        } else {
            b = new_node(value);
        }
        p = merge(merge(a, b), d);
```

```
    }
    void insert(const T &value) { insert(root, value); }
    bool erase(int &p, const T &value) { return erase_node(p, value); }
    bool erase(const T &value) { return erase(root, value); }
    // ===== 只读查询：不 split，不改变树形 =====
    int count(int p, const T &value) const {
        while (p) {
            if (comp(value, tr[p].value)) p = tr[p].ch[0];
            else if (comp(tr[p].value, value)) p = tr[p].ch[1];
            else return tr[p].cnt;
        }
        return 0;
    }
    int count(const T &value) const { return count(root, value); }
    bool contains(int p, const T &value) const { return count(p, value) != 0; }
    bool contains(const T &value) const { return contains(root, value); }
    // 严格小于 value 的元素个数
    int order_of_key(int p, const T &value) const {
        int ans = 0;
        while (p) {
            if (comp(tr[p].value, value)) {
                ans += node_size(tr[p].ch[0]) + tr[p].cnt;
                p = tr[p].ch[1];
            } else {
                p = tr[p].ch[0];
            }
        }
        return ans;
    }
    int order_of_key(const T &value) const { return order_of_key(root, value); }
    // k 为 0base
    optional<T> kth(int p, int k) const {
        if (k < 0 || k >= node_size(p)) return nullopt;
        while (p) {
            int left = node_size(tr[p].ch[0]);
            if (k < left) {
                p = tr[p].ch[0];
            } else if (k < left + tr[p].cnt) {
                return tr[p].value;
            } else {
                k -= left + tr[p].cnt;
                p = tr[p].ch[1];
            }
        }
        return nullopt;
    }
    optional<T> kth(int k) const { return kth(root, k); }
    // 严格前驱：最大的 < value
    optional<T> predecessor(int p, const T &value) const {
        int ans = 0;
        while (p) {
            if (comp(tr[p].value, value)) {
                ans = p;
                p = tr[p].ch[1];
            } else {
                p = tr[p].ch[0];
            }
        }
        if (!ans) return nullopt;
        return tr[ans].value;
    }
    optional<T> predecessor(const T &value) const { return predecessor(root, value); }
    // 严格后继：最小的 > value
    optional<T> successor(int p, const T &value) const {
        int ans = 0;
        while (p) {
            if (comp(value, tr[p].value)) {
                ans = p;
                p = tr[p].ch[0];
            } else {
                p = tr[p].ch[1];
```

```
            }
        }
        if (!ans) return nullopt;
        return tr[ans].value;
    }
    optional<T> successor(const T &value) const { return successor(root, value); }
    // ===== Treap Union: 若不需要任意两个有序多重集合并，可直接删除 =====
    // a、b 必须属于同一节点池且互不共享节点；调用后旧 a、b 不再独立使用
    int unite(int a, int b) {
        if (!a || !b) return a | b;
        if (tr[a].pri > tr[b].pri) swap(a, b);
        auto [l, ge] = split_less(b, tr[a].value);
        auto [eq, r] = split_leq(ge, tr[a].value);
        if (eq) {
            tr[a].cnt += tr[eq].cnt;
            recycle(eq);
        }
        tr[a].ch[0] = unite(tr[a].ch[0], l);
        tr[a].ch[1] = unite(tr[a].ch[1], r);
        pull(a);
        return a;
    }
};
```

接口说明

默认单棵树

- `FHQTreap<T> tr`：构造空有序多重集
- `FHQTreap<T> tr(a)`：由初始序列构造；先排序并压缩相同键
- `insert(value)`：插入一个值，期望 $O(\log d)$
- `erase(value)`：删除一个值；不存在返回 `false`，期望 $O(\log d)$
- `count(value)`：该值出现次数，期望 $O(\log d)$
- `contains(value)`：是否存在，期望 $O(\log d)$
- `order_of_key(value)`：严格小于 `value` 的元素个数，期望 $O(\log d)$
- `kth(k)`：`0base` 第 `k` 小；越界返回 `nullopt`，期望 $O(\log d)$
- `predecessor(value)`：严格前驱，期望 $O(\log d)$
- `successor(value)`：严格后继，期望 $O(\log d)$
- `size()` / `empty()`：元素总数 / 是否为空
- `reserve(nodes)`：预留节点池空间
- `clear()`：清空当前对象并保留 `vector` 容量

这里 d 表示**不同键的数量**，而不是元素总数

重复值

与旧版相比，这版不再让每个重复值占一个 Treap 节点，而是：

```
Node::cnt
```

记录该键的出现次数
因此：

```
{5, 5, 5, 5}
```

只占一个节点：

```
value = 5
cnt   = 4
size  = 4
```

这在重复值很多时明显更省空间，也让 `Treap Union` 合并重复键更加自然

同一节点池维护多棵树

所有常用接口都有接受根编号的版本：

```
int rt = 0;
tr.insert(rt, x);
tr.erase(rt, x);
```

```
tr.order_of_key(rt, x);
tr.kth(rt, k);
tr.predecessor(rt, x);
tr.successor(rt, x);
```

因此一个 `FHQTreap` 对象可以作为**共享节点池**：

```
FHQTreap<int> tr;
vector<int> root(n);
```

每个 `root[i]` 都可以表示一棵独立 Treap
这正是 `unite()` 最实用的使用方式

Treap Union / `unite`

接口

```
int c = tr.unite(a, b);
```

与普通：

```
merge(a, b)
```

完全不同
`merge(a,b)` 要求：

```
a 中所有键 < b 中所有键
```

而：

```
unite(a, b)
```

没有键域顺序限制
例如：

```
A = {1, 3, 3, 8}
B = {2, 3, 7}

unite(A, B)

⇒ {1, 2, 3, 3, 3, 7, 8}
```

相同键直接累加 `cnt`

使用方式

```
FHQTreap<int> tr;
vector<int> rt(n);
for (int i = 0; i < n; ++ i) {
    tr.insert(rt[i], a[i]);
}
rt[u] = tr.unite(rt[u], rt[v]);
rt[v] = 0;
```

调用后：

```
rt[u]
```

拥有两棵树中的全部元素
原来的：

```
rt[u] / rt[v]
```

不能再作为两棵独立树同时继续使用
尤其应当写：

```
rt[u] = tr.unite(rt[u], rt[v]);
rt[v] = 0;
```

而不要保留 `rt[v]`

适用场景

比较典型的是：

```
树上子树信息合并
DSU / 连通块合并
每个节点维护一个有序多重集
分治过程中合并若干集合
```

例如：

```
for (int v : g[u]) {
    dfs(v, u);
    rt[u] = tr.unite(rt[u], rt[v]);
    rt[v] = 0;
}
```

合并后可以直接查询：

```
auto x = tr.kth(rt[u], k);
int cnt = tr.order_of_key(rt[u], value);
```

注意事项

- 当前版本使用**每个不同键一个节点 + `cnt` 记录重数**。`size` 统计元素总数，而节点数统计不同键数量
- `Compare` 的等价关系是：

```
!comp(a, b) && !comp(b, a)
```

两个在 `Compare` 意义下等价的对象会被压缩到同一个节点

- `kth()` 使用 `0base`
- 常用题目中的 `1base` 排名为：

```
tr.order_of_key(x) + 1
```

- `predecessor()` / `successor()` 均为**严格**前驱 / 后继
- 查询均为纯遍历，不会调用 `split()`，所以查询不会改变 Treap 树形
- `split_size()` 已经删除。权值 Treap 不需要按中序前 `k` 个节点破坏性分裂；而且当前节点带有 `cnt`，按元素数量分裂可能需要把一个节点的 `cnt` 拆成两份。按排名分裂更适合放在隐式 Treap 中
- `merge(a,b)` 只用于键域严格有序的两棵树；任意集合合并必须使用 `unite(a,b)`
- `unite(a,b)` 要求两棵树：
  - 来自**同一个 `FHQTreap` 对象**
  - 节点集合互不共享
  - 调用后视为消费掉旧的 `a,b`
- `unite()` 不是持久化操作，会直接修改两棵输入树
- 节点被真正删除或 `unite()` 中出现重复键时会进入 `bin`，后续插入复用
- 随机优先级碰撞不影响正确性；树高期望为 $O(\log d)$，理论最坏仍可能达到 $O(d)$

常见用法

六种经典平衡树操作

```
FHQTreap<int> tr;
tr.insert(x);
tr.erase(x);
int rank = tr.order_of_key(x) + 1;
if (auto v = tr.kth(k - 1)) {
    cout << *v << '\n';
}
if (auto v = tr.predecessor(x)) {
    cout << *v << '\n';
}
```

```
if (auto v = tr.successor(x)) {
    cout << *v << '\n';
}
```

查询重复次数

```
cout << tr.count(x) << '\n';
if (tr.contains(x)) {
    // x 存在
}
```

初始序列批量建树

如果一开始已经有全部元素：

```
vector<int> a(n);
for (int &x : a) cin >> x;
FHQTreap<int> tr(move(a));
```

内部会：

```
ranges::sort
→ 合并重复值
→ O(d) 构造随机笛卡尔树
```

整体瓶颈为排序
如果只从空树在线插入，可以把整个 `build()` 和对应构造函数删掉

自定义键

```
struct Key {
    i64 value{};
    int id{};
};
struct KeyLess {
    bool operator()(const Key &a, const Key &b) const {
        return tie(a.value, a.id) < tie(b.value, b.id);
    }
};
FHQTreap<Key, KeyLess> tr;
tr.insert({value, id});
int rank = tr.order_of_key({value, INT_MIN}) + 1;
if (auto x = tr.kth(k - 1)) {
    cout << x->value << ' ' << x->id << '\n';
}
```

由于 `id` 参与比较：

```
(value, id1)
(value, id2)
```

是两个不同键，因此不会合并 `cnt`

树上合并

```
FHQTreap<int> tr;
vector<int> rt(n);
for (int u = 0; u < n; ++ u) {
    tr.insert(rt[u], a[u]);
}
auto dfs = Y([&](auto self, int u, int fa) -> void {
    for (int v : g[u]) if (v != fa) {
        self(v, u);
        rt[u] = tr.unite(rt[u], rt[v]);
        rt[v] = 0;
    }
});
dfs(0, -1);
```

例如查询整个子树的第 `k` 小：

```
auto ans = tr.kth(rt[u], k);
```

复杂度

设：

- n：元素总数
- d：不同键数量
- `unite()` 的两棵树含有 n_d, m_d 个不同键节点，且 $n_d \geq m_d$

操作	期望时间
`insert`	$O(\log d)$
`erase`	$O(\log d)$
`count / contains`	$O(\log d)$
`order_of_key`	$O(\log d)$
`kth`	$O(\log d)$
`predecessor / successor`	$O(\log d)$
初始批量构造	$O(n \log n)$
`unite`	$O!\left(m_d \log\left(\frac{n_d}{m_d}+1\right)\right)$ 期望

空间复杂度：

$$O(d)$$

节点回收后，长期在线插删时实际节点池规模接近“历史上同时存在的最大不同键数”，而不是总操作次数

3.8.4 隐式 FHQ Treap 序列

解决问题： 将序列下标隐式维护在 FHQ Treap 的中序遍历中，支持动态插入 / 删除、单点访问 / 修改，以及区间加、区间和、区间翻转

```
template<class T = i64>
struct ImplicitFHQTreap {
    struct Node {
        int ch[2]{}, size{};
        unsigned pri{};
        T value{}, sum{}, add{};
        bool rev{};
    };
    mt19937 rng;
    vector<Node> tr{Node{}};
    vector<int> bin;
    int root{};
    explicit ImplicitFHQTreap(
        unsigned seed = unsigned(chrono::steady_clock::now().time_since_epoch().count())
    ) : rng(seed) {}
    explicit ImplicitFHQTreap(
        const vector<T> &a,
        unsigned seed = unsigned(chrono::steady_clock::now().time_since_epoch().count())
    ) : rng(seed) {
        tr.reserve(a.size() + 1);
        root = build_tree(a);
    }
private:
    int node_size(int p) const { return tr[p].size; }
    T node_sum(int p) const { return tr[p].sum; }
    // ===== 节点创建与维护 =====
    int new_node(const T &value) {
        int p;
        if (bin.empty()) {
            p = int(tr.size());
            tr.emplace_back();
        } else {
            p = bin.back();
            bin.pop_back();
```

```
        }
        tr[p] = Node{{}, 1, unsigned(rng()), value, value, T{}, false};
        return p;
    }
    void pull(int p) {
        tr[p].size = 1 + node_size(tr[p].ch[0]) + node_size(tr[p].ch[1]);
        tr[p].sum = node_sum(tr[p].ch[0]) + tr[p].value + node_sum(tr[p].ch[1]);
    }
    // ===== 区间加：若不需要，删除 Node::add / apply_add() / range_add()
    // 同时删除 push() 中对应逻辑 =====
    void apply_add(int p, const T &value) {
        if (!p) return;
        tr[p].value += value;
        tr[p].sum += value * tr[p].size;
        tr[p].add += value;
    }
    // ===== 区间翻转：若不需要，删除 Node::rev / apply_rev() / reverse()
    // 同时删除 push() 中对应逻辑 =====
    void apply_rev(int p) {
        if (!p) return;
        swap(tr[p].ch[0], tr[p].ch[1]);
        tr[p].rev ^= 1;
    }
    // ===== 懒标记下传：只要保留 add / rev 中任意一种，就必须保留 =====
    void push(int p) {
        if (tr[p].rev) {
            apply_rev(tr[p].ch[0]);
            apply_rev(tr[p].ch[1]);
            tr[p].rev = false;
        }
        if (tr[p].add != T{}) {
            apply_add(tr[p].ch[0], tr[p].add);
            apply_add(tr[p].ch[1], tr[p].add);
            tr[p].add = T{};
        }
    }
    // ===== O(n) 笛卡尔树建树：若只从空序列单点插入，可删除
    // 同时删除 vector 构造函数和批量 insert(pos, values) =====
    int build_tree(const vector<T> &a) {
        if (a.empty()) return 0;
        vector<int> st;
        st.reserve(a.size());
        for (const T &value : a) {
            int x = new_node(value), last = 0;
            while (!st.empty() && tr[st.back()].pri > tr[x].pri) {
                last = st.back();
                st.pop_back();
            }
            tr[x].ch[0] = last;
            if (!st.empty()) tr[st.back()].ch[1] = x;
            st.push_back(x);
        }
        int rt = st.front();
        vector<int> order{rt};
        order.reserve(a.size());
        for (int i = 0; i < order.size(); ++ i) {
            int p = order[i];
            for (int c : tr[p].ch) if (c) order.push_back(c);
        }
        for (int p : order | views::reverse) pull(p);
        return rt;
    }
    // ===== 单点辅助：get / set 直接沿树下降，不需要 split =====
    T get_node(int p, int k) {
        while (p) {
            push(p);
            int left = node_size(tr[p].ch[0]);
            if (k < left) p = tr[p].ch[0];
            else if (k == left) return tr[p].value;
            else {
                k -= left + 1;
```

```
                p = tr[p].ch[1];
            }
        }
        return T{};
    }
    void set_node(int p, int k, const T &value) {
        push(p);
        int left = node_size(tr[p].ch[0]);
        if (k < left) set_node(tr[p].ch[0], k, value);
        else if (k > left) set_node(tr[p].ch[1], k - left - 1, value);
        else tr[p].value = value;
        pull(p);
    }
    // ===== 区间辅助：将 root 分成 [0,l) / [l,r) / [r,n) =====
    array<int, 3> split_range(int l, int r) {
        auto [a, c] = split(root, r);
        auto [x, b] = split(a, l);
        return {x, b, c};
    }
    void join_range(int a, int b, int c) { root = merge(merge(a, b), c); }
public:
    int size(int p) const { return node_size(p); }
    int size() const { return node_size(root); }
    bool empty() const { return root == 0; }
    void reserve(int nodes) { tr.reserve(nodes + 1); bin.reserve(nodes); }
    void clear() {
        tr.resize(1);
        tr[0] = Node{};
        bin.clear();
        root = 0;
    }
    // ===== FHQ 核心：split 按元素数量分裂，merge 按原序列顺序拼接 =====
    // 前 k 个元素 | 剩余元素
    pair<int, int> split(int p, int k) {
        if (!p) return {0, 0};
        push(p);
        int left = node_size(tr[p].ch[0]);
        if (k <= left) {
            auto [a, b] = split(tr[p].ch[0], k);
            tr[p].ch[0] = b;
            pull(p);
            return {a, p};
        }
        auto [a, b] = split(tr[p].ch[1], k - left - 1);
        tr[p].ch[1] = a;
        pull(p);
        return {p, b};
    }
    // 返回序列 a + b
    int merge(int a, int b) {
        if (!a || !b) return a | b;
        if (tr[a].pri < tr[b].pri) {
            push(a);
            tr[a].ch[1] = merge(tr[a].ch[1], b);
            pull(a);
            return a;
        }
        push(b);
        tr[b].ch[0] = merge(a, tr[b].ch[0]);
        pull(b);
        return b;
    }
    // ===== 基础序列接口：插入 / 删除 / 单点访问修改 =====
    // 在原下标 pos 之前插入
    void insert(int pos, const T &value) {
        assert(0 <= pos && pos <= size());
        auto [a, b] = split(root, pos);
        root = merge(merge(a, new_node(value)), b);
    }
    // 在 pos 前插入整个 values
    void insert(int pos, const vector<T> &values) {
```

```
        assert(0 <= pos && pos <= size());
        if (values.empty()) return;
        auto [a, b] = split(root, pos);
        root = merge(merge(a, build_tree(values)), b);
    }
    T erase(int pos) {
        assert(0 <= pos && pos < size());
        auto [a, b, c] = split_range(pos, pos + 1);
        T value = move(tr[b].value);
        join_range(a, 0, c);
        tr[b] = Node{};
        bin.push_back(b);
        return value;
    }
    // 删除 [l, r)
    void erase(int l, int r) {
        assert(0 <= l && l <= r && r <= size());
        if (l == r) return;
        if (r - l == 1) {
            erase(l);
            return;
        }
        auto part = split_range(l, r);
        root = merge(part[0], part[2]);
    }
    T get(int pos) {
        assert(0 <= pos && pos < size());
        return get_node(root, pos);
    }
    void set(int pos, const T &value) {
        assert(0 <= pos && pos < size());
        set_node(root, pos, value);
    }
    // ===== 区间修改：[l, r) =====
    void range_add(int l, int r, const T &value) {
        assert(0 <= l && l <= r && r <= size());
        if (l == r) return;
        auto [a, b, c] = split_range(l, r);
        apply_add(b, value);
        join_range(a, b, c);
    }
    void reverse(int l, int r) {
        assert(0 <= l && l <= r && r <= size());
        if (r - l <= 1) return;
        auto [a, b, c] = split_range(l, r);
        apply_rev(b);
        join_range(a, b, c);
    }
    // ===== 区间和：若不需要，删除 Node::sum / node_sum() / range_sum() / all_sum()
    // 并将 pull() 改为只维护 size =====
    T range_sum(int l, int r) {
        assert(0 <= l && l <= r && r <= size());
        if (l == r) return T{};
        auto [a, b, c] = split_range(l, r);
        T ans = node_sum(b);
        join_range(a, b, c);
        return ans;
    }
    T all_sum() const { return node_sum(root); }
    // ===== 导出序列：只用于输出 / 调试时可删除 =====
    vector<T> to_vector() {
        vector<T> a;
        a.reserve(size());
        vector<int> st;
        int p = root;
        while (p || !st.empty()) {
            while (p) {
                push(p);
                st.push_back(p);
                p = tr[p].ch[0];
            }
```

```
            p = st.back();
            st.pop_back();
            a.push_back(tr[p].value);
            p = tr[p].ch[1];
        }

        return a;
    }
};
```

接口说明

核心语义

隐式 Treap 不按 `value` 排序
节点的逻辑下标由：

```
size(left)
```

隐式决定
因此：

```
auto [a, b] = split(root, k);
```

表示：

```
a = 原序列前 k 个元素
b = 原序列剩余元素
```

也就是：

```
[0, k) | [k, n)
```

这也是它和上一份 FHQ Treap 有序多重集最本质的区别

基础接口

接口	含义	期望复杂度
`size()`	序列长度	$O(1)$
`empty()`	是否为空	$O(1)$
`insert(pos,x)`	在 `pos` 前插入 `x`	$O(\log n)$
`insert(pos,a)`	批量插入序列 `a`	$O(\|a\|)$
`erase(pos)`	删除并返回 `pos`	$O(\log n)$
`erase(l,r)`	删除 `[l,r)`	$O(\log n)$
`get(pos)`	读取单点	$O(\log n)$
`set(pos,x)`	单点赋值	$O(\log n)$
`range_add(l,r,x)`	`[l,r)` 全部加 `x`	$O(\log n)$
`reverse(l,r)`	翻转 `[l,r)`	$O(\log n)$
`range_sum(l,r)`	`[l,r)` 区间和	$O(\log n)$
`all_sum()`	整体和	$O(1)$
`to_vector()`	导出整个序列	$O(n)$

所有公开下标统一使用 `0base`，所有区间统一使用 `[l,r)`

`split` 与 `merge`

`split(p, k)`

```
auto [a, b] = tr.split(p, k);
```

切成：

```
a = 前 k 个元素
b = 剩余元素
```

例如：

```
p = [1, 2, 3, 4, 5]
k = 2
a = [1, 2]
b = [3, 4, 5]
```

`merge(a, b)`

隐式 Treap 的：

```
merge(a, b)
```

只表示：

```
序列 a + 序列 b
```

注意事项

懒标记必须在下降前 `push`

例如 `split()`：

```
push(p);
int left = size(tr[p].ch[0]);
```

顺序不能反
特别是存在 `rev` 时，如果没有先下传，逻辑左儿子和当前物理 `ch[0]` 可能不一致
`merge()` 同理，在修改某个节点的儿子之前必须先 `push()`

`apply_add()` 必须同时修改 `value` 和 `sum`

这里使用的是标准的：

```
tr[p].value += value;
tr[p].sum += value * tr[p].size;
tr[p].add += value;
```

也就是说 lazy 标记表示：

> 整棵子树的逻辑值已经更新，只是对子节点的修改尚未真正下传

所以 `sum` 在不 `push()` 的情况下依然始终正确。参考模板中的 Treap-序列同样采用区间 `add` 与 `sum` 的懒标记维护

`reverse` 立即交换左右儿子

```
swap(tr[p].ch[0], tr[p].ch[1]);
tr[p].rev ^= 1;
```

因此当前节点的物理结构已经翻转，只需要记录：

> 两个儿子的内部序列还需要继续翻转

不要只写：

```
tr[p].rev ^= 1;
```

却不交换两个儿子

`range_sum()` 会改变 Treap 内部树形

它会执行：

```
split
split
读取 sum
```

```
merge
merge
```

因此：

```
range_sum()
```

不是 `const`
不过它不会改变**逻辑序列**，只可能改变随机 Treap 的内部指针组织

单点删除回收节点，批量删除仍保留对数复杂度

`erase(pos)` 会把删除的单个节点放入 `bin`，后续插入可直接复用；单元素的 `erase(l,r)` 也会转到单点删除。

而对于多元素区间：

```
erase(l, r)
```

仍直接丢弃中间子树，因此时间仍然是期望：

$$O(\log n)$$

但**批量删除**的节点仍然留在：

```
tr
```

中，不会重新利用；单点删除的节点由 `bin` 重用
因此空间复杂度更准确地说是：

> `O(初始元素数 + 历史插入元素总数)`（批量删除后仍可能如此；只有单点删除时，节点数可随回收复用保持稳定）

而不是严格的当前 $O(n)$
这是故意的：如果为了回收 `[l,r)` 中的全部节点去遍历整棵被删除子树，则一次区间删除会变成：

$$O(\log n + r - l)$$

若题目存在极大量的“插入一大段 → 删除一大段”并且内存敏感，可以采用后面“常见用法”中的 `rebuild()` 周期性回收

`T` 的要求

完整版本要求 `T` 支持：

```
T{}
a + b
a += b
a * int
a != b
```

通常直接：

```
ImplicitFHQTreap<i64>
```

即可
如果只需要维护一般对象序列，可以裁掉 `sum / add` 部分

功能裁剪

只需要动态序列 + 翻转

删除：

```
Node::sum
Node::add
node_sum()
apply_add()
range_add()
range_sum()
all_sum()
```

然后：

```
void pull(int p) {
    tr[p].size = 1 + node_size(tr[p].ch[0]) + node_size(tr[p].ch[1]);
}
```

`push()` 只保留：

```
void push(int p) {
    if (tr[p].rev) {
        apply_rev(tr[p].ch[0]);
        apply_rev(tr[p].ch[1]);
        tr[p].rev = false;
    }
}
```

此时 `T` 不再需要任何数值运算

不需要区间翻转

删除：

```
Node::rev
apply_rev()
reverse()
```

并删除 `push()` 中对应部分

完全没有懒标记

如果既不要：

```
range_add
reverse
```

那么：

```
push()
```

以及所有：

```
push(p);
```

都可以一起删除
这会得到一份非常短的：

> 动态数组型隐式 FHQ Treap

不需要批量建树

删除：

```
build_tree()
ImplicitFHQTreap(const vector<T> &a, ...)
insert(pos, const vector<T> &values)
```

只保留单点：

```
insert(pos, value)
```

即可

常见用法

动态数组

```
ImplicitFHQTreap<i64> tr({1, 2, 3, 4});
tr.insert(2, 10);
// [1, 2, 10, 3, 4]
tr.erase(1);
// [1, 10, 3, 4]
```

```
tr.set(2, 100);
// [1, 10, 100, 4]
cout << tr.get(2) << '\n';
```

区间加、区间和、翻转

```
tr.range_add(l, r, delta);
i64 sum = tr.range_sum(l, r);
tr.reverse(l, r);
```

均使用：

```
[l, r)
```

批量插入

```
vector<i64> b{3, 1, 4, 1, 5};
tr.insert(pos, b);
```

内部不是逐个执行：

```
insert()
```

而是先对 `b` 用随机优先级构造 Cartesian Tree，再一次 `merge()`
因此复杂度是：

$$O(|b| + \log n)$$

剪切一段并删除

如果只是剪掉：

```
[l,r)
```

直接：

```
tr.erase(l, r);
```

即可
如果还需要保留被剪下来的 Treap 根，可以直接利用核心接口：

```
auto [ab, c] = tr.split(tr.root, r);
auto [a, b] = tr.split(ab, l);
tr.root = tr.merge(a, c);
// b 就是被剪下来的 [l, r)
```

需要注意：

```
b
```

仍然属于同一个 `tr` 节点池

交换两个相邻区间

将：

```
A B C D
```

中的：

```
B = [l,m)
C = [m,r)
```

交换成：

```
A C B D
```

可以：

```
auto [abc, d] = tr.split(tr.root, r);
auto [ab, c] = tr.split(abc, m);
auto [a, b] = tr.split(ab, l);

tr.root = tr.merge(tr.merge(tr.merge(a, c), b), d);
```

这本质就是：

```
[l,m) 与 [m,r) 交换
```

也是隐式 Treap 很经典的应用

区间循环移动

例如将：

```
[l,r)
```

循环右移 `k` 位

```
int len = r - l;
k %= len;
auto [abc, d] = tr.split(tr.root, r);
auto [a, bc] = tr.split(abc, l);
auto [b, c] = tr.split(bc, len - k);
tr.root = tr.merge(tr.merge(tr.merge(a, c), b), d);
```

例如：

```
[1, 2, 3, 4, 5]
[l,r) = [1,5)
k = 2

⇒ [1, 4, 5, 2, 3]
```

如果 `k == 0` 可以直接跳过

导出整个序列

```
vector<i64> a = tr.to_vector();
```

调试 / 对拍尤其方便：

```
for (i64 x : tr.to_vector()) {
    cerr << x << ' ';
}
```

大量删除后的重构

如果题目长期执行：

```
大量插入
大量区间删除
大量插入
大量区间删除
```

当前节点池可能累计得很大
可以偶尔执行：

```
auto a = tr.to_vector();
tr.clear();
tr.reserve(a.size());
tr.insert(0, a); // build_tree 是私有接口
```

不过 `build_tree()` 当前是 `private`
如果题目确实需要这种操作，可以在结构体中补一个：

```
void rebuild() {
    auto a = to_vector();
    clear();
    tr.reserve(a.size() + 1);
    root = build_tree(a);
}
```

复杂度

Treap 树高期望为 $O(\log n)$

操作	期望时间
初始 `build`	$O(n)$
`split / merge / insert(pos,x)`	$O(\log n)$
批量插入 m 个元素	$O(m+\log n)$
`erase(pos) / erase(l,r) / get / set / range_add / reverse / range_sum`	$O(\log n)$
`all_sum`	$O(1)$
`to_vector`	$O(n)$

理论最坏树高仍可能为 $O(n)$
节点池空间为：

$$O(n_0+I)$$

3.8.5 可持久化 FHQ Treap 有序多重集

解决问题： 基于任意历史版本插入 / 删除一个键，并查询各版本的排名、第 k 小、前驱、后继等有序多重集信息
相同值使用多个独立节点保存；每次修改产生一个新版本，未修改的子树与旧版本共享

```
template<class T, class Compare = less<T>>
struct PersistentFHQTreap {
    struct Node {
        int ch[2]{}, size{};
        unsigned pri{};
        T value{};
    };
    vector<Node> tr{{}};
    vector<int> root{ 0 };
    u64 rng_state;
    Compare comp;
    explicit PersistentFHQTreap(
        u64 seed = chrono::steady_clock::now().time_since_epoch().count(),
        Compare comp_ = Compare()
    ) : rng_state(seed), comp(move(comp_)) {}
    void reserve(int nodes, int versions = 0) {
        tr.reserve(nodes + 1);
        if (versions) root.reserve(versions + 1);
    }
    // ===== 节点与随机优先级：所有修改操作依赖，不建议删除 =====
    int node_size(int p) const { return tr[p].size; }
    unsigned rng() {
        u64 z = (rng_state += 0x9e3779b97f4a7c15ULL);
        z = (z ^ (z >> 30)) * 0xbf58476d1ce4e5b9ULL;
        z = (z ^ (z >> 27)) * 0x94d049bb133111ebULL;
        return unsigned(z ^ (z >> 31));
    }
    int new_node(const T &value) {
        tr.push_back({{}, 1, rng(), value});
        return int(tr.size()) - 1;
    }
    int clone(int p) {
        tr.push_back(tr[p]);
        return int(tr.size()) - 1;
    }
    void pull(int p) {
        tr[p].size = 1 + node_size(tr[p].ch[0]) + node_size(tr[p].ch[1]);
```

```
    }
    // ===== 持久化 split / merge：插入和删除依赖 =====
    // < value | ≥ value
    pair<int, int> split_less(int p, const T &value) {
        if (!p) return {0, 0};
        int q = clone(p);
        if (comp(tr[q].value, value)) {
            auto [a, b] = split_less(tr[q].ch[1], value);
            tr[q].ch[1] = a;
            pull(q);
            return {q, b};
        }
        auto [a, b] = split_less(tr[q].ch[0], value);
        tr[q].ch[0] = b;
        pull(q);
        return {a, q};
    }
    // 要求 a 中所有元素排在 b 中所有元素之前
    int merge(int a, int b) {
        if (!a || !b) return a | b;
        if (tr[a].pri < tr[b].pri) {
            int p = clone(a);
            tr[p].ch[1] = merge(tr[p].ch[1], b);
            pull(p);
            return p;
        }
        int p = clone(b);
        tr[p].ch[0] = merge(a, tr[p].ch[0]);
        pull(p);
        return p;
    }
    // ===== 修改核心：沿修改路径复制 =====
    int insert_node(int p, int x) {
        if (!p) return x;
        if (tr[x].pri < tr[p].pri) {
            T key = tr[x].value; // split_less 会扩容节点池，不能持有池内引用
            auto [a, b] = split_less(p, key);
            tr[x].ch[0] = a;
            tr[x].ch[1] = b;
            pull(x);
            return x;
        }
        int q = clone(p);
        if (comp(tr[x].value, tr[q].value)) tr[q].ch[0] = insert_node(tr[q].ch[0], x);
        else tr[q].ch[1] = insert_node(tr[q].ch[1], x);
        pull(q);
        return q;
    }
    int erase_node(int p, const T &value, bool &erased) {
        if (!p) return 0;
        if (comp(value, tr[p].value)) {
            int c = erase_node(tr[p].ch[0], value, erased);
            if (!erased) return p;
            int q = clone(p);
            tr[q].ch[0] = c;
            pull(q);
            return q;
        }
        if (comp(tr[p].value, value)) {
            int c = erase_node(tr[p].ch[1], value, erased);
            if (!erased) return p;
            int q = clone(p);
            tr[q].ch[1] = c;
            pull(q);
            return q;
        }
        erased = true;
        return merge(tr[p].ch[0], tr[p].ch[1]);
    }
    // ===== 版本修改：insert / erase 均追加一个新版本 =====
    int insert(int ver, const T &value) {
```

```
        int x = new_node(value);
        root.push_back(insert_node(root[ver], x));
        return int(root.size()) - 1;
    }
    int erase(int ver, const T &value) {
        bool erased = false;
        int r = erase_node(root[ver], value, erased);
        root.push_back(r);
        return int(root.size()) - 1;
    }
    // 若题目要求“每次操作都产生版本”，查询操作后可复制原版本
    int copy_version(int ver) {
        root.push_back(root[ver]);
        return int(root.size()) - 1;
    }
    // ===== 基础查询：均不产生新节点 =====
    int size(int ver) const { return node_size(root[ver]); }
    bool empty(int ver) const { return root[ver] == 0; }
    // Compare 意义下严格排在 value 前面的元素个数
    int order_of_key(int ver, const T &value) const {
        int p = root[ver], ans = 0;
        while (p) {
            if (comp(tr[p].value, value)) {
                ans += node_size(tr[p].ch[0]) + 1;
                p = tr[p].ch[1];
            } else {
                p = tr[p].ch[0];
            }
        }
        return ans;
    }
    // 0base 第 k 小；Compare 改变时对应第 k 个元素
    optional<T> kth(int ver, int k) const {
        if (k < 0 || k >= size(ver)) return nullopt;
        int p = root[ver];
        while (p) {
            int left = node_size(tr[p].ch[0]);
            if (k < left) p = tr[p].ch[0];
            else if (k == left) return tr[p].value;
            else {
                k -= left + 1;
                p = tr[p].ch[1];
            }
        }
        return nullopt;
    }
    // 最大的严格 < value
    optional<T> predecessor(int ver, const T &value) const {
        int p = root[ver], ans = 0;
        while (p) {
            if (comp(tr[p].value, value)) {
                ans = p;
                p = tr[p].ch[1];
            } else {
                p = tr[p].ch[0];
            }
        }
        if (!ans) return nullopt;
        return tr[ans].value;
    }
    // 最小的严格 > value
    optional<T> successor(int ver, const T &value) const {
        int p = root[ver], ans = 0;
        while (p) {
            if (comp(value, tr[p].value)) {
                ans = p;
                p = tr[p].ch[0];
            } else {
                p = tr[p].ch[1];
            }
        }
```

```
        if (!ans) return nullopt;
        return tr[ans].value;
    }
    // ===== 等值查询：若不需要 count / contains，以下代码可直接删除 =====
    int count_leq(int ver, const T &value) const {
        int p = root[ver], ans = 0;
        while (p) {
            if (!comp(value, tr[p].value)) {
                ans += node_size(tr[p].ch[0]) + 1;
                p = tr[p].ch[1];
            } else {
                p = tr[p].ch[0];
            }
        }
        return ans;
    }
    int count(int ver, const T &value) const {
        return count_leq(ver, value) - order_of_key(ver, value);
    }
    bool contains(int ver, const T &value) const {
        int p = root[ver];
        while (p) {
            if (comp(value, tr[p].value)) p = tr[p].ch[0];
            else if (comp(tr[p].value, value)) p = tr[p].ch[1];
            else return true;
        }
        return false;
    }
};
```

版本与修改

- `root[0]`：初始空版本
- `insert(ver, value)`：在版本 `ver` 中插入一个独立的 `value`，追加并返回新版本编号；期望 $O(\log n)$
- `erase(ver, value)`：删除版本 `ver` 中的一个等价值；若不存在则新版本直接共享原根，不创建 Treap 节点；期望 $O(\log n)$
- `copy_version(ver)`：直接复制版本根并产生新版本，$O(1)$
- `reserve(nodes, versions)`：预留节点池 / 版本数组容量，最好在第一次修改前调用

查询

- `size(ver)` / `empty(ver)`：版本大小 / 是否为空，$O(1)$
- `order_of_key(ver, value)`：返回满足 `comp(x, value)` 的元素数；默认 `less<T>` 时即严格小于 `value` 的元素数，期望 $O(\log n)$
- `kth(ver, k)`：返回 `0base` 的第 `k` 个元素；越界返回 `nullopt`，期望 $O(\log n)$
- `predecessor(ver, value)`：严格前驱；不存在返回 `nullopt`，期望 $O(\log n)$
- `successor(ver, value)`：严格后继；不存在返回 `nullopt`，期望 $O(\log n)$
- `count(ver, value)`：与 `value` 在 `Compare` 意义下等价的元素数，期望 $O(\log n)$
- `contains(ver, value)`：是否至少存在一个等价值，期望 $O(\log n)$

  其中等价关系为：

```
!comp(a, b) && !comp(b, a)
```

所以自定义 `Compare` 时，并不要求 `T` 提供 `==`

注意事项

- **版本编号与节点编号不是一回事。** `ver` 必须是 `root` 中已有的版本编号；内部节点编号不要拿来当版本使用
- 所有旧版本可达节点都不会被原地修改；`split_less()`、`merge()` 以及修改路径上的节点都会按需复制
- 相同值使用多个独立节点。`erase()` 只删除其中一个，但不保证对应哪一个物理节点，多重集语义不受影响
- 重复值统一向右插入，因此内部满足“左侧严格在当前值之前，右侧不在当前值之前”；不要自行修改 `insert_node()` 中相等值的分支而不同步修改 `split_less()`
- `predecessor()` / `successor()` 是**严格**前驱 / 后继；若希望“$\le x$ 最大值”或“$\ge x$ 最小值”，不能直接代替使用
- `kth()` 使用 `0base`
- 随机优先级只决定树形，不影响有序性正确性；树高及操作复杂度为期望 $O(\log n)$，理论最坏仍为 $O(n)$
- 默认随机种子来自时间；调试时若希望树形完全可复现，可以显式传固定种子
- 持久化数据结构通常会产生大量节点。如果能估计上界，建议提前 `reserve()`，避免 `vector` 多次扩容搬迁

常见用法

从旧版本分叉

```
PersistentFHQTreap<int> tree;
int v1 = tree.insert(0, 3);   // {3}
int v2 = tree.insert(v1, 5);  // {3, 5}
int v3 = tree.insert(v1, 1);  // {1, 3}
cout << *tree.kth(v2, 1) << '\n'; // 5
cout << *tree.kth(v3, 0) << '\n'; // 1
```

重复值与删除

```
int v1 = tree.insert(0, 5);
int v2 = tree.insert(v1, 5);
int v3 = tree.insert(v2, 5);
cout << tree.count(v3, 5) << '\n'; // 3
int v4 = tree.erase(v3, 5);
cout << tree.count(v4, 5) << '\n'; // 2
cout << tree.count(v3, 5) << '\n'; // 3，旧版本不变
```

排名、第 k 小、前驱与后继

```
int rank = tree.order_of_key(ver, x); // 严格小于 x 的数量
auto kth = tree.kth(ver, k);
auto pre = tree.predecessor(ver, x);
auto nxt = tree.successor(ver, x);

if (kth) cout << *kth << '\n';
if (pre) cout << *pre << '\n';
if (nxt) cout << *nxt << '\n';
```

若题目中的“排名”使用 `1base`：

```
int rank = tree.order_of_key(ver, x) + 1;
```

每次操作都必须生成版本

例如输入的第 `i` 次操作要求产生版本 `i`，即使这次只是查询：

```
int ver = old_ver;

if (op == 1) {
    ver = tree.insert(old_ver, x);
} else if (op == 2) {
    ver = tree.erase(old_ver, x);
} else {
    cout << tree.order_of_key(old_ver, x) + 1 << '\n';
    ver = tree.copy_version(old_ver);
}
```

自定义键

如果需要按 `(value, id)` 区分元素：

```
struct Key {
    i64 value{};
    int id{};
};
struct KeyLess {
    bool operator()(const Key &a, const Key &b) const {
        return tie(a.value, a.id) < tie(b.value, b.id);
    }
};
PersistentFHQTreap<Key, KeyLess> tree;
int ver = tree.insert(0, {value, id});
int rank = tree.order_of_key(ver, {value, INT_MIN});
auto kth = tree.kth(ver, k);
```

这里需要注意：一旦把 `id` 加入比较键，两个 `{value, id}` 只有完整键等价时才被视为同一个元素；如果只是想统计某个 `value` 的全部元素，应根据键域分别求边界排名

复杂度

操作	时间复杂度	新增 Treap 节点
`insert`	期望 $O(\log n)$	期望 $O(\log n)$
`erase`，值存在	期望 $O(\log n)$	期望 $O(\log n)$
`erase`，值不存在	期望 $O(\log n)$	$O(1)$
`copy_version / size / empty`	$O(1)$	0
`order_of_key / kth / predecessor / successor / count / contains`	期望 $O(\log n)$	0

若共有 V 个版本、M 次实际改变集合内容的修改，总空间期望为

$$O(V + M \log n)$$

3.8.6 可持久化隐式 FHQ Treap 序列

```
可持久化 FHQ Treap 有序多重集
    ↓
按 value / Compare 维护顺序
排名、第 k 小、前驱、后继

可持久化隐式 FHQ Treap 序列
    ↓
按中序位置维护顺序
插入、删除、翻转、区间修改、区间查询
```

```cpp
template<class T = i64>
struct PersistentImplicitTreap {
    struct Node {
        int ch[2]{}, size{}, stamp{};
        unsigned pri{};
        T value{}, sum{}, add{};
        bool rev{};
    };
    vector<Node> tr{Node{}};
    vector<int> root{0};
    u64 rng_state;
    int epoch{1};
    explicit PersistentImplicitTreap(
        u64 seed = chrono::steady_clock::now().time_since_epoch().count()
    ) : rng_state(seed) {}
    explicit PersistentImplicitTreap(
        const vector<T> &a,
        u64 seed = chrono::steady_clock::now().time_since_epoch().count()
    ) : rng_state(seed) { root[0] = build_tree(a); }
    void reserve(int nodes, int versions = 0) {
        tr.reserve(nodes + 1);
        if (versions) root.reserve(versions + 1);
    }
    // ===== 节点与 COW：所有修改操作依赖，不建议删除 =====
    int node_size(int p) const { return p ? tr[p].size : 0; }
    T node_sum(int p) const { return p ? tr[p].sum : T{}; }
    unsigned rng() {
        u64 z = (rng_state += 0x9e3779b97f4a7c15ULL);
        z = (z ^ (z >> 30)) * 0xbf58476d1ce4e5b9ULL;
        z = (z ^ (z >> 27)) * 0x94d049bb133111ebULL;
        return unsigned(z ^ (z >> 31));
    }
    int new_node(const T &value) {
        tr.push_back({{}, 1, epoch, rng(), value, value, T{}, false});
        return int(tr.size()) - 1;
    }
    // 当前操作内已经复制过则直接复用，否则复制后再修改
    int touch(int p) {
        if (!p || tr[p].stamp == epoch) return p;
        tr.push_back(tr[p]);
```

```
        tr.back().stamp = epoch;
        return int(tr.size()) - 1;
    }
    void pull(int p) {
        tr[p].size = 1 + node_size(tr[p].ch[0]) + node_size(tr[p].ch[1]);
        tr[p].sum = node_sum(tr[p].ch[0]) + tr[p].value + node_sum(tr[p].ch[1]);
    }
    void begin_modify() { ++ epoch; }
    // ===== 懒标记：range_add / reverse 及带标记的 split / merge 依赖 =====
    void apply_add(int p, const T &value) {
        if (!p) return;
        tr[p].value += value;
        tr[p].sum += value * tr[p].size;
        tr[p].add += value;
    }
    void apply_rev(int p) {
        if (!p) return;
        swap(tr[p].ch[0], tr[p].ch[1]);
        tr[p].rev ^= 1;
    }
    // 子节点仍可能属于旧版本，因此下传前必须 touch()
    void push(int p) {
        if (tr[p].rev) {
            for (int d = 0; d < 2; ++ d) if (tr[p].ch[d]) {
                tr[p].ch[d] = touch(tr[p].ch[d]);
                apply_rev(tr[p].ch[d]);
            }
            tr[p].rev = false;
        }
        if (tr[p].add != T{}) {
            T add = tr[p].add;
            for (int d = 0; d < 2; ++ d) if (tr[p].ch[d]) {
                tr[p].ch[d] = touch(tr[p].ch[d]);
                apply_add(tr[p].ch[d], add);
            }
            tr[p].add = T{};
        }
    }
    // ===== split / merge: split(p, k) 将前 k 个元素分到左侧 =====
    pair<int, int> split(int p, int k) {
        if (!p) return {0, 0};
        p = touch(p);
        push(p);
        int left = node_size(tr[p].ch[0]);
        if (left >= k) {
            auto [a, b] = split(tr[p].ch[0], k);
            tr[p].ch[0] = b;
            pull(p);
            return {a, p};
        }
        auto [a, b] = split(tr[p].ch[1], k - left - 1);
        tr[p].ch[1] = a;
        pull(p);
        return {p, b};
    }
    // 要求 a 的所有元素在序列中位于 b 之前
    int merge(int a, int b) {
        if (!a || !b) return a | b;
        if (tr[a].pri < tr[b].pri) {
            a = touch(a);
            push(a);
            tr[a].ch[1] = merge(tr[a].ch[1], b);
            pull(a);
            return a;
        }
        b = touch(b);
        push(b);
        tr[b].ch[0] = merge(a, tr[b].ch[0]);
        pull(b);
        return b;
    }
```

```
    // ===== 单点核心：比统一 split + merge 的节点常数更小 =====
    int insert_node(int p, int k, int x) {
        if (!p) return x;
        if (tr[x].pri < tr[p].pri) {
            auto [a, b] = split(p, k);
            tr[x].ch[0] = a;
            tr[x].ch[1] = b;
            pull(x);
            return x;
        }
        p = touch(p);
        push(p);
        int left = node_size(tr[p].ch[0]);
        if (k <= left) tr[p].ch[0] = insert_node(tr[p].ch[0], k, x);
        else tr[p].ch[1] = insert_node(tr[p].ch[1], k - left - 1, x);
        pull(p);
        return p;
    }
    int erase_node(int p, int k) {
        p = touch(p);
        push(p);
        int left = node_size(tr[p].ch[0]);
        if (k < left) tr[p].ch[0] = erase_node(tr[p].ch[0], k);
        else if (k > left) tr[p].ch[1] = erase_node(tr[p].ch[1], k - left - 1);
        else return merge(tr[p].ch[0], tr[p].ch[1]);
        pull(p);
        return p;
    }
    int set_node(int p, int k, const T &value) {
        p = touch(p);
        push(p);
        int left = node_size(tr[p].ch[0]);
        if (k < left) tr[p].ch[0] = set_node(tr[p].ch[0], k, value);
        else if (k > left) tr[p].ch[1] = set_node(tr[p].ch[1], k - left - 1, value);
        else tr[p].value = value;
        pull(p);
        return p;
    }
    // ===== O(n) 笛卡尔树建树：不需要批量建树时可删除 =====
    int build_tree(const vector<T> &a) {
        if (a.empty()) return 0;
        vector<int> st;
        st.reserve(a.size());
        for (const T &value : a) {
            int x = new_node(value), last = 0;
            while (!st.empty() && tr[st.back()].pri >= tr[x].pri) {
                last = st.back();
                st.pop_back();
            }
            tr[x].ch[0] = last;
            if (!st.empty()) tr[st.back()].ch[1] = x;
            st.push_back(x);
        }
        int rt = st.front();
        vector<int> order{rt};
        order.reserve(a.size());
        for (int i = 0; i < order.size(); ++ i) {
            int p{ order[i] };
            if (tr[p].ch[0]) order.push_back(tr[p].ch[0]);
            if (tr[p].ch[1]) order.push_back(tr[p].ch[1]);
        }
        for (int p : views::reverse(order)) pull(p);
        return rt;
    }
    // ===== 版本与基础修改：每次调用均追加一个版本 =====
    int copy_version(int ver) {
        root.push_back(root[ver]);
        return int(root.size()) - 1;
    }
    int size(int ver) const { return node_size(root[ver]); }
    bool empty(int ver) const { return root[ver] == 0; }
```

```
    int insert(int ver, int pos, const T &value) {
        assert(0 <= pos && pos <= size(ver));
        begin_modify();
        int x = new_node(value);
        root.push_back(insert_node(root[ver], pos, x));
        return int(root.size()) - 1;
    }
    int insert(int ver, int pos, const vector<T> &values) {
        assert(0 <= pos && pos <= size(ver));
        if (values.empty()) return copy_version(ver);
        begin_modify();
        auto [a, b] = split(root[ver], pos);
        int mid = build_tree(values);
        root.push_back(merge(merge(a, mid), b));
        return int(root.size()) - 1;
    }
    int erase(int ver, int pos) {
        assert(0 <= pos && pos < size(ver));
        begin_modify();
        root.push_back(erase_node(root[ver], pos));
        return int(root.size()) - 1;
    }
    int erase(int ver, int l, int r) {
        assert(0 <= l && l <= r && r <= size(ver));
        if (l == r) return copy_version(ver);
        begin_modify();
        auto [a, c] = split(root[ver], r);
        auto [x, b] = split(a, l);
        root.push_back(merge(x, c));
        return int(root.size()) - 1;
    }
    int set(int ver, int pos, const T &value) {
        assert(0 <= pos && pos < size(ver));
        begin_modify();
        root.push_back(set_node(root[ver], pos, value));
        return int(root.size()) - 1;
    }
    // ===== 区间修改：[l, r)；若不需要对应操作可删除公开接口 =====
    int range_add(int ver, int l, int r, const T &value) {
        assert(0 <= l && l <= r && r <= size(ver));
        if (l == r || value == T{}) return copy_version(ver);
        begin_modify();
        auto [a, c] = split(root[ver], r);
        auto [x, b] = split(a, l);
        b = touch(b);
        apply_add(b, value);
        root.push_back(merge(merge(x, b), c));
        return int(root.size()) - 1;
    }
    int reverse(int ver, int l, int r) {
        assert(0 <= l && l <= r && r <= size(ver));
        if (r - l <= 1) return copy_version(ver);
        begin_modify();
        auto [a, c] = split(root[ver], r);
        auto [x, b] = split(a, l);
        b = touch(b);
        apply_rev(b);
        root.push_back(merge(merge(x, b), c));
        return int(root.size()) - 1;
    }
    // ===== 只读查询 =====
    T get_node(int p, int k, T extra, bool flip) const {
        int lc = tr[p].ch[flip ? 1 : 0];
        int rc = tr[p].ch[flip ? 0 : 1];
        int left = node_size(lc);
        T next_extra = extra + tr[p].add;
        bool next_flip = flip ^ tr[p].rev;
        if (k < left) return get_node(lc, k, next_extra, next_flip);
        if (k == left) return tr[p].value + extra;
        return get_node(rc, k - left - 1, next_extra, next_flip);
    }
```

```
    T get(int ver, int pos) const {
        assert(0 <= pos && pos < size(ver));
        return get_node(root[ver], pos, T{}, false);
    }
    T range_sum_node(int p, int l, int r, T extra, bool flip) const {
        if (!p || l >= r) return T{};
        if (l == 0 && r == node_size(p)) return tr[p].sum + extra * tr[p].size;
        int lc = tr[p].ch[flip ? 1 : 0];
        int rc = tr[p].ch[flip ? 0 : 1];
        int left = node_size(lc);
        T next_extra = extra + tr[p].add;
        bool next_flip = flip ^ tr[p].rev;
        T ans{};
        if (l < left) {
            ans += range_sum_node(lc, l, min(r, left), next_extra, next_flip);
        }
        if (l <= left && left < r) ans += tr[p].value + extra;
        if (r > left + 1) {
            ans += range_sum_node(rc, max(0, l - left - 1),
                r - left - 1, next_extra, next_flip);
        }
        return ans;
    }
    T range_sum(int ver, int l, int r) const {
        assert(0 <= l && l <= r && r <= size(ver));
        return range_sum_node(root[ver], l, r, T{}, false);
    }
    T all_sum(int ver) const { return node_sum(root[ver]); }
};
```

核心语义

“键”不是 `value`，而是隐式下标

```
split(p, k)
```

始终表示：

```
原序列 = A + B
|A| = k
```

即：

```
A = 前 k 个元素
B = 剩余元素
```

所有公开区间统一使用 Obase 左闭右开 `[l, r)`

初始化与版本

- `PersistentImplicitTreap<T> tree`：版本 `0` 为空序列
- `PersistentImplicitTreap<T> tree(a)`：版本 `0` 为初始序列 `a`，建树 $O(n)$
- `root[ver]`：版本 `ver` 的根
- `size(ver)`：版本长度，$O(1)$
- `empty(ver)`：是否为空，$O(1)$
- `copy_version(ver)`：直接复制根，生成内容完全相同的新版本，$O(1)$
- `reserve(nodes, versions)`：预留节点 / 版本容量

单点修改

```
tree.insert(ver, pos, value);
```

在原下标 `pos` 元素之前插入：

```
0 ... pos-1 | value | pos ...
```

其中 `0 <= pos <= size(ver)`

```
tree.erase(ver, pos);
```

删除下标 `pos`

```
tree.set(ver, pos, value);
```

修改一个元素
这三个操作都返回新版本编号

区间修改

```
tree.erase(ver, l, r);
tree.range_add(ver, l, r, delta);
tree.reverse(ver, l, r);
```

均作用于 `[l, r)`，并返回新版本编号
例如：

```
[0, 1, 2, 3, 4]
reverse(ver, 1, 4)

⇒ [0, 3, 2, 1, 4]
```

查询

```
tree.get(ver, pos);
tree.range_sum(ver, l, r);
tree.all_sum(ver);
```

分别查询：

- 下标 `pos` 的值
- `[l, r)` 的区间和
- 整个版本的元素和
  这些查询不会 `split`，也不会 `push` 或复制节点，因此不会消耗持久化空间

批量插入

```
tree.insert(ver, pos, values);
```

将整个 `values` 插入到 `pos` 前
其中新插入部分使用 $O(m)$ 的笛卡尔树建树，而不是逐元素执行 `insert()`

注意事项

1. `touch()` 是持久化正确性的核心

普通 FHQ Treap 可以直接：

```
tr[p].ch[0] = ...
```

可持久化版本不行
一个节点可能仍被许多历史版本引用，因此任何写操作之前都必须保证它属于本次修改

```
p = touch(p);
```

如果：

```
tr[p].stamp ≠ epoch
```

就复制一个节点；反之说明这个节点已经是当前操作创建的副本，可以继续原地修改
所以：

> `stamp / epoch` 并不是版本号本身，而是一次修改过程的 COW 标记

2. 懒标记下传时，儿子也必须先 `touch`

这是最容易写错的地方：

```
tr[p].ch[d] = touch(tr[p].ch[d]);
apply_add(tr[p].ch[d], add);
```

不能直接：

```
apply_add(tr[p].ch[d], add); // 错
```

否则会修改旧版本共享的儿子节点

3. 为什么查询不能直接 `push`

普通隐式 Treap 的查询经常写：

```
push(p);
```

这里不能这么做，因为 `push()` 本身会修改节点和儿子
所以 `get()` / `range_sum()` 使用：

```
extra
flip
```

把尚未真正下传的 `add` / `rev` 沿递归传下去
因此纯查询：

```
tree.get(...)
tree.range_sum(...)
```

零新增节点

4. `rev` 的实现已经立即交换儿子

```
swap(tr[p].ch[0], tr[p].ch[1]);
tr[p].rev ^= 1;
```

所以查询时：

```
flip
```

表示的是“来自祖先、尚未实际作用到当前节点的翻转”
不要再把 `tr[p].rev` 当作“当前节点的左右儿子还没交换”，否则下标方向会算反

5. 持久化结构不要回收历史节点

即使执行：

```
erase(ver, l, r)
```

被删除区间的旧节点仍可能被历史版本引用，不能放回普通 freelist 重用

6. 类型要求

当前版本的 `T` 同时承担：

```
元素值
区间和
区间加标记
```

因此至少需要支持：

```
T{}
a + b
a += b
a * int
a == b
a != b
```

整数序列通常直接使用：

```
PersistentImplicitTreap<i64>
```

若值和区间和可能需要不同类型，例如元素为 `int`、和必须用 `i128`，则建议再把 `value` 与 `sum` 拆成两个模板参数，而不是强行让这里的 `T` 承担两种含义

常见用法

从历史版本分叉

```
PersistentImplicitTreap<i64> tree({1, 2, 3, 4});
int v1 = tree.insert(0, 2, 10);
// v1 = [1, 2, 10, 3, 4]
int v2 = tree.reverse(v1, 1, 4);
// v2 = [1, 3, 10, 2, 4]
int v3 = tree.range_add(0, 0, 2, 100);
// 从版本 0 分叉
// v3 = [101, 102, 3, 4]
```

历史版本完全不受影响：

```
cout << tree.get(0, 0) << '\n';  // 1
cout << tree.get(v3, 0) << '\n'; // 101
```

插入和删除

```
int ver = 0;
ver = tree.insert(ver, 0, 5);
ver = tree.insert(ver, 1, 7);
ver = tree.insert(ver, 1, 6);
// [5, 6, 7]
ver = tree.erase(ver, 1);
// [5, 7]
```

区间删除

```
int ver = tree.erase(old_ver, l, r); // 删除 [l, r)
```

区间加与区间和

```
int ver = tree.range_add(old_ver, l, r, delta);
i64 ans = tree.range_sum(ver, l, r);
```

区间翻转

```
int ver = tree.reverse(old_ver, l, r);
```

批量插入

```
vector<i64> b{3, 1, 4, 1, 5};
int ver = tree.insert(old_ver, pos, b);
```

比连续执行：

```
for (i64 x : b) {
    old_ver = tree.insert(old_ver, pos++, x);
}
```

更合适

每次输入操作都要求产生版本

```
int ver;
if (op == 1) {
    ver = tree.insert(old_ver, pos, x);
} else if (op == 2) {
    ver = tree.erase(old_ver, pos);
```

```
} else if (op == 3) {
    cout << tree.range_sum(old_ver, l, r) << '\n';
    ver = tree.copy_version(old_ver);
}
```

功能裁剪

如果不需要批量建树 / 批量插入，可以删除：

```
build_tree()
PersistentImplicitTreap(const vector<T> &a, ...)
insert(ver, pos, const vector<T> &values)
```

如果不需要区间翻转，可以删除：

```
Node::rev
apply_rev()
reverse()
```

同时删除 `push()` 中 `rev` 部分；`get_node()` / `range_sum_node()` 中的 `flip` 逻辑也可同步简化为固定访问 `ch[0]` / `ch[1]`
如果不需要区间加，可以删除：

```
Node::add
apply_add()
range_add()
```

同时删除 `push()` 中 `add` 部分，以及只读查询中的 `extra` 传递
如果区间加和区间和都不要，还能继续删除：

```
Node::sum
node_sum()
range_sum_node()
range_sum()
all_sum()
```

并把：

```
void pull(int p) {
    tr[p].size = 1 + node_size(tr[p].ch[0]) + node_size(tr[p].ch[1]);
}
```

作为唯一维护信息
最终就会退化成一个相当短的：

> 可持久化动态数组 / Rope 式序列

只保留：

```
split / merge
insert / erase / set / get
版本分叉
```

这部分裁剪关系正是这类中大型板子最值得明确记录的内容

复杂度

设当前序列长度为 n，批量插入长度为 m

操作	期望时间	新增节点
初始 `build`	$O(n)$	$O(n)$
`copy_version` / `all_sum`	$O(1)$	0
单点 `insert` / 单点 `erase` / `set` / 区间 `erase` / `range_add` / `reverse`	$O(\log n)$	$O(\log n)$
`get` / `range_sum`	$O(\log n)$	0
批量插入 m 个元素	$O(m+\log(n+m))$	$O(m+\log(n+m))$

Treap 随机平衡，因此这些均为期望复杂度；理论最坏树高仍为 $O(n)$

3.8.7 Splay 序列与区间暴露

```
template<class T>
struct SplaySeq {
    struct Node {
        int ch[2]{}, fa{}, size{};
        bool rev{};
        T value{};
    };
    vector<Node> tr{Node{}};
    vector<int> bin, stk;
    int root{};
    int guard[2]{};
    SplaySeq() { build({}); }
    explicit SplaySeq(const vector<T> &a) { build(a); }
private:
    int node_size(int p) const { return tr[p].size; }
    // ===== 节点维护：所有操作依赖，不建议删除 =====
    void pull(int p) {
        tr[p].size = node_size(tr[p].ch[0]) + 1 + node_size(tr[p].ch[1]);
    }
    int new_node(const T &value = T{}) {
        int p;
        if (bin.empty()) {
            p = tr.size();
            tr.push_back({});
        } else {
            p = bin.back();
            bin.pop_back();
            tr[p] = {};
        }
        tr[p].size = 1;
        tr[p].value = value;
        return p;
    }
    // 删除大区间时避免递归回收造成栈溢出
    void recycle(int p) {
        if (!p) return;
        stk.clear();
        stk.push_back(p);
        for (int i = 0; i < stk.size(); ++ i) {
            int x = stk[i];
            for (int c : tr[x].ch) if (c) stk.push_back(c);
        }
        for (int x : stk | views::reverse) {
            tr[x] = {};
            bin.push_back(x);
        }
        stk.clear();
    }
    // ===== 区间翻转：不需要 reverse() 时可删除 rev / apply_rev() =====
    // 并把 push() 改为空函数 =====
    void apply_rev(int p) {
        if (!p) return;
        swap(tr[p].ch[0], tr[p].ch[1]);
        tr[p].rev ^= 1;
    }
    void push(int p) {
        if (!p || !tr[p].rev) return;
        apply_rev(tr[p].ch[0]);
        apply_rev(tr[p].ch[1]);
        tr[p].rev = false;
    }
    // ===== O(n) 平衡建树：构造和批量插入依赖 =====
    int build_rec(const vector<int> &id, int l, int r, int fa) {
        if (l >= r) return 0;
        int m = (l + r) / 2, p = id[m];
        tr[p].fa = fa;
        tr[p].ch[0] = build_rec(id, l, m, p);
```

```
        tr[p].ch[1] = build_rec(id, m + 1, r, p);
        pull(p);
        return p;
    }
    int build_values(const vector<T> &a) {
        vector<int> id;
        id.reserve(a.size());
        for (const T &value : a) id.push_back(new_node(value));
        return build_rec(id, 0, id.size(), 0);
    }
    // ===== Splay 核心：rotate / splay，不建议删除 =====
    void push_path(int p, int goal = 0) {
        stk.clear();
        for (int x = p; x != goal; x = tr[x].fa) stk.push_back(x);
        for (int x : stk | views::reverse) push(x);
    }
    void rotate(int p) {
        int fa = tr[p].fa, g = tr[fa].fa;
        int d = tr[fa].ch[1] == p;
        int x = tr[p].ch[d ^ 1];
        if (g) tr[g].ch[tr[g].ch[1] == fa] = p;
        else root = p;
        tr[p].fa = g;
        tr[p].ch[d ^ 1] = fa;
        tr[fa].fa = p;
        tr[fa].ch[d] = x;
        if (x) tr[x].fa = fa;
        pull(fa);
        pull(p);
    }
    void splay(int p, int goal = 0) {
        push_path(p, goal);
        while (tr[p].fa != goal) {
            int fa = tr[p].fa, g = tr[fa].fa;
            if (g != goal) {
                if ((tr[fa].ch[1] == p) == (tr[g].ch[1] == fa)) rotate(fa);
                else rotate(p);
            }
            rotate(p);
        }
        if (!goal) root = p;
    }
    // ===== 按排名定位：expose / 单点接口依赖 =====
    int kth_node(int k) {
        assert(0 <= k && k < node_size(root));
        int p = root;
        while (true) {
            push(p);
            int left = node_size(tr[p].ch[0]);
            if (k < left)  p = tr[p].ch[0];
            else if (k == left) return p;
            else {
                k -= left + 1;
                p = tr[p].ch[1];
            }
        }
    }
    // ===== 区间暴露：将真实序列 [l,r) 暴露为右边界的左子树 =====
    int expose(int l, int r) {
        assert(0 <= l && l <= r && r <= size());
        int left = kth_node(l);
        splay(left);
        int right = kth_node(r + 1);
        splay(right, left);
        return tr[right].ch[0];
    }
    // ===== 区间拆接：erase / move / insert 依赖 =====
    int detach(int l, int r) {
        int mid = expose(l, r);
        int right = tr[root].ch[1];
        tr[right].ch[0] = 0;
```

```
        if (mid) tr[mid].fa = 0;
        pull(right);
        pull(root);
        return mid;
    }
    void attach(int pos, int p) {
        if (!p) return;
        assert(0 <= pos && pos <= size());
        int left = kth_node(pos);
        splay(left);
        int right = kth_node(pos + 1);
        splay(right, left);
        tr[right].ch[0] = p;
        tr[p].fa = right;
        pull(right);
        pull(left);
    }
public:
    // ===== 初始化与状态 =====
    void build(const vector<T> &a) {
        tr.clear();
        tr.push_back({});
        bin.clear();
        stk.clear();
        root = 0;
        vector<int> id;
        id.reserve(a.size() + 2);
        guard[0] = new_node();
        id.push_back(guard[0]);
        for (const T &value : a) id.push_back(new_node(value));
        guard[1] = new_node();
        id.push_back(guard[1]);
        root = build_rec(id, 0, id.size(), 0);
    }
    void clear() { build({}); }
    void reserve(int nodes) {
        tr.reserve(nodes + 3);
        bin.reserve(nodes);
        stk.reserve(nodes + 2);
    }
    int size() const { return node_size(root) - 2; }
    bool empty() const { return size() == 0; }
    // ===== 基础区间操作 =====
    void reverse(int l, int r) {
        assert(0 <= l && l <= r && r <= size());
        if (l < r) apply_rev(expose(l, r));
    }
    // ===== 插入：若只需要文艺平衡树，可整段删除 =====
    void insert(int pos, const T &value) {
        assert(0 <= pos && pos <= size());
        attach(pos, new_node(value));
    }
    void insert(int pos, const vector<T> &a) {
        assert(0 <= pos && pos <= size());
        if (!a.empty()) attach(pos, build_values(a));
    }
    // ===== 删除：若不需要删除，可删除 recycle / bin 以及本段 =====
    T erase(int pos) {
        assert(0 <= pos && pos < size());
        int p = detach(pos, pos + 1);
        T value = std::move(tr[p].value);
        recycle(p);
        return value;
    }
    void erase(int l, int r) {
        assert(0 <= l && l <= r && r <= size());
        if (l < r) recycle(detach(l, r));
    }
    // ===== 区间搬移：不需要时可直接删除 =====
    // pos 是移动前原序列的位置，表示把 [l,r) 移动到 pos 前
    void move(int l, int r, int pos) {
```

```
        assert(0 <= l && l <= r && r <= size());
        assert(0 <= pos && pos <= size());
        if (l >= r || (l <= pos && pos <= r)) return;
        int mid = detach(l, r);
        if (pos > r) pos -= r - l;
        attach(pos, mid);
    }
    // ===== 单点操作：不需要时可直接删除 =====
    T get(int pos) {
        assert(0 <= pos && pos < size());
        int p = kth_node(pos + 1);
        splay(p);
        return tr[p].value;
    }
    void set(int pos, const T &value) {
        assert(0 <= pos && pos < size());
        int p = kth_node(pos + 1);
        splay(p);
        tr[p].value = value;
    }
    // ===== 导出序列：输出 / 调试需要时保留 =====
    vector<T> to_vector() {
        vector<T> a;
        a.reserve(size());
        vector<int> st;
        int p = root;
        while (p || !st.empty()) {
            while (p) {
                push(p);
                st.push_back(p);
                p = tr[p].ch[0];
            }
            p = st.back();
            st.pop_back();
            if (p != guard[0] && p != guard[1]) a.push_back(tr[p].value);
            p = tr[p].ch[1];
        }
        return a;
    }
};
```

接口说明

两个哨兵

内部中序序列实际上是：

```
左哨兵 + 真实序列 + 右哨兵
```

所以真实长度直接是：

```
node_size(root) - 2
```

不再额外维护一个 `n`，这样 `detach / attach` 时也不会出现“树大小和 `n` 忘记同步”的问题
真实下标 `pos` 对应内部排名：

```
pos + 1
```

区间暴露 `expose(l,r)`

对于真实区间：

```
[l,r)
```

先找到：

```
left  = kth_node(l);
right = kth_node(r + 1);
```

然后：

```
splay(left);
splay(right, left);
```

此时结构固定为：

```
          left
            \
           right
           /
        [l,r)
```

因此：

```
tr[right].ch[0]
```

恰好就是整个目标区间
所以以后任何区间操作，本质都是：

```
int p = expose(l, r);
// 对 p 整棵子树操作
```

注意事项

- 外部统一使用 `0base` 和 `[l,r)`
- 两个哨兵永远不属于真实序列，也不会被公开接口删除或搬移
- `kth_node()` 查询的是内部中序排名，所以真实元素 `pos` 对应 `pos + 1`
- 在存在 `rev` 时，下降到儿子前必须 `push()`；`splay()` 旋转前必须把从祖先到目标节点的整条路径下传
- `reverse(l,r)` 只修改目标子树的翻转标记，不需要重新 `pull()`，因为翻转不会改变子树大小
- `move(l,r,pos)` 中 `pos` 是移动前原序列的位置。若 `pos` 位于 `[l,r]`，移动后序列不变
- `erase(l,r)` 为了回收节点，需要遍历被删除的全部元素，因此为 $O(\log n + r - l)$；如果不要求回收节点，可以直接丢弃子树，把删除降到均摊 $O(\log n)$
- Splay 单次操作可能是 $O(n)$，一串操作均摊为 $O(\log n)$
- 这是序列 Splay，不按 `value` 排序；排名、第 k 小、前驱、后继请使用前面的 FHQ Treap 有序多重集
- 与隐式 FHQ Treap 相比，Splay 最有特色的是 `splay(x, goal)` 和稳定的区间 `expose`，也是理解 LCT 的重要前置结构

常见用法

文艺平衡树

```
vector<int> a(n);
iota(a.begin(), a.end(), 1);
SplaySeq<int> sp(a);
while (m --) {
    int l, r;
    cin >> l >> r;
    sp.reverse(l - 1, r);
}
for (int x : sp.to_vector()) {
    cout << x << ' ';
}
```

动态插入和删除

```
SplaySeq<int> sp(a);
sp.insert(pos, x);
sp.insert(pos, vector<int>{1, 2, 3});
int old = sp.erase(pos);
sp.erase(l, r);
```

区间搬移

```
sp.move(l, r, pos);
```

表示：

> 把原序列 `[l,r)` 剪下来，插到原位置 `pos` 之前

例如：

```
0 1 2 3 4 5
move(1, 3, 5)
0 3 4 1 2 5
```

交换两个相邻区间

若：

```
A [l,m) [m,r) D
```

想变成：

```
A [m,r) [l,m) D
```

直接：

```
sp.move(l, m, r);
```

区间循环右移

把 `[l,r)` 循环右移 `k` 位：

```
int len = r - l;
if (len) {
    k %= len;
    if (k) sp.move(r - k, r, l);
}
```

循环左移同理：

```
if (len) {
    k %= len;
    if (k) sp.move(l, l + k, r);
}
```

单点访问

```
T x = sp.get(pos);
sp.set(pos, value);
```

常见扩展：加入区间聚合

例如需要维护区间和，只需要增加几处
`Node` 增加：

```
T sum{};
```

`new_node()` 增加：

```
tr[p].sum = value;
```

`pull()` 改成：

```
void pull(int p) {
    tr[p].size = node_size(tr[p].ch[0]) + 1 + node_size(tr[p].ch[1]);
    tr[p].sum =
        (tr[p].ch[0] ? tr[tr[p].ch[0]].sum : T{}) +
        tr[p].value +
        (tr[p].ch[1] ? tr[tr[p].ch[1]].sum : T{});
}
```

然后区间和就是：

```
T range_sum(int l, int r) {
    if (l == r) return T{};
    return tr[expose(l, r)].sum;
}
```

`set()` 中修改完：

```
tr[p].value = value;
pull(p);
```

即可
同样的方式可以维护：

```
sum
min / max
xor
最大子段和
区间哈希
```

只要信息能由左右子树和当前节点在 `pull()` 中合并即可

加入区间加

若还需要：

```
range_add(l,r,x)
range_sum(l,r)
```

再增加：

```
T add{};
```

以及：

```
void apply_add(int p, const T &x) {
    if (!p) return;
    tr[p].value += x;
    tr[p].sum += x * tr[p].size;
    tr[p].add += x;
}
```

并在 `push()` 中下传
修改整段以后，因为区间聚合值发生了变化，还应更新两个边界：

```
void range_add(int l, int r, const T &x) {
    if (l == r) return;
    int p = expose(l, r);
    apply_add(p, x);
    int right = tr[root].ch[1];
    pull(right);
    pull(root);
}
```

这就是 Splay 的一个优势：

```
expose 区间
↓
整棵目标子树打标记
↓
恢复两个边界信息
```

结构非常固定

功能裁剪

如果题目只是经典文艺平衡树，只需要保留：

```
Node::size / rev
new_node / pull
apply_rev / push
build
push_path / rotate / splay
kth_node / expose
reverse
to_vector
```

可以整个删除：

```
bin / recycle
build_values
detach / attach
insert
erase
move
get / set
```

如果连区间翻转都不需要，则再删除：

```
Node::rev
apply_rev
reverse
```

并让：

```
void push(int) {}
```

或者进一步删掉所有 `push()`

复杂度

操作	时间复杂度
`build`	$O(n)$
`kth_node` / `expose` / `reverse` / 单点 `insert` / `erase(pos)` / `move` / `get` / `set`	均摊 $O(\log n)$
批量插入 m 个元素	$O(m+\log n)$
`erase(l,r)`	$O(\log n + r - l)$
`to_vector`	$O(n)$

空间复杂度为 $O(n)$；启用节点回收后，被删除节点可以继续复用

3.9 分块、莫队与离线分治

3.9.1 数列分块

基础分块：区间加与区间和

```
template<class T = i64>
struct BlockSum {
    int n, B, m;
    vector<T> a, s, t;
    explicit BlockSum(const vector<T> &v, int bs = 0)
        : n(v.size()), B(bs ? min(bs, max(1, n)) : max(1, int(sqrt(max(1, n))))),
          m(n / B + (n % B != 0)), a(v), s(m), t(m) {
        for (int i = 0; i < n; ++ i) s[i / B] += a[i];
    }
    void add(int l, int r, T x) {
        if (l >= r) return;
        int p{ l / B }, q{ (r - 1) / B };
        if (p == q) {
            for (int i = l; i < r; ++ i) a[i] += x;
            s[p] += x * (r - l);
```

```
            return;
        }
        int pr{ min(n, (p + 1) * B) };
        for (int i = l; i < pr; ++ i) a[i] += x;
        s[p] += x * (pr - l);
        for (int b = p + 1; b < q; ++ b) t[b] += x;
        int ql{ q * B };
        for (int i = ql; i < r; ++ i) a[i] += x;
        s[q] += x * (r - ql);
    }
    T sum(int l, int r) const {
        if (l >= r) return T{};
        int p{ l / B }, q{ (r - 1) / B };
        T res{};
        if (p == q) {
            for (int i = l; i < r; ++ i) res += a[i] + t[p];
            return res;
        }
        int pr{ min(n, (p + 1) * B) };
        for (int i = l; i < pr; ++ i) res += a[i] + t[p];
        for (int b = p + 1; b < q; ++ b) {
            int len{ min(n, (b + 1) * B) - b * B };
            res += s[b] + t[b] * len;
        }
        for (int i = q * B; i < r; ++ i) res += a[i] + t[q];
        return res;
    }
};
```

成员变量

- `n`：序列长度
- `B`：块长
- `m`：块数
- `a[i]`：去掉块标记后的值
- `s[b]`：第 `b` 块的 `a[i]` 之和
- `t[b]`：第 `b` 块整体加法标记
- 实际值为 `a[i] + t[i / B]`

接口

- `BlockSum(a, bs)`：建块，`bs = 0` 时自动取约 $\sqrt{n}$ 的块长，$O(n)$
- `add(l, r, x)`：区间 `[l,r)` 加 `x`，$O(B+n/B)$
- `sum(l, r)`：查询 `[l,r)` 区间和，$O(B+n/B)$

注意事项

- 下标为 `0base`，区间为 `[l,r)`
- 完整块修改只修改 `t[b]`
- 边角块修改 `a[i]`，同时修正 `s[b]`
- `s[b]` 不包含 `t[b]`
- 这份板子主要作为数列分块的基础骨架，若题目只有区间加和区间和，线段树或树状数组通常更优

常见用法

```
BlockSum<> b(a);
b.add(l, r, x);
i64 ans = b.sum(l, r);
```

复杂度

操作	时间复杂度
构造	$O(n)$
`add`	$O(B+n/B)$
`sum`	$O(B+n/B)$

空间复杂度 $O(n)$
取 $B=\Theta(\sqrt{n})$ 时，区间操作为 $O(\sqrt{n})$

块内有序分块

```
template<class T = i64>
struct BlockOrd {
    using W = conditional_t<is_integral_v<T>, i128, T>;
    int n, B, m;
    vector<T> a, t;
    vector<vector<T>> v;
    explicit BlockOrd(const vector<T> &a_, int bs = 0)
        : n(a_.size()), B(bs ? min(bs, max(1, n)) : max(1, int(sqrt(max(1, n))))),
          m(n / B + (n % B != 0)), a(a_), t(m), v(m) {
        for (int b = 0; b < m; ++ b) pull(b);
    }
    // ===== 块内重构 =====
    void pull(int b) {
        int l{ b * B }, r{ min(n, (b + 1) * B) };
        v[b].assign(a.begin() + l, a.begin() + r);
        sort(v[b].begin(), v[b].end());
    }
    // ===== 基础接口 =====
    T get(int i) const { return a[i] + t[i / B]; }
    void set(int i, T x) {
        int b{ i / B };
        a[i] = x - t[b];
        pull(b);
    }
    void add(int l, int r, T x) {
        if (l >= r) return;
        int p{ l / B }, q{ (r - 1) / B };
        if (p == q) {
            for (int i = l; i < r; ++ i) a[i] += x;
            pull(p);
            return;
        }
        int pr{ min(n, (p + 1) * B) };
        for (int i = l; i < pr; ++ i) a[i] += x;
        pull(p);
        for (int b = p + 1; b < q; ++ b) t[b] += x;
        for (int i = q * B; i < r; ++ i) a[i] += x;
        pull(q);
    }
    int lt(int l, int r, T x) const {
        if (l >= r) return 0;
        int p{ l / B }, q{ (r - 1) / B }, res{};
        if (p == q) {
            for (int i = l; i < r; ++ i) res += a[i] + t[p] < x;
            return res;
        }
        int pr{ min(n, (p + 1) * B) };
        for (int i = l; i < pr; ++ i) res += a[i] + t[p] < x;
        for (int b = p + 1; b < q; ++ b) {
            res += lower_bound(v[b].begin(), v[b].end(), W(x) - t[b]) - v[b].begin();
        }
        for (int i = q * B; i < r; ++ i) res += a[i] + t[q] < x;
        return res;
    }
    // ===== 前驱后继 =====
    optional<T> pre(int l, int r, T x) const {
        if (l >= r) return nullopt;
        int p{ l / B }, q{ (r - 1) / B };
        optional<T> res;
        auto upd = [&](T y) {
            if (y < x && (!res || y > *res)) res = y;
        };
        if (p == q) {
            for (int i = l; i < r; ++ i) upd(a[i] + t[p]);
            return res;
        }
        int pr{ min(n, (p + 1) * B) };
```

```
        for (int i = l; i < pr; ++ i) upd(a[i] + t[p]);
        for (int b = p + 1; b < q; ++ b) {
            auto it{ lower_bound(v[b].begin(), v[b].end(), W(x) - t[b]) };
            if (it != v[b].begin()) upd(*prev(it) + t[b]);
        }
        for (int i = q * B; i < r; ++ i) upd(a[i] + t[q]);
        return res;
    }
    optional<T> suc(int l, int r, T x) const {
        if (l >= r) return nullopt;
        int p{ l / B }, q{ (r - 1) / B };
        optional<T> res;
        auto upd = [&](T y) {
            if (y > x && (!res || y < *res)) res = y;
        };
        if (p == q) {
            for (int i = l; i < r; ++ i) upd(a[i] + t[p]);
            return res;
        }
        int pr{ min(n, (p + 1) * B) };
        for (int i = l; i < pr; ++ i) upd(a[i] + t[p]);
        for (int b = p + 1; b < q; ++ b) {
            auto it{ upper_bound(v[b].begin(), v[b].end(), W(x) - t[b]) };
            if (it != v[b].end()) upd(*it + t[b]);
        }
        for (int i = q * B; i < r; ++ i) upd(a[i] + t[q]);
        return res;
    }
};
```

成员变量

- `a[i]`：去掉块标记后的值
- `t[b]`：第 `b` 块整体加法标记
- `v[b]`：第 `b` 块所有 `a[i]` 排序后的结果

接口

- `BlockOrd(a, bs)`：建块并排序，$O(n \log B)$
- `pull(b)`：重建第 `b` 块，$O(B \log B)$
- `get(i)`：查询实际值，$O(1)$
- `set(i, x)`：单点赋值，$O(B \log B)$
- `add(l, r, x)`：区间 `[l,r)` 加 `x`，$O(B \log B + n/B)$
- `lt(l, r, x)`：查询 `[l,r)` 中严格小于 `x` 的数量，$O(B + (n/B) \log B)$
- `pre(l, r, x)`：查询严格小于 `x` 的最大值，不存在返回 `nullopt`
- `suc(l, r, x)`：查询严格大于 `x` 的最小值，不存在返回 `nullopt`

注意事项

- `v[b]` 保存的是裸值 `a[i]`，不包含 `t[b]`
- 完整块二分使用宽类型计算 `x - t[b]`，避免查询阈值在 T 极值处溢出；实际值、裸值、标记及修改运算仍须可由 T 表示
- 完整块加法不改变相对顺序，因此无需 `pull()`
- 边角修改会改变块内顺序，修改后必须 `pull()`
- `lt()` 统计 `< x`，不是 `<= x`
- 若不需要前驱后继，可直接删除 `pre()`、`suc()`
- 整数值域下可以对答案二分，并用 `lt()` 求区间第 `k` 小，不单独封装 `kth()`

常见用法

区间加

```
BlockOrd<> b(a);
b.add(l, r, x);
```

查询小于 `x` 的数量

```
int cnt = b.lt(l, r, x);
```

查询前驱后继

```
auto p = b.pre(l, r, x);
auto s = b.suc(l, r, x);
if (p) {
    // *p
}
if (s) {
    // *s
}
```

复杂度

操作	时间复杂度
构造	$O(n \log B)$
`pull`	$O(B \log B)$
`get`	$O(1)$
`set`	$O(B \log B)$
`add`	$O(B \log B + n/B)$
`lt`	$O(B + (n/B) \log B)$
`pre` / `suc`	$O(B + (n/B) \log B)$

空间复杂度 $O(n)$
取 $B = \Theta(\sqrt{n})$ 时，修改与顺序统计通常为 $O(\sqrt{n} \log n)$

重构型分块

当完整块不能只靠一个标记维护时，可以将块内真实值下放，暴力修改，再重新计算块信息
常见于区间开方、区间取模、 `chmin` 、 `chmax` 等特殊修改

改造骨架

```
void push(int b) {
    if (!t[b]) return;
    int l{ b * B }, r{ min(n, (b + 1) * B) };
    for (int i = l; i < r; ++ i) a[i] += t[b];
    t[b] = 0;
}

void pull(int b) {
    int l{ b * B }, r{ min(n, (b + 1) * B) };
    // 根据 a[l..r) 重建 sum / mn / mx / v 等块信息
}
```

修改一个无法整体处理的块时通常写成

```
push(b);
for (int i = l; i < r; ++ i) {
    a[i] = f(a[i]);
}
pull(b);
```

注意事项

- `push()` 与 `pull()` 只是改造模式，不是独立通用数据结构
- 若块信息能判断本次操作对整块无影响，可直接跳过
- 若完整块能整体修改，应优先整体处理
- 若必须频繁暴力重构，需要证明总重构次数或势能下降有均摊上界
- 不能因为使用分块就默认复杂度是 $O(\sqrt{n})$

块长选择

设块长为 B，块数约为 n/B，基础分块单次操作通常为

$$O\left(B + \frac{n}{B}\right)$$

因此通常取

$$B = \Theta(\sqrt{n})$$

若块内还需要排序、重构或其他较重操作，最优块长会受修改与查询比例影响，可通过构造函数参数 `bs` 手动调整

3.9.2 根号分治与大小分类

根号分治的核心是找到一个非负规模量 w_i，设总量

$$S = \sum_i w_i$$

取阈值 B，将对象分成

- 轻对象：$w_i < B$，单个对象可在 $O(B)$ 内暴力处理
- 重对象：$w_i \geq B$，数量至多为 S/B
- 若总复杂度形如 $O(AB + C/B)$，通常取 $B = \Theta(\sqrt{C/A})$
  若操作能转化为“枚举一个轻对象”或“枚举所有重对象”，单次复杂度通常为

  $$O\left(B + \frac{S}{B}\right)$$

  取 $B = \Theta(\sqrt{S})$ 时得到 $O(\sqrt{S})$

通用大小分类

```
struct SqrtSplit {
    int B;
    vector<int> id, hi;
    explicit SqrtSplit(const vector<int> &w, int bs = 0) : id(w.size(), -1) {
        i64 s{};
        for (int x : w) s += x;
        B = bs ? bs : max(1, int(sqrt(max<i64>(1, s))));
        for (int i = 0; i < w.size(); ++ i) {
            if (w[i] ≥ B) id[i] = hi.size(), hi.push_back(i);
        }
    }
    bool high(int x) const { return id[x] =≠ -1; }
};
```

成员变量

- `B`：轻重分类阈值
- `hi`：所有重对象的原编号
- `id[x]`：对象 `x` 在 `hi` 中的编号，轻对象为 `-1`

接口

- `SqrtSplit(w, bs)`：按权值 `w[i]` 分类，`bs = 0` 时取约 $\sqrt{\sum w_i}$，$O(n)$
- `high(x)`：判断 `x` 是否为重对象，$O(1)$

注意事项

- 要求 `w[i]≥0`
- 若阈值为 `B`，重对象数量至多为 $\lfloor S/B \rfloor$
- `w[i]` 不一定是真正的权值，常见含义是出现次数、度数、集合大小、操作次数
- `B` 不一定必须取 $\sqrt{S}$，应根据轻对象和重对象两侧的真实复杂度平衡

常见用法

高频与低频元素

序列已离散化到 `[0,m)`

```
vector<int> cnt(m);
for (int x : a) ++ cnt[x];

SqrtSplit sp(cnt);
```

```
for (int h = 0; h < int(sp.hi.size()); ++ h) {
    int x = sp.hi[h];
    // 对高频值 x 预处理
}
```

高频值数量至多为 n/B，低频值出现次数小于 B
这种分类常用于：

- 高频值预处理整组答案
- 低频值枚举全部出现位置
- 区间众数与频率相关询问
- 按颜色、种类、标签的出现次数分类

复杂度

设对象数为 k，总权值为 S

操作	时间复杂度
构造	$O(k)$
`high`	$O(1)$
枚举全部重对象	$O(S/B)$

空间复杂度 $O(k)$

小参数预处理与大参数暴力

另一类常见根号分治不是按对象频率，而是按询问参数 k 分类
若 $k < B$，参数种类只有 $O(B)$ 个，可以预处理
若 $k \geq B$，并且每次暴力处理都会至少前进或消耗 k 个单位，则单次最多处理

$$O\left(\frac{S}{k}\right) \leq O\left(\frac{S}{B}\right)$$

典型结构为

```
k < B
    对每个小 k 预处理答案

k ⩾ B
    直接按 k 跳跃 / 枚举
    操作次数 ⩽ S / k ⩽ S / B
```

例如每次按照步长 `k` 访问位置：

```
if (k < B) {
    // 读取预处理的 f[k]
} else {
    for (int i = st; i < n; i += k) {
        // 暴力处理
    }
}
```

注意事项

- “大参数暴力更快”必须有类似 S/k 的上界才能成立
- 小参数预处理复杂度需要单独计算，不能机械认为是 $O(SB)$
- 若小参数预处理为 $O(PB)$、大参数单次为 $O(S/B)$，应根据 P, S, q 平衡 B
- `B=sqrt(S)` 只是最常见选择，不一定最优

度数分治

无向图中令

$$w_u = \deg(u), \quad S = \sum_u \deg(u) = 2m$$

则

- 轻点满足 $\deg(u) < B$
- 重点数量至多为 $2m/B$
  这是根号分治非常典型的应用
  下面的板子维护动态点权，支持：
- 单点权值增加
- 查询一个点所有邻点的权值和
  轻点查询直接枚举邻边，重点查询预处理答案

```
template<class T = i64>
struct DegSum {
    const vector<vector<int>> &g;
    int n, B;
    vector<int> id, hi;
    vector<vector<pair<int, int>>> hs;
    vector<T> a, s;
    explicit DegSum(const vector<vector<int>> &g_, vector<T> a_ = {}, int bs = 0)
        : g(g_), n(g.size()), id(n, -1), hs(n), a(move(a_)) {
        if (a.empty()) a.assign(n, T{});
        i64 d{};
        for (const auto &e : g) d += e.size();
        B = bs ? bs : max(1, int(sqrt(max<i64>(1, d))));
        for (int u = 0; u < n; ++ u) {
            if (g[u].size() >= B) {
                id[u] = hi.size();
                hi.push_back(u);
            }
        }
        s.assign(hi.size(), T{});
        // 统计 u 与每个重点之间的边数，重边只保留一项
        vector<int> cnt(hi.size());
        for (int u = 0; u < n; ++ u) {
            vector<int> used;
            for (int v : g[u]) {
                int h{ id[v] };
                if (h != -1 && cnt[h] ++ == 0) used.push_back(h);
            }
            for (int h : used) {
                hs[u].push_back({ h, cnt[h] });
                cnt[h] = 0;
            }
        }
        for (int h = 0; h < hi.size(); ++ h) {
            for (int v : g[hi[h]]) s[h] += a[v];
        }
    }
    void add(int u, T x) {
        a[u] += x;
        for (auto [h, c] : hs[u]) s[h] += x * c;
    }
    T ask(int u) const {
        if (id[u] != -1) return s[id[u]];
        T res{};
        for (int v : g[u]) res += a[v];
        return res;
    }
};
```

成员变量

- `B`：度数分类阈值
- `id[u]`：重点 `u` 的编号，轻点为 `-1`
- `hi`：所有重点
- `hs[u]`：`{h,c}` 表示 `u` 与重点 `hi[h]` 之间有 `c` 条边
- `a[u]`：点权
- `s[h]`：重点 `hi[h]` 所有邻点的点权和

接口

- `DegSum(g, a, bs)`：预处理轻重点，`a` 为空时初始点权全为 `0`

- `add(u, x)`：令点 `u` 的权值增加 `x`
- `ask(u)`：查询 `u` 所有邻点的点权和

注意事项

- `g` 应为无向图邻接表，每条边双向加入
- `g` 的生命周期必须覆盖 `DegSum`
- 若传入 `a`，要求 `a.size() == g.size()`
- 默认不支持动态加边删边，图结构变化后轻重分类可能失效
- 若存在重边，查询会按邻接表中的边重复计贡献
- `ask(u)` 不包含 `u` 自身，除非图中存在自环
- 单点赋值可以写成 `add(u, x - a[u])`

常见用法

```
DegSum<> d(g, a);

d.add(u, x);
i64 ans = d.ask(u);
```

复杂度

设无向图有 m 条边，阈值为 B
重点数量为

$$O\left(\frac{m}{B}\right)$$

因此

操作	时间复杂度
构造	$O(n+m)$
`add`	$O(m/B)$
轻点 `ask`	$O(B)$
重点 `ask`	$O(1)$

空间复杂度 $O(n+m)$
取

$$B=\Theta(\sqrt{m})$$

时单次操作最坏为

$$O(\sqrt{m})$$

常见分类方式

分类对象	规模量 `w`	重对象数量上界
数值 / 颜色	出现次数	$O(n/B)$
图上节点	度数	$O(m/B)$
集合	集合大小	$O(S/B)$
参数 `k`	参数本身	通常用 `k<B` 与 `k⩾B` 分类

注意事项

- 根号分治本质上不是固定算法，而是利用“轻对象单个便宜、重对象数量少”平衡复杂度
- 在写复杂度前先明确总量 S 是什么，不能看到“大小分类”就直接写 $O(\sqrt{n})$
- 高频/低频分类时通常应先离散化，避免值域过大
- 重对象预处理如果需要两两关系，空间可能达到 $O((S/B)^2)$，应提前检查
- 若轻对象处理不是 $O(B)$，或重对象数量不能由 S/B 控制，就需要重新选择阈值或换算法
- 与 3.9.1 的数列分块相比，这一节的“块”通常不是连续下标区间，而是按频率、度数、参数大小进行分类

> Tip

赛场上判断能不能根号分治，优先找一个总量受限的量 S，再问两个问题：小于 B 的对象能否暴力处理，大于等于 B 的对象是否只有 $O(S/B)$ 个
两个条件同时成立，基本就找到了大小分类的切入点

3.9.3 莫队基础与带修改

普通莫队按左端点分块，并让相邻块中的右端点反向排序以减少指针移动，常用块长包括 `n / min(n, sqrt(q))` 与约 $\sqrt{n}$
带修改莫队在询问中增加时间维，按左端点块、右端点块、时间排序，推荐块长可取 `pow(n, 0.6666)`

普通莫队

```
struct Mo {
    struct Q {
        int l, r, id;
    };
    int n, B;
    vector<Q> q;
    explicit Mo(int n_, int bs = 0) : n(n_), B(bs) {}
    void ask(int l, int r, int id) { q.push_back({ l, r, id }); }
    template<class Add, class Del, class Save>
    void run(Add add, Del del, Save save) {
        if (q.empty()) return;
        if (!B) {
            int k{ max(1, min(n, int(sqrt(q.size())))) };
            B = max(1, n / k);
        }
        sort(q.begin(), q.end(), [&](const Q &a, const Q &b) {
            int x{ a.l / B }, y{ b.l / B };
            if (x != y) return x < y;
            return x & 1 ? a.r < b.r : a.r > b.r;
        });
        int l{}, r{};
        for (auto [ql, qr, id] : q) {
            while (l > ql) add(-- l);
            while (r < qr) add(r ++);
            while (l < ql) del(l ++);
            while (r > qr) del(-- r);
            save(id);
        }
    }
};
```

成员变量

- `n`：序列长度
- `B`：块长，传 `0` 时根据询问数自动选择
- `q`：所有离线询问
- `Q::l,r`：询问区间 `[l,r)`
- `Q::id`：询问编号

接口

- `Mo(n, bs)`：构造莫队，`bs = 0` 时自动选择块长
- `ask(l, r, id)`：加入区间 `[l,r)` 的询问，$O(1)$
- `run(add, del, save)`：排序询问并依次处理
- `add(p)`：将位置 `p` 加入当前区间
- `del(p)`：将位置 `p` 移出当前区间
- `save(id)`：当前状态即询问 `id` 的答案，在此保存答案

注意事项

- 下标为 `0base`，区间统一使用 `[l,r)`
- 所有询问必须提前得到，属于离线算法
- `add()`、`del()` 应能快速维护当前答案，通常要求 $O(1)$
- `del()` 必须能够撤销一个元素的贡献，若删除难以维护，应使用后面的回滚莫队
- 莫队本身不处理离散化，值域过大时应提前离散化

- 默认块长参考资料中的 `n / sqrt(q)` 型选择，相比固定 $\sqrt{n}$ 在询问数较少时通常更合适
- 奇偶块内让 `r` 交替升降不会改变正确性，只用于减少常数

常见用法

区间不同数个数

```
vector<int> cnt(m), ans(qn);
int cur{};
Mo mo(n);
for (int i = 0; i < qn; ++ i) {
    int l, r;
    cin >> l >> r;
    mo.ask(l, r, i);
}
auto add = [&](int p) {
    if (cnt[a[p]] ++ == 0) ++ cur;
};
auto del = [&](int p) {
    if (-- cnt[a[p]] == 0) -- cur;
};
mo.run(add, del, [&](int id) {
    ans[id] = cur;
});
```

复杂度

设有 q 次询问，块长为 B，且 `add()`、`del()` 均为 $O(1)$
按上述排序方式估算，总指针移动量为

$$O\left(qB+\frac{n^2}{B}\right)$$

取

$$B\approx\frac{n}{\sqrt{q}}$$

得到

$$O(n\sqrt{q})$$

再加排序复杂度 $O(q\log q)$
当 $q=\Theta(n)$ 时即常见的

$$O(n\sqrt{n})$$

空间复杂度 $O(q)$，不计题目自身维护的信息

带修改莫队

普通莫队只移动左右端点，带修改莫队额外维护一个时间指针 `t`
每个询问记录

```
(l, r, t)
```

其中 `t` 表示该询问之前已经发生的修改次数

```
struct MoMod {
    struct Q {
        int l, r, t, id;
    };
    int n, B;
    vector<Q> q;
    explicit MoMod(int n_, int bs = 0) : n(n_), B(bs) {}
    void ask(int l, int r, int t, int id) { q.push_back({ l, r, t, id }); }
    template<class Add, class Del, class Chg, class Save>
```

```
    void run(Add add, Del del, Chg chg, Save save) {
        if (q.empty()) return;
        if (!B) B = max(1, int(pow(max(1, n), 2.0 / 3)));
        ranges::sort(q, [&](const Q &a, const Q &b) {
            int x{ a.l / B }, y{ b.l / B };
            if (x != y) return x < y;
            int u{ a.r / B }, v{ b.r / B };
            if (u != v) return x & 1 ? u > v : u < v;
            return u & 1 ? a.t > b.t : a.t < b.t;
        });
        int l{}, r{}, t{};
        for (auto [ql, qr, qt, id] : q) {
            while (l > ql) add(--l);
            while (r < qr) add(r++);
            while (l < ql) del(l++);
            while (r > qr) del(--r);
            while (t < qt) chg(t++, l, r);
            while (t > qt) chg(--t, l, r);
            save(id);
        }
    }
};
```

成员变量

- `n`：序列长度
- `B`：块长
- `Q::l,r`：查询区间 `[l,r)`
- `Q::t`：该询问之前的修改次数
- `Q::id`：询问编号

接口

- `MoMod(n, bs)`：构造，`bs = 0` 时默认取约 $n^{2/3}$ 的块长
- `ask(l, r, t, id)`：加入询问，其中 `t` 为此前修改次数
- `run(add, del, chg, save)`：运行带修改莫队
- `add(p)` / `del(p)`：加入或删除当前版本中位置 `p` 的元素
- `chg(k, l, r)`：切换第 `k` 次修改，要求同一函数既能应用也能撤销
- `save(id)`：保存当前询问答案

可逆修改

```
struct C {
    int p, x;
};
vector<C> c;
auto chg = [&](int k, int l, int r) {
    auto &[p, x] = c[k];
    if (l <= p && p < r) del(p);
    swap(a[p], x);
    if (l <= p && p < r) add(p);
};
```

执行一次后

- `a[p]` 变成修改后的值
- `c[k].x` 保存修改前的值
  再次执行同一个 `chg(k)` 就会恢复原状态

常见用法

动态区间不同数个数

假设初始值与所有修改值已经一起离散化

```
struct C {
    int p, x;
};
vector<C> c;
MoMod mo(n);
int qid{};
```

```
for (int i = 0; i < q; ++ i) {
    char op;
    cin >> op;
    if (op == 'Q') {
        int l, r;
        cin >> l >> r;
        mo.ask(l, r, c.size(), qid ++);
    } else {
        int p, x;
        cin >> p >> x;
        c.push_back({ p, x });
    }
}
vector<int> cnt(m), ans(qid);
int cur{};
auto add = [&](int p) {
    if (cnt[a[p]] ++ == 0) ++ cur;
};
auto del = [&](int p) {
    if (-- cnt[a[p]] == 0) -- cur;
};
auto chg = [&](int k, int l, int r) {
    auto &[p, x] = c[k];
    if (l <= p && p < r) del(p);
    swap(a[p], x);
    if (l <= p && p < r) add(p);
};
mo.run(add, del, chg, [&](int id) {
    ans[id] = cur;
});
```

注意事项

- 只适用于可以离线的修改与询问
- 这里的修改模型是单点赋值
- `Q::t` 必须等于该询问之前发生的修改数
- 时间移动时若修改位置位于当前 `[l,r)` 内，必须先删除旧贡献，再加入新贡献
- `chg()` 必须可逆，使用 `swap` 是最简写法
- 使用 `swap` 后修改数组 `c` 会随时间移动发生变化，莫队结束后不应再认为 `c[k].x` 是最初读入的新值
- 若需要再次运行同一批数据，应重新复制初始数组与修改数组
- 带修改莫队常数明显大于普通莫队，没有修改时不要使用这一版本

复杂度

带修改莫队的块长取约

$$B = n^{2/3}$$

设有 q 次询问、u 次修改，按上述三维排序估算移动量为

$$O\left(qB + \frac{n^2}{B} + \frac{un^2}{B^2}\right)$$

当

$$q, u = O(n)$$

且

$$B = \Theta(n^{2/3})$$

时为

$$O(n^{5/3})$$

另有排序复杂度 $O(q \log q)$
空间复杂度 $O(q + u)$，不计题目自身维护的信息

3.9.4 回滚、树上与二次离线莫队

三类扩展分别解决普通莫队的不同限制

方法	适用情况
回滚莫队	`add` 容易，`del` 很难，但状态可以撤销
树上莫队	查询对象是树上路径
二次离线莫队	移动端点的贡献本身无法 $O(1)$ 计算，但能转成前缀离线统计

情况	优先选择
加入、删除都容易	普通莫队
有单点修改	带修改莫队
加入容易、删除难，但可回滚	回滚莫队
树上路径询问	树上莫队
树上路径 + 单点修改	双欧拉序 + 带修改莫队
端点贡献较慢，但能转成前缀离线事件	二次离线莫队

回滚莫队

普通莫队要求左右端点能够反复加入和删除
回滚莫队只让跨块询问的右端点单调增加，左端点每次临时加入，求完答案后恢复快照
同一块内部的短询问直接暴力

```
struct RollbackMo {
    struct Q {
        int l, r, id;
    };
    int n, B;
    vector<Q> q;
    explicit RollbackMo(int n_, int bs = 0) : n(n_), B(bs) {}
    void ask(int l, int r, int id) {
        q.push_back({ l, r, id });
    }
    template<class Add, class Snap, class Roll, class Save>
    void run(Add add, Snap snap, Roll roll, Save save) {
        if (q.empty()) return;
        if (!B) {
            int k{ max(1, min(n, int(sqrt(q.size())))) };
            B = max(1, n / k);
        }
        sort(q.begin(), q.end(), [&](const Q &a, const Q &b) {
            int x{ a.l / B }, y{ b.l / B };
            if (x != y) return x < y;
            return a.r < b.r;
        });
        for (int s = 0; s < q.size(); ) {
            int b{ q[s].l / B }, e{ s };
            while (e < int(q.size()) && q[e].l / B == b) ++ e;
            int br{ min(n, (b + 1) * B) };
            auto base{ snap() };
            int k{ s };
            // 同块询问：长度不超过 B，直接暴力
            while (k < e && q[k].r <= br) {
                auto h{ snap() };
                for (int i = q[k].l; i < q[k].r; ++ i) add(i);
                save(q[k].id);
                roll(h);
                ++ k;
            }
            // 跨块询问：右端点单调增加，左端点临时加入
            int r{ br };
            for (; k < e; ++ k) {
                while (r < q[k].r) add(r ++);
```

```
                auto h{ snap() };
                for (int i = br - 1; i >= q[k].l; -- i) add(i);
                save(q[k].id);
                roll(h);
            }
            roll(base);
            s = e;
        }
    }
};
```

接口说明

- `RollbackMo(n, bs)`：构造，`bs = 0` 时自动选择块长
- `ask(l, r, id)`：加入 `[l,r)` 的询问
- `add(p)`：将位置 `p` 加入当前状态
- `snap()`：返回当前状态快照
- `roll(s)`：恢复到快照 `s`
- `save(id)`：保存当前答案

常见用法

例如维护

$$\max_x cnt_x \cdot val_x$$

以下示例要求 val[x] >= 0，空区间答案为 0。删除某个值后最大值很难恢复，但增加和撤销都很简单

```
vector<int> cnt(m);
vector<pair<int, i64>> his;
i64 cur{};
auto add = [&](int p) {
    int x{ a[p] };
    his.push_back({ x, cur });
    ++ cnt[x];
    cur = max(cur, 1LL * cnt[x] * val[x]);
};
auto snap = [&]() {
    return his.size();
};
auto roll = [&](size_t s) {
    while (his.size() > s) {
        auto [x, old]{ his.back() };
        his.pop_back();
        -- cnt[x];
        cur = old;
    }
};
RollbackMo mo(n);
for (int i = 0; i < q; ++ i) {
    mo.ask(l[i], r[i], i);
}
mo.run(add, snap, roll, [&](int id) {
    ans[id] = cur;
});
```

注意事项

- 下标为 `0base`，区间为 `[l,r)`
- 不需要实现 `del()`
- `add()` 产生的全部状态变化都必须可以被 `roll()` 恢复
- `snapshot()` 通常直接返回历史栈大小
- 跨块询问内部 `r` 必须保持单调，因此不能像普通莫队一样使用奇偶块反转 `r`
- 同块询问不能直接混入跨块状态，否则会破坏右侧永久部分
- 如果 `del()` 本来就能 $O(1)$ 完成，普通莫队更简单
- 典型适用状态是“增加容易，删除后答案难恢复”，例如最大值类统计

复杂度

设块长为 B，`add / rollback` 单次修改为 $O(1)$
总修改次数为

$$O\left(qB+\frac{n^2}{B}\right)$$

取

$$B\approx\frac{n}{\sqrt{q}}$$

时为

$$O(n\sqrt{q})$$

当 $q=\Theta(n)$ 时为

$$O(n\sqrt{n})$$

另有排序复杂度 $O(q\log q)$

树上莫队

树上路径不能直接表示为普通 DFS 序中的一个区间
树上莫队为每个节点记录两次：

- 进入节点时记录一次
- 离开节点时再记录一次
  于是节点在当前欧拉区间中出现奇数次时，恰好表示它属于当前路径状态
  设

$$p=\mathrm{lca}(u,v)$$

并调整到 `in[u] ⩽ in[v]`
若 `p = u`

$$[u,v]\longrightarrow[in_u,in_v]$$

否则

$$[u,v]\longrightarrow[out_u,in_v]$$

并额外加入 `p`
依赖 3.9.3 普通莫队 与已有 LCA

```
struct TreeMo {
    struct Q {
        int l, r, p, id;
    };
    int n, B;
    vector<int> in, out, e;
    vector<Q> q;
    explicit TreeMo(int n_, int bs = 0) : n(n_), B(bs), in(n), out(n) {}
    // ===== 双欧拉序 =====
    void build(const vector<vector<int>> &g, int root = 0) {
        e.clear();
        e.reserve(2 * n);
        if (!n) return;
        vector<pair<int, int>> st{{root, -1}};
        while (!st.empty()) {
            auto [u, fa]{ st.back() };
            st.pop_back();
            if (u < 0) {
                u = ~u;
                out[u] = e.size();
                e.push_back(u);
                continue;
            }
            in[u] = e.size();
            e.push_back(u);
```

```
            st.push_back({~u, fa});
            for (int v : g[u]) if (v != fa) st.push_back({v, u});
        }
    }
    // ===== 路径转莫队区间 =====
    template<class Lca>
    void ask(int u, int v, int id, Lca lca) {
        if (in[u] > in[v]) swap(u, v);
        int p{ lca(u, v) };
        if (p == u) q.push_back({ in[u], in[v] + 1, -1, id });
        else q.push_back({ out[u], in[v] + 1, p, id });
    }
    // ===== 复用普通莫队 =====
    template<class Tog, class Save>
    void run(Tog tog, Save save) {
        Mo mo(e.size(), B);
        for (int i = 0; i < int(q.size()); ++ i) {
            mo.ask(q[i].l, q[i].r, i);
        }
        auto move = [&](int p) {
            tog(e[p]);
        };
        mo.run(move, move, [&](int i) {
            auto &[l, r, p, id]{ q[i] };
            if (p != -1) tog(p);
            save(id);
            if (p != -1) tog(p);
        });
    }
};
```

接口说明

- `TreeMo(n, bs)`：构造
- `build(g, root)`：生成长度为 `2n` 的双欧拉序
- `ask(u, v, id, lca)`：加入路径询问
- `run(tog, save)`：使用普通莫队处理所有路径
- `tog(u)`：切换节点 `u` 是否属于当前状态
- `save(id)`：保存当前路径答案

常见用法

路径不同颜色数

```
TreeMo tm(n);
tm.build(g, 0);
for (int i = 0; i < q; ++ i) {
    tm.ask(u[i], v[i], i, [&](int x, int y) {
        return hld.lca(x, y);
    });
}
vector<int> on(n), cnt(m), ans(q);
int cur{};
auto tog = [&](int u) {
    int x{ col[u] };
    if (on[u]) {
        if (-- cnt[x] == 0) -- cur;
    } else {
        if (cnt[x] ++ == 0) ++ cur;
    }
    on[u] ^= 1;
};
tm.run(tog, [&](int id) {
    ans[id] = cur;
});
```

注意事项

- 依赖 3.9.3 的 `Mo`
- LCA 直接复用已有板子，不在这里重复实现

- 每个节点在双欧拉序中恰好出现两次
- `tog(u)` 必须是自反操作，执行两次恢复原状态
- 非祖先路径的 LCA 不在对应莫队区间内，因此保存答案前临时加入
- `build()` 不保存 `g`，可以直接复用已有树结构的邻接表
- 当前版本处理点信息
- 若处理边信息，通常将边权挂到较深端点，但 LCA 是否需要排除会改变，不能直接照搬点权版
- 使用显式栈生成双欧拉序，避免极深链递归栈溢出

树上带修改莫队

树上带修改不需要再造一套全新思想
路径仍按上面的双欧拉序转换，排序部分换成 3.9.3 `MoMod`
修改节点 `u` 时，不应根据欧拉位置判断是否在区间中，而应直接看当前节点开关 `on[u]`

```
struct C {
    int u, x;
};
vector<C> c;
auto chg = [&](int k, int, int) {
    auto &[u, x]{ c[k] };
    bool in{ bool(on[u]) };
    if (in) tog(u);
    swap(col[u], x);
    if (in) tog(u);
};
```

也就是说

```
树上带修改莫队
=
双欧拉序路径转换
+
节点 toggle
+
3.9.3 带修改莫队的时间维
```

通常不需要再独立维护一份完整模板

复杂度

欧拉序构造为

$$O(n)$$

路径转换需要额外的 LCA 复杂度

$$O(qT_{\text{lca}})$$

随后是在长度 `2n` 的序列上运行普通莫队
若 `tog()` 为 $O(1)$，当 $q = \Theta(n)$ 时通常为

$$O(n\sqrt{n})$$

二次离线莫队

普通莫队要求每移动一次端点，就能立刻算出这一步产生的答案变化
二次离线莫队处理的是：

> 单次端点贡献本身不够快，但这份贡献可以写成某种前缀状态对一个区间的统计

于是第一次莫队只负责记录：

```
在前缀 p 的状态下
查询区间 [l,r]
贡献符号为 +1 / -1
```

第二次统一扫描前缀，再批量处理这些事件
参考资料中的经典模型为：

> 查询 [l,r] 内满足 i < j 且 popcount(a[i] xor a[j]) = k 的数对数量

其实现先记录 range[p] 事件，之后重新扫描数组并统一处理
这一技巧高度依赖具体贡献公式，因此不强行封装成万能类

XOR 数对二次离线莫队

查询 [l,r) 内满足

$$i < j, \quad \text{popcount}(a_i \oplus a_j) = k$$

的数对数量

```
vector<i64> mo2_xor(const vector<int> &src, const vector<pair<int, int>> &qs, int k, int W = 14, int bs = 0) {
    struct Q {
        int l, r, id;
        i64 d{};
    };
    struct E {
        int id, l, r, s;
    };
    int n{ int(src.size()) }, m{ int(qs.size()) };
    if (!m) return {};
    if (!n) return vector<i64>(m);
    int S{ 1 << W };
    vector<int> a(n + 2), mask;
    for (int i = 1; i <= n; ++ i) {
        a[i] = src[i - 1];
    }
    for (int x = 0; x < S; ++ x) {
        if (__builtin_popcount(unsigned(x)) == k) {
            mask.push_back(x);
        }
    }
    // f[i]: a[i + 1] 与前缀 a[1..i] 构成的合法数对数
    vector<int> f(n + 1), g(S);
    for (int i = 1; i <= n; ++ i) {
        for (int x : mask) ++ g[a[i] ^ x];
        if (i < n) f[i] = g[a[i + 1]];
    }
    vector<Q> q;
    q.reserve(m);
    for (int i = 0; i < m; ++ i) {
        auto [l, r]{ qs[i] };
        q.push_back({ l + 1, r, i, 0 });
    }
    int B;
    if (bs) B = bs;
    else B = max(1, n / min(n, max(1, int(sqrt(m)))));
    sort(q.begin(), q.end(), [&](const Q &a, const Q &b) {
        int x{ (a.l - 1) / B }, y{ (b.l - 1) / B };
        if (x != y) return x < y;
        return x & 1 ? a.r < b.r : a.r > b.r;
    });
    vector<vector<E>> ev(n + 1);
    int L{ 1 }, R{};
    for (int i = 0; i < m; ++ i) {
        int l{ q[i].l }, r{ q[i].r };
        if (R < r) ev[L - 1].push_back({ i, R + 1, r, -1 });
        while (R < r) q[i].d += f[R ++];
        if (R > r) ev[L - 1].push_back({ i, r + 1, R, 1 });
        while (R > r) q[i].d -= f[-- R];
        if (L < l) ev[R].push_back({ i, L, l - 1, -1 });
        while (L < l) {
            q[i].d += f[L - 1] + (k == 0);
            ++ L;
```

```
        }
        if (L > l) ev[R].push_back({ i, l, L - 1, 1 });
        while (L > l) {
            q[i].d -= f[L - 2] + (k == 0);
            -- L;
        }
    }
    // ===== 第二次离线：扫描前缀并处理所有记录事件 =====
    ranges::fill(g, 0);
    for (int i = 1; i <= n; ++ i) {
        for (int x : mask) ++ g[a[i] ^ x];
        for (auto [id, l, r, s] : ev[i]) {
            for (int x = l; x <= r; ++ x) {
                q[id].d += 1LL * g[a[x]] * s;
            }
        }
    }
    // q[i].d 是相邻莫队状态的答案变化量
    for (int i = 1; i < m; ++ i) {
        q[i].d += q[i - 1].d;
    }
    vector<i64> ans(m);
    for (auto [l, r, id, d] : q) {
        ans[id] = d;
    }
    return ans;
}
```

接口说明

- `mo2_xor(a, qs, k, W, bs)`：回答所有 XOR 数对询问
- `a[i]`：要求 `0 ≤ a[i] < 2^W`
- `qs[i] = {l,r}`：查询 `[l,r)`
- `k`：要求 XOR 后二进制中恰有 `k` 个 `1`
- `W`：值域二进制位数，要求 0 <= W < 31，且计数数组可以放入内存
- `bs = 0`：自动选择莫队块长
- 返回值按原询问顺序排列

核心变量

- `mask`：所有 `popcount(mask) = k` 的掩码
- `f[i]`：`a[i+1]` 与前缀 `[1,i]` 中元素构成的合法数对数
- `q[i].d`：第 `i` 个莫队状态相对前一个状态的答案增量
- `ev[p]`：扫描到前缀 `p` 时需要计算的第二批离线事件
- `s = ±1`：事件贡献符号

常见用法

```
vector<pair<int, int>> qs(q);
for (auto &[l, r] : qs) {
    cin >> l >> r;
    -- l;
}
auto ans = mo2_xor(a, qs, k, 14);
for (i64 x : ans) {
    cout << x << '\n';
}
```

注意事项

- 这是二次离线莫队的经典具体模型，不是万能通用类
- 外部接口统一使用 `0base + [l,r)`
- 内部为了方便使用来源中的前缀公式，转换成 `1base`
- 要求 `0 ≤ k ≤ W`
- `2^W` 不能过大，否则枚举 `mask` 和计数数组本身不可接受
- 当 `k = 0` 时自身异或满足条件，因此左端点移动公式需要额外的 `(k == 0)`
- `q[i].d` 保存的是莫队排序后相邻状态之间的变化量，最后必须做前缀累加
- 第二次离线的事件不是普通询问，而是第一次莫队移动端点产生的“待计算贡献”
- 奇偶块排序只减少端点移动次数，不改变事件公式

什么时候能二次离线

若移动端点需要计算

$$\sum_{i=l}^{r-1} F(a_i, x)$$

而存在可以离线维护的前缀函数

$$P(p, x) = \sum_{i=0}^{p-1} F(a_i, x)$$

那么一次端点变化可以改写为若干

```
(prefix p, interval [l,r), sign)
```

事件
此时就可以考虑二次离线莫队
核心不是“跑两遍莫队”，而是：

```
第一次莫队
    ↓
把无法快速算出的端点贡献记录下来
    ↓
第二次按某个前缀维统一扫描
    ↓
批量计算这些贡献
```

复杂度

令

$$C = \binom{W}{k}$$

即 `mask.size()`
掩码枚举

$$O(2^W)$$

前缀预处理与第二次扫描中维护关系计数

$$O(nC)$$

莫队产生并处理的区间总长度为

$$O\left(qB + \frac{n^2}{B}\right)$$

因此总复杂度为

$$O\left(2^W + nC + q\log q + qB + \frac{n^2}{B}\right)$$

取

$$B \approx \frac{n}{\sqrt{q}}$$

时，莫队部分为

$$O(n\sqrt{q})$$

当 $q = \Theta(n)$ 时总复杂度为

$$O(2^W + nC + n\sqrt{n})$$

空间复杂度

$$O(2^W + n + q)$$

3.9.5 离线分治

离线分治主要保留两类值得独立成板子的算法

- 整体二分：对一批询问同时二分答案，将事件稳定划分到左右答案区间
- CDQ 分治：按时间、下标或某一维分治，只计算左半对右半的贡献

整体二分

对所有询问共同维护答案区间 `[vl,vr)`，每层取 `vm`

- 修改值 `< vm` 时加入数据结构
- 查询当前区间中 `< vm` 的元素数 `c`
- 若 `c ⩾ k`，答案进入左半
- 否则进入右半，并令 `k -= c`
  事件划分后必须保持原来的时间顺序

动态区间第 k 小

支持单点赋值与区间第 `k` 小，所有操作离线后统一计算

```
template<class T = int>
struct PBS {
    struct E {
        int tp, l, r, k, id, d, v;
        T x;
    };
    int n, qn{};
    vector<T> a, val, ans;
    vector<E> e, w, tmp;
    vector<int> bit;
    vector<char> go;
    explicit PBS(vector<T> a_) : n(a_.size()), a(move(a_)), val(a) {
        for (int i = 0; i < n; ++ i) {
            e.push_back({ 0, i, 0, 0, -1, 1, -1, a[i] });
        }
    }
    // ===== 加入操作 =====
    void set(int p, T x) {
        e.push_back({ 0, p, 0, 0, -1, -1, -1, a[p] });
        e.push_back({ 0, p, 0, 0, -1, 1, -1, x });
        a[p] = x;
        val.push_back(x);
    }

    int ask(int l, int r, int k) {
        int id{ qn ++ };
        e.push_back({ 1, l, r, k, id, 0, -1, T{} });
        return id;
    }
    // ===== BIT: 统计当前值域左半中的位置 =====
    void add(int p, int x) {
        for (++ p; p ⩽ n; p += p & -p) bit[p] += x;
    }
    int sum(int p) const {
        int s{};
        for (; p; p -= p & -p) s += bit[p];
        return s;
    }
    // ===== 整体二分 =====
    void solve(int l, int r, int vl, int vr) {
```

```
        if (l >= r) return;
        if (vr - vl == 1) {
            for (int i = l; i < r; ++ i) {
                if (w[i].tp) ans[w[i].id] = val[vl];
            }
            return;
        }
        int vm{ (vl + vr) / 2 }, nl{};
        for (int i = l; i < r; ++ i) {
            auto &z{ w[i] };
            if (!z.tp) {
                go[i] = z.v < vm;
                if (go[i]) add(z.l, z.d);
            } else {
                int c{ sum(z.r) - sum(z.l) };
                go[i] = c >= z.k;
                if (!go[i]) z.k -= c;
            }
            nl += go[i];
        }
        // 撤销本层 BIT 修改
        for (int i = l; i < r; ++ i) {
            if (!w[i].tp && go[i]) add(w[i].l, -w[i].d);
        }
        // 稳定划分，保持操作的时间顺序
        int x{ l }, y{ l + nl };
        for (int i = l; i < r; ++ i) {
            tmp[go[i] ? x ++ : y ++] = move(w[i]);
        }
        for (int i = l; i < r; ++ i) w[i] = move(tmp[i]);
        solve(l, l + nl, vl, vm);
        solve(l + nl, r, vm, vr);
    }
    vector<T> run() {
        sort(val.begin(), val.end());
        val.erase(unique(val.begin(), val.end()), val.end());
        for (auto &z : e) if (!z.tp) {
            z.v = lower_bound(val.begin(), val.end(), z.x) - val.begin();
        }
        w = e;
        tmp.resize(w.size());
        go.resize(w.size());
        bit.assign(n + 1, 0);
        ans.assign(qn, T{});
        solve(0, w.size(), 0, val.size());
        return ans;
    }
};
```

成员变量

- `e`：按真实时间顺序保存的修改与询问事件
- `tp = 0`：修改事件
- `tp = 1`：询问事件
- `d = 1/-1`：插入 / 删除一个值
- `v`：修改值离散化后的编号
- `k`：当前递归中还需要寻找的排名
- `bit`：维护当前答案左半值域中各位置出现次数

接口

- `PBS(a)`：加入初始数组
- `set(p, x)`：单点赋值 `a[p] = x`
- `ask(l, r, k)`：加入 `[l,r)` 第 `k` 小询问，返回询问编号
- `run()`：离线求出所有询问答案

常见用法

```
PBS<i64> pbs(a);
int x = pbs.ask(l, r, k);
```

```
pbs.set(p, v);
int y = pbs.ask(l2, r2, k2);
auto ans = pbs.run();
cout << ans[x] << '\n';
cout << ans[y] << '\n';
```

注意事项

- 下标为 `0base`，区间为 `[l,r)`
- `k` 从 `1` 开始，要求 `1 ⩽ k ⩽ r - l`
- `set()` 和 `ask()` 必须严格按照真实时间顺序加入
- 单点赋值拆成删除旧值和插入新值两个事件
- 所有初始值和修改值统一离散化
- 事件划分必须稳定，否则会破坏修改与询问的时间关系
- 进入右半答案区间时必须执行 `k -= c`
- 每层递归结束前必须撤销该层加入 BIT 的修改
- `run()` 后得到的是最终离线答案，不能在线输出询问
- 若只有静态区间第 `k` 小，主席树、小波矩阵等通常更直接

复杂度

设修改次数为 u、询问次数为 q、不同值数量为 V
事件数量

$$E = n + 2u + q$$

每个事件经过 $O(\log V)$ 层，每层修改或查询 BIT 为 $O(\log n)$

$$O(E \log V \log n)$$

空间复杂度

$$O(E + n + V)$$

整体二分的其他常见模型

同一套思想还可以处理

- 第一个满足条件的时刻
- 第 `k` 个发生的事件
- 动态排名与第 `k` 小
- 多个询问共享一个具有单调性的判定过程
  若答案不是一个离散值，而是“最早时刻”，只需把递归值域改成时间轴，并重新设计每层的 `check`

CDQ 分治

CDQ 将事件按一个天然有序的维度分成左右两半，只统计

$$\text{左半} \to \text{右半}$$

的跨区间贡献
处理多维偏序时，可以用递归消掉一个维度，再用排序、树状数组等结构处理剩余维度

三维偏序

给定点

$$p_i = (x_i, y_i, z_i)$$

求每个点满足

$$x_j \le x_i, \quad y_j \le y_i, \quad z_j \le z_i$$

的点 `j` 数量，包括自身
经典实现先合并重复点，然后按第二维归并，同时用树状数组维护第三维

```
vector<i64> cdq3(const vector<array<int, 3>> &p) {
    struct P {
        int x, y, z, w{ 1 };
        i64 s{};
        vector<int> id;
    };
    int n{ int(p.size()) };
    if (!n) return {};
    vector<int> o(n);
    iota(o.begin(), o.end(), 0);
    sort(o.begin(), o.end(), [&](int i, int j) {
        return p[i] < p[j];
    });
    // ===== 合并完全相同的点 =====
    vector<P> a;
    for (int i : o) {
        if (!a.empty() &&
            array<int, 3>{ a.back().x, a.back().y, a.back().z } == p[i]) {
            ++ a.back().w;
            a.back().id.push_back(i);
        } else {
            a.push_back({ p[i][0], p[i][1], p[i][2], 1, 0, { i } });
        }
    }
    // ===== 第三维离散化 =====
    vector<int> z;
    z.reserve(a.size());
    for (auto &v : a) z.push_back(v.z);
    sort(z.begin(), z.end());
    z.erase(unique(z.begin(), z.end()), z.end());
    for (auto &v : a) {
        v.z = lower_bound(z.begin(), z.end(), v.z) - z.begin() + 1;
    }
    vector<int> bit(z.size() + 1);
    auto add = [&](int p, int x) {
        for (; p < int(bit.size()); p += p & -p) bit[p] += x;
    };
    auto sum = [&](int p) {
        int s{};
        for (; p; p -= p & -p) s += bit[p];
        return s;
    };
    vector<P> tmp(a.size());
    // ===== 左半贡献给右半，同时按 y 归并 =====
    auto cdq = [&](auto &&self, int l, int r) -> void {
        if (r - l <= 1) return;
        int m{ (l + r) / 2 };
        self(self, l, m);
        self(self, m, r);
        int i{ l }, j{ m }, k{ l };
        while (i < m || j < r) {
            if (j == r || (i < m && a[i].y <= a[j].y)) {
                add(a[i].z, a[i].w);
                tmp[k ++] = a[i ++];
            } else {
                a[j].s += sum(a[j].z);
                tmp[k ++] = a[j ++];
            }
        }
        for (int t = l; t < m; ++ t) add(a[t].z, -a[t].w);
        for (int t = l; t < r; ++ t) a[t] = move(tmp[t]);
    };
    cdq(cdq, 0, a.size());
    vector<i64> ans(n);
    for (auto &v : a) {
        for (int id : v.id) ans[id] = v.s + v.w;
    }
```

```
    return ans;
}
```

成员含义

- `x,y,z`：三维坐标
- `w`：完全相同点的出现次数
- `s`：其他坐标组对当前点产生的贡献
- `id`：这一坐标组对应的原输入编号

接口

- `cdq3(p)`：返回每个点三维均不大于它的点数
- 返回值包含点自身以及与其完全相同的其他点

常见用法

每个点的偏序计数

```
auto ans = cdq3(p);
for (i64 x : ans) cout << x << '\n';
```

三维偏序计数直方图

若需要统计“恰有 `k` 个其他点三维均不大于它”的点数

```
auto d = cdq3(p);
vector<int> ans(p.size());
for (i64 x : d) ++ ans[x - 1];
```

注意事项

- 第一维先按 `{x,y,z}` 排序
- 完全相同的点必须先合并，否则相等点之间的贡献容易漏算或重复
- CDQ 返回上一层时，当前区间必须保持按 `y` 有序
- `y` 相等时左半元素必须先加入 BIT，才能统计 `≤`
- 第三维查询使用前缀 `sum(z)`，同样表示 `≤`
- 每层结束后必须撤销加入 BIT 的左半贡献
- 若要求严格 `<`，需要同时调整排序、重复点处理以及归并时的相等关系
- CDQ 不一定处理几何坐标，三个维度也可以分别表示时间、位置、权值

复杂度

初始排序与离散化

$$O(n \log n)$$

CDQ 共 $O(\log n)$ 层，每层每个点进行 $O(\log n)$ 的 BIT 操作

$$O(n \log^2 n)$$

空间复杂度

$$O(n)$$

CDQ 的其他常见模型

若把第一维换成操作时间，CDQ 可以处理很多离线动态问题，例如

```
时间 t
    ↓ CDQ
只统计过去的修改对未来询问的贡献
    ↓
剩余二维关系交给 BIT / 线段树
```

常见形式包括

- 单点增加 + 二维矩形计数
- 动态二维偏序
- 动态逆序对的部分模型
- 时间维修改 + 区间 / 前缀统计
  这类题通常不值得再造一个所谓“万能 CDQ 类”，因为事件含义与贡献方向差异很大，保留三维偏序板子和下面的核心骨架更实用

```
void cdq(int l, int r) {
    if (r - l <= 1) return;
    int m = (l + r) / 2;
    cdq(l, m);
    // 计算 [l,m) 中事件对 [m,r) 的贡献
    // 常用排序 + BIT / 双指针
    // 撤销临时状态
    cdq(m, r);
}
```

选择

方法	分治对象	关键条件	常见用途
整体二分	答案值域	判定随答案单调	动态第 k 小、最早时刻
CDQ	时间 / 下标 / 偏序维	贡献可拆成左半对右半	多维偏序、离线动态统计

可以直接记成

```
整体二分：这一批答案分别在哪一半
CDQ：左半事件对右半事件贡献多少
```

3.9.6 时间线段树分治与可撤销维护

时间线段树

```
template<class E>
struct TimeSeg {
    int q, base{ 1 };
    vector<vector<E>> seg;
    explicit TimeSeg(int q_) : q(q_) {
        while (base < q) base <<= 1;
        seg.resize(base << 1);
    }
    // ===== 基础接口：将事件加入整个生效区间 [l, r) =====
    void add(int l, int r, const E &e) {
        if (l >= r) return;
        for (l += base, r += base; l < r; l >>= 1, r >>= 1) {
            if (l & 1) seg[l ++].push_back(e);
            if (r & 1) seg[-- r].push_back(e);
        }
    }
private:
    // ===== 核心 DFS：进入节点应用事件，退出节点恢复快照 =====
    template<class State, class Apply, class F>
    void run(int p, State &st, Apply &apply, F &answer) {
        auto snap{ st.snapshot() };
        for (const auto &e : seg[p]) apply(st, e);

        if (p >= base) {
            int t{ p - base };
            if (t < q) answer(t, st);
        } else {
            run(p << 1, st, apply, answer);
            run(p << 1 | 1, st, apply, answer);
        }

        st.rollback(snap);
    }
public:
    template<class State, class Apply, class F>
```

```
    void run(State &st, Apply apply, F answer) {
        if (q) run(1, st, apply, answer);
    }
};
```

成员变量

- `q`：时间点数量，时间为 `0 ... q - 1`
- `base`：不小于 `q` 的最小二次幂
- `seg[p]`：在线段树节点 `p` 对应整个时间区间内始终有效的事件

接口

- `TimeSeg<E>(q)`：建立时间线段树，事件类型为 `E`
- `add(l, r, e)`：事件 `e` 在整个 `[l,r)` 内有效，$O(\log q)$
- `run(st, apply, answer)`：遍历所有时间点
- `apply(st, e)`：将事件 `e` 加入当前可撤销状态
- `answer(t, st)`：回答时间 `t` 的询问
  传入的 `State` 至少需要提供

```
auto snapshot() const;
void rollback(/* snapshot 返回的类型 */ snap);
```

`snapshot()` 的返回类型不限，只需能够传回对应的 `rollback()`

时间定义

推荐统一定义：

> 时间 `t` 表示第 `t` 个操作执行后的状态

若对象在时间 `l` 加入，在时间 `r` 删除，并且删除操作执行后立即失效，则生命周期为

$$[l, r)$$

因此

- 时间 `l` 包含该对象
- 时间 `r` 不包含该对象
  如果题目询问的是**操作执行前**的状态，需要相应平移生命周期端点

常见用法

已知所有事件生命周期

```
struct E {
    int x;
};
TimeSeg<E> seg(q);
for (auto [l, r, x] : intervals) {
    seg.add(l, r, { x });
}
State st;
seg.run(st,
    [&](State &cur, const E &e) {
        cur.add(e.x);
    },
    [&](int t, State &cur) {
        // cur 恰好表示时间 t 的状态
    }
);
```

注意事项

- 这是离线算法，运行前必须得到每个事件完整的生效区间
- 时间线段树只负责**调度事件生命周期**，不负责具体维护内容
- `apply()` 可以是并查集合并、加入异或约束、插入线性基或其他可撤销操作
- `run()` 已经负责节点级 `snapshot()` / `rollback()`，回调中通常不要自行回滚
- 状态必须支持撤销，不能直接使用只支持永久修改的数据结构
- 所有时间区间统一为 `[l,r)`
- 要求 `0 ⩽ l ⩽ r ⩽ q`

- `q = 0` 时不会调用 `answer()`
- 时间线段树与普通线段树不同，不维护区间聚合信息，只利用线段树把一个生命周期拆成 $O(\log q)$ 个节点
- DFS 深度仅为 $O(\log q)$

复杂度

设共有 m 个生命周期区间
每个 `[l,r)` 被拆成 $O(\log q)$ 个节点，因此事件记录数为

$$O(m \log q)$$

若单次 `apply()` 代价为 A，对应撤销总代价与修改量同阶，则总时间为

$$O(q + m \log q \cdot A)$$

不计状态自身空间时，时间线段树空间为

$$O(q + m \log q)$$

生命周期预处理

当输入给出的是

```
加入对象
删除对象
```

而不是直接给 `[l,r)` 时，可以先把操作转换成生命周期
下面的 `Life` 对任意可排序的对象类型都适用

```
template<class T>
struct Life {
    struct Seg {
        int l, r;
        T x;
    };
    struct State {
        int cnt{}, l{};
    };
    vector<Seg> segs;
    map<T, State> opened;
    // ===== 生命周期维护 =====
    void add(int t, const T &x) {
        auto &s{ opened[x] };
        if (s.cnt ++ == 0) s.l = t;
    }
    void erase(int t, const T &x) {
        auto it{ opened.find(x) };
        auto &[cnt, l]{ it->second };
        if (-- cnt) return;
        if (l < t) segs.push_back({ l, t, x });
        opened.erase(it);
    }
    // ===== 收尾：读取完全部操作后调用 =====
    void close_all(int q) {
        for (auto &[x, s] : opened) {
            if (s.l < q) segs.push_back({ s.l, q, x });
        }
        opened.clear();
    }
};
```

接口说明

- `add(t, x)`：时间 `t` 增加一份对象 `x`
- `erase(t, x)`：时间 `t` 删除一份对象 `x`
- `close_all(q)`：将操作结束后仍然存在的对象补为 `[l,q)`
- `segs`：所有生成的 `{l,r,x}` 生命周期

注意事项

- `T` 需要支持 `operator<`，以便作为 `map` 的键
- `erase()` 要求删除前至少存在一份对应对象
- 当前实现按多重集合语义维护出现次数
- `0 → 1` 时开始生命周期
- `1 → 0` 时结束生命周期
- 中间的重复加入、删除不会产生重叠区间
- 若题目为普通集合语义，重复加入已存在对象应当无效，需要在读入阶段过滤
- 若对象具有等价表示，应先规范化后再交给 `Life`
  例如无向边需要保证

```
auto edge = [](int u, int v) {
    if (u > v) swap(u, v);
    return array{ u, v };
};
```

动态连通性

依赖 3.2.6 可撤销 DSU

完整增删边与连通性询问

```
using Edge = array<int, 2>;
auto edge = [](int u, int v) {
    if (u > v) swap(u, v);
    return Edge{ u, v };
};
Life<Edge> life;
for (int t = 0; t < q; ++ t) {
    if (op[t] == '+') {
        life.add(t, edge(u[t], v[t]));
    } else if (op[t] == '-') {
        life.erase(t, edge(u[t], v[t]));
    }
}
life.close_all(q);
TimeSeg<Edge> seg(q);
for (auto &[l, r, e] : life.segs) {
    seg.add(l, r, e);
}
RollbackDSU dsu(n);
vector<bool> ans(q);
seg.run(dsu,
    [](RollbackDSU &cur, const Edge &e) {
        cur.unite(e[0], e[1]);
    },
    [&](int t, RollbackDSU &cur) {
        if (op[t] == '?') {
            ans[t] = cur.same(u[t], v[t]);
        }
    }
);
for (int t = 0; t < q; ++ t) {
    if (op[t] == '?') {
        cout << (ans[t] ? "YES\n" : "NO\n");
    }
}
```

已知边生命周期

若题目已经直接给出

```
[l,r) : 边 (u,v) 存在
```

则 `Life` 可以直接删除

```
using Edge = array<int, 2>;

TimeSeg<Edge> seg(q);
for (auto [l, r, u, v] : intervals) {
```

```
    seg.add(l, r, { u, v });
}

RollbackDSU dsu(n);

seg.run(dsu,
    [](RollbackDSU &cur, const Edge &e) {
        cur.unite(e[0], e[1]);
    },
    [&](int t, RollbackDSU &cur) {
        // 查询时间 t 的图
    }
);
```

可维护的信息

除 `same(u,v)` 外，只要可撤销 DSU 本身维护了相应信息，同一个框架还可以回答

```
当前连通块数量
u 所在连通块大小
u、v 是否连通
加入边后是否发生合并
```

复杂度

设生命周期数量为 m，连通性询问数量为 k
每条生命周期被加入 $O(\log q)$ 个时间线段树节点，因此执行

$$O(m \log q)$$

次合并
若可撤销 DSU 使用按大小合并且不路径压缩，则树高为 $O(\log n)$，单次 `find / unite / same` 为 $O(\log n)$
总时间复杂度

$$O\left(q + m \log q \log n + k \log n\right)$$

若使用 `Life` 的 `map` 处理 q 次增删操作，还需要

$$O(q \log q)$$

量级的预处理时间
总空间复杂度

$$O(n + q + m \log q)$$

动态二分图与异或关系

动态二分图并不是另一套时间线段树
对于无向边 `(u,v)`，二分图约束等价于

$$col_u \oplus col_v = 1$$

因此只需把事件从普通边

```
struct Edge {
    int u, v;
};
```

扩展为关系约束

```
struct E {
    int u, v, w;
};
```

其中

```
w = 0 : u、v 关系相同
w = 1 : u、v 关系不同
```

然后使用 3.2.8 可撤销异或关系与二分图 DSU 作为状态
时间线段树部分完全不变

```
TimeSeg<E> seg(q);
for (auto [l, r, u, v, w] : intervals) {
    seg.add(l, r, { u, v, w });
}
RollbackXorDSU dsu(n);
vector<bool> ans(q);
seg.run(dsu,
    [](auto &cur, const E &e) {
        cur.add_relation(e.u, e.v, e.w);
    },
    [&](int t, auto &cur) {
        ans[t] = cur.valid();
    }
);
```

这里不重复实现关系 DSU，核心要求是：

- 父子关系变化可以撤销
- 异或势能变化可以撤销
- 冲突数量或合法状态也必须可以撤销
- `snapshot()` 后发生的所有状态变化都必须能由 `rollback()` 恢复

> Tip
>
> 动态二分图中每条普通边直接视为 `w = 1` 的异或约束即可

其他可撤销维护

时间线段树要求的并不是“必须使用 DSU”，而只是状态满足

```
snapshot()
apply(event)
rollback(snapshot)
```

因此可以替换成其他可撤销结构

状态结构	常见用途
可撤销 DSU	动态连通性、连通块统计
可撤销异或 / 带权 DSU	动态二分图、异或关系、势能约束
可撤销线性基	动态最大异或、线性可表示性
可撤销计数结构	生命周期集合统计
自定义历史栈	其他区间生效的动态状态

自定义可撤销状态骨架

若状态比较简单，不必专门设计持久化结构，只需要记录每次修改前的信息

```
struct State {
    struct Change {
        // 恢复本次修改所需的信息
    };
    vector<Change> his;
    int snapshot() const {
        return his.size();
    }
    void add(/* event */) {
        // his.push_back(...)
        // 修改当前状态
    }
    void rollback(int snap) {
```

```
        while (int(his.size()) > snap) {
            auto c{ his.back() };
            his.pop_back();
            // 根据 c 恢复状态
        }
    }
};
```

注意事项

- 不要为了撤销而复制整个状态，通常只记录真正被修改的字段
- `snapshot()` 通常就是历史栈大小
- 一次 `apply()` 修改多个位置时，需要完整记录这些修改
- “什么都没有改变”的操作是否需要压入历史，要和 `rollback()` 的实现保持一致
- 若维护额外全局量，例如连通块数、冲突数、答案值，也必须一起恢复
- 可撤销结构通常不能使用无法恢复的路径压缩
- 按大小 / 按秩合并非常适合与 rollback 搭配

初始始终有效的事件

若某个约束从最开始就存在，并在全部时间内有效

```
seg.add(0, q, e);
```

若初始图已有边，同样可以直接加入 `[0,q)`
如果它之后会被删除，则应将其生命周期设为

```
[0, erase_time)
```

而不是同时再通过 `Life::add(0, e)` 重复加入

时间端点常见情况

设第 `t` 个操作发生在时间 `t`

操作含义	生命周期处理
`t` 时加入，操作后立即生效	左端点为 `t`
`t` 时删除，操作后立即失效	右端点为 `t`
查询操作后的状态	直接查询时间 `t`
查询操作前的状态	需要调整时间定义或将询问平移
初始已存在	左端点为 `0`
操作结束仍存在	`close_all(q)` 补到 `q`

最容易写错的不是线段树本身，而是这里的时间定义

与其他算法的区别

方法	事件模型	核心操作
带修改莫队	修改与查询混合，时间指针前后移动	应用 / 撤销单次修改
CDQ 分治	过去事件向未来事件产生贡献	左半贡献右半
时间线段树分治	对象在一段时间内持续有效	区间挂事件 + DFS 回滚
可持久化结构	需要保留并访问多个版本	保存版本而不是撤销

如果题目中反复出现

```
加入 x
删除 x
询问当前状态
```

并且能够离线，就应优先检查：

```
x 的存在时间能否拆成若干 [l,r)
        ↓
状态能否 snapshot + rollback
```

```
        ↓
时间线段树分治
```

复杂度总结

设

- `q`：时间点数量
- `m`：生命周期区间数量
- `A`：应用一个事件的复杂度
- `Q`：回答一个时间点询问的复杂度
  则通用时间线段树复杂度为

操作	时间复杂度
构造	$O(q)$
单个 `add(l,r,e)`	$O(\log q)$
存储全部生命周期	$O(m \log q)$
DFS 应用全部事件	$O(m \log q \cdot A)$
回答全部时间点	$O(qQ)$

因此总时间复杂度为

$$O\left(q + m \log q \cdot A + qQ\right)$$

空间复杂度为

$$O(q + m \log q + S)$$

其中 S 为可撤销状态及其历史栈空间

3.10 其他高级结构

3.10.1 选择树（锦标赛树）

⊗ 在功能上基本可以被线段树替代，常数也可以被 ZKW线段树 逼近，只展示一种思想以及最简洁的实现

败者树

内部节点只保存该场比赛的败者编号，全局额外保存最终胜者：

```
template<class T, class Compare = less<T>>
struct LoserTree {
    int n, base{ 1 }, win{ -1 };
    Compare better;
    vector<T> val;
    vector<int> tr;
    explicit LoserTree(vector<T> val_, Compare better_ = Compare()) : n(val_.size()), better(better_), val(move(val_)) {
        while (base < n) base *= 2;
        tr.assign(2 * base, -1);
        if (!n) return;
        vector<int> w(2 * base, -1);
        for (int i = 0; i < n; ++ i) w[base + i] = i;
        for (int p = base - 1; p > 0; -- p) {
            int x = w[2 * p], y = w[2 * p + 1];
            if (x == -1 || y == -1) {
                w[p] = x == -1 ? y : x;
            } else {
                w[p] = best(x, y);
                tr[p] = w[p] == x ? y : x;
            }
        }
        win = w[1];
    }
    int best(int x, int y) const {
        if (x == -1) return y;
```

```
        if (y == -1) return x;
        if (better(val[x], val[y])) return x;
        if (better(val[y], val[x])) return y;
        return min(x, y);
    }
    int winner() const { return win; }
    const T &top() const { return val[win]; }
    void replace_winner(const T &v) {
        int x = win;
        val[x] = v;
        for (int p = (base + x) / 2; p > 0; p /= 2) {
            int y = tr[p];
            if (y == -1) continue;
            int z = best(x, y);
            if (z == x) tr[p] = y;
            else {
                tr[p] = x;
                x = y;
            }
        }
        win = x;
    }
};
```

这个版本有一个很重要的设计选择：

> 只提供 `replace_winner()`，不提供任意 `set(id, v)`

成员变量

- `val[id]`：编号 `id` 当前对应的值
- `tr[p]`：比赛节点 `p` 中输掉的元素编号
- `win`：当前整个锦标赛的胜者编号
- `base`：不小于 `n` 的最小 2 的幂
- `-1`：空位置 / 轮空
- `better(x, y)`：判断 `x` 是否比 `y` 更优
  - 默认 `less<T>`：较小值获胜
  - `greater<T>`：较大值获胜

  若两个元素在 `Compare` 意义下等价，则编号较小者获胜，使结果确定

接口

- `LoserTree(val)`：由初始序列构造败者树，$O(n)$
- `winner()`：返回当前胜者的原编号，要求非空，$O(1)$
- `top()`：返回当前胜者的值，要求非空，$O(1)$
- `replace_winner(v)`：将**当前胜者位置**的值替换为 `v`，并重新比赛，$O(\log n)$

核心思想

胜者树的内部节点记录：*两个儿子谁赢了*
败者树则记录：*当前胜者一路打上来时，每一轮击败了谁*
例如当前胜者为 `x`，沿根路径可能记录：

```
x 击败 a
x 击败 b
x 击败 c
```

当 `x` 被取走并换成新值后，它无需重新和所有叶子比赛，只需要重新依次挑战：

```
a → b → c
```

也就是原来沿途的败者
因此 `replace_winner()` 的核心就是：

```
for (int p = (base + x) / 2; p > 0; p >>= 1) {
    // 当前候选 x 与这一层原败者重新比赛
}
```

这也是“记录败者”比单纯重复记录胜者更有信息含义的地方

注意事项

- `winner()`、`top()`、`replace_winner()` 要求结构非空
- 默认 `less<T>` 表示较小元素获胜，即维护最小值
- 败者树中的叶子位置固定，不支持动态增加或减少路数
- `replace_winner(v)` **只能修改当前胜者对应的位置**
  这一点尤其重要
  不能写：

```
tree.val[id] = x;
tree.replace_winner(...);
```

  去试图修改任意 `id`
  因为内部只记录了每场比赛的败者，没有保存所有子树当前胜者的信息；如果一个原本失败的元素被直接修改得更优，它可能需要从某个内部节点重新向上挑战，而仅靠当前败者信息无法从任意叶子直接恢复。
  如果题目需要：

```
任意修改 val[id]
然后查询全局最优
```

应使用前面的 `WinnerTree::set(id, v)`

> Tip
>
> 因此胜者树和败者树并不是简单的“哪个更高级”：
>
> - **任意点修改** → 胜者树
> - **反复替换当前最优元素** → 败者树

常见用法

查询最小值及来源

```
LoserTree<int> t(a);
int id{ t.winner() };
int x{ t.top() };
```

其中：`id` 仍然是原数组位置

替换当前最小值

```
int id{ t.winner() };
// 第 id 路取出当前元素，并换成下一个元素
t.replace_winner(next);
```

更新后新的：`t.winner()` 就是新的全局最小值来源

多路有序序列归并

这是败者树最值得放进板子的场景
假设有 k 个已经升序的序列：

```
vector<vector<int>> a(k);
vector<int> pos(k);
```

先取每一路当前元素构造败者树
如果使用一个足够大的哨兵值：

```
vector<int> cur(k, INF);
for (int i = 0; i < k; ++ i) {
    if (!a[i].empty()) cur[i] = a[i][0];
}
LoserTree<int> t(cur);
```

不断取当前胜者：

```
while (t.top() != INF) {
    int id{ t.winner() };
```

```
    cout << t.top() << ' ';
    ++ pos[id];
    if (pos[id] < a[id].size()) t.replace_winner(a[id][pos[id]]);
    else t.replace_winner(INF);
}
```

这里每次变化的恰好都是：`winner()`
对应的那一路，所以和败者树的接口完全吻合

大根败者树

```
LoserTree<int, greater<int>> t(a);
```

此时每轮胜者是最大值

与胜者树对比

特性	胜者树	败者树
内部节点记录	胜者	败者
全局胜者	`tr[1]`	单独保存 `win`
查询胜者	$O(1)$	$O(1)$
任意位置修改	$O(\log n)$	不适合
替换当前胜者	$O(\log n)$	$O(\log n)$
多路归并	可以	**更自然**
保留原来源编号	是	是
每层含义	子树胜者	当前胜者曾击败的对手

特别是在：

```
k 路归并
外部排序
多个有序流合并
```

这些场景中，优先抄败者树

胜者树

胜者树是一棵锦标赛树：叶子表示固定位置的元素，每个内部节点记录两个儿子中更优元素的编号
适合：

- 元素位置固定
- 需要频繁修改某个位置的值
- 每次查询全局最优元素
- 希望同时得到最优元素的**原编号**

```
template<class T, class Compare = less<T>>
struct WinnerTree {
    int n, base{ 1 };
    Compare better;
    vector<T> val;
    vector<int> tr;
    explicit WinnerTree(vector<T> val_, Compare better_ = Compare()) : n(val_.size()), better(better_), val(move(val_)) {
        while (base < n) base *= 2;
        tr.assign(2 * base, -1);
        for (int i = 0; i < n; ++ i) tr[base + i] = i;
        for (int p = base - 1; p > 0; -- p) pull(p);
    }
    int win(int x, int y) const {
        if (x == -1) return y;
        if (y == -1) return x;
        if (better(val[x], val[y])) return x;
        if (better(val[y], val[x])) return y;
        return min(x, y);
    }
    void pull(int p) { tr[p] = win(tr[2 * p], tr[2 * p + 1]); }
    void set(int id, const T &v) {
        val[id] = v;
```

```
        for (int p = (base + id) / 2; p > 0; p /= 2) pull(p);
    }
    int winner() const { return tr[1]; }
};
```

成员变量

- `n`：实际元素数量
- `base`：不小于 `n` 的最小 2 的幂
- `val[id]`：编号 `id` 当前对应的值
- `tr[p]`：锦标赛节点 `p` 当前胜者的原编号
- `-1`：不存在的叶子
- `better(x, y)`：判断 `x` 是否优于 `y`
  若两个值等价，则：

```
return min(x, y);
```

即编号较小者获胜

接口

- `WinnerTree(val)`：由初始数组建树，$O(n)$
- `win(x, y)`：返回两个编号中的胜者，$O(1)$
- `set(id, v)`：把 `val[id]` 修改为 `v` 并维护到根，$O(\log n)$
- `winner()`：返回当前全局最优元素的编号，$O(1)$
  当前最优值：

```
int id{ t.winner() };
auto x = t.val[id];
```

注意事项

- 默认 `less<T>` 表示较小值获胜，因此默认维护最小值
- `set(id, v)` 要求 `0 ≤ id < n`
- `winner()` 返回的是元素编号，不是元素值
- 当 `n = 0` 时，`winner()` 为 `-1`
- 胜者树维护的是固定的 `n` 个位置，没有直接提供动态插入 / 删除接口
- 如果需要“删除”某个位置，可以在具体题目中将其修改为一个永远不会获胜的哨兵值，但泛型板子本身不提供哨兵
- 相同值时按编号较小者优先，因此结果具有确定性

与堆的区别

需求	更适合
动态插入元素、删除堆顶	二叉堆
高效合并两个堆	左偏树 / 斜堆
固定位置单点修改，查询全局最优编号	胜者树

例如：

```
100000 个选手
每次修改其中一个选手成绩
询问当前第一名是谁
```

胜者树就非常自然

常见用法

动态维护最小值及其位置

```
WinnerTree<int> t(a);
int id{ t.winner() };
cout << id << ' ' << t.val[id] << '\n';
t.set(3, 10);
id = t.winner();
```

维护最大值

```
WinnerTree<int, greater<int>> t(a);
```

多路归并

胜者树经典用途之一是维护每一路当前候选：

```
int id{ t.winner() };
// 取走第 id 路的当前元素后
t.set(id, next_value);
```

它比只保存值的普通堆更自然地保留“来自哪一路”的编号

3.10.2 Trie 多重集

```
template<int B = 31>
struct BinaryTrie {
    static_assert(1 ⩽ B && B ⩽ 64);
    struct Node {
        int ch[2]{ -1, -1 };
        int siz{};
    };
    vector<Node> tr;
    vector<int> bin;
    explicit BinaryTrie(int want = 0) {
        tr.reserve(1LL + 1LL * want * B);
        tr.push_back({});
    }
    int size() const { return tr[0].siz; }
    bool empty() const { return size() == 0; }
    // ===== 节点创建：以下代码被 add() 依赖，不建议删除 =====
    int new_node() {
        if (!bin.empty()) {
            int p = bin.back();
            bin.pop_back();
            tr[p] = Node{};
            return p;
        }
        tr.push_back({});
        return int(tr.size()) - 1;
    }
    // ===== 基础修改与计数 =====
    void add(u64 x) {
        int p{ 0 };
        ++ tr[p].siz;
        for (int bit = B - 1; bit ⩾ 0; -- bit) {
            int b{ int(x >> bit & 1ULL) };
            if (tr[p].ch[b] == -1) tr[p].ch[b] = new_node();
            p = tr[p].ch[b];
            ++ tr[p].siz;
        }
    }
    int count(u64 x) const {
        int p{ 0 };
        for (int bit = B - 1; bit ⩾ 0; -- bit) {
            int b{ int(x >> bit & 1ULL) };
            p = tr[p].ch[b];
            if (p == -1 || tr[p].siz == 0) return 0;
        }
        return tr[p].siz;
    }
    bool erase(u64 x) {
        if (count(x) == 0) return false;
        array<int, B + 1> path{};
        path[0] = 0;
        -- tr[0].siz;
        for (int i = 0; i < B; ++ i) {
            int bit = B - 1 - i;
            path[i + 1] = tr[path[i]].ch[x >> bit & 1ULL];
```

```
            -- tr[path[i + 1]].siz;
        }
        for (int i = B; i > 0 && tr[path[i]].siz == 0; -- i) {
            int bit = B - i;
            tr[path[i - 1]].ch[x >> bit & 1ULL] = -1;
            tr[path[i]] = Node{};
            bin.push_back(path[i]);
        }
        return true;
    }
    // ===== 异或序统计：若只需要插入 / 删除 / 计数，以下代码可直接删除 =====
    // 统计满足 (x ^ y) < k 的 y 的个数
    int count_xor_less(u64 x, u64 k) const {
        if constexpr (B < 64) { if (k >> B) return size(); }
        int p{ 0 }, ans{ 0 };
        for (int bit = B - 1; bit >= 0 && p != -1; -- bit) {
            int xb{ int(x >> bit & 1ULL) }, kb{ int(k >> bit & 1ULL) };
            if (kb) {
                int q{ tr[p].ch[xb] };
                if (q != -1) ans += tr[q].siz;
                p = tr[p].ch[xb ^ 1];
            } else {
                p = tr[p].ch[xb];
            }
        }
        return ans;
    }
    // 返回第 k 小的 x ^ y，k 为 0base
    optional<u64> kth_xor(u64 x, int k) const {
        if (k < 0 || k >= size()) return nullopt;
        int p{ 0 };
        u64 ans{};
        for (int bit = B - 1; bit >= 0; -- bit) {
            int b{ int(x >> bit & 1ULL) }, q{ tr[p].ch[b] };
            int cnt{ q == -1 ? 0 : tr[q].siz };
            if (k < cnt) {
                p = q;
            } else {
                k -= cnt;
                ans |= u64(1) << bit;
                p = tr[p].ch[b ^ 1];
            }
        }
        return ans;
    }
    optional<u64> min_xor(u64 x) const { return kth_xor(x, 0); }
    optional<u64> max_xor(u64 x) const { return kth_xor(x, size() - 1); }
};
```

- `BinaryTrie<B>(want)`：构造空 Trie；`B` 为参与比较的二进制位数，`want` 为预计插入次数，仅用于预留空间
- `size()`：当前元素总数，包含重复元素，$O(1)$
- `empty()`：判断是否为空，$O(1)$
- `add(x)`：插入一个 `x`，$O(B)$
- `count(x)`：返回 `x` 的出现次数，$O(B)$
- `erase(x)`：删除一个 `x`；不存在时返回 `false`，$O(B)$
- `count_xor_less(x,k)`：返回满足 `x ^ y < k` 的元素 `y` 的个数，$O(B)$
- `kth_xor(x,k)`：返回第 `k` 小的 `x ^ y`，`k` 为 `0base`；不存在返回 `nullopt`，$O(B)$
- `min_xor(x)`：返回最小的 `x ^ y`；空集合返回 `nullopt`，$O(B)$
- `max_xor(x)`：返回最大的 `x ^ y`；空集合返回 `nullopt`，$O(B)$

令 `x = 0` 后，异或序统计就退化成普通整数序统计：

- `count_xor_less(0,k)`：集合中 `< k` 的元素数量
- `kth_xor(0,k)`：集合第 `k` 小元素

可持久化 01 Trie

按顺序执行 `add(a[i])` 后，`root[r] - root[l]` 表示插入序列区间 `[l,r)` 中的多重集

```cpp
template<int B = 31>
struct PersistentBinaryTrie {
    static_assert(1 <= B && B <= 64);
    struct Node {
        int ch[2]{};
        int siz{};
    };
    vector<Node> tr{ {} };
    vector<int> root{ 0 };
    explicit PersistentBinaryTrie(int want = 0) { tr.reserve(1LL + 1LL * want * (B + 1)); }
    int size() const { return int(root.size()) - 1; }
    // ===== 节点复制与插入：可持久化核心，不建议删除 =====
    int clone(int p) {
        tr.push_back(tr[p]);
        return int(tr.size()) - 1;
    }
    // 从版本 pre 插入 x，返回新根；不会自动加入 root
    int insert(int pre, u64 x) {
        int rt{ clone(pre) }, p{ rt };
        ++ tr[p].siz;
        for (int bit = B - 1; bit >= 0; -- bit) {
            int b{ int(x >> bit & 1ULL) }, q{ clone(tr[pre].ch[b]) };
            tr[p].ch[b] = q;
            p = q;
            pre = tr[pre].ch[b];
            ++ tr[p].siz;
        }
        return rt;
    }
    // 在最后一个前缀版本上插入 x
    void add(u64 x) { root.push_back(insert(root.back(), x)); }
    // ===== 两版本作差查询：以下函数均作用于 (left_root, right_root] 表示的多重集 =====
    int count(int left_root, int right_root, u64 x) const {
        for (int bit = B - 1; bit >= 0; -- bit) {
            int b{ int(x >> bit & 1ULL) };
            left_root = tr[left_root].ch[b];
            right_root = tr[right_root].ch[b];
        }
        return tr[right_root].siz - tr[left_root].siz;
    }
    int count_xor_less(int left_root, int right_root, u64 x, u64 k) const {
        int total{ tr[right_root].siz - tr[left_root].siz };
        if constexpr (B < 64) { if (k >> B) return total; }
        int ans{ 0 };
        for (int bit = B - 1; bit >= 0; -- bit) {
            int xb{ int(x >> bit & 1ULL) }, kb{ int(k >> bit & 1ULL) };
            if (kb) {
                int l{ tr[left_root].ch[xb] }, r{ tr[right_root].ch[xb] };
                ans += tr[r].siz - tr[l].siz;
                left_root = tr[left_root].ch[xb ^ 1];
                right_root = tr[right_root].ch[xb ^ 1];
            } else {
                left_root = tr[left_root].ch[xb];
                right_root = tr[right_root].ch[xb];
            }
        }
        return ans;
    }
    // 返回第 k 小的 x ^ y，k 为 0base
    optional<u64> kth_xor(int left_root, int right_root, u64 x, int k) const {
        int total{ tr[right_root].siz - tr[left_root].siz };
        if (k < 0 || k >= total) return nullopt;
        u64 ans{};
        for (int bit = B - 1; bit >= 0; -- bit) {
            int b{ int(x >> bit & 1ULL) }, l{ tr[left_root].ch[b] };
            int r{ tr[right_root].ch[b] }, cnt{ tr[r].siz - tr[l].siz };
            if (k < cnt) {
                left_root = l;
                right_root = r;
            } else {
```

```
                k -= cnt;
                ans |= u64(1) << bit;
                left_root = tr[left_root].ch[b ^ 1];
                right_root = tr[right_root].ch[b ^ 1];
            }
        }
        return ans;
    }
    optional<u64> min_xor(int left_root, int right_root, u64 x) const {
        return kth_xor(left_root, right_root, x, 0);
    }
    optional<u64> max_xor(int left_root, int right_root, u64 x) const {
        int siz{ tr[right_root].siz - tr[left_root].siz };
        return kth_xor(left_root, right_root, x, siz - 1);
    }
    // ===== 前缀版本区间接口：若只手动管理版本根，以下代码可直接删除 =====
    int range_count(int l, int r, u64 x) const { return count(root[l], root[r], x); }
    int range_count_xor_less(int l, int r, u64 x, u64 k) const {
        return count_xor_less(root[l], root[r], x, k);
    }
    optional<u64> range_kth_xor(int l, int r, u64 x, int k) const {
        return kth_xor(root[l], root[r], x, k);
    }
    optional<u64> range_min_xor(int l, int r, u64 x) const {
        return min_xor(root[l], root[r], x);
    }
    optional<u64> range_max_xor(int l, int r, u64 x) const {
        return max_xor(root[l], root[r], x);
    }
};
```

- `PersistentBinaryTrie<B>(want)`：构造空结构，`root[0]` 为空版本
- `size()`：使用 `add()` 建立的前缀版本个数，即已插入元素数，$O(1)$
- `insert(pre,x)`：从根 `pre` 复制路径并插入一个 `x`，返回新版本根；新增 `B+1` 个节点，$O(B)$
- `add(x)`：在 `root.back()` 上插入 `x`，并把新根加入 `root`，$O(B)$
- `count(left_root,right_root,x)`：查询两版本作差后的多重集中 `x` 的出现次数，$O(B)$
- `count_xor_less(left_root,right_root,x,k)`：两版本作差后统计 `x ^ y < k` 的元素数量，$O(B)$
- `kth_xor(left_root,right_root,x,k)`：两版本作差后查询第 `k` 小异或值，`k` 为 `0base`，$O(B)$
- `min_xor/max_xor(left_root,right_root,x)`：两版本作差后的最小 / 最大异或值，$O(B)$
- `range_count(l,r,x)`：查询插入序列 `[l,r)` 中 `x` 的出现次数，$O(B)$
- `range_count_xor_less(l,r,x,k)`：查询 `[l,r)` 中满足 `x ^ y < k` 的元素数量，$O(B)$
- `range_kth_xor(l,r,x,k)`：查询 `[l,r)` 中第 `k` 小的 `x ^ y`，$O(B)$
- `range_min_xor/range_max_xor(l,r,x)`：查询 `[l,r)` 中最小 / 最大的 `x ^ y`，$O(B)$

注意事项

- `B` 表示参与 Trie 的**位数**，遍历范围为 `[B-1,0]`。例如所有数均满足 $0 \le x < 2^{31}$ 时使用 `BinaryTrie<31>`
- 当 `B < 64` 时，插入值和查询值应满足 $x < 2^B$；高于第 `B-1` 位的部分不会被 Trie 保存
- 默认使用 `u64` 按无符号整数处理。若题目中的数可能为负数，需要先明确题目要求的是二进制异或还是有符号整数顺序
- `min_xor/max_xor/kth_xor` 返回的是**异或结果** `x ^ y`，不是被选择的原数 `y`；若需要原数，可由 `y = x ^ result` 得到
- `kth_xor()` 的 `k` 为 `0base`，因此最小值对应 `k = 0`
- 普通版本是多重集，同一个值可以插入多次；`erase(x)` 每次只删除一次
- 普通 `BinaryTrie::erase()` 在计数减至 0 后自底向上回收空路径节点，后续插入优先复用空闲节点；可持久化版本不回收历史版本节点
- 可持久化版本的 `root[i]` 表示前 `i` 次 `add()` 后的状态，因此 `root[r] - root[l]` 正好对应 `[l,r)`
- 只有沿同一插入序列得到的前缀版本才能直接解释为区间差；若使用 `insert(pre,x)` 从任意历史版本分叉，新根仍然合法，但两个无祖先关系的版本之差通常没有“集合差”的意义
- 可持久化版本使用 `0` 号空节点作为所有空儿子，因此查询过程中可以直接访问 `tr[0]`

常见用法

动态集合最大异或

```
BinaryTrie<31> trie(n);
for (u64 x : a) trie.add(x);
if (auto ans = trie.max_xor(x)) {
    cout << *ans << '\n';
}
```

删除与重复元素

```
trie.add(x);
trie.add(x);
cout << trie.count(x) << '\n'; // 2
trie.erase(x);
cout << trie.count(x) << '\n'; // 1
```

第 k 小异或

```
int k; // 0base
cin >> k;
if (auto ans = trie.kth_xor(x, k)) {
    cout << *ans << '\n';
}
```

统计异或值小于 k 的元素数量

```
int cnt = trie.count_xor_less(x, k);
```

因此还可以统计半开异或值域 `[L,R)`：

```
int cnt = trie.count_xor_less(x, R) - trie.count_xor_less(x, L);
```

插入序列区间最大异或

```
PersistentBinaryTrie<31> trie(n);
for (u64 x : a) trie.add(x);
if (auto ans = trie.range_max_xor(l, r, x)) {
    cout << *ans << '\n';
}
```

其中查询范围为 `[l,r)`

插入序列区间第 k 小异或

```
if (auto ans = trie.range_kth_xor(l, r, x, k)) {
    cout << *ans << '\n';
}
```

`k` 为 `0base`

复杂度

设 Trie 位数为 `B`

01 Trie 多重集

操作	时间复杂度
`size` / `empty`	$O(1)$
`add` / `count` / `erase` / `count_xor_less` / `kth_xor` / `min_xor` / `max_xor`	$O(B)$

同时存在 n 个不同路径时最坏需要 $O(nB)$ 个有效节点；普通版回收空路径，累计申请的内存由历史峰值决定，而不是历史插入次数之和

可持久化 01 Trie

操作	时间复杂度	新增节点
`insert` / `add`	$O(B)$	`B + 1`
`count` / `count_xor_less` / `kth_xor` / `min_xor` / `max_xor` / 各 `range_*` 查询	$O(B)$	0

插入 n 次后的空间复杂度为 $O(nB)$

3.10.3 Patricia Trie（压缩位 Trie / crit-bit）

解决问题： 用压缩后的二进制 Trie 维护整数多重集，只保留真正发生分叉的二进制位，在保持异或最值和异或第 k 小查询的同时，将有效节点数从普通 01 Trie 的最坏 $O(nB)$ 降到 $O(D)$，其中 D 为不同键数量

```
struct PatriciaTrie {
    struct Node {
        int ch[2]{};
        int bit{ -1 }; // -1 表示叶子，否则表示当前分叉位
        u64 value{};
        int cnt{}, siz{};
    };
private:
    vector<Node> tr{ {} };
    vector<int> free_nodes;
    int root{}, distinct{};
    // ===== 节点创建与回收：插入 / 删除依赖，不建议删除 =====
    int new_node(const Node &u) {
        if (!free_nodes.empty()) {
            int p{ free_nodes.back() };
            free_nodes.pop_back();
            tr[p] = u;
            return p;
        }
        tr.push_back(u);
        return int(tr.size()) - 1;
    }
    int new_leaf(u64 x) {
        Node u;
        u.value = x;
        u.cnt = u.siz = 1;
        return new_node(u);
    }
    int new_branch(int bit, int a, int b) {
        Node u;
        u.ch[0] = a;
        u.ch[1] = b;
        u.bit = bit;
        u.siz = tr[a].siz + tr[b].siz;
        return new_node(u);
    }
    void release(int p) { free_nodes.push_back(p); }
    // ===== 基础查找：count / add 依赖，不建议删除 =====
    static int direction(u64 x, int bit) { return x >> bit & 1ULL; }
    int find_leaf(u64 x) const {
        int p{ root };
        while (p && tr[p].bit >= 0) {
            p = tr[p].ch[direction(x, tr[p].bit)];
        }
        return p;
    }
public:
    explicit PatriciaTrie(int want = 0) {
        tr.reserve(1LL + 2LL * want);
        free_nodes.reserve(2LL * want);
    }
    int size() const { return root ? tr[root].siz : 0; }
    int distinct_size() const { return distinct; }
    bool empty() const { return root == 0; }
    // ===== 基础多重集接口：插入 / 查询 / 删除 =====
    int count(u64 x) const {
        int p{ find_leaf(x) };
        return p && tr[p].value == x ? tr[p].cnt : 0;
    }
    void add(u64 x) {
        if (!root) {
            root = new_leaf(x);
            distinct = 1;
            return;
        }
        int same{ find_leaf(x) };
        // 重复元素只增加叶子及祖先的计数，不新增节点
        if (tr[same].value == x) {
            int p{ root };
            while (tr[p].bit >= 0) {
```

```
            ++ tr[p].siz;
            p = tr[p].ch[direction(x, tr[p].bit)];
        }
        ++ tr[p].cnt;
        ++ tr[p].siz;
        return;
    }
    // x 与已有叶子最高的不同位就是新的分叉位
    int bit{ 63 - __builtin_clzll(x ^ tr[same].value) };
    // Patricia 要求根到叶子的分叉位严格递减
    int parent{}, side{}, p{ root };
    while (tr[p].bit > bit) {
        ++ tr[p].siz;
        parent = p;
        side = direction(x, tr[p].bit);
        p = tr[p].ch[side];
    }
    int q{ new_leaf(x) }, b{ direction(x, bit) };
    int branch{ b ? new_branch(bit, p, q) : new_branch(bit, q, p) };
    if (!parent) root = branch;
    else tr[parent].ch[side] = branch;
    ++ distinct;
}
bool erase(u64 x) {
    if (!root) return false;
    // 一条路径最多经过 64 个分叉位，再加一个叶子
    array<int, 66> path{};
    int len{}, p{ root };
    while (tr[p].bit >= 0) {
        path[len ++] = p;
        p = tr[p].ch[direction(x, tr[p].bit)];
    }
    path[len ++] = p;
    if (tr[p].value != x) return false;
    // 仍有重复元素，只减少路径计数
    if (tr[p].cnt > 1) {
        -- tr[p].cnt;
        for (int i = 0; i < len; ++ i) -- tr[path[i]].siz;
        return true;
    }
    // 整棵树只有这个叶子
    if (len == 1) {
        release(root);
        root = 0;
        distinct = 0;
        return true;
    }
    // 删除叶子后，其父分叉只剩一个儿子，应一起压缩掉
    int parent{ path[len - 2] }, side{ tr[parent].ch[1] == p };
    int sibling{ tr[parent].ch[side ^ 1] };
    for (int i = 0; i + 2 < len; ++ i) -- tr[path[i]].siz;
    if (len == 2) {
        root = sibling;
    } else {
        int grand{ path[len - 3] };
        int parent_side{ tr[grand].ch[1] == parent };
        tr[grand].ch[parent_side] = sibling;
    }
    release(p);
    release(parent);
    -- distinct;
    return true;
}
// ===== 异或序统计：若不需要异或查询，以下代码可直接删除 =====
// 返回第 k 小的 x ^ y，k 为 0base
optional<u64> kth_xor(u64 x, int k) const {
    if (k < 0 || k >= size()) return nullopt;
    int p{ root };
    while (tr[p].bit >= 0) {
        int b{ direction(x, tr[p].bit) };
        // 当前分叉位异或为 0 的一侧一定更小
```

```
            int q{ tr[p].ch[b] };
            if (k < tr[q].siz) {
                p = q;
            } else {
                k -= tr[q].siz;
                p = tr[p].ch[b ^ 1];
            }
        }
        return x ^ tr[p].value;
    }
    optional<u64> min_xor(u64 x) const { return kth_xor(x, 0); }
    optional<u64> max_xor(u64 x) const { return kth_xor(x, size() - 1); }
};
```

成员含义

- `Node::bit`：内部节点的分叉位，范围为 `0..63`；叶子固定为 `-1`
- `Node::value`：叶子保存的完整 `u64` 键值
- `Node::cnt`：该叶子对应值的出现次数
- `Node::siz`：该节点子树中的元素总数，包含重复元素
- `root`：根节点编号，`0` 表示空树
- `distinct`：当前不同键的数量

接口

- `PatriciaTrie(want)`：构造空树，`want` 为预计同时存在的不同键数量，仅用于预留空间
- `size()`：返回元素总数，包含重复元素，$O(1)$
- `distinct_size()`：返回不同元素数量，$O(1)$
- `empty()`：判断是否为空，$O(1)$
- `count(x)`：返回 `x` 的出现次数，$O(B)$
- `add(x)`：插入一个 `x`，允许重复，$O(B)$
- `erase(x)`：删除一个 `x`；不存在时返回 `false`，$O(B)$
- `kth_xor(x,k)`：返回第 `k` 小的 `x ^ y`，`k` 为 `0base`；不存在返回 `nullopt`，$O(B)$
- `min_xor(x)`：返回最小的 `x ^ y`；空集合返回 `nullopt`，$O(B)$
- `max_xor(x)`：返回最大的 `x ^ y`；空集合返回 `nullopt`，$O(B)$

  这里的 $B = 64$，但实际只访问当前树中存在的分叉位

  令 `x = 0` 后：
- `kth_xor(0,k)` 就是集合中的第 `k` 小元素
- `min_xor(0)` 就是集合最小值
- `max_xor(0)` 就是集合最大值

注意事项

- Patricia Trie 不像普通 01 Trie 那样逐位建节点。**内部节点只表示真正产生分叉的最高不同位，完整键只保存在叶子中**
- 根到叶子的 `bit` 必须严格递减，这是 Patricia Trie 最重要的结构不变量
- 若两个不同值为 `x,y`，其新分叉位为：

```
63 - __builtin_clzll(x ^ y)
```

调用时保证 `x ≠ y`，因此不会对 `0` 调用 `__builtin_clzll`

- 一个内部节点的两个儿子在 `bit` 位必然分别为 `0` 和 `1`；该子树内所有键在更高位均相同，因此异或查询仍然可以按照最高分叉位贪心
- `kth_xor()` 的顺序也是按完整 `u64` 异或结果排序。未显式出现的压缩位在当前子树内全部相同，不影响子树内部的大小关系
- `kth_xor()` 的 `k` 为 `0base`
- `min_xor/max_xor/kth_xor` 返回的是 `x ^ y` **的值**，而不是被选择的原数 `y`；需要原数时可用：

```
u64 y = x ^ result;
```

- 删除最后一个 `x` 时，不仅要删除叶子，还必须把只剩一个儿子的父分叉节点一起压缩掉
- 本实现会把删除掉的叶子和分叉节点加入 `free_nodes`，之后插入时优先复用，避免反复插入删除导致节点池持续增长
- 不要在 `tr.push_back()` 前保存 `Node &` 或指针；`vector` 扩容会使其失效。本实现始终保存节点编号
- 键按 `u64` 处理。如果直接把负的有符号整数转换成 `u64`，其二进制表示仍可用于异或，但普通数值大小关系会变成无符号顺序
- Patricia Trie **不需要像普通 01 Trie 那样选择** `B=31/60/64`；本版本天然处理完整 `u64`

与普通 01 Trie 的选择

普通 01 Trie 更简单，常数通常也更小：

```
普通 01 Trie:
每个键最多新增 B 个节点
空间 O(DB)

Patricia Trie:
只为不同键建立叶子和真正的分叉
有效节点恰为 2D-1
空间 O(D)
```

因此：

- `B` 较小、内存足够：优先普通 01 Trie
- `u64` 键很多、公共前缀很长或普通 Trie 节点数过大：考虑 Patricia Trie
- 需要持久化版本：普通可持久化 01 Trie 通常更直接
- 只需要 `set/multiset` 的普通前驱后继，不涉及位结构：优先直接使用平衡树

常见用法

动态多重集

```cpp
PatriciaTrie trie(n);
trie.add(x);
trie.add(x);
cout << trie.count(x) << '\n'; // 2
trie.erase(x);
cout << trie.count(x) << '\n'; // 1
```

最大异或

```cpp
if (auto ans = trie.max_xor(x)) {
    cout << *ans << '\n';
}
```

若还需要取得被选择的原数：

```cpp
if (auto ans = trie.max_xor(x)) {
    u64 y = x ^ *ans;
}
```

第 k 小异或

```cpp
int k; // 0base
cin >> k;
if (auto ans = trie.kth_xor(x, k)) {
    cout << *ans << '\n';
}
```

作为有序多重集查询第 k 小

```cpp
if (auto ans = trie.kth_xor(0, k)) {
    cout << *ans << '\n';
}
```

复杂度

设 D 为当前不同键数量，$B = 64$

操作	时间复杂度
`size / distinct_size / empty`	$O(1)$
`count / add / erase / kth_xor / min_xor / max_xor`	$O(B)$

实际上每次操作只经过存在的分叉节点，路径上的 `bit` 严格递减，因此路径长度同时不超过 B 和 D
若当前存在 $D > 0$ 个不同键：

- 叶子恰有 D 个
- 内部分叉节点恰有 $D - 1$ 个

- 有效节点总数恰为 $2D-1$
  节点池支持删除后复用，因此空间复杂度按历史同时存在的最大不同键数 $D_{\max}$ 计为 $O(D_{\max})$

3.10.4 可持久化 01 Trie

```
template<class U = uint32_t, int B = numeric_limits<U>::digits>
struct Persistent01Trie {
    static_assert(is_integral_v<U> && is_unsigned_v<U> && !is_same_v<U, bool>);
    static_assert(1 <= B && B <= numeric_limits<U>::digits);
    struct Node {
        int ch[2]{};
        int cnt{};
    };
    vector<Node> tr{ Node{} };
    vector<int> root{ 0 };
    Persistent01Trie() = default;
    explicit Persistent01Trie(const vector<U> &a) {
        reserve_updates(a.size());
        for (U x : a) append(x);
    }
private:
    static void check(U x) {
        if constexpr (B < numeric_limits<U>::digits) assert((x >> B) == 0);
    }
    int clone(int p) {
        tr.push_back(tr[p]);
        return int(tr.size()) - 1;
    }
    U xor_best_root(int a, int b, U x, bool mx) const {
        check(x);
        assert(size_root(a, b) > 0);
        U ans{};
        for (int i = B - 1; i >= 0; --i) {
            int v = int((x >> i) & U(1));
            int d = v ^ int(mx);
            if (tr[tr[b].ch[d]].cnt == tr[tr[a].ch[d]].cnt) d ^= 1;
            if (d != v) ans |= U(1) << i;
            a = tr[a].ch[d];
            b = tr[b].ch[d];
        }
        return ans;
    }
public:
    // ===== 路径复制与版本管理 =====
    void reserve_updates(size_t q) { tr.reserve(tr.size() + q * (B + 1)); }
    int add_root(int p, U x, int delta = 1) {
        check(x);
        if (delta == 0) return p;
        int q = clone(p), u = q;
        assert(tr[u].cnt + delta >= 0);
        tr[u].cnt += delta;
        for (int i = B - 1; i >= 0; --i) {
            int d = int((x >> i) & U(1));
            int v = tr[p].ch[d], w = clone(v);
            tr[u].ch[d] = w;
            assert(tr[w].cnt + delta >= 0);
            tr[w].cnt += delta;
            p = v;
            u = w;
        }
        return q;
    }
    void append(U x) { root.push_back(add_root(root.back(), x)); }
    int size_root(int a, int b) const { return tr[b].cnt - tr[a].cnt; }
    int size(int l, int r) const {
        assert(0 <= l && l <= r && r < int(root.size()));
        return size_root(root[l], root[r]);
    }
    // ===== 最大 / 最小异或 =====
```

```
    U max_xor_root(int a, int b, U x) const { return xor_best_root(a, b, x, true); }
    U min_xor_root(int a, int b, U x) const { return xor_best_root(a, b, x, false); }
    U max_xor(int l, int r, U x) const {
        assert(size(l, r) > 0);
        return max_xor_root(root[l], root[r], x);
    }
    U min_xor(int l, int r, U x) const {
        assert(size(l, r) > 0);
        return min_xor_root(root[l], root[r], x);
    }
    // ===== 按异或值排序第 k 小 =====
    U kth_xor_root(int a, int b, U x, int k) const {
        check(x);
        assert(0 <= k && k < size_root(a, b));
        U ans{};
        for (int i = B - 1; i >= 0; --i) {
            int d = int((x >> i) & U(1));
            int ca = tr[a].ch[d], cb = tr[b].ch[d];
            int cnt = tr[cb].cnt - tr[ca].cnt;
            if (k >= cnt) {
                k -= cnt;
                d ^= 1;
                ans |= U(1) << i;
            }
            a = tr[a].ch[d];
            b = tr[b].ch[d];
        }
        return ans;
    }
    U kth_xor(int l, int r, U x, int k) const {
        assert(0 <= k && k < size(l, r));
        return kth_xor_root(root[l], root[r], x, k);
    }
    // ===== 异或值小于 limit 的个数 =====
    int count_xor_less_root(int a, int b, U x, U limit) const {
        check(x);
        if constexpr (B < numeric_limits<U>::digits) {
            if (limit >> B) return size_root(a, b);
        }
        int ans = 0;
        for (int i = B - 1; i >= 0; --i) {
            int d = int((x >> i) & U(1));
            if ((limit >> i) & U(1)) {
                ans += tr[tr[b].ch[d]].cnt - tr[tr[a].ch[d]].cnt;
                d ^= 1;
            }
            a = tr[a].ch[d];
            b = tr[b].ch[d];
        }
        return ans;
    }
    int count_xor_less(int l, int r, U x, U limit) const {
        assert(0 <= l && l <= r && r < int(root.size()));
        return count_xor_less_root(root[l], root[r], x, limit);
    }
    // ===== 树上路径四版本最大异或 =====
    // a + b - c - d，四个参数是版本根，不是序列下标
    U max_xor_path(int a, int b, int c, int d, U x) const {
        check(x);
        assert(tr[a].cnt + tr[b].cnt - tr[c].cnt - tr[d].cnt > 0);
        U ans{};
        for (int i = B - 1; i >= 0; --i) {
            int v = int((x >> i) & U(1)), t = v ^ 1;
            int ca = tr[a].ch[t], cb = tr[b].ch[t];
            int cc = tr[c].ch[t], cd = tr[d].ch[t];
            int cnt = tr[ca].cnt + tr[cb].cnt - tr[cc].cnt - tr[cd].cnt;
            if (cnt == 0) t ^= 1;
            if (t != v) ans |= U(1) << i;
            a = tr[a].ch[t];
            b = tr[b].ch[t];
            c = tr[c].ch[t];
```

```
            d = tr[d].ch[t];
        }
        return ans;
    }
};
```

成员变量

- `tr`：持久化节点池；`tr[0]` 为空节点，`Node::ch[0/1]` 为子节点，`Node::cnt` 为经过该节点的元素出现次数。
- `root[i]`：使用 `append()` 依次插入时，保存前缀 `[0, i)` 的版本根；`root[0] = 0`。
- `U`：无符号整数类型，默认 `uint32_t`；`B`：维护的有效低位数，默认 `numeric_limits<U>::digits`。

接口

接口	用途	时间复杂度
`Persistent01Trie(a)`	从 `vector<U>` 建立全部前缀版本	$O(nB)$
`Persistent01Trie()`	建立空结构	$O(1)$
`reserve_updates(q)`	为后续 `q` 次非零增量的版本更新预留节点容量	重新分配时 $O(\text{当前节点数})$
`add_root(p, x, delta=1)`	在版本根 `p` 中改变 `x` 的出现次数，返回新根；不自动加入 `root`	$O(B)$
`append(x)`	在最新前缀版本后插入 `x`，追加一个版本根	$O(B)$
`size_root(a, b)`	两个版本差分的元素总数 `cnt(b)-cnt(a)`	$O(1)$
`size(l, r)`	前缀版本之差 `[l,r)` 的元素总数	$O(1)$
`max_xor_root(a,b,x)` / `min_xor_root(a,b,x)`	两版本差分集合中最大 / 最小的 `y ^ x`	$O(B)$
`max_xor(l,r,x)` / `min_xor(l,r,x)`	`[l,r)` 中最大 / 最小的 `a[i] ^ x`	$O(B)$
`kth_xor_root(a,b,x,k)`	两版本差分中按 `y ^ x` 升序的第 `k` 个**异或值**	$O(B)$
`kth_xor(l,r,x,k)`	`[l,r)` 中按 `a[i] ^ x` 升序的第 `k` 个**异或值**	$O(B)$
`count_xor_less_root(a,b,x,limit)`	两版本差分中满足 `(y ^ x) < limit` 的元素个数	$O(B)$
`count_xor_less(l,r,x,limit)`	`[l,r)` 中满足 `(a[i] ^ x) < limit` 的元素个数	$O(B)$
`max_xor_path(a,b,c,d,x)`	四版本组合 `a+b-c-d` 对应集合中的最大异或值	$O(B)$

其中 `k` 从 0 开始；带 `_root` 的接口传版本根编号，其余 `[l,r)` 接口传数组下标。

注意事项

- **值域限制**：只接受无符号整数 `U`；当 `B` 小于类型位宽时，插入的 `x`、异或查询参数 `x` 都必须小于 `2^B`。`limit` 允许超出 `B` 位范围，此时计数直接返回区间总数。
- **返回异或值**：`max_xor`、`min_xor`、`kth_xor` 返回 `y ^ x`，不是元素 `y`。若要取得对应元素，使用 `y = result ^ x`；有重复值时返回的元素值仍唯一确定，但不能区分原始下标。
- **合法版本差分**：`root[r] - root[l]` 仅在 `l ≤ r` 时表示非负频数集合。对任意两个自定义版本根，须保证差分后**每个值的频数均非负**，不能简单对两个无关版本相减。
- **版本删除**：`add_root(p, x, delta < 0)` 仅在旧版本中 `x` 的数量至少为 `-delta` 时合法；必须保证所有版本的频数始终非负。`delta=0` 原样返回 `p`。代码在启用断言时会检查沿路径各节点的计数不会减成负数。
- **树上路径**：路径点权前缀根 `rt[u]` 包含根至 `u`；应传入 `rt[u], rt[v], rt[lca], rt[parent(lca)]`，根节点无父节点时使用 `0` 号空根。
- **内存**：每次非零 `add_root` 恰好增加 `B+1` 个节点；`Node` 通常是 12 字节。`B=32`、`n=2e5` 时约 660 万节点，仅节点池就约 79 MB（不计 `vector` 容量冗余），内存紧张时设小有效位宽，且优先 `reserve_updates()`。
- **裁剪**：只需要最大异或时，可删除整个 `kth_xor`、`count_xor_less`、`max_xor_path` 功能分区以及 `min_xor_root()` / `min_xor()`；基础建树及 `xor_best_root()` 必须保留。

常见用法

静态区间最大 / 最小异或

```
vector<uint32_t> a = {3, 8, 2, 7, 5};
Persistent01Trie<uint32_t, 4> trie(a);
auto mx = trie.max_xor(1, 5, 6);  // max(8^6, 2^6, 7^6, 5^6) = 14
auto mn = trie.min_xor(1, 5, 6);  // 7^6 = 1
auto y = mx ^ 6U;                 // 获得产生最大异或的原值 8
```

异或第 k 小与异或值域计数

```
auto kth = trie.kth_xor(1, 5, 6, 2);          // 异或结果升序第 3 个（0base）
int cnt = trie.count_xor_less(1, 5, 6, 8U);  // 异或结果 < 8 的个数
```

手动建立历史版本

```
Persistent01Trie<uint32_t, 20> trie;
int v1 = trie.add_root(0, 7);
int v2 = trie.add_root(v1, 11);
int v3 = trie.add_root(v2, 7, -1);  // 删除一个 7，v2 仍可用
uint32_t ans = trie.max_xor_root(v1, v2, 2); // 只查询新增的 11
```

树上路径最大异或

```
Persistent01Trie<uint32_t, 31> trie;
trie.reserve_updates(n);
vector<int> rt(n);
// DFS 遍历树：rt[u] = trie.add_root(fa == -1 ? 0 : rt[fa], w[u]);
int p = lca(u, v);
int rp = parent[p] == -1 ? 0 : rt[parent[p]];
auto ans = trie.max_xor_path(rt[u], rt[v], rt[p], rp, x);
```

复杂度

令 n 为插入次数，B 为维护的二进制位数。

操作	时间复杂度	新增节点
初始化空结构	$O(1)$	1 个空节点
建立 `n` 个前缀版本	$O(nB)$	$n(B+1)$
非零增量的 `add_root` / `append`	$O(B)$	$B+1$
两版本差分求元素总数	$O(1)$	0
最大 / 最小异或、异或第 k 小、异或值小于阈值的数量	$O(B)$	0
四版本树上路径最大异或	$O(B)$	0

空间复杂度：$O(nB)$；每个版本根仅需一个整数下标。`B=32` 或 `64` 时，查询只沿一条从最高位到最低位的路径下降。

3.10.5 FastSet（分层位集）

```
struct FastSet {
    using u64 = uint64_t;
    int n;
    vector<vector<u64>> a;
    explicit FastSet(int n_ = 0) : n(n_) {
        assert(n >= 0);
        int m = max(n, 1);
        do {
            m = (m - 1) / 64 + 1;
            a.emplace_back(m, 0);
        } while (m > 1);
    }
    // ===== 基础操作：重复插入 / 删除不存在元素均无影响 =====
    bool empty() const { return a.back()[0] == 0; }
    bool contains(int x) const {
        assert(0 <= x && x < n);
        return (a[0][x >> 6] >> (x & 63)) & 1;
    }
    void insert(int x) {
        assert(0 <= x && x < n);
        for (auto &v : a) {
            u64 &w = v[x >> 6], mask = 1ULL << (x & 63);
            if (w & mask) break;
            w |= mask;
            if (w != mask) break;
            x >>= 6;
        }
    }
    void erase(int x) {
        assert(0 <= x && x < n);
        for (auto &v : a) {
            u64 &w = v[x >> 6], mask = 1ULL << (x & 63);
            if (!(w & mask)) break;
```

```
            w &= ~mask;
            if (w) break;
            x >>= 6;
        }
    }
    // ===== 前驱后继：next ≥ x，prev ≤ x；不存在返回 n / -1 =====
    int next(int x) const {
        x = max(x, 0);
        if (x >= n) return n;
        for (int d = 0; d < int(a.size()); ++d) {
            int q = x >> 6;
            if (q >= int(a[d].size())) break;
            u64 w = a[d][q] >> (x & 63);
            if (w) {
                x += countr_zero(w);
                for (int e = d; e-- > 0;) {
                    x = (x << 6) | countr_zero(a[e][x]);
                }
                return x;
            }
            x = q + 1;
        }
        return n;
    }
    int prev(int x) const {
        x = min(x, n - 1);
        for (int d = 0; d < int(a.size()) && x >= 0; ++d) {
            int q = x >> 6;
            u64 w = a[d][q] << (63 - (x & 63));
            if (w) {
                x -= countl_zero(w);
                for (int e = d; e-- > 0;) {
                    x = (x << 6) | (63 - countl_zero(a[e][x]));
                }
                return x;
            }
            x = q - 1;
        }
        return -1;
    }
    // ===== 批量建集 =====
    explicit FastSet(const string &s) : FastSet(int(s.size())) { build(s); }
    void build(const string &s) {
        assert(int(s.size()) == n);
        fill(a[0].begin(), a[0].end(), 0);
        for (int i = 0; i < n; ++i) {
            if (s[i] == '1') a[0][i >> 6] |= 1ULL << (i & 63);
        }
        for (int d = 1; d < int(a.size()); ++d) {
            fill(a[d].begin(), a[d].end(), 0);
            for (int i = 0; i < int(a[d - 1].size()); ++i) {
                if (a[d - 1][i]) a[d][i >> 6] |= 1ULL << (i & 63);
            }
        }
    }
};
```

成员变量

- `n`：整数全集 `[0, n)` 的大小，不是当前元素数量。
- `a[d]`：第 `d` 层的 `uint64_t` 块；`a[0]` 存储实际元素，更高层的第 `i` 位标记下层第 `i` 个块是否非空。

构造与基础操作

接口	作用	时间复杂度
`FastSet(n)`	创建全集为 `[0, n)` 的空集合	$O(1 + n/64)$
`empty()`	集合是否为空	$O(1)$
`contains(x)`	判断 `x` 是否存在	$O(1)$
`insert(x)`	插入 `x`；重复插入无影响	$O(log_{64}(n+1))$

接口	作用	时间复杂度
`erase(x)`	删除 `x`；不存在时无影响	$O(log_{64}(n+1))$
`next(x)`	最小的 $\geqslant$ `x` 的元素；不存在返回 `n`	$O(log_{64}(n+1))$
`prev(x)`	最大的 $\leqslant$ `x` 的元素；不存在返回 `-1`	$O(log_{64}(n+1))$

可删除的批量建集接口

- `FastSet(s)`：`s` 为 `0/1` 字符串，令 `n = s.size()`，将 `s[i] == '1'` 的下标加入集合，`O(n)`。
- `build(s)`：用长度为 `n` 的 `0/1` 字符串**覆盖**当前集合，重新构造所有层；`O(n)`。
- 不需要按字符串建集时，可直接删除 `FastSet(const string &)` 与 `build()`，其余代码无需修改。

注意事项

- 集合元素为 **`0base` 的非负整数**，合法范围 `[0, n)`；`insert`、`erase`、`contains` 的参数必须在此范围内。
- `next(x)`、`prev(x)` **都是包含 `x` 的查询**。严格后继用 `next(x + 1)`，严格前驱用 `prev(x - 1)`（这里 `x` 通常在 `[0, n)`）。
- `next(x)` 的无答案标记是 `n`，**不是** `-1`；`prev(x)` 的无答案标记是 `-1`。
- `next(x)` 支持 `x < 0`，视为从 `0` 开始；`x ⩾ n` 返回 `n`。`prev(x)` 支持 `x ⩾ n`，视为从 `n - 1` 开始；`x < 0` 返回 `-1`。
- `n = 0` 时允许构造空集合，`empty()` 为真，`next(x) == 0`、`prev(x) == -1`；此时不可调用 `contains/insert/erase`。
- 仅支持**集合语义**，不记录重复次数；不提供元素第 `k` 小、集合元素数量或区间统计接口。
- 采用稠密全集预分配，适合 `n` 已知且适中的整数键；对于稀疏的超大 / 64 位键域，改用 `set`、整数 Trie 或先离散化。**离散化后无法无代价地在线加入未预知的新值。**

常见用法

插入、删除和前驱后继

```
FastSet st(100);           // 元素范围 [0, 100)
st.insert(3);
st.insert(10);
st.insert(10);            // 重复插入无影响
st.erase(3);

bool has = st.contains(10); // true
int a = st.next(4);       // 10: 最小的 ⩾ 4
int b = st.prev(9);       // -1: 没有 ⩽ 9 的元素
int c = st.next(11);      // 100: 无答案
```

严格前驱与严格后继

```
int pre = st.prev(x - 1);  // 最大的 < x
int suc = st.next(x + 1);  // 最小的 > x
if (pre == -1) { /* 无严格前驱 */ }
if (suc == st.n) { /* 无严格后继 */ }
```

按顺序枚举元素

```
for (int x = st.next(0); x < st.n; x = st.next(x + 1)) {
    // 访问 x，按从小到大的顺序
}
```

从 01 字符串建集（Library Checker: Predecessor Problem）

```
string s;
cin >> s;
FastSet st(s);                // s[i] == '1' 表示包含 i

int x;
cin >> x;
int ans = st.next(x);
if (ans == st.n) ans = -1;    // 题目输出要求不存在时为 -1
cout << ans << '\n';
```

复杂度

设整数全集大小为 `n`，层数 `L = max(1, ceil(log_64(max(n, 1))))`（仅按数量级理解）

操作	时间复杂度
构造空集	`O(1 + n / 64)`
单点插入、删除	`O(L)`，遇到非空块可提前终止
成员查询、判空	`O(1)`
前驱、后继	`O(L)`
字符串批量建集	`O(n)`

空间复杂度为 `O(1 + n / 64)` 个 `uint64_t` 块，即 `O(n)` bit；上层所需块数逐层按 64 倍缩小。

3.10.6 DLX（精确覆盖与重复覆盖）

```
struct DLX {
    struct Node {
        int l, r, u, d, c, row;
    };
    int m, rows{};
    vector<Node> tr;
    vector<int> sz, path, ans;
    explicit DLX(int columns, int ones_hint = 0) : m(columns), tr(m + 1), sz(m + 1) {
        assert(m >= 0 && ones_hint >= 0);
        tr.reserve(size_t(m) + 1 + ones_hint);
        for (int i = 0; i <= m; ++ i) {
            tr[i] = {i - 1, i + 1, i, i, i, -1};
        }
        tr[0].l = m;
        tr[m].r = 0;
    }
    // ===== 建立稀疏 01 矩阵：列号 0base，行内列号不得重复 =====
    void add_row(const vector<int> &cols, int row_id = -1) {
        int id = row_id < 0 ? rows : row_id;
        ++ rows;
        int first = 0;
        for (int x : cols) {
            assert(0 <= x && x < m);
            int c = x + 1, p = int(tr.size());
            tr.push_back({p, p, tr[c].u, c, c, id});
            tr[tr[c].u].d = p;
            tr[c].u = p;
            ++ sz[c];
            if (!first) {
                first = p;
            } else {
                int last = tr[first].l;
                tr[p].l = last;
                tr[p].r = first;
                tr[last].r = p;
                tr[first].l = p;
            }
        }
    }
    int choose_column() const {
        int chosen = tr[0].r;
        for (int c = tr[chosen].r; c != 0; c = tr[c].r) {
            if (sz[c] < sz[chosen]) chosen = c;
        }
        return chosen;
    }
    // ===== 精确覆盖：以下部分可与重复覆盖部分二选一 =====
    void cover(int c) {
        tr[tr[c].r].l = tr[c].l;
        tr[tr[c].l].r = tr[c].r;
        for (int i = tr[c].d; i != c; i = tr[i].d) {
            for (int j = tr[i].r; j != i; j = tr[j].r) {
                tr[tr[j].d].u = tr[j].u;
                tr[tr[j].u].d = tr[j].d;
                -- sz[tr[j].c];
```

```
        }
    }
}
void uncover(int c) {
    for (int i = tr[c].u; i != c; i = tr[i].u) {
        for (int j = tr[i].l; j != i; j = tr[j].l) {
            ++ sz[tr[j].c];
            tr[tr[j].d].u = j;
            tr[tr[j].u].d = j;
        }
    }
    tr[tr[c].r].l = c;
    tr[tr[c].l].r = c;
}
bool dfs_exact() {
    if (tr[0].r == 0) {
        ans = path;
        return true;
    }
    int c = choose_column();
    if (!sz[c]) return false;
    cover(c);
    for (int i = tr[c].d; i != c; i = tr[i].d) {
        path.push_back(tr[i].row);
        for (int j = tr[i].r; j != i; j = tr[j].r) cover(tr[j].c);
        bool ok = dfs_exact();
        for (int j = tr[i].l; j != i; j = tr[j].l) uncover(tr[j].c);
        path.pop_back();
        if (ok) {
            uncover(c);
            return true;
        }
    }
    uncover(c);
    return false;
}
bool solve_exact() {
    ans.clear();
    path.clear();
    return dfs_exact();
}
// ===== 重复覆盖：只删除列头，行的环形链表不变；不用时可整段删除 =====
vector<unsigned char> active;
vector<int> changed, seen;
int stamp{}, best{};
void remove_column(int c) {
    active[c] = false;
    tr[tr[c].r].l = tr[c].l;
    tr[tr[c].l].r = tr[c].r;
    changed.push_back(c);
}
void restore_columns(int checkpoint) {
    while (int(changed.size()) > checkpoint) {
        int c = changed.back();
        changed.pop_back();
        tr[tr[c].r].l = c;
        tr[tr[c].l].r = c;
        active[c] = true;
    }
}
// 贪心找两两不能被同一行覆盖的列，作为还需选择行数的下界
int lower_bound(int limit) {
    if (stamp == INT_MAX) {
        fill(seen.begin(), seen.end(), 0);
        stamp = 0;
    }
    ++ stamp;
    int need = 0;
    for (int c = tr[0].r; c != 0; c = tr[c].r) {
        if (seen[c] == stamp) continue;
        if (++ need >= limit) return need;
```

```
            for (int i = tr[c].d; i != c; i = tr[i].d) {
                int j = i;
                do {
                    seen[tr[j].c] = stamp;
                    j = tr[j].r;
                } while (j != i);
            }
        }
        return need;
    }
    bool dfs_repeat(bool first_only) {
        if (tr[0].r == 0) {
            best = int(path.size());
            ans = path;
            return first_only;
        }
        if (lower_bound(best - int(path.size())) >= best - int(path.size())) return false;
        int c = choose_column();
        if (!sz[c]) return false;
        for (int i = tr[c].d; i != c; i = tr[i].d) {
            int checkpoint = changed.size();
            path.push_back(tr[i].row);
            int j = i;
            do {
                int col = tr[j].c;
                if (active[col]) remove_column(col);
                j = tr[j].r;
            } while (j != i);
            bool ok = dfs_repeat(first_only);
            restore_columns(checkpoint);
            path.pop_back();
            if (ok) return true;
        }
        return false;
    }
    void prepare_repeat(int limit) {
        ans.clear();
        path.clear();
        changed.clear();
        active.assign(m + 1, true);
        seen.assign(m + 1, 0);
        stamp = 0;
        best = min(rows, limit < 0 ? rows : limit) + 1;
    }
    // 返回所需的最少行数；无解或超过 limit 返回 -1，答案保存在 ans
    int min_cover(int limit = -1) {
        prepare_repeat(limit);
        dfs_repeat(false);
        return ans.empty() && m != 0 ? -1 : int(ans.size());
    }
    // 只判断是否能用不超过 limit 行覆盖；找到一组就结束
    bool cover_at_most(int limit) {
        assert(limit >= 0);
        prepare_repeat(limit);
        return dfs_repeat(true);
    }
};
```

成员变量

- `m`：约束列数，外部列号为 `[0, m)`，内部列头为 `[1, m]`；`0` 为总表头。
- `tr`：十字链表节点池，`Node::{l,r,u,d}` 为上下左右四向链接，`c` 为所属列头编号，`row` 为原始行编号。
- `sz[c]`：当前列的候选行数。**精确覆盖**中随删列更新；**重复覆盖**中保留原始行数，便于选择分支列。
- `rows`：调用 `add_row()` 的次数（包括空行），`ans`：成功时选中行编号组成的一个解，按搜索顺序排列。
- `path`：当前搜索路径；`active/changed/seen/stamp/best` 仅供重复覆盖搜索使用。

构造与公开接口

接口	作用	复杂度
`DLX(m, ones_hint = 0)`	建立 `m` 列空矩阵，按需预留数据节点	`O(m)`

接口	作用	复杂度
`add_row(cols, row_id = -1)`	加入一行，列号 `0base`；默认自动编号	`O(
`solve_exact()`	查找每列恰好出现一次的任意解；成功返回 `true`，答案写入 `ans`	指数级搜索
`min_cover(limit = -1)`	选最少行，使每列至少出现一次；若无解或最优解超过 `limit`，返回 `-1`；成功返回最小行数，答案写入 `ans`	指数级搜索
`cover_at_most(limit)`	只判断能否用至多 `limit` 行覆盖所有列；找到一组即退出，成功时 `ans` 给出方案	指数级搜索

- `ones_hint` 是矩阵中 1 的总个数的容量预估，不是行数；设置不足不影响正确性。
- 精确覆盖只能选择互不冲突的行；重复覆盖允许已覆盖的列被其他选中行再次覆盖。
- `limit = -1` 仅对 `min_cover()` 表示不限制行数；`cover_at_most(limit)` 要求 `limit ⩾ 0`。

注意事项

- 建图语义：每行表示一个候选方案，每列表示一个约束；每次 `add_row(cols)` 的 `cols` 必须是互不相同的列编号（无需排序）。重复列会破坏十字链表的搜索逻辑，应在外部去重。
- `row_id` 若省略，采用该行加入时的 `0base` 编号；指定后原样写入 `ans`。希望方案中行号唯一时，不要传入重复的自定义 `row_id`。
- 空行不会帮助覆盖任何列，可以加入但不会被选中；若有一列从未被任何行包含，则两类问题均无解。
- `m = 0` 视为空约束：精确覆盖成功、最少重复覆盖数为 `0`、限制非负时可以覆盖，`ans` 为空。
- 两种覆盖的删链过程不同，不能混用：
  - 精确覆盖的 `cover/uncover` 会删掉冲突行在其他列中的垂直节点，同时维护 `sz`，恢复时必须逆序。
  - 重复覆盖的 `remove_column/restore_columns` 仅摘除/恢复列头，保留各行的水平环与各列的垂直链，所以允许行交叠。`lower_bound()` 通过列间可能共享的行构造仍需选行数的下界。
- 两类搜索都会在返回前完整恢复链表；同一对象可重复查询，甚至先 `solve_exact()` 再 `min_cover()`，但不要在递归搜索过程中并发调用其它搜索或修改矩阵。
- 只支持无权覆盖：`min_cover()` 最小化的是选中行数。带权集合覆盖需要更换代价与剪枝下界，不可直接将 `best` 当权重使用。
- `ans` 只提供一组解；需要枚举全部精确覆盖解时，应修改 `dfs_exact()` 的成功分支与返回方式，不能直接复用当前的提前返回逻辑。
- 该实现默认所有列都是必选约束。部分列“至多选一次但可以不选”（secondary columns）属于扩展模型，需要修改表头初始化和精确覆盖建模，不是重复覆盖。
- 使用 C++20 环境（本实现自身的语法兼容 C++17），默认外部已有 `<bits/stdc++.h>` 和 `using namespace std;`。

功能裁剪

- 仅精确覆盖：删除 `active/changed/seen/stamp/best` 和从 `remove_column()` 到 `cover_at_most()` 的整段重复覆盖实现；`Node/tr/sz/add_row/choose_column` 不变。
- 仅重复覆盖：删除 `cover()`、`uncover()`、`dfs_exact()`、`solve_exact()`；共用建图、`choose_column()` 和重复覆盖部分保留。
- 只需最少行数：可删除 `cover_at_most()`；保留 `min_cover()`、`prepare_repeat()`、`dfs_repeat()` 与 `lower_bound()`。
- 只需限制行数可行性：可删除 `min_cover()`；保留 `cover_at_most()` 和重复覆盖辅助函数。
- 若只想获得解是否存在，`ans` / `path` 仍需要用于搜索路径，不能直接删除；可以仅取消成功分支对 `ans` 的拷贝，并相应修改接口。

常见用法

精确覆盖：每列恰好覆盖一次

```
DLX dlx(3, 6); // 3 个约束，预计 6 个 1

dlx.add_row({0, 2}, 10);
dlx.add_row({1}, 20);
dlx.add_row({0, 1}, 30);
dlx.add_row({2}, 40);

if (dlx.solve_exact()) {
    for (int id : dlx.ans) {
        // 一组可行方案：{10,20} 或 {30,40}，顺序不保证
    }
}
```

重复覆盖：最少选多少行

```
DLX dlx(3);
dlx.add_row({0, 1});
dlx.add_row({1, 2});

bool exact = dlx.solve_exact();   // false：列 1 会重叠
int k = dlx.min_cover();          // 2：选两行覆盖三列
bool ok = dlx.cover_at_most(1);   // false：1 行不够
```

二分答案的判定函数

当题目在某个参数下建立可覆盖关系，且参数越大可选择的行覆盖范围越大，可以每次重新建矩阵，用 `cover_at_most(k)` 判断能否用不超过 `k` 行覆盖全部约束：

```
DLX dlx(m, ones_hint);
for (int i = 0; i < n; ++ i) {
    vector<int> cols;
    // 收集第 i 个方案能够覆盖的列
    dlx.add_row(cols, i);
}
bool ok = dlx.cover_at_most(k);
```

`dlx` 需要在矩阵关系变化后重新构建，不能仅通过修改 `sz` 代替增删矩阵中的 1。

数独：把填数方案转为四类精确覆盖约束

普通 9×9 数独有 `324` 列约束（81 个格子、81 个行数字、81 个列数字、81 个宫数字），每个填数候选方案恰好关联四列：

```
DLX dlx(324, 729 * 4);
for (int r = 0; r < 9; ++ r) {
    for (int c = 0; c < 9; ++ c) {
        for (int d = 0; d < 9; ++ d) {
            // 若 (r,c) 是已填格且不等于 d+1，跳过
            int b = (r / 3) * 3 + c / 3;
            dlx.add_row({r * 9 + c, 81 + r * 9 + d,
                         162 + c * 9 + d, 243 + b * 9 + d},
                        (r * 9 + c) * 9 + d);
        }
    }
}
if (dlx.solve_exact()) {
    for (int id : dlx.ans) {
        int d = id % 9, c = (id / 9) % 9, r = id / 81;
        // a[r][c] = d + 1;
    }
}
```

复杂度

记列数为 `m`、行数为 `n`、稀疏矩阵中 1 的个数为 `E`，当前分支所剩约束数为 `m'`。

操作	时间复杂度
构造	`O(m)`
加入一行	`O(行内 1 的个数)`（均摊）
选分支列	`O(m')`；采用当前候选行数最小列启发式
精确覆盖 `cover/uncover`	按涉及的十字链表节点数线性增量处理
重复覆盖一次列摘除/恢复	`O(1)`；一整行的覆盖操作为 `O(行内 1 的个数)`
重复覆盖贪心下界 `lower_bound()`	最坏 `O(m E)` 每个搜索节点，通常更少
两类求解总体	最坏指数级；分支数量取决于约束稀疏程度与剪枝

- 空间：`O(m + E + n)`（节点池、列大小、候选路径与重复覆盖工作空间）。
- 每个矩阵 1 使用一个 `Node`（通常 24 字节），外加 `m + 1` 个表头；预留 `ones_hint` 减少扩容。
- 在重复覆盖中，`lower_bound()` 是用于优化搜索的正确下界；在极少分支的小问题中它也会引入额外扫描成本，可按题型决定是否简化剪枝。

3.10.7 可持久化队列（Persistent Queue）

```
template<class T = i64>
struct PersistentQueue {
    struct Node {
        T val{};
        int dep{}, off{};
    };
    struct Version {
        int tail{}, len{};
```

```
    };
    vector<Node> tr{ Node{} };
    vector<int> up;
    vector<Version> ver{ Version{} };
    // ===== 倍增祖先：队首及随机访问依赖本函数 =====
    int ancestor(int p, int k) const {
        assert(0 <= k && k <= tr[p].dep);
        while (k) {
            int b = countr_zero(unsigned(k));
            p = up[tr[p].off + b];
            k &= k - 1;
        }
        return p;
    }
    // ===== 持久化入队 / 出队：版本 0 为空队列，旧版本始终不变 =====
    int push(int v, T x) {
        assert(0 <= v && v < int(ver.size()));
        int p = ver[v].tail, d = tr[p].dep + 1;
        int q = int(tr.size());
        tr.push_back({move(x), d, int(up.size())});
        up.push_back(p);
        for (int j = 1; (1LL << j) <= d; ++ j) {
            int mid = up.back();
            up.push_back(up[tr[mid].off + j - 1]);
        }
        ver.push_back({q, ver[v].len + 1});
        return int(ver.size()) - 1;
    }
    int pop(int v) {
        assert(0 <= v && v < int(ver.size()) && ver[v].len > 0);
        auto [tail, len] = ver[v];
        ver.push_back({len == 1 ? 0 : tail, len - 1});
        return int(ver.size()) - 1;
    }

    // ===== 基础查询：队首 O(log N)，队尾 / 大小 O(1) =====
    int size(int v) const {
        assert(0 <= v && v < int(ver.size()));
        return ver[v].len;
    }
    bool empty(int v) const { return size(v) == 0; }
    T front(int v) const {
        assert(size(v) > 0);
        return tr[ancestor(ver[v].tail, ver[v].len - 1)].val;
    }
    T back(int v) const {
        assert(size(v) > 0);
        return tr[ver[v].tail].val;
    }
    // ===== 可选：按队列顺序随机访问 =====
    T at(int v, int k) const {
        assert(0 <= k && k < size(v));
        return tr[ancestor(ver[v].tail, ver[v].len - 1 - k)].val;
    }
    // ===== 可选：预留空间 / 原样复制版本 =====
    void reserve(int pushes, int versions) {
        assert(pushes >= 0 && versions >= 0);
        tr.reserve(tr.size() + pushes);
        ver.reserve(ver.size() + versions);
        if (pushes) {
            int lg = bit_width(unsigned(tr.size() + pushes));
            up.reserve(up.size() + size_t(pushes) * lg);
        }
    }
    int copy(int v) {
        assert(0 <= v && v < int(ver.size()));
        ver.push_back(ver[v]);
        return int(ver.size()) - 1;
    }
};
```

成员变量

- `tr[p]`：第 `p` 个历史尾节点；`val` 为入队值，`dep` 为该节点在历史链上的深度，`off` 为其倍增表在 `up` 中的起始下标。
- `up`：扁平化存储各节点的 `2^j` 级祖先；无需固定 `LOG` 或为每个节点分配 `vector`。
- `ver[v] = {tail, len}`：版本 `v` 的尾节点和当前队列长度；仅 `len` 个尾部历史节点属于该队列。
- 版本 `0` 是空队列。每次 `push`、`pop` 或 `copy` 都追加一个版本并返回新版本编号；旧版本永远不变。

基础接口

接口	作用	时间复杂度
`push(v, x)`	从版本 `v` 的队尾插入 `x`，返回新版本编号	$O(logP)$
`pop(v)`	从非空版本 `v` 的队首出队，返回新版本编号	$O(1)$
`front(v)`	返回版本 `v` 的队首值（不出队）	$O(logP)$
`back(v)`	返回版本 `v` 的队尾值	$O(1)$
`size(v)`	当前队列长度	$O(1)$
`empty(v)`	当前队列是否为空	$O(1)$

`P` 表示执行过的 `push` 总次数。`front(v)` 是对 `ver[v].tail` 跳 `ver[v].len - 1` 级祖先。

可选接口

接口	作用	时间复杂度
`at(v, k)`	队首起第 `k` 个值，`k` 为 0base	$O(logP)$
`copy(v)`	原样复制版本 `v`，返回新版本编号	$O(1)$
`reserve(pushes, versions)`	预留额外入队数和版本数的容量	预分配成本与预留量成正比

裁剪：不需要随机访问可直接删 `at()`；不需要手动容量预留 / 空操作版本可直接删除 `reserve()` / `copy()`。`ancestor()`、`push()`、`pop()` 和基础查询保留。

注意事项

- 所有版本编号均为 0base，`0` 是初始空队列；修改返回的是新版本编号，不是新的队首或队尾下标。
- `pop(v)`、`front(v)`、`back(v)` 要求版本 `v` 非空；`at(v, k)` 要求 `0 ≤ k < size(v)`。不合法操作由 `assert` 检查。
- 出队不删除历史节点。例如 `[1,2,3] → pop → [2,3] → push(4) → [2,3,4]`，新版本仍会共享旧链上曾经出队的节点；通过 `len` 定位有效后缀，功能正确。
- 队列完全出空时，将新版本 `tail` 置为 `0`，之后再次入队会从空链开始，避免历史链无意义地变长。
- 本结构是完全持久化：可以反复以任意旧版本为父版本生成分支；同一 `PersistentQueue` 对象内的节点与版本共享内存。
- `T` 默认为已有的 `i64`，也可以改为 `int`；节点 `0` 使用 `T{}`，所以要求 `T` 可默认构造，常用数值类型均满足。`front/back/at` 按值返回。
- `reserve()` 是性能优化，不影响正确性；如果知道最多有多少次入队和一共多少次产生新版本，可以提前调用。
- 这版采用历史尾链 + 倍增，优先考虑短代码、无需复杂引用计数，以及分支版本正确性。理论上还存在更复杂的 $O(1)$ 持久化实时队列；如果题目对操作量与内存特别敏感，应按约束选择。

常见用法

从任意历史版本分叉

```
PersistentQueue<int> q;
int v1 = q.push(0, 3);     // v1: [3]
int v2 = q.push(v1, 5);    // v2: [3, 5]
int v3 = q.pop(v2);        // v3: [5]
int v4 = q.push(v3, 7);    // v4: [5, 7]
int v5 = q.push(v1, 11);   // v5: [3, 11]

int x = q.front(v4);       // 5
int y = q.back(v5);        // 11
int z = q.at(v2, 1);       // 5, v2 不会被后续操作修改
```

先取队首，再生成出队版本

```
int x = q.front(v2);
int next_version = q.pop(v2);
// x 是旧版本队首；next_version 是删除队首后的新版本
```

Library Checker: Persistent Queue

题目将空初始版本标为 `t = -1`，第 `i` 个询问执行后形成版本 `i`。本结构的版本编号统一加 `1`：题目中的 `t` 映射为 `t + 1`。

```
int Q;
cin >> Q;
PersistentQueue<int> q;
q.reserve(Q, Q);
for (int i = 0; i < Q; ++ i) {
    int op, t;
    cin >> op >> t;
    int v = t + 1;
    if (op == 0) {
        int x;
        cin >> x;
        q.push(v, x);          // 返回的新版本必为 i + 1
    } else {
        cout << q.front(v) << '\n';
        q.pop(v);
    }
}
```

- 测试入口：Library Checker — Persistent Queue。
- `copy()` 只在“需要生成一个不改变内容的新版本”时才用；若仅保留引用，直接保存原版本编号即可。

复杂度

设 `P` 为总入队次数、`V` 为创建的版本总数（含空版本）。

操作	时间复杂度
`push`	$O(logP)$（新建一个节点和其倍增祖先）
`pop`、`back`、`size`、`empty`、`copy`	$O(1)$
`front`、`at`	$O(logP)$

空间复杂度：$O(PlogP+V)$
更精确地，每次入队只创建 1 个 `Node`，并追加 `floor(log2(dep)) + 1` 个 `int` 祖先索引；每次操作额外创建 1 个 `Version`。没有按版本复制队列元素，也不需要固定最大层数的稠密倍增数组。

3.11 多维与嵌套数据结构

3.11.1 BIT 套动态权值线段树

外层按位置维护 Fenwick 树，每棵内部动态权值树支持计数与共同下降求第 k 小。

```
struct BITValueTree {
    struct Node {
        int ch[2]{};
        int sum{};
    };
    int n;
    i64 vl, vr;
    vector<int> root;
    vector<Node> tr{{}};
    vector<int> bin;
    BITValueTree(int n_, i64 vl_, i64 vr_)
        : n(n_), vl(vl_), vr(vr_), root(n + 1) {
        assert(0 <= n && vl < vr);
    }
    BITValueTree(const vector<i64> &a, i64 vl_, i64 vr_)
        : BITValueTree(int(a.size()), vl_, vr_) {
        for (int i = 0; i < n; ++ i) add(i, a[i], 1);
    }
    void reserve(size_t nodes) { tr.reserve(nodes + 1); }
private:
    // ===== 节点复用 =====
    int new_node() {
        if (bin.empty()) {
            tr.push_back({});
            return int(tr.size()) - 1;
        }
        int p = bin.back();
```

```
        bin.pop_back();
        tr[p] = {};
        return p;
    }
    int add_node(int p, i64 l, i64 r, i64 x, int delta) {
        if (!p) {
            assert(delta > 0);
            p = new_node();
        }
        tr[p].sum += delta;
        assert(tr[p].sum >= 0);
        if (l < r - 1) {
            i64 m = midpoint(l, r);
            int d = x >= m;
            tr[p].ch[d] = add_node(tr[p].ch[d], d ? m : l,
                                   d ? r : m, x, delta);
        }
        if (tr[p].sum == 0) {
            assert(!tr[p].ch[0] && !tr[p].ch[1]);
            tr[p] = {};
            bin.push_back(p);
            return 0;
        }
        return p;
    }
    int prefix_node(int p, i64 l, i64 r, i64 x) const {
        if (!p || x <= l) return 0;
        if (r <= x) return tr[p].sum;
        i64 m = midpoint(l, r);
        return prefix_node(tr[p].ch[0], l, m, x)
             + prefix_node(tr[p].ch[1], m, r, x);
    }
public:
    // ===== 基础修改与值域计数 =====
    void add(int pos, i64 x, int delta) {
        assert(0 <= pos && pos < n && vl <= x && x < vr);
        for (++ pos; pos <= n; pos += pos & -pos) {
            root[pos] = add_node(root[pos], vl, vr, x, delta);
        }
    }
    void modify(int pos, i64 old_value, i64 new_value) {
        if (old_value == new_value) return;
        add(pos, old_value, -1);
        add(pos, new_value, 1);
    }
    int prefix_count(int pos, i64 value) const {
        int res = 0;
        for (; pos > 0; pos -= pos & -pos) {
            res += prefix_node(root[pos], vl, vr, value);
        }
        return res;
    }
    int count(int l, int r, i64 x) const {
        return prefix_count(r, x) - prefix_count(l, x);
    }
    int count_range(int l, int r, i64 lo, i64 hi) const {
        return count(l, r, hi) - count(l, r, lo);
    }
    int size(int l, int r) const {
        auto pref = [&](int x) {
            int s = 0;
            for (; x > 0; x -= x & -x) s += tr[root[x]].sum;
            return s;
        };
        return pref(r) - pref(l);
    }
    int rank(int l, int r, i64 x) const { return count(l, r, x) + 1; }
    // ===== 第 k 小：若仅需要计数，可删除此函数 =====
    i64 kth(int l, int r, int k) const {
        assert(1 <= k && k <= size(l, r));
        array<int, 32> a{}, b{};
```

```
        int na = 0, nb = 0;
        for (int x = r; x > 0; x -= x & -x) a[na ++] = root[x];
        for (int x = l; x > 0; x -= x & -x) b[nb ++] = root[x];
        i64 L = vl, R = vr;
        while (L < R - 1) {
            int cnt = 0;
            for (int i = 0; i < na; ++ i) cnt += tr[tr[a[i]].ch[0]].sum;
            for (int i = 0; i < nb; ++ i) cnt -= tr[tr[b[i]].ch[0]].sum;
            i64 M = midpoint(L, R);
            int d = k > cnt;
            if (d) k -= cnt, L = M;
            else R = M;
            for (int i = 0; i < na; ++ i) a[i] = tr[a[i]].ch[d];
            for (int i = 0; i < nb; ++ i) b[i] = tr[b[i]].ch[d];
        }
        return L;
    }
};
```

接口说明

- `BITValueTree(n, vl, vr)` / `BITValueTree(a, vl, vr)`：空建或从数组建树，值域 `[vl, vr)`。
- `add(pos, x, delta)`：在位置 `pos` 增删权值 `x` 的出现次数，$O(\log n \log V)$。
- `modify(pos, old, now)`：单个位置换值，$O(\log n \log V)$。
- `prefix_count(pos, x)` / `count(l,r,x)`：统计位置前缀或 `[l,r)` 中权值 `< x` 的个数，$O(\log n \log V)$。
- `count_range(l,r,lo,hi)`：位置 `[l,r)` 且权值 `[lo,hi)` 的个数，$O(\log n \log V)$。
- `size(l,r)`：元素个数，$O(\log n)$；`rank(l,r,x)`：`< x` 的个数加一，$O(\log n \log V)$。
- `kth(l,r,k)`：区间第 `k` 小，`k` 为 1base，$O(\log n \log V)$。
- `reserve(nodes)`：预分配节点池容量，避免大规模增删时反复扩容。

注意事项

- 数组位置采用 `0base`，区间统一 `[l,r)`；`add` 的频率必须始终非负，删除不存在的值违反前提。
- 权值域必须满足 `vl ≤ x < vr`。`midpoint(l,r)` 无有符号加法溢出风险；须包含 `<numeric>`（通用 `bits/stdc++.h` 已包含）。
- 单点换值时自行维护原数组。空子树会回收到 `bin`，但容器 `tr` 的容量不会自动缩小。
- 无法预先列举权值且需要 **动态第 k 小** 时优先本结构；能够离线登记所有修改值时也可以选择 `3.11.3`。
- 节点 `sum` 使用 `int`，总元素数量不得超过 `INT_MAX`；满 64 位有符号整数域的右开端点无法表示 `LLONG_MAX+1`。

常见用法

```
BITValueTree bit(a, -1000000000LL, 1000000001LL);
bit.modify(pos, a[pos], x);
a[pos] = x;
int k = bit.count(l, r, x);            // < x 的数量
if (k) i64 pre = bit.kth(l, r, k);     // 严格前驱
if (k < bit.size(l,r)) {
    i64 ge = bit.kth(l, r, k + 1);   // ≥ x 的最小值
}
```

复杂度

所有增删与计数/第 k 小操作为 $O(\log n \log V)$，`size` 为 $O(\log n)$。令 U 为历史更新次数，节点池的最坏上界 $O(U \log n \log V)$；回收后活跃节点随当前使用的权值路径数变化，但 `tr` 已分配容量按历史峰值保留。

3.11.2 线段树套 FHQ Treap

当值域不是方便做权值线段树的整数域，或主要需求为动态区间严格前驱/后继时使用。
依赖：3.8.3 FHQ Treap 有序多重集 中的完整 `FHQTreap<T, Compare>`。

```
template<class T, class Compare = less<T>>
struct SegTreeTreap {
    int n, base = 1;
    vector<T> a;
    vector<int> root;
    FHQTreap<T, Compare> tr;
    explicit SegTreeTreap(const vector<T> &a_) : n(a_.size()), a(a_) {
        while (base < n) base <<= 1;
        root.assign(2 * base, 0);
```

```
        for (int i = 0; i < n; ++ i) {
            for (int p = base + i; p; p >>= 1) tr.insert(root[p], a[i]);
        }
    }
    void modify(int pos, const T &value) {
        assert(0 <= pos && pos < n);
        if (!tr.comp(a[pos], value) && !tr.comp(value, a[pos])) {
            a[pos] = value;
            return;
        }
        for (int p = base + pos; p; p >>= 1) {
            bool ok = tr.erase(root[p], a[pos]);
            assert(ok);
            tr.insert(root[p], value);
        }
        a[pos] = value;
    }
    int count_less(int l, int r, const T &value) const {
        int ans = 0;
        for (l += base, r += base; l < r; l >>= 1, r >>= 1) {
            if (l & 1) ans += tr.order_of_key(root[l ++], value);
            if (r & 1) ans += tr.order_of_key(root[-- r], value);
        }
        return ans;
    }
    int count_range(int l, int r, const T &lo, const T &hi) const {
        return count_less(l, r, hi) - count_less(l, r, lo);
    }
    int rank(int l, int r, const T &x) const {
        return count_less(l, r, x) + 1;
    }
    optional<T> predecessor(int l, int r, const T &x) const {
        optional<T> ans;
        auto use = [&](int p) {
            auto v = tr.predecessor(p, x);
            if (v && (!ans || tr.comp(*ans, *v))) ans = v;
        };
        for (l += base, r += base; l < r; l >>= 1, r >>= 1) {
            if (l & 1) use(root[l ++]);
            if (r & 1) use(root[-- r]);
        }
        return ans;
    }
    optional<T> successor(int l, int r, const T &x) const {
        optional<T> ans;
        auto use = [&](int p) {
            auto v = tr.successor(p, x);
            if (v && (!ans || tr.comp(*v, *ans))) ans = v;
        };
        for (l += base, r += base; l < r; l >>= 1, r >>= 1) {
            if (l & 1) use(root[l ++]);
            if (r & 1) use(root[-- r]);
        }
        return ans;
    }
};
```

接口说明

- `SegTreeTreap(a)`：构造，$O(n\log^2 n)$ 期望。
- `modify(pos,x)`：将 `a[pos]` 改为 `x`，内部自动维护原值，$O(\log^2 n)$ 期望。
- `count_less(l,r,x)`：区间内小于 `x` 的数量；`count_range(l,r,lo,hi)`：值域 `[lo,hi)` 的数量，均 $O(\log^2 n)$ 期望。
- `rank(l,r,x)`：严格小于 `x` 的数量加一，$O(\log^2 n)$ 期望。
- `predecessor(l,r,x)` / `successor(l,r,x)`：**严格**前驱/后继，返回 `optional<T>`，$O(\log^2 n)$ 期望。
- `tr`：复用 3.8.3 的节点池；`root[p]`：位置线段树节点 `p` 内的 Treap 根。

注意事项

- **必须先复制 `3.8.3 FHQ Treap 有序多重集 的 FHQTreap 定义`**；本页不重复 Treap 代码。
- 所有区间为 `[l,r)`，修改下标为 `0base`；`T` 必须可用 `Compare` 比较且可复制。
- 本版本刻意不提供第 `k` 小：未知连续值域下用 Treap 做第 k 小需额外二分及值域假设；需要它请优先使用 `3.11.1` 或 `3.11.3`。

- Treap 高度为随机期望界；最坏单次不保证 $O(\log^2 n)$。大量不同键时内存约 $O(n \log n)$ 个 Treap 节点。

常见用法

```
SegTreeTreap<i64> seg(a);
seg.modify(pos, x);
int less = seg.count_less(l, r, x);
auto pre = seg.predecessor(l, r, x);
if (pre) cout << *pre << '\n';
```

复杂度

初始化 $O(n \log^2 n)$ 期望；单点修改、计数、前驱和后继均为 $O(\log^2 n)$ 期望；空间 $O(n \log n)$ 个内部 Treap 节点。

3.11.3 离线 BIT 动态顺序统计

复用 `OfflineBIT2D<int>`：位置为第一维，权值为第二维。适合**提前已知所有修改值**的动态顺序统计。
依赖：3.3.3 离线二维树状数组 中的 `OfflineBIT2D<T>`。

```
struct OfflineKthBIT {
    int n;
    vector<i64> a, xs;
    OfflineBIT2D<int> bit;
    OfflineKthBIT(vector<i64> a_, const vector<pair<int, i64>> &updates)
        : n(a_.size()), a(move(a_)), xs(a), bit(n) {
        for (int i = 0; i < n; ++ i) bit.reserve_point(i, a[i]);
        for (auto [pos, x] : updates) {
            bit.reserve_point(pos, x);
            xs.push_back(x);
        }
        ranges::sort(xs);
        xs.erase(unique(xs.begin(), xs.end()), xs.end());
        bit.build();
        for (int i = 0; i < n; ++ i) bit.add(i, a[i], 1);
    }
    void modify(int pos, i64 x) {
        if (a[pos] == x) return;
        bit.add(pos, a[pos], -1);
        bit.add(pos, x, 1);
        a[pos] = x;
    }
    int count_less(int l, int r, i64 x) const {
        return bit.prefix(r, x) - bit.prefix(l, x);
    }
    int count_range(int l, int r, i64 lo, i64 hi) const {
        return count_less(l, r, hi) - count_less(l, r, lo);
    }
    int rank(int l, int r, i64 x) const { return count_less(l, r, x) + 1; }
    i64 kth(int l, int r, int k) const {
        assert(1 <= k && k <= r - l);
        int low = 0, high = int(xs.size());
        while (low + 1 < high) {
            int mid = (low + high) / 2;
            if (count_less(l, r, xs[mid]) < k) low = mid;
            else high = mid;
        }
        return xs[low];
    }
};
```

接口说明

- `OfflineKthBIT(a, updates)`：建树前预登记所有可能发生的 `(位置, 修改后权值)`；时间 $O((n+q)\log n \log(n+q))$ 量级。
- `modify(pos,x)`：单点赋值，$O(\log^2 n)$（把对数中的内部坐标规模记为 $O(n+q)$ 时）。
- `count_less(l,r,x)` / `count_range(l,r,lo,hi)` / `rank(l,r,x)`：动态区间计数/排名，$O(\log n \log(n+q))$。
- `kth(l,r,k)`：1base 第 `k` 小，$O(\log \sigma \log n \log(n+q))$，$\sigma = |xs|$。
- `a`：当前位置的值；`xs`：全部可能权值的升序去重表。

注意事项

- 必须预先收集修改后的 `(pos,new_value)`；每个位置的初始值由构造器自动登记。查询的阈值无需登记。
- `OfflineBIT2D` 的 `add()` 默认**不检查坐标是否登记**；漏登记修改值会悄悄写错，因此须先离线扫描全部操作。
- 与 `3.11.1` 相比，`kth` 多一层二分，通常更慢；但第二维压缩后内存布局连续，在更新坐标稀疏且完全已知时值得选用。

常见用法

```
vector<pair<int, i64>> updates;
for (auto [pos, x] : all_future_modifications) updates.push_back({pos, x});
OfflineKthBIT bit(a, updates);
bit.modify(pos, x);
int rank = bit.rank(l, r, x);
i64 kth = bit.kth(l, r, k);
```

复杂度

设 $M = n + q$ 为初始元素数与预登记操作数，σ 为不同权值数量。空间 $O(M \log n)$，初始化 $O(M \log n \log M)$，单点修改和计数 $O(\log n \log M)$，第 k 小 $O(\log \sigma \log n \log M)$。

3.11.4 二维线段树

已有二维 BIT 不支持一般最值的单点赋值，因此补充一份短的二维迭代线段树作为动态矩形最值后端。

```
template<class T, class Op>
struct SegTree2D {
    int n, m, sx = 1, sy = 1;
    T e;
    Op op;
    vector<vector<T>> tr;
    SegTree2D(int n_, int m_, T e_, Op op_ = Op())
        : n(n_), m(m_), e(e_), op(op_) {
        while (sx < n) sx <<= 1;
        while (sy < m) sy <<= 1;
        tr.assign(2 * sx, vector<T>(2 * sy, e));
    }
    SegTree2D(const vector<vector<T>> &a, T e_, Op op_ = Op())
        : SegTree2D(int(a.size()), a.empty() ? 0 : int(a[0].size()), e_, op_) {
        for (int i = 0; i < n; ++ i) {
            for (int j = 0; j < m; ++ j) tr[sx + i][sy + j] = a[i][j];
            for (int j = sy - 1; j; -- j) {
                tr[sx + i][j] = op(tr[sx + i][j << 1], tr[sx + i][j << 1 | 1]);
            }
        }
        for (int i = sx - 1; i; -- i) {
            for (int j = 1; j < 2 * sy; ++ j) {
                tr[i][j] = op(tr[i << 1][j], tr[i << 1 | 1][j]);
            }
        }
    }
    void set(int x, int y, const T &v) {
        int p = sx + x, q = sy + y;
        tr[p][q] = v;
        for (int j = q >> 1; j; j >>= 1) {
            tr[p][j] = op(tr[p][j << 1], tr[p][j << 1 | 1]);
        }
        for (p >>= 1; p; p >>= 1) {
            tr[p][q] = op(tr[p << 1][q], tr[p << 1 | 1][q]);
            for (int j = q >> 1; j; j >>= 1) {
                tr[p][j] = op(tr[p][j << 1], tr[p][j << 1 | 1]);
            }
        }
    }
    T query_y(int p, int l, int r) const {
        T left = e, right = e;
        for (l += sy, r += sy; l < r; l >>= 1, r >>= 1) {
            if (l & 1) left = op(left, tr[p][l ++]);
            if (r & 1) right = op(tr[p][-- r], right);
        }
        return op(left, right);
    }
```

```
    T query(int x1, int y1, int x2, int y2) const {
        T ans = e;
        for (x1 += sx, x2 += sx; x1 < x2; x1 >>= 1, x2 >>= 1) {
            if (x1 & 1) ans = op(ans, query_y(x1 ++, y1, y2));
            if (x2 & 1) ans = op(ans, query_y(-- x2, y1, y2));
        }
        return ans;
    }
};
```

接口说明

- `SegTree2D(n,m,e,op)`：空矩阵（初始值为单位元 `e`），$O(nm)$；`SegTree2D(a,e,op)`：用矩阵线性建树，$O(nm)$。
- `set(x,y,value)`：单点赋值，$O(\log n \log m)$。
- `query(x1,y1,x2,y2)`：矩形聚合，$O(\log n \log m)$。
- `e`：聚合单位元；`op`：**交换且结合**的二元运算。

注意事项

- 适用范围：二维单点赋值 + 矩形 `min/max/gcd/sum` 等**可交换结合**的聚合。这里不支持一般非交换运算。
- 单点加 + 矩形和请优先使用 `3.3.2 BIT2D`；矩形加 + 矩形和请复用同页的四棵 BIT；静态矩形最值可选 `3.4.2` 二维 ST 表。
- 下标 `0base`，矩形 `[x1,x2) × [y1,y2)`。空间为 $4s_x s_y$ 个 `T`（其中 s_x, s_y 为二次幂扩容维度），大矩阵可能较耗内存。

常见用法

```
struct MinOp { int operator()(int a, int b) const { return min(a, b); } };
SegTree2D<int, MinOp> seg(a, INT_MAX);
seg.set(x, y, v);
int res = seg.query(x1, y1, x2, y2);
```

复杂度

线性初始化 $O(nm)$；单点赋值、矩形查询 $O(\log n \log m)$；空间 $O(nm)$，具体为 $4s_x s_y$ 个元素，介于 $4nm$ 与约 $16nm$ 之间（非空矩阵）。

3.11.5 KD Tree（静态与动态）

二维空间最近邻和矩形计数；动态版支持插入、按编号删除、自动重构

静态二维 KD Tree

```
struct KDTree {
    using i64 = long long;
    using i128 = __int128_t;
    static constexpr i128 INF = numeric_limits<i128>::max();
    struct Point {
        i64 x[2];
        int id;
    };
    struct Node {
        Point p{};
        int ch[2]{};
        i64 mn[2]{}, mx[2]{};
        int size{1};
    };
    vector<Node> tr{{}};
    int root{0};
    explicit KDTree(vector<Point> p) { root = build(p, 0, int(p.size()), 0); }
    static i128 sq(i128 x) { return x * x; }
    static i128 distance(const Point &a, const Point &b) { return sq(i128(a.x[0]) - b.x[0]) + sq(i128(a.x[1]) - b.x[1]); }
    int new_node(const Point &p) {
        int u = int(tr.size());
        tr.push_back({p});
        for (int d = 0; d < 2; ++d) tr[u].mn[d] = tr[u].mx[d] = p.x[d];
        return u;
    }
    void pull(int u) {
        tr[u].size = 1;
```

```
        for (int c : tr[u].ch) if (c) tr[u].size += tr[c].size;
        for (int d = 0; d < 2; ++d) {
            tr[u].mn[d] = tr[u].mx[d] = tr[u].p.x[d];
            for (int c : tr[u].ch) if (c) {
                tr[u].mn[d] = min(tr[u].mn[d], tr[c].mn[d]);
                tr[u].mx[d] = max(tr[u].mx[d], tr[c].mx[d]);
            }
        }
    }
    int build(vector<Point> &p, int l, int r, int depth) {
        if (l == r) return 0;
        int d = depth & 1, m = (l + r) / 2;
        nth_element(p.begin() + l, p.begin() + m, p.begin() + r,
            [d](const Point &a, const Point &b) {
                return pair{a.x[d], a.id} < pair{b.x[d], b.id};
            });
        int u = new_node(p[m]);
        tr[u].ch[0] = build(p, l, m, depth + 1);
        tr[u].ch[1] = build(p, m + 1, r, depth + 1);
        pull(u);
        return u;
    }
    i128 min_box(int u, const Point &q) const {
        i128 ans = 0;
        for (int d = 0; d < 2; ++d) {
            if (q.x[d] < tr[u].mn[d]) ans += sq(i128(tr[u].mn[d]) - q.x[d]);
            if (q.x[d] > tr[u].mx[d]) ans += sq(i128(q.x[d]) - tr[u].mx[d]);
        }
        return ans;
    }
    i128 max_box(int u, const Point &q) const {
        i128 ans = 0;
        for (int d = 0; d < 2; ++d) {
            ans += max(sq(i128(q.x[d]) - tr[u].mn[d]), sq(i128(q.x[d]) - tr[u].mx[d]));
        }
        return ans;
    }
    void nearest(int u, const Point &q, int ban, pair<i128, int> &best) const {
        if (!u) return;
        if (tr[u].p.id != ban) {
            best = min(best, pair{distance(tr[u].p, q), tr[u].p.id});
        }
        int a = tr[u].ch[0], b = tr[u].ch[1];
        if (a && b && min_box(a, q) > min_box(b, q)) swap(a, b);
        if (a && min_box(a, q) <= best.first) nearest(a, q, ban, best);
        if (b && min_box(b, q) <= best.first) nearest(b, q, ban, best);
    }
    void farthest(int u, const Point &q, int ban, pair<i128, int> &best) const {
        if (!u) return;
        if (tr[u].p.id != ban) {
            best = max(best, pair{distance(tr[u].p, q), -tr[u].p.id});
        }
        int a = tr[u].ch[0], b = tr[u].ch[1];
        if (a && b && max_box(a, q) < max_box(b, q)) swap(a, b);
        if (a && max_box(a, q) >= best.first) farthest(a, q, ban, best);
        if (b && max_box(b, q) >= best.first) farthest(b, q, ban, best);
    }
    void k_nearest(int u, const Point &q, int ban, int k,
                   priority_queue<pair<i128, int>> &heap) const {
        if (!u) return;
        if (tr[u].p.id != ban) {
            pair<i128, int> cand{distance(tr[u].p, q), tr[u].p.id};
            if (int(heap.size()) < k) heap.push(cand);
            else if (cand < heap.top()) {
                heap.pop();
                heap.push(cand);
            }
        }
        int a = tr[u].ch[0], b = tr[u].ch[1];
        if (a && b && min_box(a, q) > min_box(b, q)) swap(a, b);
        auto possible = [&](int v) {
```

```
            return v && (int(heap.size()) < k || min_box(v, q) <= heap.top().first);
        };
        if (possible(a)) k_nearest(a, q, ban, k, heap);
        if (possible(b)) k_nearest(b, q, ban, k, heap);
    }
    // ===== 静态点集矩形计数 =====
    int count_rect(int u, i64 x1, i64 y1, i64 x2, i64 y2) const {
        if (!u || tr[u].mx[0] < x1 || tr[u].mn[0] >= x2
               || tr[u].mx[1] < y1 || tr[u].mn[1] >= y2) return 0;
        if (x1 <= tr[u].mn[0] && tr[u].mx[0] < x2
         && y1 <= tr[u].mn[1] && tr[u].mx[1] < y2) return tr[u].size;
        const auto &p = tr[u].p;
        int ans = (x1 <= p.x[0] && p.x[0] < x2 && y1 <= p.x[1] && p.x[1] < y2);
        for (int c : tr[u].ch) ans += count_rect(c, x1, y1, x2, y2);
        return ans;
    }
    int count_rect(i64 x1, i64 y1, i64 x2, i64 y2) const {
        return count_rect(root, x1, y1, x2, y2);
    }
    pair<i128, int> nearest(const Point &q, int ban = -1) const {
        pair<i128, int> ans{INF, numeric_limits<int>::max()};
        nearest(root, q, ban, ans);
        return ans;
    }
    pair<i128, int> farthest(const Point &q, int ban = -1) const {
        pair<i128, int> ans{-1, numeric_limits<int>::min()};
        farthest(root, q, ban, ans);
        if (ans.first >= 0) ans.second = -ans.second;
        return ans;
    }
    vector<pair<i128, int>> k_nearest(const Point &q, int k, int ban = -1) const {
        if (k <= 0) return {};
        priority_queue<pair<i128, int>> heap;
        k_nearest(root, q, ban, k, heap);
        vector<pair<i128, int>> ans;
        while (!heap.empty()) ans.push_back(heap.top()), heap.pop();
        reverse(ans.begin(), ans.end());
        return ans;
    }
};
```

动态二维 KD Tree

```
struct DynamicKDTree {
    using i64 = long long;
    using i128 = __int128_t;
    static constexpr i128 INF = numeric_limits<i128>::max();
    struct Point { i64 x[2]; int id; };
    struct Node {
        Point p{};
        int ch[2]{}, size{1}, dim{0};
        i64 mn[2]{}, mx[2]{};
        bool alive{true};
    };
    vector<Node> tr{{}};
    vector<int> bin;
    unordered_map<int, int> where;
    int root{0}, live_count{0};
    static i128 sq(i128 x) { return x * x; }
    static i128 distance(const Point &a, const Point &b) { return sq(i128(a.x[0]) - b.x[0]) + sq(i128(a.x[1]) - b.x[1]); }
    int size(int u) const { return u ? tr[u].size : 0; }
    int new_node(Point p, int dim) {
        int u;
        if (bin.empty()) {
            u = int(tr.size());
            tr.push_back({});
        } else {
            u = bin.back();
```

```
            bin.pop_back();
        }
        tr[u] = Node{p, {}, 1, dim};
        for (int d = 0; d < 2; ++d) tr[u].mn[d] = tr[u].mx[d] = p.x[d];
        where[p.id] = u;
        ++live_count;
        return u;
    }
    void pull(int u) {
        tr[u].size = 1;
        for (int d = 0; d < 2; ++d) tr[u].mn[d] = tr[u].mx[d] = tr[u].p.x[d];
        for (int c : tr[u].ch) if (c) {
            tr[u].size += tr[c].size;
            for (int d = 0; d < 2; ++d) {
                tr[u].mn[d] = min(tr[u].mn[d], tr[c].mn[d]);
                tr[u].mx[d] = max(tr[u].mx[d], tr[c].mx[d]);
            }
        }
    }
    void collect(int u, vector<int> &ids) {
        if (!u) return;
        collect(tr[u].ch[0], ids);
        if (tr[u].alive) ids.push_back(u);
        else bin.push_back(u);
        collect(tr[u].ch[1], ids);
    }
    int rebuild(vector<int> &ids, int l, int r, int depth) {
        if (l == r) return 0;
        int d = depth & 1, m = (l + r) / 2;
        nth_element(ids.begin() + l, ids.begin() + m, ids.begin() + r,
            [&](int a, int b) {
                return pair{tr[a].p.x[d], tr[a].p.id}
                     < pair{tr[b].p.x[d], tr[b].p.id};
            });
        int u = ids[m];
        tr[u].dim = d;
        tr[u].ch[0] = rebuild(ids, l, m, depth + 1);
        tr[u].ch[1] = rebuild(ids, m + 1, r, depth + 1);
        pull(u);
        return u;
    }
    int rebuild_subtree(int u, int depth) {
        vector<int> ids;
        collect(u, ids);
        return rebuild(ids, 0, int(ids.size()), depth);
    }
    bool unbalanced(int u) const { return 4LL * max(size(tr[u].ch[0]), size(tr[u].ch[1])) > 3LL * tr[u].size; }
    int insert(int u, Point p, int depth) {
        if (!u) return new_node(p, depth & 1);
        int d = tr[u].dim;
        int side = pair{p.x[d], p.id} > pair{tr[u].p.x[d], tr[u].p.id};
        tr[u].ch[side] = insert(tr[u].ch[side], p, depth + 1);
        pull(u);
        if (unbalanced(u)) u = rebuild_subtree(u, depth);
        return u;
    }
    void insert(Point p) {
        assert(!where.contains(p.id));
        root = insert(root, p, 0);
    }
    bool erase(int id) {
        auto it = where.find(id);
        if (it == where.end()) return false;
        tr[it->second].alive = false;
        where.erase(it);
        --live_count;
        if (root && 2LL * live_count < size(root)) root = rebuild_subtree(root, 0);
        return true;
    }
    i128 min_box(int u, const Point &q) const {
        i128 ans = 0;
```

```
        for (int d = 0; d < 2; ++d) {
            if (q.x[d] < tr[u].mn[d]) ans += sq(i128(tr[u].mn[d]) - q.x[d]);
            if (q.x[d] > tr[u].mx[d]) ans += sq(i128(q.x[d]) - tr[u].mx[d]);
        }
        return ans;
    }
    void nearest(int u, const Point &q, int ban, pair<i128, int> &best) const {
        if (!u) return;
        if (tr[u].alive && tr[u].p.id != ban) {
            best = min(best, pair{distance(tr[u].p, q), tr[u].p.id});
        }
        int a = tr[u].ch[0], b = tr[u].ch[1];
        if (a && b && min_box(a, q) > min_box(b, q)) swap(a, b);
        if (a && min_box(a, q) <= best.first) nearest(a, q, ban, best);
        if (b && min_box(b, q) <= best.first) nearest(b, q, ban, best);
    }
    pair<i128, int> nearest(Point q, int ban = -1) const {
        pair<i128, int> ans{INF, numeric_limits<int>::max()};
        nearest(root, q, ban, ans);
        return ans;
    }
};
```

接口说明

- 静态 `KDTree(points)`：构造，通常 $O(n\log n)$；`nearest(q,ban)`：最近点，`farthest(q,ban)`：最远点，返回 `{距离平方,id}`。
- 静态 `k_nearest(q,k,ban)`：按 `{距离平方,id}` 升序输出至多 `k` 个点；`count_rect(x1,y1,x2,y2)`：统计半开矩形中的点数。
- 动态 `DynamicKDTree::insert({{x,y},id})`：插入新点；`erase(id)`：成功删除返回 `true`，未找到返回 `false`；`nearest(q,ban)`：当前活跃点中最近点。
- 两版最近邻不存在候选时均返回 `{INF,INT_MAX}`；静态最远点不存在时返回 `{-1,INT_MIN}`；`ban` 默认 `-1`，用于排除指定 `id`。

注意事项

- 必须在坐标差与乘方前转换为 `i128`；已修复 `i64` 先做差导致的溢出。仍须保证最终两维平方和不超过 `i128` 上限。
- `id` 应使用非负整数，不能与默认排除哨兵 `-1` 冲突；同一活动点集中不允许重复 `id`。
- 距离相同按较小 `id` 决胜，因此矩形剪枝必须保留相等边界。
- 动态版的删除为惰性删除：节点失衡或死亡节点过多时重构；被重构丢弃的节点编号进入 `bin` 以供复用。
- 动态版仅封装最近邻，矩形查询是静态版提供的可选功能；不要误认为动态版也实现了 `count_rect`。
- KD-Tree 的近邻查询最坏 $O(n)$；均衡构造和包围盒剪枝仅改善通常情形，不能保证每次对数复杂度。

常见用法

```
KDTree kd(points);
auto [dist2, id] = kd.nearest(query, ban);
auto near = kd.k_nearest(query, k);
int cnt = kd.count_rect(x1, y1, x2, y2);

DynamicKDTree dyn;
dyn.insert({{x, y}, id});
bool ok = dyn.erase(id);
auto [d2, who] = dyn.nearest(query);
```

复杂度

静态构造通常 $O(n\log n)$、空间 $O(n)$。静态最近邻/最远邻/k 近邻与矩形查询均依赖剪枝效果，最坏可达 $O(n)$（`k_nearest` 另有结果堆操作成本）。动态插入包含偶发重构，无法保证对数最坏时间；全局重构 $O(n\log n)$，节点池按历史峰值分配。

3.11.6 可回收频率与权值和树

计数 `cnt` + 任意附加权重 `sum` 的独立后端。和 `3.6.4` 的 `sum += x * delta` 变式不同，本结构允许同一权值对应任意附加权重；若只维护频率，可直接复用 `3.6.4`，不复制本结构。

```
struct RecyclableValueTree {
    struct Node {
        int ch[2]{};
        int cnt{ 0 };
        i64 sum{ 0 };
    };
    int vl, vr;
```

```
    vector<Node> tr{{}};
    vector<int> bin;
    RecyclableValueTree(int vl_, int vr_) : vl(vl_), vr(vr_) { assert(vl < vr); }
    int new_node() {
        if (bin.empty()) {
            tr.push_back({});
            return int(tr.size()) - 1;
        }
        int p = bin.back();
        bin.pop_back();
        tr[p] = {};
        return p;
    }
    int add(int p, int l, int r, int x, int dc, i64 ds) {
        if (!p) {
            assert(dc > 0);
            p = new_node();
        }
        tr[p].cnt += dc;
        tr[p].sum += ds;
        assert(tr[p].cnt >= 0);
        if (l < r - 1) {
            int m = midpoint(l, r);
            if (x < m) tr[p].ch[0] = add(tr[p].ch[0], l, m, x, dc, ds);
            else tr[p].ch[1] = add(tr[p].ch[1], m, r, x, dc, ds);
        }
        if (tr[p].cnt == 0) {
            assert(!tr[p].ch[0] && !tr[p].ch[1]);
            assert(tr[p].sum == 0);
            tr[p] = {};
            bin.push_back(p);
            return 0;
        }
        return p;
    }
    pair<int, i64> query(int p, int l, int r, int ql, int qr) const {
        if (!p || qr <= l || r <= ql) return {0, 0};
        if (ql <= l && r <= qr) return {tr[p].cnt, tr[p].sum};
        int m = midpoint(l, r);
        auto a = query(tr[p].ch[0], l, m, ql, qr);
        auto b = query(tr[p].ch[1], m, r, ql, qr);
        return {a.first + b.first, a.second + b.second};
    }
    int add(int root, int x, int dc, i64 ds) {
        assert(vl <= x && x < vr);
        return add(root, vl, vr, x, dc, ds);
    }
    pair<int, i64> query(int root, int l, int r) const { return query(root, vl, vr, l, r); }
};
```

接口说明

- `RecyclableValueTree(vl,vr)`：整数权值域 `[vl,vr)`。
- `add(root,x,dc,ds)`：在 `x` 处同时调整频率 `dc` 与附加权重和 `ds`，返回新根，$O(\log V)$。
- `query(root,l,r)`：返回值域 `[l,r)` 的 `{cnt,sum}`，$O(\log V)$。
- `tr`：共享节点池，`bin`：可复用节点编号；一个实例可通过多个根维护多个集合。

注意事项

- 插入元素 `(x,w)` 使用 `add(root,x,+1,+w)`，删除使用 `add(root,x,-1,-w)`。
- 任意时刻每个权值的频率必须非负，删除对象必须真实存在；`cnt==0` 时对应整棵子树必须确实为空且 `sum==0`。
- 更新得到的新根必须写回。后续操作不得再使用已经被回收的旧根。
- 若只做单棵普通区间和，不必引入这份结构，直接复用 `3.6.1 DynamicSegTree<Info>` 即可。

常见用法

```
RecyclableValueTree seg(0, V);
int root = 0;
root = seg.add(root, x, 1, weight);
```

```
root = seg.add(root, x, -1, -weight);
auto [cnt, sum] = seg.query(root, lo, hi);
```

复杂度

增删与查询 $O(\log V)$；活跃节点数取决于非空路径，节点池已分配内存保持历史峰值容量。